



**TRƯỜNG ĐẠI HỌC
KINH TẾ - KỸ THUẬT CÔNG NGHIỆP**
UNIVERSITY OF ECONOMICS- TECHNOLOGY FOR INDUSTRIES

KHOA CÔNG NGHỆ THÔNG TIN

Trần Thị Hương (Chủ biên)

Mai Mạnh Trùng

Trần Minh Đức



TÀI LIỆU HỌC TẬP CÔNG NGHỆ PHẦN MỀM NGÀNH CÔNG NGHỆ THÔNG TIN

HÀ NỘI: 2022

LƯU HÀNH NỘI BỘ

MỤC LỤC

DANH MỤC HÌNH ẢNH	I
DANH MỤC BẢNG BIỂU	II
DANH MỤC CÁC TỪ VIẾT TẮT	III
LỜI NÓI ĐẦU.....	1
CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ PHẦN MỀM	2
1.1 Giới thiệu chung về phần mềm.....	2
1.1.1. Khái niệm về phần mềm.....	2
1.1.2. Các đặc tính của phần mềm.....	2
1.1.3. Tiêu chuẩn của sản phẩm phần mềm.....	3
1.1.4. Phân loại phần mềm	6
1.2 Giới thiệu về Công nghệ phần mềm.....	8
1.2.1. Khái niệm Công nghệ phần mềm	8
1.2.2. Lịch sử phát triển ngành Công nghệ phần mềm.....	9
1.2.3. Vòng đời phần mềm	10
1.2.4. Phương pháp phát triển phần mềm.....	10
1.2.5. Một số mô hình phát triển phần mềm.....	11
CHƯƠNG 2: KHẢO SÁT – PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU.....	18
2.1 Tìm hiểu, xác định yêu cầu.....	18
2.1.1. Khảo sát, tìm hiểu yêu cầu	18
2.1.2. Đánh giá các yêu cầu	19
2.2 Phân tích yêu cầu.....	20
2.3 Đặc tả yêu cầu.....	22
2.3.1. Đặc tả yêu cầu	22
2.3.2. Nguyên lý đặc tả.....	24
2.4 Tư liệu hóa yêu cầu phần mềm.....	25
2.5 Đặc tính dữ liệu và các kỹ thuật để thu thập dữ liệu	27
2.5.1. Đặc tính dữ liệu	27
2.5.2. Các kỹ thuật để thu thập dữ liệu cho ứng dụng.....	30
CHƯƠNG 3. THIẾT KẾ PHẦN MỀM.....	42
3.1 Khái niệm thiết kế phần mềm.....	42
3.2 Đặc điểm và nguyên tắc thiết kế phần mềm.....	42
3.2.1. Đặc điểm.....	42
3.2.2. Nguyên tắc.....	43
3.3 Các hoạt động của quá trình thiết kế phần mềm	44
3.4 Một số yêu cầu thiết kế.....	48
3.5 Kỹ thuật thiết kế	49

3.5.1. Thiết kế hướng chức năng	49
3.5.2. Thiết kế hướng đối tượng	50
3.5.3. Thiết kế có cấu trúc	53
3.5.4. Thiết kế thời gian thực.....	54
3.5.5. Thiết kế dữ liệu.....	56
3.5.6. Thiết kế thủ tục.....	59
3.5.7. Thiết kế giao diện người sử dụng	60
CHƯƠNG 4. CÀI ĐẶT PHẦN MỀM	65
4.1 Nền tảng của ngôn ngữ lập trình	65
4.1.1. Kiểu dữ liệu, định nghĩa kiểu dữ liệu và kiểm tra kiểu dữ liệu	65
4.1.2. Chương trình con.....	66
4.1.3. Cấu trúc điều khiển.....	67
4.1.4. Vào và ra dữ liệu	68
4.1.5. Quản lý bộ nhớ	69
4.1.6. Quản lý lỗi	69
4.2 Phong cách cài đặt phần mềm	69
4.2.1. Tài liệu chương trình	69
4.2.2. Khai báo dữ liệu	70
4.2.3. Xây dựng câu lệnh.....	71
4.2.4. Vào/ra	71
4.3 Các đặc trưng của ngôn ngữ cài đặt	72
4.4 Phân lớp và đánh giá về ngôn ngữ cài đặt.....	73
4.4.1. Các lớp ngôn ngữ	73
4.4.2. So sánh, đánh giá về một số ngôn ngữ cài đặt	74
4.4.3. Chọn ngôn ngữ cho ứng dụng	77
4.5 Hiệu quả của chương trình và tầm quan trọng của ngôn ngữ cài đặt.....	78
4.6 Một số vấn đề trong cải tiến hiệu suất.....	79
4.6.1. Tốc độ xử lý.....	79
4.6.2. Không gian bộ nhớ	86
4.6.3. Lựa chọn hệ thống và phần cứng	88
4.7 Công cụ trợ giúp và phân loại	88
4.7.1. Công cụ CASE	88
4.7.2. Phân loại các công cụ CASE.....	92
4.7.3. Một số công cụ được cung cấp tự động cho việc sinh mã	97
CHƯƠNG 5: KIỂM THỬ PHẦN MỀM.....	101
5.1 Độ tin cậy của phần mềm	101
5.1.1. Chất lượng phần mềm và việc đảm bảo chất lượng phần mềm	101
5.1.2. Độ tin cậy của phần mềm	103
5.2 Kiểm thử và các chiến lược kiểm thử phần mềm.....	109
5.2.1. Kiểm thử phần mềm, khái niệm, vai trò và lợi ích của kiểm thử phần mềm.....	109
5.2.2. Các chiến lược kiểm thử phần mềm.	110
5.3 Kỹ thuật kiểm thử phần mềm	119

5.3.1. Kỹ thuật kiểm thử phần mềm là gì? Phân loại?	119
5.3.2. Kỹ thuật Phân vùng tương đương	120
5.3.3. Kỹ thuật Phân tích giá trị biên.....	121
5.3.4. Bảng quyết định.....	123
5.3.5. Đoán lỗi	123
5.4 Quy trình kiểm thử phần mềm.....	124
5.4.1. Requirement analysis - Phân tích yêu cầu.....	124
5.4.2. Test planning - Lập kế hoạch kiểm thử	125
5.4.3. Test case development - Thiết kế kịch bản kiểm thử	126
5.4.4. Test environment set up - Thiết lập môi trường kiểm thử	127
5.4.5. Test execution - Thực hiện kiểm thử.....	127
5.4.6. Test cycle closure - Đóng chu trình kiểm thử	128
CHƯƠNG 6: BẢO TRÌ PHẦN MỀM VÀ QUẢN LÝ THAY ĐỔI PHẦN MỀM ..	133
6.1 Định nghĩa bảo trì phần mềm và phân loại	133
6.1.1. Định nghĩa bảo trì phần mềm:	133
6.1.2. Phân loại bảo trì phần mềm:	133
6.2 Đặc điểm của bảo trì phần mềm	135
6.2.1. Bảo trì có cấu trúc đối với bảo trì không cấu trúc.	135
6.2.2. Giá thành bảo trì	136
6.2.3. Một số vấn đề khác.....	137
6.3 Công việc bảo trì phần mềm và một số hiệu ứng	138
6.3.1. Khả năng bảo trì	138
6.3.2. Các công việc bảo trì	139
6.3.3. Một số hiệu ứng lề của công việc bảo trì	141
6.4 Một số hình thức bảo trì phần mềm.....	142
6.4.1. Bảo trì mã chương trình xa lạ.....	142
6.4.2. Công nghệ phản hồi và công nghệ tái sử dụng.....	143
6.4.3. Bảo trì phòng ngừa	144
6.4.4. Chiến lược phần mềm thành phần.....	144
6.5 Quản lý thay đổi phần mềm.....	145
6.5.1. Các thủ tục quản lý thay đổi	145
6.5.2. Ghi quyết định theo thời gian.....	147
6.5.3. Quản lý thay đổi tài liệu	148
TÀI LIỆU THAM KHẢO.....	150

DANH MỤC HÌNH ẢNH

Hình 1.1 Mô hình ba tầng của Công nghệ phần mềm.....	9
Hình 1.2 Sơ đồ mô hình tuyến tính	11
Hình 1.3 Sơ đồ mô hình mẫu.....	12
Hình 1.4 Sơ đồ mô hình xoắn ốc	13
Hình 1.5 Sơ đồ mô hình đài phun nước.....	14
Hình 1.6 Sơ đồ mô hình phát triển dựa trên thành phần	15
Hình 2.1 Quy trình xây dựng các chức năng hệ thống	21
Hình 3.1 Mô hình có thiết kế và không có thiết kế	43
Hình 3.2 Môi liên hệ thiết kế phần mềm và Công nghệ phần mềm.....	44
Hình 3.3 Môi quan hệ giữa khía cạnh quản lý và khía cạnh kỹ thuật	45
Hình 3.4 Quan hệ giữa thiết kế và đặc tả	46
Hình 3.5 Các hoạt động thiết kế và sản phẩm của thiết kế [3].....	46
Hình 3.6 Thiết kế nên theo chiều sâu	48
Hình 3.7 Hiệu quả phạm vi module.....	49
Hình 3.8 Luồng thông tin	53
Hình 3.9 Luồng giao tác	54
Hình 3.10 Cấu trúc chương trình và các chỉ số	59
Hình 4.1 Môi trường CASE	89
Hình 4.2 Biên dịch ngôn ngữ thể hệ thứ tư	94
Hình 4.3 Bộ sinh ứng dụng.....	94
Hình 4.4 Bộ sinh mã.....	95
Hình 4.5 Các công cụ mẫu	96
Hình 5.1 Các thông số của Kiểm thử phi chức năng.....	113
Hình 6.1 Chi phí của việc phát triển phần mềm không có phương pháp	137
Hình 6.2 Các công việc chính của quá trình bảo trì	139

DANH MỤC BẢNG BIỂU

Bảng 2-1 Bảng mối quan hệ giữa các kiểu ứng dụng và đặc tính dữ liệu.....	28
Bảng 2-2 Ưu, nhược điểm các loại phỏng vấn	32
Bảng 2-3 Bảng tham khảo các hành vi và hoạt động tương ứng.....	33
Bảng 2-4 Tính phù hợp của các kỹ thuật thu thập dữ liệu	38
Bảng 2-5 Tính phù hợp của các kỹ thuật ứng dụng với các kiểu ứng dụng.....	39
Bảng 4-1 Minh họa lưu trữ mảng 2 chiều 200x150	86
Bảng 4-2 Một số CASE thông dụng.....	98
Bảng 5-1 Quy trình kiểm thử phần mềm.....	128

DANH MỤC CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Viết chi tiết	Ý nghĩa
1	AIP	Application Programming Interface	Giao diện chương trình ứng dụng
2	ANSI	American National Standards Institute	Viện tiêu chuẩn quốc gia Hoa Kỳ
3	BA	Business Analysis	Phân tích kinh doanh
4	CASE	Computer-Aided Software Engineering	Công cụ hỗ trợ phát triển phần mềm
5	CPU	Central Processing Unit	Đơn vị xử lý trung tâm
6	DFD	Data Flow Diagram	Sơ đồ luồng dữ liệu.
7	GUI	Graphical User Interface	Giao diện đồ họa người dùng
8	HTML	Hyper Text Markup Language	Ngôn ngữ đánh dấu siêu văn bản
9	I&T	Integration and Testing	Tích hợp và kiểm thử
10	I/O	Input/Output	Vào ra dữ liệu
11	IDE	Integrated Development Environment	Môi trường phát triển tích hợp
12	IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử
13	PM	Project Manager	Quản lý dự án
14	PTHĐT		Phân tích hướng đối tượng
15	Q&A	Question and Answer	Câu hỏi và trả lời
16	QA	Quality Assurance	Đảm bảo chất lượng
17	RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
18	SCM	Supply Chain Management	Quản lý chuỗi cung ứng
19	SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
20	STLC	Software testing life circle	Vòng đời kiểm thử phần mềm
21	TCP/IP	Transmission Control Protocol/ Internet Protocol	Giao thức TCP/IP
22	XSS	Cross-site scripting	Đoạn mã nguy hiểm

LỜI NÓI ĐẦU

Ngày 15 tháng 1 năm 1945 máy tính điện tử đầu tiên có tên là ENIAC đầu tiên ra đời tại Mỹ. Đến những năm 60, những chương trình điều khiển máy tính đầu tiên đã ra đời, số lượng phần mềm cũng tăng lên rất nhanh, lúc bấy giờ đã xuất hiện “cuộc khủng hoảng phần mềm”, và người ta đã nhận ra nguyên nhân sâu xa là do chưa có các phương pháp phát triển phần mềm. Vì vậy cần phải có các phương pháp, các kỹ thuật và các quy trình trong việc phát triển các phần mềm.

Công nghệ phần mềm là ngành kỹ thuật liên quan đến tất cả các khía cạnh sản xuất phần mềm. Các hoạt động nền tảng của công nghệ phần mềm là đặc tả phần mềm, phát triển phần mềm, kiểm nghiệm phần mềm, và tiến hóa phần mềm.

Học phần Công nghệ phần mềm là học phần trang bị cho sinh viên hệ thống các khái niệm, các thuật ngữ chuyên môn trong lĩnh vực công nghệ phần mềm. Giúp sinh viên hiểu được các công việc của toàn bộ quá trình phát triển một sản phẩm phần mềm trong thực tế, đồng thời giúp sinh viên biết cách vận dụng các kiến thức đã học, đã trau dồi được sử dụng như thế nào trong công việc phát triển phần mềm.

Tài liệu học tập gồm 6 chương

Chương 1: Tổng quan về Công nghệ phần mềm

Chương 2: Khảo sát - phân tích và đặc tả yêu cầu

Chương 3: Thiết kế phần mềm

Chương 4: Cài đặt phần mềm

Chương 5: Kiểm thử phần mềm

Chương 6: Bảo trì và quản lý thay đổi phần mềm

Như vậy, Tài liệu học tập đã bao trùm đầy đủ các vấn đề theo đề cương môn học. Mỗi chương đều có các phần theo đúng yêu cầu, có các khái niệm và các phương pháp, các quy trình liên quan đến một giai đoạn cụ thể của quá trình phát triển phần mềm thực tế.

Tài liệu học tập là công sức, tâm huyết, là sản phẩm lao động khoa học nghiêm túc của nhóm Giảng viên giảng dạy môn Công nghệ phần mềm khoa Công nghệ Thông tin, trường Đại học Kinh tế Kỹ thuật Công nghiệp. Chúng tôi hy vọng, cuốn tài liệu sẽ giúp ích cho sinh viên, giảng viên học tập và giảng dạy môn Công nghệ phần mềm nói riêng và giúp ích cho sinh viên trong quá trình học tập cũng như làm đồ án tốt nghiệp và trong công việc sau khi ra trường.

Hà Nội, ngày... tháng 8 năm 2022

NHÓM BIÊN SOẠN

Trần Thị Hương – Mai Mạnh Trùng – Trần Minh Đức

CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ PHẦN MỀM

Mục tiêu của chương:

Mục tiêu của chương này giới thiệu chung về Công nghệ phần mềm như khái niệm về phần mềm, Công nghệ phần mềm, các đặc tính cũng như tiêu chuẩn của sản phẩm phần mềm. Kế đến chúng ta sẽ tìm hiểu về lịch sử phát triển ngành Công nghệ phần mềm và một số mô hình phát triển phần mềm.

1.1 Giới thiệu chung về phần mềm

1.1.1. Khái niệm về phần mềm

Chương trình máy tính là một trình tự các chỉ thị (lệnh) để hướng dẫn máy tính làm việc nhằm hoàn thành một công việc nào đó do con người yêu cầu.

Phần mềm là một hệ thống các chương trình có thể thực hiện trên máy tính nhằm hỗ trợ các nhà chuyên môn trong từng lĩnh vực chuyên ngành thực hiện tốt nhất các thao tác nghiệp vụ của mình. Nhiệm vụ chính yếu của phần mềm là cho phép các công việc của họ trên máy tính dễ dàng và nhanh chóng hơn so với khi thực hiện cùng công việc đó trong thế giới thực.

Hoạt động của mọi phần mềm là sự mô phỏng lại các hoạt động của thế giới thực trong một góc độ thu hẹp nào đó trên máy tính. Quá trình sử dụng một phần mềm chính là quá trình người dùng thực hiện các công việc trên máy tính để hoàn tất một công việc tương đương trong thế giới thực.

1.1.2. Các đặc tính của phần mềm

Các đặc tính phần mềm là tất cả các điểm chung cho mọi ứng dụng: dữ liệu, các tiến trình, các ràng buộc, và các giao diện.

1.1.2.1 Dữ liệu

Đầu vào: Dữ liệu vào là dữ liệu ở bên ngoài máy tính, và chúng được đưa vào bằng cách sử dụng một thiết bị đầu vào. Thiết bị đầu vào được sử dụng để đưa dữ liệu vào máy tính có thể là: bàn phím, máy quét, hoặc được truyền từ một máy tính khác.

Đầu ra: Dữ liệu ra ngược lại so với dữ liệu vào ở chỗ, đầu ra là các dữ liệu được đưa ra ngoài máy tính. Một số các thiết bị đầu ra như máy in, màn hình hiển thị, một máy tính khác...

Sự lưu trữ dữ liệu và sự tìm kiếm dữ liệu: Dữ liệu được mô tả ở dạng vật lý, trong một máy có thể đọc được các khuôn dạng dữ liệu. Việc tìm kiếm dữ liệu được hiểu là bạn có thể truy nhập vào dữ liệu ở dạng lưu trữ của nó. Việc lưu trữ và tìm kiếm luôn đi cùng với nhau (cả ở mức quan niệm lẫn trong các chương trình phần mềm). Việc lưu trữ dữ liệu đòi hỏi hai kiểu định nghĩa dữ liệu là kiểu vật lý và kiểu logic.

1.1.2.2 Xử lý

Xử lý bao gồm một chuỗi các lệnh hoặc các sự kiện có liên quan với nhau làm việc với các dữ liệu. Kết quả của một xử lý có thể là: làm thay đổi cơ sở dữ liệu, đưa dữ liệu trả

lời ra thiết bị đầu cuối, máy in hoặc in ra giấy, có thể là những yêu cầu về các trang thiết bị, sản sinh những chương trình, hoặc lưu trữ những luật, những thông tin mới, được suy diễn ra về các tình huống, các phần tử.

1.1.2.3 Ràng buộc

Ràng buộc bao gồm: ràng buộc thứ tự trước, ràng buộc thứ tự sau, ràng buộc thời gian, ràng buộc cấu trúc, ràng buộc điều khiển và cả ràng buộc về tham chiếu.

- Ràng buộc về thứ tự trước: Bắt buộc về thứ tự trước là điều kiện đầu tiên phải được đáp ứng để có thể bắt đầu quá trình xử lý.
- Ràng buộc về thứ tự sau: Ràng buộc loại này là điều kiện cần phải thỏa mãn để quá trình xử lý có thể hoàn thành được. Cụm câu lệnh này được đưa vào cuối quá trình xử lý.
- Ràng buộc về thời gian: Bao gồm ràng buộc về thời gian xử lý, thời gian phân chia cho một quá trình xử lý, thời gian yêu cầu đối với các quá trình xử lý bên ngoài, thời gian xử lý đồng bộ, thời gian trả lời cho quá trình xử lý với giao diện ngoài.
- Ràng buộc về mặt cấu trúc: Có thể hiểu là bao gồm việc xác định loại đầu vào và đầu ra của dữ liệu nào được cho phép, quá trình xử lý được thực hiện như thế nào và mối quan hệ giữa các quá trình với nhau.
- Ràng buộc về điều khiển: Liên quan đến việc duy trì mối quan hệ về dữ liệu.
- Ràng buộc về suy diễn: Đó là những khả năng có thể xảy ra từ một ứng dụng, dựa vào các kết quả trước đó, hoặc có thể dựa vào các quan hệ về dữ liệu, ta có thể dẫn đến một kết quả khác.

1.1.2.4 Giao diện

Quan trọng nhất là giao diện người sử dụng - là phương tiện giao tiếp giữa người sử dụng và chương trình. Sau đó là giao diện thủ công - là các mẫu báo cáo, ... và một số giao diện đã được chuẩn hóa như giao diện về mạng LAN của Institute of Electrical and Electronic Engineers, mô hình OSI (Open Systems Interconnection Reference Model) của International Standards Organization, ...

1.1.3. Tiêu chuẩn của sản phẩm phần mềm

Để đánh giá được sản phẩm của một nền công nghệ là tốt hay xấu, chúng ta phải nghiên cứu để đưa ra được những tiêu chuẩn đánh giá chúng. Chất lượng của sản phẩm phần mềm bao gồm nhiều yếu tố dựa trên các tiêu chuẩn đã được tổng kết.

1.1.3.1 Tính đúng

Một sản phẩm thực hiện được gọi là đúng nếu nó thực hiện chính xác những chức năng đã đặc tả và thỏa mãn các mục đích công việc của khách hàng.

Như vậy, một sản phẩm phải được so sánh chuẩn đặt ra để kiểm tra tính đúng và điều này dẫn đến có nhiều bậc thang về tính đúng.

Liệt kê theo thang giảm dần, tính đúng của phần mềm có thể:

-
- Tuyệt đối đúng,
 - Đúng,
 - Có lỗi,
 - Có nhiều lỗi, ...

Ví dụ: Một hệ thống xử lý dữ liệu không chạy được khi file cơ sở dữ liệu rỗng hoặc có quá 10^4 bảng ghi... là những hệ thống vi phạm tính đúng.

1.1.3.2 Tính khoa học

Tính khoa học của phần mềm được thể hiện qua các mặt

- Khoa học về cấu trúc.
- Khoa học về nội dung.
- Khoa học về hình thức thao tác.

1.1.3.3 Tính tin cậy

Tính tin cậy của sản phẩm phần mềm thể hiện ở sản phẩm được trông chờ thực hiện các chức năng dự kiến của nó với độ chính xác được yêu cầu.

1.1.3.4 Tính kiểm thử được

Phần mềm có thể kiểm thử được là phần mềm mà nó có cách dễ dàng để có thể kiểm tra được. Đảm bảo rằng nó thực hiện đúng các chức năng dự định.

1.1.3.5 Tính hữu hiệu

Tính hữu hiệu của phần mềm được xác định qua các tiêu chuẩn sau:

- Hiệu quả kinh tế hoặc ý nghĩa; giá trị thu được do áp dụng sản phẩm đó.
- Tốc độ xử lý sản phẩm.
- Giới hạn tối đa của sản phẩm hoặc miền xác định của chương trình được xác định qua khối lượng tối đa của các đối tượng mà sản phẩm đó quản lý.

1.1.3.6 Tính sáng tạo

Một sản phẩm phần mềm có tính sáng tạo khi nó thỏa mãn một trong các tính chất sau:

- Sản phẩm được thiết kế và cài đặt đầu tiên.
- Sản phẩm được phục vụ cho những đặc thù riêng.
- Sản phẩm có những đặc điểm khác về mặt nguyên lý so với các sản phẩm hiện hành.
- Sản phẩm có những ưu thế nổi bật so với sản phẩm hiện hành.

1.1.3.7 Tính an toàn

Tính an toàn của sản phẩm phần mềm được đánh giá thông qua:

- Có cơ chế bảo mật và bảo vệ các đối tượng do hệ thống phát sinh hoặc quản lý.

- Bản thân sản phẩm được đặt trong một cơ chế bảo mật nhằm chống sao chép trộm hoặc làm biến dạng sản phẩm đó.

1.1.3.8 Tính toàn vẹn

Sản phẩm phần mềm có tính toàn vẹn khi nó:

- Có cơ chế ngăn ngừa việc thâm nhập bất hợp pháp vào phần mềm hay dữ liệu và ngăn ngừa việc phát sinh ra những đối tượng (dữ liệu, đơn thể...) sai quy cách hoặc mâu thuẫn với các đối tượng sẵn có.

- Không gây ra nhập nhằng trong thao tác. Đảm bảo nhất quán về cú pháp.

- Có cơ chế phục hồi lại toàn bộ hoặc một phần những đối tượng thuộc toàn bộ hoặc một phần những đối tượng thuộc diện quản lý của sản phẩm trong trường hợp có sự cố như hỏng máy, mất điện đột ngột.

1.1.3.9 Tính đối xứng và đầy đủ chức năng

Sản phẩm cung cấp đủ các chức năng cho người sử dụng và các chức năng của sản phẩm có các cặp loại trừ lẫn nhau, ví dụ các chức năng đối xứng thường gặp:

- Tạo lập - Hủy bỏ,
- Thêm - Bớt (xem - xóa),
- Tăng - Giảm,
- Dịch chuyển lên - xuống; phải - trái,
- Quay xuôi - ngược chiều kim đồng hồ, ...

1.1.3.10 Tính tiêu chuẩn và tính chuẩn

Sản phẩm phần mềm cần đạt được một số tiêu chuẩn tối thiểu được thừa nhận trong thị trường hoặc trong khoa học, và có thể chuyển đổi dạng cấu trúc dữ liệu riêng của hệ thống sang chuẩn và ngược lại.

Tính chuẩn của phần mềm thể hiện ở sản phẩm đó phù hợp với các chuẩn quốc gia hoặc quốc tế.

Trong khi xây dựng phần mềm, cần tuân theo nguyên tắc chuẩn hoá sau:

- Chỉ thiết kế và xây dựng phần mềm sau khi đã xác định được chuẩn.
- Mọi thành phần của phần mềm phải được thiết kế và cài đặt theo cùng một chuẩn (tối thiểu thì các chuẩn phải tương thích nhau).

1.1.3.11 Tính độc lập

Phần mềm cần và nên đảm bảo được tính độc lập với các đối tượng sau:

- Độc lập với thiết bị,
- Độc lập với cấu trúc của đối tượng mà sản phẩm đó quản lý,
- Độc lập với nội dung của đối tượng mà sản phẩm đó quản lý.

1.1.3.12 Tính dễ phát triển, hoàn thiện

Thể hiện ở phần mềm có thể mở rộng cho các phương án khác hoặc mở rộng, tăng cường về mặt chức năng một cách rõ ràng.

1.1.3.13 Một số tính chất khác

Ngoài các tính chất trên, tùy theo công dụng mà sản phẩm phần mềm cần phải được bổ sung các tính chất sau:

1. Tính phổ dụng: có thể áp dụng cho nhiều lĩnh vực theo nhiều chế độ làm việc khác nhau.
2. Tính đơn giản: mang những yếu tố tâm lý: dễ thao tác, dễ học, dễ hoàn thiện kỹ năng khai thác sản phẩm, trong sáng, dễ hiểu, dễ nhớ...
3. Tính liên tác: là tính chất cần có để có thể gắn hệ thống này với hệ thống khác.
4. Tính súc tích: là độ gọn của chương trình tính theo số mã dòng lệnh.
5. Tính dung thứ sai lầm: tức là những hỏng hóc xuất hiện khi chương trình gặp phải lỗi được chấp nhận.
6. Tính module: là sự độc lập chức năng của các thành phần trong chương trình.
7. Tính đầy đủ hồ sơ: hệ thống phải có đầy đủ hồ sơ pháp lý khi xây dựng.
8. Tính theo dõi được, tính dễ vận hành, ...

1.1.4. Phân loại phần mềm

Phân loại phần mềm được định nghĩa như sự định hướng các công việc của một ứng dụng, ví dụ như theo kiểu hướng giao dịch, hỏi đáp, trợ giúp quyết định, ...

1.1.4.1 Ứng dụng hướng giao dịch

Ứng dụng hướng giao dịch còn có tên là hệ thống xử lý giao dịch (TPS – Transaction Processing Systems) được sử dụng nhằm hỗ trợ các hoạt động hằng ngày của một công việc, bao gồm: xử lý đơn hàng, quản lý kiểm kê, ghi quỹ, ... Chúng được đặc trưng như là các ứng dụng mà trong đó các yêu cầu, các dữ liệu và quá trình xử lý được biết rõ và có cấu trúc tốt. Theo nghĩa được biết rõ, chức năng đó phải có tính lặp lại, thân thiện và rõ ràng. Theo nghĩa cấu trúc tốt, vấn đề đó phải có thể được xác định một cách đầy đủ và rõ ràng. Các yêu cầu có thể được định danh bởi đội ngũ xây dựng phần mềm.

1.1.4.2 Ứng dụng cơ sở dữ liệu

Ứng dụng cơ sở dữ liệu được sử dụng như một ứng dụng xử lý câu hỏi về dữ liệu. Ngôn ngữ truy vấn dữ liệu chuẩn SQL cho phép người sử dụng đặt câu hỏi dưới dạng: họ biết họ cần gì nhưng không biết làm cách nào để lấy được dữ liệu đó. Các phần mềm máy tính đưa ra các phương pháp xử lý và truy cập tối ưu để thực hiện các thao tác đó.

Ở đây, có ba loại câu hỏi chính:

1. Tương tác: dữ liệu sử dụng xong là không cần nữa?
2. Dữ liệu được lưu trữ để sử dụng lại và thay đổi trong tương lai?

3. Dữ liệu được lưu trữ để sử dụng thường xuyên trong một số quá trình lặp lại?

Ứng dụng truy vấn hỗ trợ một khái niệm là kho chứa dữ liệu (data warehouse). Đó là một sơ đồ lưu trữ xây dựng trên quan điểm: hầu hết dữ liệu cần phải giữ lại cho các truy nhập truy vấn trực tuyến. Tại đây lưu lại các phiên bản cũ của phần lớn các phần tử trong cơ sở dữ liệu, các lần vào ra giao dịch và các bản ghi liên quan đến một số quá trình hoạt động.

1.1.4.3 Ứng dụng hỗ trợ quyết định (Decision Supports System - DSS)

DSS làm nhiệm vụ xác định và giải quyết bài toán. Khác với một ứng dụng truy vấn mà những người chuyên nghiệp và các nhà quản lý sử dụng để tìm kiếm và tổng hợp các dữ liệu về một quá trình hoạt động (như ở ví dụ trên), với ứng dụng hỗ trợ quyết định, họ phân tích, xác định các xu hướng, thực hiện các phân tích dữ liệu về mặt thống kê hay toán học từ đó giải các bài toán không cấu trúc. Dữ liệu dùng cho DSS thường lấy từ các ứng dụng sử dụng giao dịch.

Vì thông tin thường không đầy đủ, trong DSS thường giải bài toán bằng phương pháp lặp, áp dụng mô hình toán học hoặc thống kê để đi tới quyết định. Dữ liệu hỗ trợ và/hoặc hiệu chỉnh thường được đưa trở lại quá trình mô hình hoá để làm mịn các phân tích.

Ta thường gặp một số hệ thống được xem là một sản phẩm phụ của DSS như:

- **Hệ thống thông tin thi hành** (Executive Information System - EIS) là một sản phẩm phụ của DSS. EIS hỗ trợ quyết định thực hiện và cung cấp khả năng tìm kiếm trong các môi trường một cách tự động. Các hệ thi hành hàng đầu phải xử lý được các vấn đề với thông tin không đầy đủ, không chính xác, không rõ ràng và có liên quan đến tương lai. EIS tích hợp thông tin từ cơ sở dữ liệu bên ngoài với ứng dụng nội bộ để tạo ra khả năng mô hình hoá và tìm kiếm thông tin tự động. Sự khác nhau cơ bản của EIS với DSS là ở đây dữ liệu không hoàn chỉnh, không rõ ràng và thậm chí không chính xác.

- **Hệ thống hỗ trợ quyết định theo nhóm** (Group DSS - GDSS) là một dạng đặc biệt của ứng dụng DSS. GDSS có một nhật ký ghi lại quá trình xây dựng một quyết định để hỗ trợ một nhóm những người có trách nhiệm ra quyết định (decision maker). GDSS tập trung chủ yếu vào các quá trình tương tác có ít hoặc không có phân tích thống kê hoặc mô hình hoá dữ liệu trong nhóm. Các phần mềm cơ sở dữ liệu trong GDSS có xu hướng ít được xây dựng hơn đối với DSS, nhưng có thể bao gồm một số bảng tính và các thủ tục biểu diễn tổng kết về các bên tham gia dưới dạng số hoặc đồ thị.

1.1.4.4 . Hệ chuyên gia (Expert Systems - ES)

Các ứng dụng hệ chuyên gia là các ứng dụng tin học tự động hoá tri thức và khả năng lập luận của một hoặc nhiều chuyên gia trong một lĩnh vực cụ thể nào đó. ES phân tích các đặc trưng của một tình huống để đưa ra một lời khuyên, một khuyến nghị hoặc phác hoạ một kết luận bằng các quá trình lập luận tự động. Một hệ ES bao gồm bốn thành

phần chính: hệ thống thu thập tri thức, cơ sở tri thức, mô tơ suy diễn (còn gọi là cơ sở luật) và hệ thống diễn giải.

- Hệ thống thu thập tri thức là phương tiện xây dựng cơ sở tri thức. Nói chung, càng nhiều tri thức thì hệ thống càng “thông minh” hơn. Hệ thống thu thập tri thức phải cung cấp các sự kiện khởi đầu, các quy tắc mẹo mực (heuristic rules of thumb) và có thể dễ dàng bổ sung tri thức mới.

Thông thường, chúng ta lập luận mà không biết làm cách nào để đi tới phương án. Từ phân tích quá trình con người suy nghĩ khi phân tích một vấn đề có thể áp dụng để xây dựng một ứng dụng. Tại sao ta lại làm việc đó? Đó là cả một quá trình suy diễn nội tâm và rất khó diễn giải. Khó khăn này không của riêng ai. Suy luận thông tin từ tri thức của các chuyên gia là khó khăn cơ bản để xây dựng một hệ chuyên gia có hiệu quả.

- Cơ sở tri thức là một phiên bản tự động hệ thống hoá tri thức chuyên gia cộng với các mẹo áp dụng tri thức đó. Thiết kế cơ sở tri thức cũng khó như suy luận thông tin vì dù nó được thiết kế như thế nào thì cũng bị giới hạn bởi hệ thống cài đặt nó. Vì vậy, một ngôn ngữ đặt biệt cho ES đã được thiết kế, nó cho phép xác định mối quan hệ giữa các mẫu thông tin và sử dụng một cách mềm dẻo các thông tin đó trong lập luận.

- Vì mục đích của lập luận là tìm một giải pháp khả dĩ nhất cho một tình huống, ES sử dụng lập luận và suy diễn để xây dựng nhiều giải pháp có thể cho một tình huống cho trước. Một vài giải pháp có thể được đưa ra khi thông tin không hoàn chỉnh hoặc khi mới lập luận một phần. Xác suất chính xác của giải pháp do hệ thống đưa ra thường được đo bằng mức độ hữu ích của giải pháp đó. Các vấn đề liên quan đến quy tắc hoặc đạo đức thường được xét đến trong các ES hơn so với trong các ứng dụng khác.

- Thành phần quan trọng cuối cùng của một ES là khả năng diễn giải các lập luận cho người sử dụng. Tìm lại quá trình suy diễn là điều rất quan trọng giúp người sử dụng có được kinh nghiệm sử dụng hệ thống và xác định mức độ tin cậy vào kết quả do ES đưa ra.

1.1.4.5 Các hệ thống nhúng (Embedded systems)

Đây là các ứng dụng vốn là một phần của hệ thống lớn hơn. Thường, bản thân ứng dụng thì rất đơn giản nhưng sự phức tạp của chúng là ở giao diện để tạo ra một độ chính xác, tính theo thời gian thực (*real - time*) trong phạm vi đời sống của hệ thống lớn hơn. Việc phát triển các ứng dụng kết hợp này là địa phận của các nhà thiết kế theo học ngành khoa học máy tính hơn là những nhà thiết kế hệ thống thông tin.

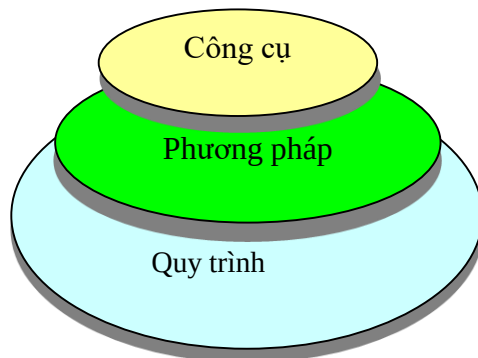
1.2 Giới thiệu về Công nghệ phần mềm

1.2.1. Khái niệm Công nghệ phần mềm

Công nghệ phần mềm (software engineering): là việc áp dụng các công cụ, các kỹ thuật một cách hệ thống trong việc phát triển các ứng dụng dựa trên máy tính. Đó chính là việc áp dụng các quan điểm, các tiến trình có kỷ luật và lượng hoá được, có bài bản và hệ thống để phát triển, vận hành và bảo trì phần mềm [2].

Theo quan điểm của nhiều nhà nghiên cứu, có thể nhìn công nghệ phần mềm là một mô hình được phân theo ba tầng mà tất cả các tầng này đều nhằm tới mục tiêu chất lượng, chi phí, thời hạn phát triển phần mềm.

Mô hình được phân theo ba tầng của công nghệ phần mềm được mô tả như sau:



Hình 1.1 Mô hình ba tầng của Công nghệ phần mềm

Ở đây tầng quy trình (process) liên quan tới vấn đề quản trị phát triển phần mềm như lập kế hoạch, quản trị chất lượng, tiến độ, chi phí, mua bán sản phẩm phụ, cấu hình phần mềm, quản trị sự thay đổi, quản trị nhân sự (trong môi trường làm việc nhóm), việc chuyển giao, đào tạo, tài liệu [2];

Tầng phương pháp (methods) hay cách thức, công nghệ, kỹ thuật để làm phần mềm: liên quan đến tất cả các công đoạn phát triển hệ thống như nghiên cứu yêu cầu, thiết kế, lập trình, kiểm thử và bảo trì. Phương pháp dựa trên những nguyên lý cơ bản nhất cho tất cả các lĩnh vực công nghệ kể cả các hoạt động mô hình hoá và kỹ thuật mô tả [2];

Tầng công cụ (tools) liên quan đến việc cung cấp các phương tiện hỗ trợ tự động hay bán tự động cho các tầng quá trình và phương pháp (công nghệ) [2].

Qua sơ đồ trên, ta thấy rõ công nghệ phần mềm là một khái niệm đề cập không chỉ tới các công nghệ và công cụ phần mềm mà còn tới cả cách thức phối hợp công nghệ, phương pháp và công cụ theo các quy trình nghiêm ngặt để làm ra sản phẩm có chất lượng

1.2.2. Lịch sử phát triển ngành Công nghệ phần mềm

Công nghệ phần mềm có một lịch sử khá sớm. Các công cụ được dùng cũng như các ứng dụng được viết đã tham gia vào Công nghệ phần mềm theo các mốc thời gian:

- Thập niên 1940: Các chương trình cho máy tính được viết bằng tay.
- Thập niên 1950: Các công cụ đầu tiên xuất hiện như phần mềm biên dịch Macro Assembler và phần mềm thông dịch đã được tạo ra và sử dụng rộng rãi để nâng cao năng suất và chất lượng. Các trình dịch được tối ưu hóa lần đầu tiên ra đời.
- Thập niên 1960: Các công cụ của thế hệ thứ hai như các trình dịch tối ưu hóa và công việc kiểm tra mẫu đã được dùng để nâng cao sản phẩm và chất lượng. Khái niệm công nghệ phần mềm đã được bàn thảo rộng rãi.

-
- Thập niên 1970: Các công cụ phần mềm, chẳng hạn trong UNIX các vùng chứa mã lệnh, lệnh make ... được kết hợp với nhau. Số lượng doanh nghiệp nhỏ về phần mềm và số lượng máy tính cỡ nhỏ tăng nhanh.
 - Thập niên 1980: các PC và máy trạm ra đời. Cùng lúc có sự xuất hiện của mô hình dự toán khả năng. Lượng phần mềm tiêu thụ tăng mạnh.
 - Thập niên 1990: Phương pháp lập trình hướng đối tượng ra đời. Các quá trình nhanh như là lập trình cực hạn được chấp nhận rộng rãi. Trong thập niên này, WWW và các thiết bị máy tính cầm tay phổ biến rộng rãi.
 - Hiện nay: Các phần mềm biên dịch và quản lý như là .NET, PHP, và Java làm cho việc viết phần mềm trở nên dễ dàng hơn nhiều.

1.2.3. Vòng đời phần mềm

Vòng đời phần mềm là thời kỳ tính từ khi phần mềm được sinh (tạo) ra cho đến khi chết đi (từ lúc hình thành đáp ứng yêu cầu, vận hành, bảo dưỡng cho đến khi loại bỏ không sử dụng) [3].

Vòng đời phần mềm được phân chia thành các pha chính: phân tích, thiết kế, cài đặt, kiểm thử, bảo trì. Biểu diễn các pha có thể khác nhau theo từng mô hình.

Mọi sản phẩm phần mềm đều có vòng đời. Vòng đời thường khá dài, một số sản phẩm phần mềm đã “tồn tại” được 30 năm và vòng đời có thể được rút ngắn do tiến bộ công nghệ.

Các pha trong vòng đời phần mềm: tất cả các sản phẩm phần mềm đều trải qua ít nhất các giai đoạn sau:

- Yêu cầu: xác định nhu cầu của khách hàng và các ràng buộc của sản phẩm.
- Thiết kế: xác định cấu trúc/ tổ chức của hệ thống phần mềm.
- Cài đặt: viết phần mềm
- Kiểm thử: vận hành hệ thống để tìm và loại bỏ các khiếm khuyết
- Bảo trì: sửa chữa và nâng cấp sản phẩm sau khi phần mềm được triển khai.

1.2.4. Phương pháp phát triển phần mềm

Khác với thời kỳ đầu của tin học, các chương trình phụ thuộc nhiều vào thiết bị và người ta chỉ quan tâm đến các "mẹo vặt" lập trình, thì ngày nay người ta quan tâm đến nguyên lý và phương pháp để phát triển phần mềm. Các nguyên lý và phương pháp được đề xuất nhằm nâng cao năng suất lao động cho nhóm phát triển phần mềm. Năng suất ở đây bao gồm tính đúng đắn của sản phẩm, tính dễ đọc, dễ sửa đổi, dễ thực hiện, tận dụng được tối đa khả năng của thiết bị mà vẫn không bị phụ thuộc vào thiết bị.

Có nhiều phương pháp được đề cập như: phương pháp hướng chức năng, phương pháp hướng đối tượng, phương pháp ngữ nghĩa, ... và thậm chí là không phương pháp để phát triển phần mềm. Bên cạnh các phương pháp để chỉ định cho việc tạo một bản phân

tích và thiết kế, người ta còn chú ý đến phương pháp làm thế nào để đưa người dùng tham gia vào quy trình gọi là phương pháp luận xã hội.

1.2.5. Một số mô hình phát triển phần mềm

1.2.5.1. Mô hình tuyến tính (*The linear sequential model*)

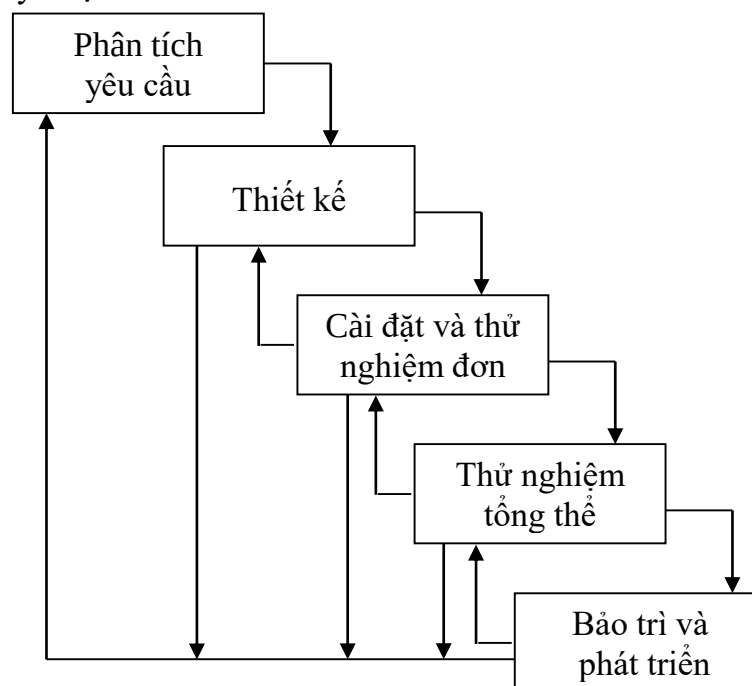
Đôi lúc còn được gọi là mô hình kinh điển (classic model) hay mô hình thác nước (waterfall model). Mô hình này xem quá trình xây dựng một sản phẩm phần mềm bao gồm nhiều giai đoạn tách biệt, sau khi hoàn tất một giai đoạn thì chuyển đến giai đoạn sau.

Có hai hoạt động phổ biến được thực hiện trong mỗi giai đoạn là: kiểm tra - phê chuẩn và quản lý cấu hình. Tổng kết mỗi giai đoạn là sự kiểm tra, phê chuẩn và quản lý cấu hình đây chính là mục tiêu của sản phẩm. Việc kiểm tra đưa ra khuôn mẫu đúng đắn tương ứng giữa sản phẩm phần mềm và các đặc tính của nó. Sự phê chuẩn đưa ra chuẩn mực về sự phù hợp hay chất lượng của sản phẩm phần mềm đối với mục đích của quá trình hoạt động.

Tuy vậy, thường thì các dự án có hàng ngàn trang tài liệu mà không ai ngoại trừ tác giả đọc đến nó. Thông tin ứng dụng chỉ nằm trong đầu mọi người và việc trao đổi thông tin là một trở ngại lớn để có được thành công của hệ thống. Kết luận là văn bản không phải là một phương tiện tốt để mô tả các yêu cầu phức tạp của ứng dụng. Thêm vào đó, mô hình bộc lộ một số nhược điểm quan trọng như:

- Mối qua hệ giữa các giai đoạn không được thể hiện.
- Hệ thống phải được kết thúc ở từng giai đoạn do vậy rất khó thực hiện được đầy đủ những yêu cầu của khách hàng...

Mô hình này được tóm tắt như sau

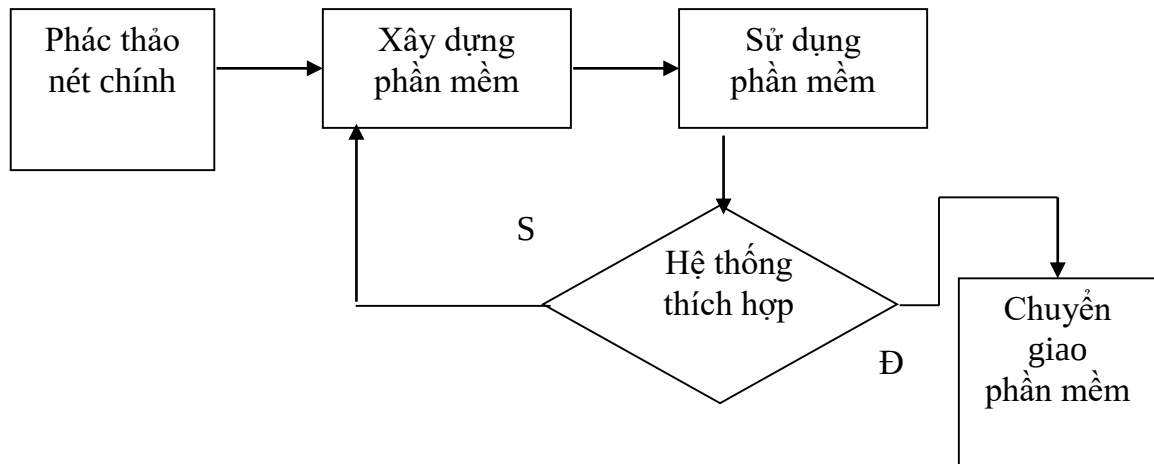


Hình 1.2 Sơ đồ mô hình tuyến tính

1.2.5.2. Mô hình mẫu (Prototyping model)

Thông thường, khách hàng sẽ đưa ra mục tiêu của họ một cách chung chung mà họ không biết hoặc không đưa ra một cách cụ thể những cái vào, cái ra và các tiến trình xử lý chúng. Thêm vào đó, chúng ta cũng không thể không quan tâm đến thuật toán sử dụng, tính tương thích của sản phẩm phần mềm với môi trường của nó như: phần cứng, hệ điều hành... Trong trường hợp này, mô hình mẫu có thể là sự lựa chọn tốt hơn cho người lập trình.

Những điểm chính của mô hình mẫu được tóm tắt theo sơ đồ sau:



Hình 1.3 Sơ đồ mô hình mẫu

Mô hình mẫu là một cách để phá vỡ sự khắt khe, cứng nhắc trong chu trình tuần tự của dự án. Tuy vậy, trong mô hình mẫu, sử dụng sai làm hỏng phân tích và thiết kế, không bao giờ hoàn thiện được mẫu thành các ứng dụng thực sự là các vấn đề cần quan tâm. Thêm vào đó là hệ thống có thể không bao giờ được chuẩn hóa, chi tiết của việc xử lý, việc kiểm tra tính hợp lệ của dữ liệu và các đòi hỏi kiểm toán có thể bị bỏ quên trong việc đưa mẫu vào sản xuất.

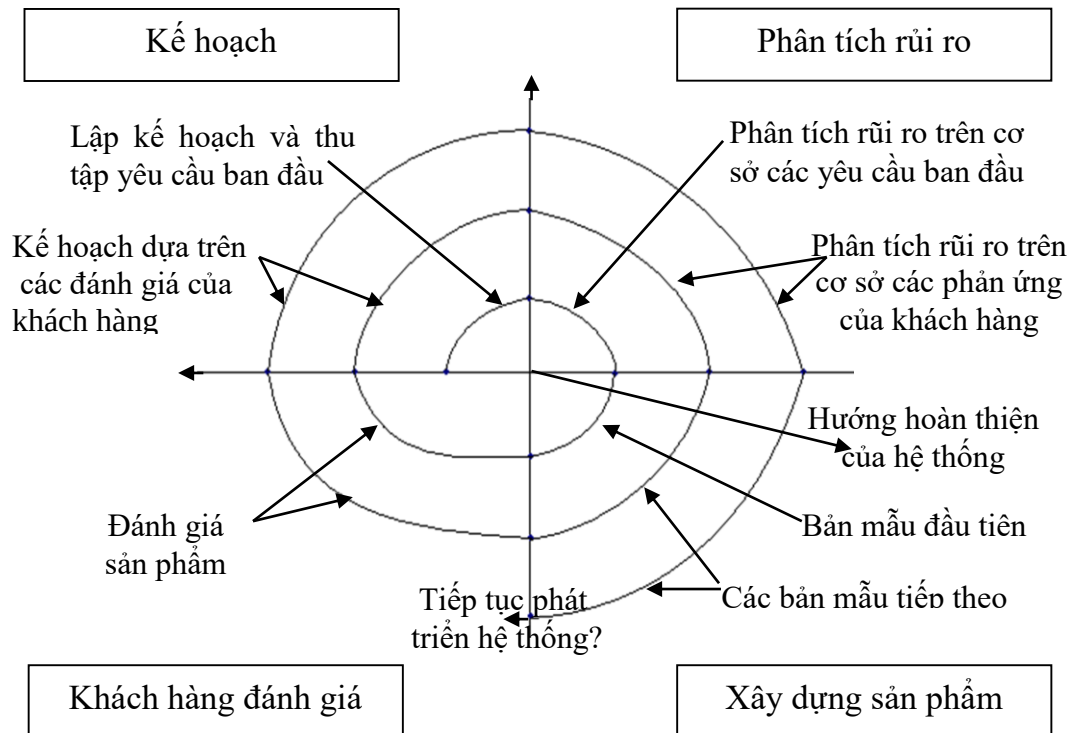
Trong tương lai, tạo mẫu thích hợp với đánh giá thiết kế, cải tiến cách dùng phần cứng và phần mềm mới. Tạo mẫu thường đi đôi với các ngôn ngữ lập trình bậc cao và ngày càng có nhiều công cụ đặt mẫu sẽ được tích hợp với CASE.

1.2.5.3. Mô hình xoắn ốc (The spiral model)

Mô hình này được Boehm đưa ra nên đôi lúc còn được gọi là mô hình Boehm's (The Boehm's spiral model). Nó có thể xem là sự kết hợp giữa mô hình thác nước và mô hình mẫu và đồng thời thêm một thành phần mới - phân tích rủi ro. Bao gồm bốn hoạt động chính:

- Planning: Xác định mục tiêu, tương tác và ràng buộc.
- Risk analysis: Phân tích các lựa chọn và các chỉ định/giải quyết rủi ro.
- Engineering: Phát triển sản phẩm.
- Customer evaluation: Đánh giá kết quả xây dựng.

Mô hình được tóm tắt như sau:



Hình 1.4 Sơ đồ mô hình xoắn ốc

Trong vòng đầu tiên của xoắn ốc, mục đích, lựa chọn, các ràng buộc được định nghĩa và các nguy cơ được xác định và phân tích. Nếu phân tích các lỗi chỉ ra rằng có một vài yêu cầu không chắc chắn, tạo mẫu có thể được tiến hành để giúp đỡ nhà phát triển và khách hàng. Mô phỏng và các mô hình khác có thể được sử dụng để xác định vấn đề và làm mịn các yêu cầu.

Khách hàng đánh giá công việc và đưa ra các gợi ý. Trên cơ sở ý kiến đó, phân tiếp theo của lập kế hoạch và phân tích lỗi xuất hiện.

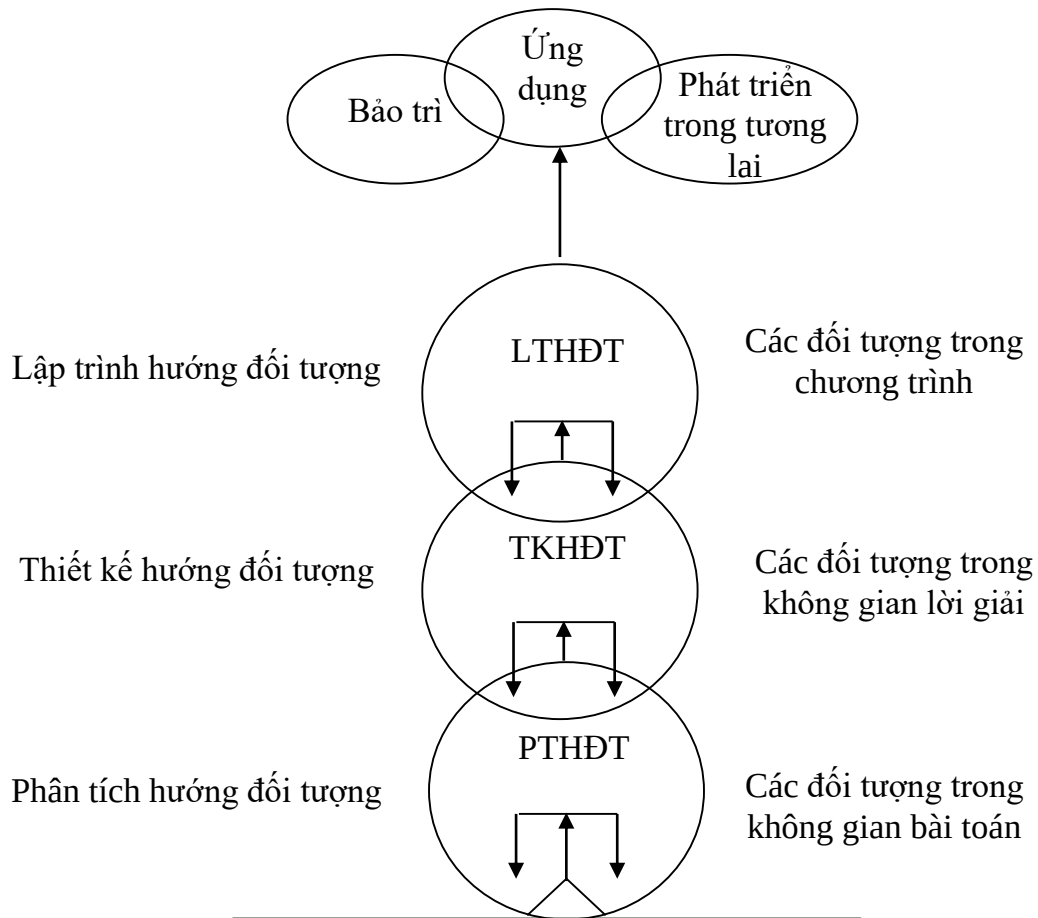
Mô hình xoắn ốc hiện nay là mô hình hướng tiếp cận hiện thực nhất để phát triển các hệ thống lớn. Nó sử dụng mô hình mẫu như là cơ chế loại trừ lỗi, cho phép nhà phát triển áp dụng mô hình mẫu tại mỗi chu trình phát triển. Nó kế thừa cách tiếp cận hệ thống từng bước từ chu kỳ sống cổ điển nhưng kết hợp với quá trình lặp lại phù hợp với thực tế.

Giống như các quy trình khác, mô hình xoắn ốc không phải là công cụ vạn năng. Đối với những hệ thống lớn, khó có thể điều khiển sự tiến hóa của phần mềm. Nó đòi hỏi phải có kỹ năng đánh giá lỗi. Cuối cùng là cần phải có thêm thời gian để kiểm nghiệm phương pháp mới này.

1.2.5.4. Mô hình đài phun nước

Đây là mô hình của cách tiếp cận hướng đối tượng, hệ thống được xem như là một hệ thống các thực thể tác động qua lại để đạt được một mục đích nào đó. Mô hình này tương ứng với mô hình thác nước trong cách tiếp cận hướng thủ tục ở trên. Ở đây, ta thấy trong có những phân lặp và giao nhau giữa các bước phân tích, thiết kế và cài đặt.

Các điểm chính của mô hình được tóm tắt như sau



Hình 1.5 Sơ đồ mô hình đài phun nước

1.2.5.5. Mô hình phát triển dựa trên thành phần

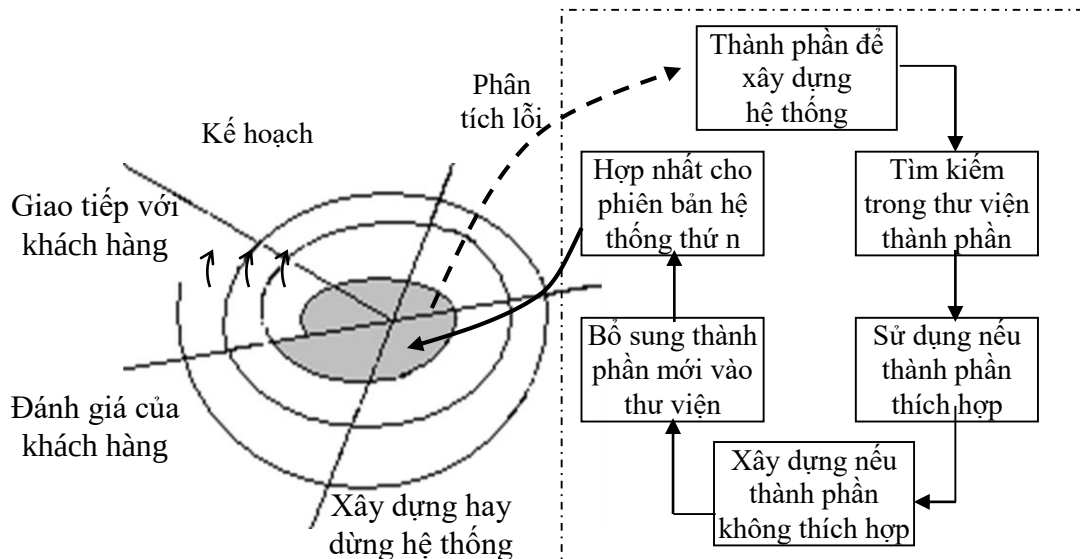
Xuất phát từ quan điểm: "Buy do not build", tư tưởng của phát triển dựa trên thành phần là lắp ráp hệ thống từ những thành phần đã có. Do vậy, kiến trúc phần mềm của hệ thống dựa vào kiến trúc phần mềm của các thành phần phần mềm tiêu chuẩn nên hệ thống đạt chất lượng cao hơn.

Phương pháp phát triển dựa trên thành phần gần tương tự như phương pháp phát triển hướng đối tượng. Hoạt động công nghệ bắt đầu với sự chỉ ra các lớp tham dự để phát triển hệ thống. Nếu các lớp này được tìm thấy trong thư viện và sự thích nghi là tốt, chúng sẽ được lấy ra và phát triển hệ thống. Ngược lại, chúng sẽ được phát triển để sử dụng và bổ sung vào thư viện sử dụng lại.

Thành phần phần mềm được sử dụng lại có độ chính xác cao và có thể nói là không chứa lỗi. Mặc dầu không thường xuyên được chứng minh về mặt hình thức nhưng với việc sử dụng lại, lỗi được tìm thấy và loại trừ; chất lượng của thành phần được cải thiện như là một kết quả.

Khi những thành phần sử dụng lại được ứng dụng thông qua tiến trình phần mềm, chúng ta ít tốn thời gian để tạo ra kế hoạch, mô hình, tài liệu, mã và dữ liệu mà chúng là cần thiết để tạo ra hệ thống. Thêm vào, chức năng cùng mức được phân phối cho người sử dụng với đầu vào ít công sức hơn, do vậy, hiệu suất phần mềm được cải thiện.

Những điểm chính của mô hình được tóm tắt như sau:



Hình 1.6 Sơ đồ mô hình phát triển dựa trên thành phần

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

Câu 1. Các đặc tính của phần mềm. Việc phân loại phần mềm có đem lại lợi ích gì trong việc phát triển phần mềm.

Câu 2. Phân tích các đặc điểm của các mô hình phát triển phần mềm. Khi xây dựng một phần mềm, người ta dựa vào các yếu tố nào để chọn mô hình phát triển.

Câu 3. Phân tích vòng đời phát triển phần mềm.

Câu 4. Mô tả lịch sử phát triển của công nghệ phần mềm.

Câu 5. Trình bày các tiêu chuẩn để có 1 phần mềm tốt.

Câu hỏi trắc nghiệm:

Câu 1. Trong các mô hình phát triển phần mềm sau, mô hình nào được xem là mô hình của cách tiếp cận hướng đối tượng:

- a. Mô hình xoắn ốc
- b. Mô hình tuyến tính
- c. Mô hình mẫu.
- d. Mô hình đài phun nước

Câu 2. Khi xây dựng một phần mềm, người ta ít quan tâm đến yếu tố nào để chọn mô hình phát triển?

-
- a. Phạm vi của tổ chức
 - b. Thời gian xây dựng phần mềm
 - c. Trình độ quản lý và kỹ năng của đội ngũ nhân viên
 - d. Loại sản phẩm thực hiện

Câu 3. Mô hình phát triển phần mềm nào có 4 mục tiêu sau:

Planning: Xác định mục tiêu, tương tác và ràng buộc.

Risk analysis: Phân tích các lựa chọn và các chỉ định/giải quyết rủi ro.

Engineering: Phát triển sản phẩm

Customer evaluation: Đánh giá kết quả xây dựng.

- a. Mô hình xoắn ốc
- b. Mô hình tuyến tính
- c. Mô hình thác nước cổ điển
- d. Mô hình đài phun nước

Câu 4. Tầng quy trình trong quy trình 3 tầng (quy trình, phương pháp, công cụ) liên quan đến các công việc:

- a. Cách thức, công nghệ, kỹ thuật để làm phần mềm
- b. Quản trị phát triển phần mềm
- c. Cung cấp các phương tiện hỗ trợ tự động hay bán tự động trong quá trình sản xuất phần mềm
- d. Cung cấp các ngôn ngữ lập trình để viết code

Câu 5. Công nghệ phần mềm (software engineering) là:

- a. Việc sử dụng các phần mềm
- b. Việc áp dụng các công cụ, các kỹ thuật một cách hệ thống trong việc phát triển các ứng dụng dựa trên máy tính.
- c. Việc cài đặt và sử dụng các phần mềm một cách có hiệu quả trên máy vi tính
- d. Việc cài đặt và bảo trì các phần mềm một cách có hiệu quả trên máy vi tính

Bài tập vận dụng:

Bài 1: Bạn được giao nhiệm vụ phát triển phần mềm “Quản lý sinh viên của một trường Đại học”, bạn hãy phân tích và lựa chọn mô hình phát triển phù hợp để thực hiện nhiệm vụ này?

Bài 2: Bạn được giao nhiệm vụ kiểm thử phần mềm “Game 3D”, bạn hãy phân tích và lựa chọn mô hình phát triển phù hợp để thực hiện nhiệm vụ này?

Bài 3: Bạn được giao nhiệm vụ kiểm thử phần mềm “Website quản lý bán hàng online của 1 hệ thống bán quần áo”, bạn hãy phân tích và lựa chọn mô hình phát triển phù hợp để thực hiện nhiệm vụ này?

Bài 4: Bạn được giao nhiệm vụ kiểm thử ứng dụng dạng “Diệt virus”, bạn hãy phân tích và lựa chọn mô hình phát triển phù hợp để thực hiện nhiệm vụ này?

Bài 5: Hãy chọn một phần mềm mà bạn từng tham gia phát triển hoặc bạn có định hướng tìm hiểu, bạn hãy phân tích và lựa chọn mô hình phát triển phù hợp để thực hiện nhiệm vụ này?

CHƯƠNG 2: KHẢO SÁT – PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU

Mục tiêu của chương:

Mục tiêu của chương này là giới thiệu về quy trình khảo sát, phân tích và đặc tả yêu cầu của phần mềm. Với sản phẩm phần mềm được xây dựng, việc hiểu đầy đủ các đặc điểm của nó là điều không dễ. Quá trình xác định các chức năng và các ràng buộc của hệ thống gọi là tìm hiểu và xác định yêu cầu. Để có được điều này thì cần phải trả lời câu hỏi "cái gì-what" chứ không phải là "như thế nào-how". Tìm hiểu, xác định và phân tích yêu cầu là bước hình thành bài toán, do vậy các yêu cầu của bài toán cần phải được tìm hiểu và phân tích theo chiều rộng (ngang) và theo chiều sâu (dọc).

2.1 Tìm hiểu, xác định yêu cầu

2.1.1. Khảo sát, tìm hiểu yêu cầu

Khi một công ty muốn ký một hợp đồng cho một dự án phát triển một phần mềm, công ty sẽ phát biểu các yêu cầu ở mức trừu tượng để không bắt buộc định nghĩa trước các giải pháp. Các yêu cầu phải được viết sao cho các nhà phát triển phần mềm có thể đưa ra các giải pháp khác nhau. Sau khi đã trúng thầu và ký hợp đồng, yêu cầu phải được làm rõ hơn để khách hàng có thể hiểu và đánh giá được phần mềm. Cả hai tài liệu nói trên đều gọi là tài liệu yêu cầu người dùng.

Theo mức độ chi tiết có thể chia ra các loại tài liệu yêu cầu:

- Xác định yêu cầu: đây là một khẳng định, bằng ngôn ngữ tự nhiên hơn là các sơ đồ, về các dịch vụ hệ thống cần cung cấp và các ràng buộc mà hệ thống phải tuân theo. Tài liệu này cung cấp cho các thành phần: người quản lý của bên khách hàng, người dùng cuối của hệ thống, kỹ sư của khách hàng, người quản lý ký kết hợp đồng, các kiến trúc sư hệ thống.
- Đặc tả yêu cầu: là tài liệu được cấu trúc mô tả hệ thống các dịch vụ chi tiết hơn. Đôi khi tài liệu này được gọi là đặc tả chức năng. Đây có thể coi là hợp đồng ký kết giữa người mua và kẻ bán phần mềm. Tài liệu này cung cấp cho các thành phần: người dùng cuối của hệ thống, kỹ sư của khách hàng, các kiến trúc sư hệ thống, người phát triển phần mềm.
- Đặc tả phần mềm: là mô tả trừu tượng hơn của phần mềm làm cơ sở cho thiết kế và triển khai. Tài liệu này cung cấp cho các thành phần: kỹ sư của khách hàng, các kiến trúc sư hệ thống, người phát triển phần mềm.

Xác định yêu cầu: là mô tả trừu tượng các dịch vụ mà hệ thống được mong đợi phải cung cấp và các ràng buộc mà hệ thống phải tuân thủ khi vận hành. Nó chỉ có các đặc tả phẩm hạnh bên ngoài của hệ thống mà không liên quan đến các đặc tính thiết kế. Nó phải được viết sao cho người ta có thể hiểu được mà không cần một kiến thức chuyên môn đặc biệt nào.

Các yêu cầu của phần mềm được chia thành hai loại:

-
1. *Các yêu cầu hệ thống chức năng*: các dịch vụ mà hệ thống phải cung cấp.
 2. *Các yêu cầu không chức năng*: các ràng buộc mà hệ thống phải tuân theo.

Về nguyên tắc các yêu cầu của một hệ thống phải là vừa đầy đủ, vừa tránh kiện. Đầy đủ có nghĩa là mọi yêu cầu đều phải được đặc tả. Tránh kiện có nghĩa là các yêu cầu không gây ra mâu thuẫn. Thực tế đối với các hệ lớn và phức tạp thì thực là không thể đạt được tính đầy đủ và tính tránh kiện cho phiên bản đầu của tư liệu yêu cầu phần mềm. Vấn đề là khi duyệt lại hoặc trong các pha sau này của vòng đời phần mềm, người ta phát hiện ra các sự không thỏa mãn đó thì tư liệu yêu cầu phải được chỉnh lý lại [3].

Về bản chất, chúng ta phải hiểu và xác định rõ những yêu cầu của khách hàng. Tuy nhiên, thường bài toán được khách hàng phát biểu bằng ngôn ngữ tự nhiên cộng thêm với việc dùng các bảng các biểu đồ cho các người dùng dễ hiểu (xem là người dùng không biết các khái niệm chuyên môn công nghệ thông tin). Không may là ngôn ngữ được dùng này lại thường là không chính xác và mơ hồ, đôi khi có sự lẫn lộn giữa các biểu thị khái niệm và các biểu thị chi tiết làm cho việc mô tả chứa các thông tin hỏ lớn được biểu diễn ở nhiều mức chi tiết khác nhau.

Ở đây, chúng ta cần chú ý rằng người đặt hàng có thể vì không hiểu biết gì về tin học nên họ không thể phát biểu chính xác và đầy đủ các yêu cầu của họ, đôi lúc thì những cái mà người sử dụng yêu cầu và những cái mà họ cần là không giống nhau. Thêm vào đó, chúng ta lại không hiểu biết đầy đủ về đối tượng, địa bàn cho nên không thể thu thập đầy đủ và chính xác các thông tin của đối tượng và đây chính là một trong những mâu thuẫn giữa khách hàng và chúng ta. Vì vậy, trong thực tế, đối với các hệ thống lớn và phức tạp, rất khó có thể đạt được tính đầy đủ và thống nhất của tài liệu yêu cầu.

Các yêu cầu được tìm hiểu còn chứa các mâu thuẫn:

- Thiếu rõ ràng: Rất khó sử dụng ngôn ngữ tự nhiên mô tả chính xác không nhầm lẫn mà không làm khó khăn cho người đọc.
- Nhầm lẫn yêu cầu: Các yêu cầu chức năng, các ràng buộc, mục đích của hệ thống và các thông tin thiết kế không được phân biệt rõ ràng.
- Trộn lẫn yêu cầu: Một số các yêu cầu khác nhau có thể được thể hiện như là một yêu cầu đơn.

Giải quyết mâu thuẫn này, chúng ta phải: trên cơ sở nghiên cứu kỹ lĩnh vực ứng dụng và thảo luận với người sử dụng để định nghĩa chính xác các yêu cầu của bài toán. Xác định rõ và đầy đủ bài toán là yếu tố quan trọng góp phần đảm bảo thành công của dự án [3]. Nhiệm vụ của giai đoạn này là xây dựng được các hồ sơ mô tả chi tiết về các yêu cầu, nhiệm vụ, chức năng của hệ thống dự kiến.

2.1.2. Đánh giá các yêu cầu

Đánh giá các yêu cầu phần mềm liên quan với việc cho biết các yêu cầu thực sự định nghĩa hệ thống đáp ứng đòi hỏi của khách hàng. Nếu việc đánh giá này không chính xác, các lỗi trong phần yêu cầu sẽ truyền tới thiết kế hệ thống và triển khai hệ thống. Chi

phí sửa chữa lỗi sẽ rất lớn. Sự thay đổi về yêu cầu ngụ ý rằng việc thiết kế và triển khai cũng phải thay đổi theo. Một số khía cạnh của yêu cầu cần phải kiểm chứng:

- Giá trị: người dùng có thể nghĩ rằng hệ thống cần một số chức năng, tuy nhiên sau một số phân tích, có thể xác định các chức năng khác cần được đưa vào. Do hệ thống có nhiều loại người sử dụng nên có các yêu cầu khác nhau và không thể tránh khỏi sự thỏa hiệp các nhu cầu đó.

- Chắc chắn: mọi yêu cầu không được mâu thuẫn với các yêu cầu khác.
- Hoàn chỉnh: định nghĩa cần phải bao gồm mọi chức năng và các ràng buộc
- Hiện thực: không có các yêu cầu đặc biệt đến mức phi hiện thực. Có thể dự đoán trước các phát triển phần cứng, tuy nhiên phát triển phần mềm thì khó dự đoán hơn.

- Mẫu: là một mô hình chạy được của hệ thống được trình bày với người sử dụng. Đây là một kỹ thuật đánh giá yêu cầu hiệu quả. Nó cho phép người dùng thử nghiệm với hệ thống. Việc đánh giá lại yêu cầu không nên được coi là công việc tiếp theo của tư liệu hóa yêu cầu sau khi đã hoàn thành. Các xem xét về yêu cầu định kỳ liên quan với người dùng và kỹ sư phần mềm luôn cần thiết.

Các xem xét yêu cầu có thể là hình thức hoặc phi hình thức. Xem xét phi hình thức liên quan việc các người ký hợp đồng thảo luận các yêu cầu với khách hàng. Nhiều vấn đề có thể được giải quyết dễ dàng bất ngờ khi tham khảo trực tiếp với khách hàng. Đối với yêu cầu xem xét chính thức, đội phát triển phải dẫn dắt khách hàng thông qua các yêu cầu hệ thống, giải thích các triển khai của mỗi yêu cầu. Nhóm rà soát phải kiểm tra mỗi yêu cầu về độ thống nhất, hoàn chỉnh cho toàn bộ tài liệu. Họ có thể phải kiểm tra:

- Có khả năng kiểm tra: tài liệu có thể kiểm tra thực tế được không?
- Khả năng hiểu biết: tài liệu có được khách hàng hiểu biết thấu đáo hay không?
- Lưu vết: nguồn gốc của tài liệu có được xác định rõ ràng hay không? Rất có thể phải quay lại nguồn gốc ban đầu để đánh giá ảnh hưởng của sự thay đổi.
- Tính thích hợp: các yêu cầu đã phù hợp hay chưa? Có thể thay đổi các yêu cầu mà không làm ảnh hưởng lớn đến toàn bộ hệ thống không.

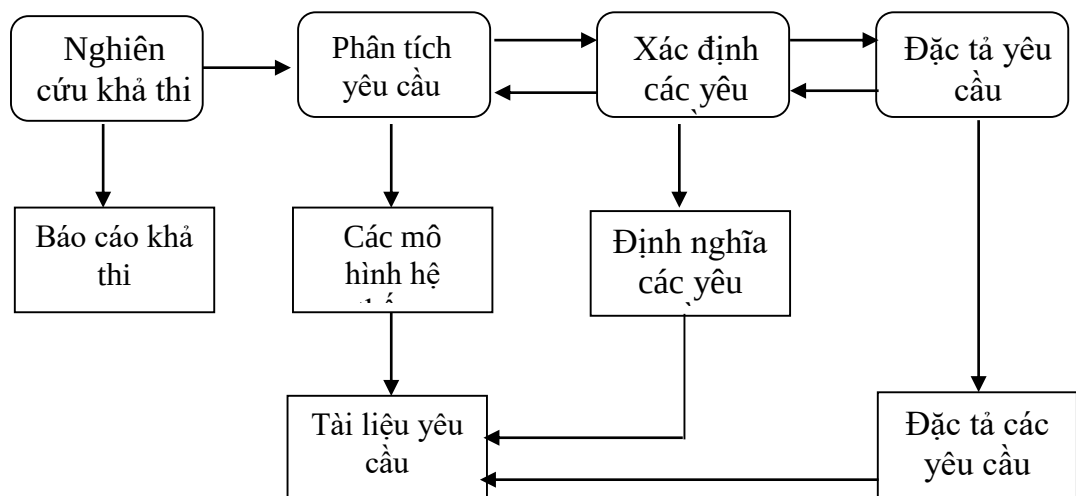
2.2 Phân tích yêu cầu

Nghiên cứu kỹ các yêu cầu của người sử dụng và của hệ thống phần mềm để xây dựng các đặc tả về hệ thống là cần thiết, nó sẽ xác định hành vi của hệ thống [3]. Nhiệm vụ của giai đoạn này là phải trả lời được các câu hỏi sau:

- Đầu vào của hệ thống là gì?
- Những quá trình cần xử lý trong hệ thống, hay hệ thống phần mềm sẽ phải xử lý những cái gì?
- Đầu ra: kết quả xử lý của hệ thống là gì?
- Những ràng buộc trong hệ thống, chủ yếu là mối quan hệ giữa đầu vào và đầu ra như thế nào?

Trả lời được câu hỏi trên, nghĩa là phải xác định được chi tiết các yêu cầu làm cơ sở để đặc tả hệ thống. Đó là kết quả của sự trao đổi, thống nhất giữa người đầu tư, người sử dụng với người xây dựng hệ thống. Mục tiêu là xây dựng các hồ sơ mô tả chi tiết các yêu cầu của bài toán nhằm nêu bật được hành vi, chức năng cần thực hiện của hệ thống dự kiến.

Như vậy, phân tích yêu cầu là quá trình suy luận các yêu cầu hệ thống thông qua quan sát hệ thống hiện tại, thảo luận với các người sử dụng, phân tích công việc. Việc này có thể liên quan với việc tạo một hay nhiều mô hình khác nhau. Nó giúp các phân tích viên hiểu biết hệ thống. Các mẫu hệ thống cũng có thể được phát triển để mô tả các yêu cầu. Ta có quy trình để có các chức năng của hệ thống:



Hình 2.1 Quy trình xây dựng các chức năng hệ thống

Trong quá trình phân tích cần lưu ý đến tính khả thi của dự án.

- Khả thi về kinh tế: chi phí phát triển phải cân xứng với lợi ích mà hệ thống đem lại, gồm có:

+ Chi phí:

- Mua sắm: thiết bị, vật tư (phần cứng), tư vấn, cài đặt thiết bị, quản lý và phục vụ, ...
- Chi phí cho khởi công: phần mềm phục vụ cho hệ thống, hệ thống liên lạc (truyền dữ liệu), nhân sự ban đầu: đào tạo - huấn luyện, cải tổ tổ chức cho phù hợp, ...
- Chi phí liên quan: chi phí nhân công phục vụ thu nhập dữ liệu, sửa đổi, cập nhật hệ thống, chuẩn bị tài liệu, ...
- Chi phí liên tục là tổn kém nhất, gồm: bảo trì, thuê bao, khấu hao phần cứng, chi phí phục vụ cho vận hành, ...

+ Lợi nhuận do sử dụng hệ thống

- Nhiệm vụ xử lý thông tin: giảm chi phí do xử lý tự động, tăng độ chính xác và kết quả tốt hơn, thời gian trả lời rút ngắn, ...

-
- Có được từ hệ thống: thu thập và lưu trữ dữ liệu tự động, đầy đủ, dữ liệu được chuẩn hóa, bảo đảm an toàn và an ninh dữ liệu, tương thích và chuyển đổi giữa các bộ phận, truy cập và tìm kiếm nhanh, kết nối và trao đổi diện rộng, ...

- Khả thi về kỹ thuật: đây là vấn đề cần lưu ý vì các mục tiêu, chức năng và hiệu suất của hệ thống theo một cách nào đó là còn "mơ hồ" do vậy xét:

- + Rủi ro xây dựng: các phần tử hệ thống (chức năng, hiệu suất) khi thiết kế và phân tích có tương đương hay không?

- + Có sẵn tài nguyên: có sẵn con người và tài nguyên cần thiết để phát triển hệ thống?

- + Công nghệ: các công nghệ liên quan cho việc phát triển hệ thống đã có sẵn hay chưa?

- Khả thi về hợp pháp: có sự xâm phạm, vi phạm hay khó khăn nào gây ra khi xây dựng hệ thống hay không.

- Các phương án: đánh giá về phương án tiếp cận đến việc xây dựng hệ thống.

2.3 Đặc tả yêu cầu

2.3.1. Đặc tả yêu cầu

Khi đã xác định rõ bài toán thì bước tiếp theo là tìm hiểu xem hệ thống dự kiến sẽ yêu cầu làm cái gì. Điều quan trọng ở đây là phải xây dựng được danh sách các yêu cầu của người sử dụng. Dựa trên những yêu cầu của người sử dụng, người phát triển đưa ra các đặc tả cho hệ thống.

Người xây dựng hệ thống phải trả lời được các yêu cầu sau đây:

- Đầu ra của hệ thống là cái gì
- Hệ thống sẽ phải làm cái gì để có kết quả mong muốn, nghĩa là phải xử lý những cái gì
- Những tài nguyên mà hệ thống yêu cầu là gì

Hiểu rõ nguồn gốc, các dạng thông tin cần cung cấp cho hệ thống hoạt động. Hệ thống sẽ phải giải quyết những vấn đề gì, những kết quả cần phải có là gì. Xác định được mối quan hệ giữa cái vào và cái ra cho quá trình hoạt động của hệ thống. Các đặc tả chi tiết phục vụ cho việc xây dựng và trắc nghiệm về hệ thống để kiểm tra xem những nhiệm vụ đã đặt ra có hoàn tất được hay không.

Ở đây, chúng ta cần chú ý là trong một số trường hợp, sẽ nảy sinh những yêu cầu mới mà có thể là ta phải xây dựng lại hệ thống, tất nhiên điều này sẽ làm chậm tiến trình xây dựng và làm tăng giá thành do một vài lý do để không thể hoàn chỉnh các đặc tả đối với các hệ thống như:

- Các hệ thống phần mềm lớn luôn đòi hỏi cải tiến từ hiện trạng. Mặc dù các khó khăn của hệ thống hiện tại có thể xác định được nhưng các ảnh hưởng và hiệu ứng của hệ thống mới khó có thể dự đoán trước được.

- Hệ thống lớn thường có nhiều cộng đồng sử dụng khác nhau. Họ có các yêu cầu và ưu tiên khác nhau. Các yêu cầu hệ thống cuối cùng không tránh khỏi các thỏa hiệp.

- Người trả tiền cho hệ thống và người sử dụng thường khác nhau. Các yêu cầu đưa ra do ràng buộc của các tổ chức và tài chính có thể tranh chấp với yêu cầu của người sử dụng.

Do các đặc tả yêu cầu thêm các thông tin vào định nghĩa yêu cầu nên các đặc tả thường được biểu diễn cùng với các mô hình hệ thống được phát triển trong quá trình phân tích yêu cầu. Nó cần bao gồm mọi thông tin cần thiết về yêu cầu chức năng và ràng buộc của hệ thống. Phân tích yêu cầu được tiếp tục xác định và đặc tả khi các yêu cầu mới nảy sinh. Đây là tài liệu thường xuyên thay đổi và nên được kiểm soát chặt chẽ.

Ngôn ngữ tự nhiên không hoàn toàn thuận tiện cho các thiết kế viên hoặc các hợp đồng giữa các khách hàng và cán bộ phát triển hệ thống vì có một số lý do như sau:

- Nhầm lẫn do cách hiểu các khái niệm khác nhau giữa hai bên.
- Đặc tả yêu cầu ngôn ngữ tự nhiên quá mềm dẻo. Một vấn đề có thể được mô tả bằng quá nhiều cách khác nhau.

- Các yêu cầu không được phân hoạch tốt, khó tìm các mối quan hệ, ...

Do vậy người ta thường dùng các thay thế khác để đặc tả các yêu cầu như:

- Ngôn ngữ tự nhiên có cấu trúc,
- Ngôn ngữ mô tả thiết kế, giống ngôn ngữ lập trình nhưng có mức trừu tượng cao hơn,

- Ngôn ngữ đặc tả yêu cầu,

- Ghi chép graphic,

- Đặc tả toán học, ...

Có thể chia đặc tả yêu cầu ra làm hai loại: đặc tả phi hình thức (ngôn ngữ tự nhiên) và đặc tả hình thức (dựa trên kiến trúc toán học).

1. Đặc tả phi hình thức

Đặc tả phi hình thức là đặc tả sử dụng ngôn ngữ tự nhiên. Tuy nó không được chặt chẽ bằng đặc tả hình thức nhưng được nhiều người biết và có thể dùng để trao đổi với nhau để làm chính xác hóa các điểm chưa rõ, chưa thống nhất giữa các bên phát triển hệ thống.

2. Đặc tả hình thức

Đặc tả hình thức là đặc tả mà ở đó các từ ngữ, cú pháp, ngữ nghĩa được định nghĩa hình thức dựa vào toán học. Đặc tả hình thức có thể coi là một phần của hoạt động đặc tả

phần mềm. Các đặc tả yêu cầu được phân tích chi tiết. Các mô tả trừu tượng của các chức năng chương trình có thể được tạo ra để làm rõ yêu cầu.

Đặc tả phần mềm hình thức là một đặc tả được trình bày trên một ngôn ngữ bao gồm: từ vựng, cú pháp và ngữ nghĩa được định nghĩa. Định nghĩa ngữ nghĩa đảm bảo ngôn ngữ đặc tả không phải là ngôn ngữ tự nhiên mà dựa trên toán học. Các chức năng nhận các đầu vào trả lại các kết quả. Các chức năng có thể định ra các điều kiện tiên tố và hậu tố. Điều kiện tiên tố là điều kiện cần thỏa mãn để có dữ liệu vào, điều kiện hậu tố là điều kiện cần thỏa mãn sau khi có kết quả.

Có hai hướng tiếp cận đặc tả hình thức để phát triển các hệ thống tương đối phức tạp.

- Tiếp cận đại số, hệ thống được mô tả dưới dạng các toán tử và các quan hệ.
- Tiếp cận mô hình, mô hình hệ thống được cấu trúc sử dụng các thực thể toán học như là các tập hợp và các thứ tự.

Sử dụng đặc tả hình thức, ta có các thuận lợi:

- Cho phép chúng ta thấy và hiểu được bản chất bên trong của các yêu cầu, đây là cách tốt nhất để làm giảm các lỗi, các thiếu sót có thể xảy ra và giúp cho công việc thiết kế được thuận lợi.

- Do chúng ta sử dụng toán học cho việc đặc tả nên có thể dựa vào các công cụ toán học khi phân tích và điều này làm tăng thêm tính chắc chắn và tính đầy đủ của hệ thống.

- Đặc tả hình thức, bản thân nó cho chúng ta một cách thức cho việc kiểm tra hệ thống sau này.

Tuy vậy, đặc tả hình thức cũng bộc lộ một vài khó khăn:

- Quản lý phần mềm có tính bảo thủ cố hữu của nó, không sẵn sàng chấp nhận các kỹ thuật mới.

- Chi phí cho việc đặc tả hình thức thường cao hơn so với các đặc tả khác (tuy phần cài đặt sẽ thấp hơn), nên khó để chứng minh rằng chi phí tương đối cao cho đặc tả sẽ làm giảm tổng chi phí dự án.

- Phần lớn, những người đặc tả hệ thống không được đào tạo một cách chính quy về việc sử dụng đặc tả hình thức cho việc đặc tả hệ thống mà dựa trên thói quen của họ.

- Thông thường, nhiều thành phần của hệ thống là khó cho việc đặc tả bằng ngôn ngữ hình thức. Thêm vào đó là khách hàng không thể hiểu được nó.

- Khách hàng không thích các đặc tả toán học.

2.3.2. Nguyên lý đặc tả

Nguyên lý 1: Phân tách chức năng với cài đặt: đặc tả là mô tả điều mong muốn chứ không phải cách thức thực hiện (cài đặt). Kết quả thu được theo dạng cái gì chứ không phải là thế nào.

Nguyên lý 2: Cần tới ngôn ngữ đặc tả hệ thống hướng tiến trình: cần thiết đặc biệt trong trường hợp môi trường là động và sự thay đổi của nó ảnh hưởng tới hành vi của thực thể nào đó tương tác với môi trường nào đó.

Nguyên lý 3: Đặc tả phải bao gồm hệ thống có phần mềm là một thành phần: vì hệ thống bao gồm các thành phần tương tác với nhau, chỉ bên trong hoàn cảnh của toàn bộ hệ thống và tương tác giữa các thành phần của nó thì hành vi của một thành phần mới có thể xác định.

Nguyên lý 4: Đặc tả phải bao gồm cả môi trường mà hệ thống vận hành.

Nguyên lý 5: Đặc tả hệ thống phải là một mô hình nhận thức: không phải là mô hình thiết kế hay cài đặt. Nó phải mô tả một hệ thống như cộng đồng người sử dụng cảm nhận thấy (các sự vật mà nó thao tác phải tương ứng với các sự vật của lĩnh vực đó; các tác nhân phải mô hình cho các cá nhân, tổ chức, trang thiết bị; các hành động họ thực hiện thì mô hình cho những hoạt động thực tế xuất hiện trong lĩnh vực; ...)

Nguyên lý 6: Đặc tả phải vận hành: phải đầy đủ và hình thức để có thể được dùng trong việc xác định liệu một cài đặt được đề nghị có thỏa mãn đặc tả trong những trường hợp kiểm thử tùy ý hay không.

Nguyên lý 7: Đặc tả hệ thống phải dung sai về tính không đầy đủ và tính nâng cao. Đặc tả không thể hoàn toàn đầy đủ do môi trường phức tạp

- Đặc tả là mô hình - sự trừu tượng hóa - của tình huống thực nên không đầy đủ.
- Đặc tả sẽ tồn tại ở nhiều mức chi tiết
- Các công cụ phân tích được sử dụng để giúp cho đặc tả và kiểm thử đặc tả phải có khả năng xử lý với tính không đầy đủ.

Nguyên lý 8: Đặc tả phải được cục bộ hóa và được ghép lồng lẻo: Đặc tả làm cơ sở cho thiết kế và cài đặt, không phải tĩnh mà là một sự vật động, đang trải qua thay đổi đáng kể nên nội dung và cấu trúc phải phù hợp. Sự thay đổi khi cần sửa đổi là tối thiểu, chỉ một phần nhỏ các thành phần có thể thâm vào hay loại bớt một cách dễ dàng.

2.4 Tư liệu hóa yêu cầu phần mềm

Các yêu cầu hệ thống được trình bày trong tài liệu các yêu cầu phần mềm cho biết những thứ cần bộ phát triển hệ thống cần biết. Tài liệu này bao gồm các định nghĩa về yêu cầu và các đặc tả về các yêu cầu. Trong một số trường hợp, chúng không được trình bày riêng biệt mà được tích hợp làm một. Đôi khi, định nghĩa yêu cầu được trình bày như là một giới thiệu tới đặc tả yêu cầu. Cách tiếp cận hiệu quả nhất là trình bày các đặc tả chi tiết như là phụ lục của yêu cầu.

Tài liệu yêu cầu phần mềm không phải tài liệu đặc tả. Nó cần phải mô tả cái hệ thống cần phải làm chứ không phải làm thế nào. Tài liệu này cần dễ dàng được đặc tả và ánh xạ sang các phản tương ứng của thiết kế hệ thống. Nếu các dịch vụ, ràng buộc và các

đặc tả thuộc tính trong tài liệu yêu cầu phần mềm được thỏa mãn bởi thiết kế thì thiết kế này được coi là giải pháp thích hợp với vấn đề.

Về nguyên tắc, các yêu cầu cần được hoàn chỉnh và chắc chắn. Mọi chức năng hệ thống cần được đặc tả và các yêu cầu không được mâu thuẫn. Tuy nhiên các thiếu sót là không thể tránh khỏi, do vậy tài liệu nên được cấu trúc để cho việc thay đổi. Nội dung nên được chia thành các chương. Sáu yêu cầu cần được thỏa mãn là:

- Nó cần mô tả các hành vi hệ thống bên ngoài
- Nó cần mô tả các ràng buộc về thực hiện
- Nó cần phải dễ thay đổi
- Nó phải là công cụ tham chiếu cho người bảo trì hệ thống
- Nó cần ghi được vòng đời của hệ thống
- Nó cần biểu thị được các đáp ứng chấp nhận được với các sự kiện không dự kiến

Cấu trúc chung của tài liệu yêu cầu phần mềm gồm các phần như sau:

- Giới thiệu: mô tả sự cần thiết của hệ thống. Nó cần sự mô tả sơ lược các chức năng của mình và giải thích cách làm việc với các hệ thống khác. Nó cũng cần mô tả làm thế nào hệ thống đáp ứng được toàn bộ các mục tiêu chiến lược và nghiệp vụ.

- Thuật ngữ: nó cần định nghĩa các khái niệm kỹ thuật được sử dụng trong tài liệu này. Không được giả định người đọc đã có kinh nghiệm.

- Mô hình hệ thống: phần này lập một hoặc nhiều mô hình hệ thống cho biết các quan hệ giữa các cấu thành hệ thống với hệ thống và môi trường của nó. Nó cần bao gồm các mô hình đối tượng, mô hình luồng dữ liệu và ngữ nghĩa dữ liệu.

- Định nghĩa yêu cầu chức năng: các dịch vụ cung cấp cho người dùng cần được mô tả trong mục này. Mô tả có thể dùng ngôn ngữ tự nhiên, sơ đồ hoặc các dạng ghi chép khác cho phép khách hàng có thể hiểu được.

Các dịch vụ cung cấp cho người dùng cần được mô tả trong mục này. Mô tả có thể dùng ngôn ngữ tự nhiên, sơ đồ hoặc các dạng ghi chép khác cho phép khách hàng có thể hiểu được.

- Định nghĩa yêu cầu phi chức năng: các ràng buộc về phần mềm và các hạn chế đối với thiết kế cần phải được mô tả trong phần này. Nó có thể bao gồm các chi tiết của biểu diễn dữ liệu, thời gian đáp ứng và yêu cầu bộ nhớ, ... Các tiêu chuẩn về sản phẩm và quy trình cần tuân thủ cũng được mô tả.

- Tiến triển hệ thống: phần này mô tả các giả thiết căn bản làm cơ sở cho hệ thống và dự đoán các thay đổi về phát triển phần cứng, yêu cầu người dùng

- Đặc tả yêu cầu: mô tả các yêu cầu cơ bản chi tiết hơn. Nếu cần các chi tiết hơn có thể được thêm vào các yêu cầu phi chức năng, ví dụ giao diện với các hệ thống có thể được định nghĩa.

- Ngoài ra, tài liệu yêu cầu phần mềm có thể bao gồm thêm các phần sau:

+ Phần cứng: nếu hệ thống được phát triển trên một phần cứng đặc biệt, phần cứng này và giao diện cần được mô tả. Nếu phần cứng bán sẵn được sử dụng, các cấu hình cực tiểu và cực đại phải được mô tả.

+ Yêu cầu dữ liệu: tổ chức logic của dữ liệu được sử dụng bởi hệ thống và các quan hệ giữa chúng được mô tả, có thể dùng sơ đồ thực thể liên kết.

+ Chỉ mục có thể được cung cấp. Ví dụ chỉ mục theo chữ cái, chỉ mục theo chương, theo chức năng....

Do hệ thống được vận hành trong thời gian dài, nên môi trường hệ thống và mục đích nghiệp vụ có thể thay đổi. Khi đó tài liệu yêu cầu cũng cần phải thay đổi. Với mục đích tiến triển, tài liệu yêu cầu thường được chia theo hai phân loại:

- Các yêu cầu ổn định: được suy dẫn từ các hoạt động cốt lõi của tổ chức tương đối liên quan trực tiếp tới miền hệ thống.

- Các yêu cầu bất thường: các yêu cầu có thể thay đổi khi phát triển hệ thống sau này như: các yêu cầu xuất hiện như là sự hiểu biết của khách hàng về sự phát triển của hệ thống trong quá trình xây dựng hệ thống, các yêu cầu được sinh ra do sự xuất hiện của việc tin học hóa làm thay đổi các quy trình nghiệp vụ, ...

2.5 Đặc tính dữ liệu và các kỹ thuật để thu thập dữ liệu

2.5.1. Đặc tính dữ liệu

Một ứng dụng thành công là một ứng dụng đáp ứng được đầy đủ các yêu cầu của người sử dụng. Trong quá trình xác định yêu cầu, các dữ liệu thu được của bài toán chứa một số tính chất mà ta gọi là đặc tính dữ liệu như:

- Tính định hướng thời gian,
- Tính cấu trúc,
- Tính đầy đủ,
- Nhập nhằng,
- Ngữ nghĩa,
- Độ lớn của dữ liệu, ...

Mỗi yếu tố trên đều quan trọng trong việc xác định các đặc tả của ứng dụng bởi vì chúng mang đến các chỉ dẫn cho kỹ sư phần mềm biết số lượng và kiểu thông tin nên được chọn. Cũng vậy, các kiểu dữ liệu khác nhau có liên quan tới các loại ứng dụng khác nhau và đòi hỏi các kỹ thuật khai thác thông tin khác nhau. Không chú ý tới các đặc tính dữ liệu sẽ gây lỗi phân tích và thiết kế. Mối liên hệ giữa các kiểu ứng dụng và các đặc tính dữ liệu của nó được thể hiện ở bảng sau:

Bảng 2-1 Bảng mối quan hệ giữa các kiểu ứng dụng và đặc tính dữ liệu

Đặc tính dữ liệu Loại ứng dụng	Tính thời gian	Cấu trúc	Mức đầy đủ	Nhập nhàng	Ngữ nghĩa	Độ lớn
TPS	Hiện tại	Cấu trúc	Đầy đủ	Thấp	Cố định	Các loại
CSDL	Quá khứ hiện tại	Cấu trúc	Đầy đủ	Thấp	Cố định	Các loại
DSS	Các loại	Cấu trúc	Thay đổi	Thấp, trung bình	Thay đổi	Trung bình cao
GDSS	Hiện tại tương lai	Phi cấu trúc	Không đầy đủ	Trung bình cao	Thay đổi	Thấp
EIS	Tương lai	Phi cấu trúc	Không đầy đủ	Trung bình cao	Thay đổi	Thấp trung bình
ES	Hiện tại (<i>dựa quá khứ</i>)	Bán cấu trúc	Không đầy đủ	Trung bình cao	Thay đổi	Thấp

2.5.1.1 Tính định hướng thời gian

Tính hướng thời gian của dữ liệu đề cập tới quá khứ, hiện tại hoặc các đòi hỏi tương lai của ứng dụng đã đề ra.

Các dữ liệu quá khứ: có thể mô tả công việc đã được biến đổi thế nào qua thời gian, các quy định ảnh hưởng thế nào tới nhiệm vụ, vị trí của nó trong tổ chức và nhiệm vụ. Các thông tin quá khứ là chính xác, đầy đủ và xác đáng [3].

Các thông tin hiện tại: là các thông tin về cái gì đang xảy ra. Ví dụ, thông tin ứng dụng hiện tại liên quan tới quá trình hoạt động của công ty, số lượng của các lệnh được thực hiện trong ngày hoặc số lượng các hàng hoá được sản xuất, các chính sách, sản phẩm, đòi hỏi nghiệp vụ, yêu cầu pháp quy hiện tại hoặc các ràng buộc khác cũng rất cần thiết

cho phát triển ứng dụng. Các thông tin hiện tại nên được tư liệu hoá theo cách thích hợp với đội ngũ phát triển để tăng trí thức của họ về ứng dụng và phạm vi bài toán.

Các đòi hỏi trong tương lai: liên quan tới các sự thay đổi sẽ xảy ra, chúng không chính xác và rất khó kiểm tra. Ví dụ: các dự đoán kinh tế, khuynh hướng tiếp thị, bán hàng, ...

2.5.1.2 Tính cấu trúc

Cấu trúc của thông tin định hướng về phần mở rộng theo đó thông tin có thể được phân loại theo cách nào đó. Cấu trúc có thể tham chiếu tới các hàm, môi trường hoặc dạng dữ liệu hay dạng xử lý. Các thông tin thay đổi từ phi cấu trúc cho tới cấu trúc mà phần cấu trúc được xác định bởi kỹ sư phần mềm. Cấu trúc là đặc biệt quan trọng bởi vì thiếu nó ta có thể tạo ứng dụng sai.

2.5.1.3 Tính đầy đủ

Tính đầy đủ thể hiện ở chỗ các thông tin cần thiết phải được biểu diễn. Một kiểu ứng dụng đòi hỏi một mức độ đầy đủ khác nhau. Các hệ thống xử lý giao dịch luôn tiếp cận các thông tin đầy đủ và chính xác, trong khi các hệ hỗ trợ quyết định đòi hỏi thông tin ít đầy đủ hơn. Các hệ thông tin điều hành, hệ chuyên gia, hoặc là các ứng dụng trí tuệ nhân tạo có mức độ cao nhất về tính không đầy đủ trong phạm vi của ứng dụng.

Đối với các ứng dụng phải giải quyết các thông tin không đầy đủ, một thách thức đối với nhóm phát triển là phải quyết định thông tin đã đủ để sử dụng hay chưa. Đôi khi quyết định này được tiến hành từ phía người dùng, đôi khi nó được tiến hành bên trong ứng dụng và cần phải có luật để xác định mức độ đầy đủ.

2.5.1.4 Nhập nhằng

Tính nhập nhằng là một thuộc tính của dữ liệu, thể hiện ở chỗ không trong sáng về nghĩa hoặc có nhiều nghĩa một cách hữu ý. Tính này liên quan nhiều đến mức độ ngữ nghĩa. Vấn đề này nảy sinh khi gặp một vấn đề có thể được hiểu theo nhiều cách - ví dụ câu phát biểu: "Ông cụ già đi nhanh quá!". Để giải quyết tính nhập nhằng cần căn cứ vào ngữ cảnh.

2.5.1.5 Ngữ nghĩa

Ngữ nghĩa là một tập hợp các định nghĩa được chia sẻ cho biết các thuật ngữ, chính sách hoặc các hành động được hiểu như thế nào cho mọi người trong một tổ chức nào đó.

Ngữ nghĩa rất quan trọng trong phát triển ứng dụng và đối với bản thân ứng dụng. Nếu mọi người dùng chung một thuật ngữ nhưng có quan niệm khác nhau sẽ xuất hiện sự không hiểu và không trao đổi thông tin được. Đối với bản thân ứng dụng nếu dữ liệu bị nhập nhằng về ý nghĩa có thể sẽ không bao giờ được xử lý cho đến khi người sử dụng hiểu được ý nghĩa của dữ liệu. Các ứng dụng sẽ có ngữ nghĩa cố định với các mục dữ liệu được định tính thông qua việc đào tạo và quá trình sử dụng lâu dài. Khi đánh mất ngữ nghĩa của thông tin có thể gây tổn thất rất lớn đối với các bên liên quan.

2.5.1.6 Độ lớn của dữ liệu

Độ lớn của dữ liệu là số lượng các sự kiện nghiệp vụ hệ thống phải tiến hành trong vài chu kỳ nào đó. Độ lớn của tạo mới hoặc thay đổi khách hàng được tiến hành theo tháng hoặc năm, trong khi độ lớn của giao dịch nghiệp vụ được tiến hành theo ngày hoặc giờ và độ lớn tối đa. Độ lớn tối đa là số lượng các giao dịch hoặc các sự kiện nghiệp vụ được xử lý trong thời kỳ bận nhất. Thời kỳ cao điểm có thể theo năm hoặc cuối vài tháng, ví dụ chuẩn bị cho báo cáo nộp thuế. Độ lớn của dữ liệu là một nguồn thông tin phức tạp bởi vì số lượng thời gian cần thiết xử lý một giao dịch đơn có thể trở thành rất quan trọng đối với lượng lớn dữ liệu cần xử lý.

2.5.2. Các kỹ thuật để thu thập dữ liệu cho ứng dụng

Do mỗi giai đoạn phát triển hệ thống đều đòi hỏi có sự trao đổi giữa nhà phát triển và người dùng để nhận được thông tin có ích. Mỗi giai đoạn cần tìm kiếm một dải rộng các câu hỏi về ứng dụng. Khi phân tích tính khả thi, các câu hỏi tương đối rộng và tổng quát như: đâu là phạm vi của vấn đề, cách tốt nhất để tự động hoá là gì, công ty có cố gắng phát triển ứng dụng này không, công ty có thể hỗ trợ việc phát triển ứng dụng không? ...thì khi phân tích yêu cầu chúng ta tìm hiểu các thông tin liên quan ứng dụng là gì, như: các dữ liệu cần thiết là gì, các xử lý nào được tiến hành và các thông tin chi tiết liên quan?... Như vậy, làm thế nào để chúng ta thu thập đầy đủ các thông tin để phục vụ cho phát triển ứng dụng?

2.5.2.1 Một số kỹ thuật để thu thập dữ liệu

1. Phỏng vấn

Phỏng vấn là việc tập hợp một số lượng ít người - thường với một hoặc hai người - cho một thời gian cố định với một mục đích cụ thể. Trong quá trình phỏng vấn, các câu hỏi có thể được thay đổi. Bạn có thể đánh giá được cảm nhận của họ, động cơ và thói quen đối với các phòng ban, quá trình quản lý, ứng dụng hoặc các thực thể khác đáng chú ý. Kiểu phỏng vấn được xác định bởi kiểu thông tin yêu cầu. Phỏng vấn nên được dẫn dắt sao cho cả hai bên tham gia đều cảm thấy thoải mái với kết quả. Được chuẩn bị kỹ đồng nghĩa là hiểu rõ người đang được phỏng vấn, do đó bạn không làm họ bối rối và bạn có thể có vài câu hỏi ban đầu được chuẩn bị cho dù không phải là tất cả.

Một phỏng vấn tốt có bắt đầu, đoạn giữa và kết thúc:

- Lúc bắt đầu, bạn tự giới thiệu và đặt các câu hỏi đơn giản. Nên bắt đầu với các câu hỏi tổng quát và không đòi hỏi các trả lời về động cơ cá nhân. Hãy chú ý tới các câu trả lời để tìm ra đầu mối các câu hỏi khác và tính trung thực, thái độ của người được phỏng vấn.

- Vào giữa buổi nên tập trung vào chủ đề. Hãy lấy mọi thông tin bạn cần lưu ý, sử dụng các kỹ thuật các bạn đã chọn ban đầu. Nếu xuất hiện một vài thông tin quan trọng, hãy hỏi xem bạn có thể được thảo luận nó sau này.

- Vào lúc kết thúc, hãy tóm tắt tất cả các thứ bạn đã nghe và nói cho các phỏng vấn cái gì sẽ xảy ra tiếp. Bạn có thể ghi chép và đề nghị người được hỏi xem xét lại. Tốt nhất là trong thời gian khoảng 48 giờ và có sự chấp thuận của người dùng theo ngày xác định.

Phỏng vấn sử dụng hai kiểu câu hỏi:

- Câu hỏi mở là câu hỏi cho nhiều câu trả lời khác nhau. Câu hỏi mở thích hợp cho các mô tả các chức năng ứng dụng hiện tại cũng như đang đề nghị và cho việc xác định cảm nhận ý kiến, và mong đợi về ứng dụng được đề ra. Một ví dụ là: “Ông có thể nói cho tôi về...” “Ông nghĩ thế nào về...” “Ông có thể mô tả làm thế nào...”

- Câu hỏi đóng là câu hỏi mà câu trả lời là “có” hoặc “không” hoặc một câu trả lời cụ thể. Các câu hỏi đóng tốt cho khai thác thông tin thực tế hoặc bắt người dùng tập trung vào phỏng vấn. Ví dụ cho câu hỏi đóng là: “Bạn có dùng các báo cáo hàng tháng không”. Các câu trả lời “Có” có thể được tiếp nối bởi các câu hỏi mở “Ông có thể giải thích...”

Các bước thường được tiến hành cho cuộc phỏng vấn

- Tiến hành đặt cuộc hẹn phù hợp với thời gian của phỏng vấn
- Chuẩn bị tốt, tìm hiểu kỹ về người được phỏng vấn
- Đúng giờ
- Có kế hoạch cho cuộc phỏng vấn
 - + Giới thiệu bản thân và mục đích
 - + Sử dụng câu hỏi mở để bắt đầu
 - + Luôn chú ý vào trả lời
 - + Có kế hoạch cho nội dung chính
 - + Kết hợp câu hỏi đóng và mở
 - + Luôn bám sát các trình bày và phát triển chi tiết
 - + Luôn cung cấp các thông tin phản hồi, ví dụ: “Cho phép tôi trình bày lại điều ông vừa nói...”
 - + Hạn chế ghi chép nếu thấy không tiện
 - + Có kế hoạch kết thúc
 - + Tóm tắt nội dung, yêu cầu hiệu chỉnh
 - + Yêu cầu làm chính xác, đánh giá lại ghi chép
 - + Cho biết ngày tháng họ sẽ nhận được báo cáo
 - + Thống nhất lại ngày lấy lại bản hiệu chỉnh
 - + Xác nhận lại lịch làm việc

Các câu hỏi có thể được đưa ra theo kiểu câu trúc hoặc phi cấu trúc.

- Phỏng vấn có cấu trúc là phỏng vấn trong đó người phỏng vấn đã có danh mục các mục cần duyệt qua, các câu hỏi xác định và các thông tin cần biết xác định.

- Phỏng vấn không cấu trúc là phỏng vấn được định hướng bởi câu trả lời. Các câu hỏi phần lớn là câu hỏi mở. Không có một kế hoạch ban đầu, do vậy người phỏng vấn biết

các thông tin cần thiết sẽ dùng các câu trả lời từ các câu hỏi mở để phát triển mọi câu hỏi chi tiết hơn về chủ đề.

Phỏng vấn có cấu trúc thích hợp khi bạn biết về các thông tin cần thiết trước khi phỏng vấn. Ngược lại phỏng vấn không cấu trúc thích hợp khi bạn không đoán trước được chủ đề.

Các trường hợp điển hình của phỏng vấn là người khách hàng bắt đầu với phỏng vấn phi cấu trúc để cho bạn nhận thức về miền bài toán. Sau đó bài toán dần dần trở thành có cấu trúc và tập trung vào các thông tin bạn cần để hoàn chỉnh phân tích.

Các thuận lợi và khó khăn của phỏng vấn có cấu trúc và phỏng vấn không có cấu trúc được thể hiện như sau:

Bảng 2-2 Ưu, nhược điểm các loại phỏng vấn

	Phỏng vấn có cấu trúc	Phỏng vấn không có cấu trúc
Ưu điểm	- Dùng dạng chuẩn cho nhiều câu hỏi - Dễ quản lý và đánh giá - Đánh giá được nhiều mục đích - Không cần đào tạo nhiều - Có kết quả trong các phỏng vấn	- Có khả năng mềm dẻo nhất - Cần chăm chú nghe và có kỹ năng mở rộng câu hỏi - Có thể bao hết các thông tin chưa biết - Đòi hỏi có thực hành
Nhược điểm	.- Chi phí chuẩn bị lớn - Tính cấu trúc có thể không thích hợp cho mọi tình huống - Giảm tính chủ động của các người phỏng vấn	- Lãng phí thời gian phỏng vấn - Người phỏng vấn có thể định kiến với các câu hỏi - Tốn thời gian lựa chọn và phân tích thông tin

Ưu nhược điểm của các loại phỏng vấn

Một kỹ năng tốt là phát triển các sơ đồ như là một phần của tài liệu phỏng vấn. Khi bắt đầu lần phỏng vấn mới, chúng ta nên bàn bạc các sơ đồ và đưa cho họ bản ghi chép để họ có thể kiểm tra sau này. Chúng ta sẽ nhận ngay ý kiến phản hồi về tính chính xác của sơ đồ và hiểu biết của mình về ứng dụng. Lợi ích của cách tiếp cận này thể hiện cả mặt kỹ năng và tâm lý. Từ khía cạnh kỹ thuật, chúng ta thường xuyên được kiểm tra lại các vấn đề được nghe. Cho tới khi thời gian phân tích kết thúc, chúng ta và khách hàng đều tin chắc rằng quá trình xử lý ứng dụng là đúng và đầy đủ. Từ khía cạnh tâm lý, chính chúng ta làm tăng niềm tin của khách hàng vào khả năng phân tích bằng cách trình bày các hiểu biết của mình. Mỗi khi chúng ta cải thiện sơ đồ và đi sâu vào phân tích sẽ làm tăng niềm tin của người sử dụng rằng chúng ta sẽ xây dựng được ứng dụng đáp ứng được nhu cầu của họ.

Các kết quả sử dụng người phỏng vấn nên luôn được trao đổi lại tới người được phỏng vấn trong một thời gian ngắn. Các người được phỏng vấn nên được báo cho một

thời hạn đối với việc phỏng vấn. Tuy nhiên chúng ta có thể xin bổ trí bổ sung phỏng vấn trong trường hợp còn nhiều điều cần hỏi hoặc còn nhiều người cần gặp.

Ưu điểm của phương pháp phỏng vấn:

- Phỏng vấn thích hợp cho việc nhận các thông tin đảm bảo cả số lượng lẫn chất lượng:

- Các kiểu thông tin định tính là các ý kiến, niềm tin, thói quen, chính sách, và mô tả.

- Các kiểu về thông tin định lượng bao gồm tần suất, số lượng, định lượng các mục được dùng trong ứng dụng.

- Nhược điểm của phương pháp phỏng vấn:

- Phỏng vấn và các dạng khác của thu thập dữ liệu có thể làm chúng ta lạc lối, thiếu chính xác, hoặc thông tin không liên quan, không thích hợp.

- Chúng ta cần học cách đọc ngôn ngữ cử chỉ và thói quen để quyết định được các điều cần thiết cho cùng một thông tin.

- Đòi hỏi kỹ năng

- Có thể có kết quả thiên vị

- Đòi hỏi ba người để kiểm tra kết quả và thường không thích hợp với số lượng lớn người

Do vậy, khi sử dụng phương pháp phỏng vấn để thu thập dữ liệu, người phỏng vấn phải hiểu các hành động của người được phỏng vấn để có thể có các hành động tương ứng cho phù hợp. Sau đây là bảng tham khảo các hành vi của người được phỏng vấn và hoạt động tương ứng của người phỏng vấn.

Bảng 2-3 Bảng tham khảo các hành vi và hoạt động tương ứng

Hành vi của người được phỏng vấn	Đáp ứng của người phỏng vấn
Đoán các câu trả lời chứ không thừa nhận không biết	Sau phỏng vấn, kiểm tra chéo các câu trả lời
Cố nói những điều lọt tai người phỏng vấn, sai sự thật	Tránh hỏi các câu dễ đoán câu trả lời, kiểm tra chéo các câu trả lời
Cho thông tin không đầy đủ	Kiên trì hỏi để đạt mục đích
Dừng trình bày khi người phỏng vấn ghi chép	Ghi nhanh nhất có thể, chỉ hỏi các câu quan trọng
Vội vã	Đề nghị bổ trí buổi khác
Không muốn thay đổi môi trường hiện tại	Động viên cải thiện môi trường hiện tại và so sánh hai khuynh hướng

Hành vi của người được phỏng vấn	Đáp ứng của người phỏng vấn
Thể hiện không quan tâm, trả lời ngắn quăng	Nói chuyện vui sau đó hướng về chủ đề chính
Không hợp tác, từ chối trả lời	Lấy nguồn tin khác, và hỏi “Ông có quan tâm về những điều người khác nói về ông hay không?”. Nếu câu trả lời là không thì thôi phỏng vấn
Phàn nàn về vị trí công tác, lương, ...	Tìm ra mấu chốt vấn đề. Có dẫn dắt về chủ đề chính. Ví dụ “Dường như cơ quan ông có rất nhiều vấn đề, có thể ứng dụng mới được đề xuất sẽ giải quyết mọi vấn đề trên”
Là người ưa chuộng công nghệ	Chọn lọc các thông tin cần thiết, không để lôi cuốn vào các chiến lược công nghệ

2. *Họp nhóm*

Họp nhóm là việc tập hợp ba hoặc nhiều hơn một số người cho một thời hạn nhất định để thảo luận một số chủ đề. Họp nhóm có thể vừa bổ sung và thay thế phỏng vấn. Nó bổ sung phỏng vấn bằng cách cho phép nhóm kiểm tra lại các kết quả phỏng vấn cá nhân. Nó có thể thay thế cho phỏng vấn bằng cách cung cấp một diễn đàn cho các người sử dụng cùng tìm ra các đòi hỏi và các giải pháp cho ứng dụng.

Họp nhóm cũng có thể làm lãng phí thời gian. Nói chung, họp nhóm càng lớn thì càng ít ý kiến nhất trí và thời gian quyết định càng kéo dài. Do vậy, nên có một kế hoạch ban đầu cho họp nhóm. Lịch trình nên được cung cấp trước cho các thành viên. Số lượng các chủ đề nên giữ ở mức độ 1 đến 5 và không mời những người không thích hợp để tránh lãng phí thời gian. Họp nhóm nên có một thời gian cố định và có điểm thống nhất cụ thể với các quyết định cần thiết, không nên kéo dài hơn 2 giờ để có thể đảm bảo được sự chú ý của thành viên. Họp nhóm phù hợp với việc đòi hỏi các tổng quát, tìm kiếm đồng lòng, và tiếp nhận cả thông tin chi tiết và thông tin tóm tắt.

Vậy, ưu điểm của phương pháp họp nhóm:

- Có thể tạo ra quyết định,
- Nhận được cả thông tin tổng hợp và chi tiết,
- Phương pháp tốt cho các yêu cầu bên ngoài,
- Tập hợp được nhiều người dùng liên quan, ...

Nhược điểm của phương pháp họp nhóm:

- Nếu số đại biểu nhiều sẽ tốn thời gian cho việc ra quyết định,
- Tốn thời gian,

-
- Các ngắt quãng làm phân tán sự chú ý,
 - Mời không đúng thành viên dẫn tới chậm có kết quả,
 - Dễ chuyển sang các chủ đề không liên quan như là chính trị, thể thao, ...

3. Quan sát

Quan sát là hoạt động có thể tiến hành thủ công hoặc tự động như sau:

- Theo cách thủ công, người quan sát ngồi tại một chỗ và ghi chép các hoạt động, các bước xử lý công việc. Ghi chép hoặc băng ghi hình được dùng để phân tích cho các sự kiện, các mô tả động từ chính, hoặc các hoạt động chỉ rõ lý do, công việc hoặc các thông tin về công việc.

- Theo cách tự động, máy tính sẽ lưu giữ các chương trình thường trú, lưu lại vết của các chương trình được sử dụng, email và các hoạt động khác được xử lý bởi máy. Các

- Theo cách tự động, máy tính sẽ lưu giữ các chương trình thường trú, lưu lại vết của các chương trình được sử dụng, email và các hoạt động khác được xử lý bởi máy. Các file nhật ký của máy sẽ được phân tích để mô tả công việc.

Quan sát có các nhược điểm:

- Thời gian của quan sát có thể không biểu diễn cho các công việc diễn ra thông thường,

- Ý nghĩ là đang bị quan sát có thể làm thay đổi thói quen thường ngày của người bị quan sát,

- Tốn thời gian.

Tuy nhiên, quan sát có các ưu điểm:

- Kỹ sư phần mềm có thể nhận được sự hiểu biết tốt về môi trường công tác hiện tại và quá trình xử lý thông qua quan sát.

- Kỹ sư phần mềm có thể tập trung vào vấn đề, mà không bị ảnh hưởng bởi người khác.

- Các ngăn cách giữa kỹ sư phần mềm và các người được phỏng vấn sẽ được vượt qua bởi quan sát.

Để quan sát có hiệu quả, nên xác định cái gì sẽ được quan sát và xác định thời gian cần thiết cho việc quan sát. Đồng thời, hãy xin sự chấp thuận của cả người quản lý và cá nhân trước khi tiến hành quan sát.

4. *Án định công việc tạm thời*

Với một công việc tạm thời, ta có được nhận thức đầy đủ hơn về các nhiệm vụ, đồng thời nó cung cấp cho ta kinh nghiệm thực tế. Đầu tiên, ta học các thuật ngữ và ngữ cảnh sử dụng nó. Thời gian tiến hành thường từ 2 tuần đến 1 tháng đủ dài để ta có thể quen thuộc với phần lớn các công việc thông thường và các tình huống ngoại lệ nhưng không được quá dài để trở thành chuyên gia thực sự đối với công việc.

Công việc tạm thời cho ta cơ sở hình thức hoá các câu hỏi về các chức năng nào của phương pháp hiện thời của công việc sẽ được lưu giữ lại và cái nào sẽ bị loại trừ hoặc thay đổi. Nó cũng là cách thức để trả lời các câu hỏi không thể thực hiện được

Bất lợi của công việc tạm thời là tốn thời gian và sự lựa chọn về thời gian có thể làm tối thiểu hoá vấn đề. Thêm vào đó, người kỹ sư phần mềm có thể có thiên kiến về quá trình xử lý công việc, nội dung làm ảnh hưởng tới công việc thiết kế sau này.

5. *Điều tra qua câu hỏi*

Điều tra qua câu hỏi là xây dựng các câu hỏi để phỏng vấn trên giấy hoặc máy tính. Các câu hỏi được dùng để nhận các thông tin từ số lượng lớn người sử dụng và thường ở dạng khả năng lựa chọn, người trả lời chỉ việc đánh dấu. Các mục câu hỏi, như là phỏng vấn, có thể là câu hỏi mở hoặc câu hỏi đóng nhưng không chỉ rõ tên, dẫn đến các câu trả lời trung thực hơn nhiều phỏng vấn.

Ưu điểm của phương pháp điều tra:

- Các trả lời không cần biết tên nên quan điểm và cảm nhận thu được là trung thực,
- Có thể tiến hành với nhiều người,
- Thích hợp với các câu hỏi đóng và hữu hạn,
- Phù hợp với công ty đa văn hoá và có thể tuỳ biến với quy ước địa phương, ...

Hạn chế của phương pháp điều tra:

- Khó thực hiện lại được,
- Các câu hỏi không có trả lời có nghĩa là không thu được thông tin,
- Các câu hỏi có thể khó hiểu,
- Thực hiện và đánh giá có thể chậm,
- Không thể thêm các thông tin khi đã tiến hành công việc,
- Thông tin thu được hạn chế trong một phạm vi hẹp,
- Chỉ dùng nó như một phương pháp bổ sung, ...

6. *Xem xét tài liệu*

Khái niệm tài liệu ám chỉ tới các cẩm nang, quy định, các thao tác chuẩn mà tổ chức cung cấp như là hướng dẫn cho các nhà quản lý và nhân viên.

Các tài liệu thực sự hữu ích cho các kỹ sư phần mềm để học về các lĩnh vực mà họ chưa từng có kinh nghiệm. Nó có thể hữu ích cho việc xác định các câu hỏi về quá trình thao tác và sản xuất. Tài liệu đưa ra các thông tin khách quan.

Tuy nhiên, các tài liệu không phải luôn nằm trong công ty, nó có thể là các ấn phẩm kỹ thuật, các báo cáo nghiên cứu, ... nên khó cho việc tìm kiếm. Nó còn gây thành kiến và không không cung cấp để có thể nhận biết được quan điểm, động cơ, ...

7. *Xem xét phần mềm*

Khi các ứng dụng cũ phải được thay thế các phần mềm mới, việc nghiên cứu các phần mềm đã tồn tại cung cấp cho chúng ta các thông tin về quá trình xử lý công việc hiện thời và các mở rộng có ràng buộc bởi thiết kế phần mềm.

Khuyết điểm chính của việc nhận thông tin từ quan sát phần mềm là tài liệu có thể không chính xác hoặc kịp thời. Thêm vào đó, thời gian có thể bị lãng phí nếu ứng dụng đang bị xoá bỏ.

2.5.2.2 Đánh giá tính phù hợp của các kỹ thuật thu thập dữ liệu đối với đặc tính của dữ liệu

Phòng vấn và họp nhóm là phù hợp với mọi loại kiểu dữ liệu do đó chúng thường xuyên được sử dụng.

Quan sát chỉ cung cấp các định hướng thô về mặt độ lớn, nhưng bị hạn chế bởi thời gian. Bởi vì thông tin từ quan sát không có cấu trúc, một vài kỹ năng được dùng cho kỹ sư phần mềm để tạo nên cấu trúc phù hợp với các tình huống. Thông tin có thể không được đầy đủ.

Kỹ thuật đặt câu hỏi có thể hỏi các câu hỏi có cấu trúc. Nếu câu hỏi là mở thì độ đầy đủ có thể thấp. Mức độ nhập nhằng của kỹ thuật đặt câu hỏi nên thấp nhưng ngữ nghĩa câu hỏi có thể không được biên dịch bởi người trả lời. Câu hỏi về độ lớn tại phòng ban hoặc tổ chức thường không thích hợp. Các thông tin về độ lớn của giao dịch hoặc thời gian cho xử lý giao dịch cho các nhân viên nên nhận được các thông tin có nghĩa.

Làm việc tạm thời tương tự quan sát ở chỗ có mức độ không chắc chắn cao kết hợp với các thông tin nhận được. Các thông tin có khuynh hướng hiện tại, phi cấu trúc, và không đầy đủ phụ thuộc vào các thời kỳ công tác. Các nhập nhằng thay đổi từ mức thấp đến mức trung bình phụ thuộc mức độ công việc được tổ chức và định nghĩa tốt đến mức nào. Nội dung ngữ nghĩa có thể thay đổi phụ thuộc các định nghĩa dùng chung trong nhóm làm việc.

Tài liệu cung cấp các thông tin không đầy đủ, phi cấu trúc. Thời gian nghiên cứu thay đổi phụ thuộc vào tài liệu nằm bên trong hay bên ngoài công ty. Các tài liệu bên trong thường liên quan tới các thông tin cũ trong khi tài liệu bên ngoài thường hướng về thông tin hiện tại và tương lai. Các tài liệu bên ngoài mang tính phổ quát và thống nhất trong khi tài liệu bên trong thường thay đổi theo phòng ban.

Phần mềm cung cấp các thông tin cũ, đôi khi là hiện tại được cấu trúc bởi vì đã được tự động hoá. Mức độ nhập nhằng có thể từ thấp đến trung bình. Các thông tin về độ lớn có thể được biểu diễn nhưng nên có kiểm tra chéo khi sử dụng các phương pháp khác.

Tính phù hợp của các kỹ thuật thu thập dữ liệu đối với đặc tính của dữ liệu được tóm tắt ở bảng sau:

Bảng 2-4 Tính phù hợp của các kỹ thuật thu thập dữ liệu

Đặc tính dữ liệu Kỹ thuật	Tính thời gian	Cấu trúc	Mức đầy đủ	Nhập nhằng	Ngữ nghĩa	Độ lớn
Phỏng vấn	Các loại	Các loại	Các loại	Các loại	Thay đổi	Các loại
Hợp nhóm	Các loại	Các loại	Các loại	Các loại	Thay đổi	Các loại
Quan sát	Hiện tại	Phi cấu trúc	Không đầy đủ	Thay đổi	Thay đổi	Thô
Câu hỏi	Các loại	Cấu trúc	Đầy đủ	Thấp	Cố định (phụ thuộc cách hiểu)	Cá nhân
Công việc tạm thời	Hiện tại	Phi cấu trúc	Không đầy đủ	Thấp, trung bình	Thay đổi	Phụ thuộc cách hiểu
Tài liệu	Quá khứ hiện tại	Phi cấu trúc	Không đầy đủ	Thấp, trung bình	Thay đổi, hoặc cố định	Có thể, hoặc không
Xem xét phần mềm	Quá khứ Hiện tại	Cấu trúc	Đầy đủ	Thấp, trung bình	Cố định	Có thể

2.5.2.3 Đánh giá tính phù hợp của các kỹ thuật thu thập dữ liệu đối với các kiểu ứng dụng

- Hệ xử lý giao dịch và các ứng dụng hỏi đáp có thể dùng mọi kỹ thuật.

Hợp nhóm và phỏng vấn có ưu thế vượt trội bởi vì chúng gợi ra một phạm vi rộng nhất các đáp ứng trong thời gian ngắn nhất.

Quan sát và ấn định công việc tạm thời hữu dụng cho việc nhận các thông tin nền tảng về vấn đề hiện thời, nhưng cần phải được sử dụng với chỉ định để không gây định kiến với thiết kế ứng dụng.

Các câu hỏi thích hợp với số lượng nhân viên tương đối lớn - lớn hơn 50 - và việc nhận ra các tính chất của người dùng đã xác định, ví dụ, yêu cầu đào tạo của người dùng trong khi phân tích các đặc tính của tổ chức. Nó yêu cầu về màn hình, ví dụ, các kiểu màu sắc khác nhau, các câu hỏi có thể thích hợp cho việc biểu diễn một tập nhỏ các lựa chọn cho người dùng.

- Hệ hỗ trợ quyết định cũng được coi là có thể dùng cho mọi kỹ thuật, nhưng không phải mọi kỹ thuật đều thích hợp cho mọi tình huống. Hệ hỗ trợ quyết định nói chung được phát triển dành cho các công việc mang tính chất riêng rẽ. Do đó, quan sát hoặc làm việc cùng một hoặc hai người đại diện có thể dẫn tới cách nhìn thành kiến về yêu cầu ứng dụng. Đối với tài liệu là các báo cáo thống kê có thể phù hợp cung cấp cho các ví dụ của các kiểu phân tích cần thiết của hệ hỗ trợ quyết định. Các tài liệu khác, như là chính sách, thủ tục nói chung là không thích hợp. Đối với các hệ hỗ trợ quyết định mục đích chung

cùng với số lượng lớn người dùng, các câu hỏi là cách thích hợp để xác định phạm vi của vấn đề và các kỹ thuật phân tích cần thiết cho hệ hỗ trợ quyết định. Các thông tin này có thể được bổ sung bởi phỏng vấn và họp nhóm để xác định thêm các chi tiết.

- Hệ hỗ trợ quyết định theo nhóm thường xuyên là một hệ cho khách hàng xây dựng các gói phần mềm cung cấp các kiểu hỗ trợ khác nhau cho các họp nhóm tự động. Các kỹ sư phần mềm làm việc trên môi trường hỗ trợ quyết định theo nhóm cần biết các kiểu dữ liệu, số lượng người tham gia, cũng như các kiểu suy luận và kỹ thuật thống nhất ý kiến của nhóm.

Các khối hỗ trợ quyết định theo nhóm không phải là các kiến thức chung mà cũng không được sử dụng thường xuyên. Chúng ta có thể xây dựng một hệ hỗ trợ quyết định theo nhóm cho một người. Do đó, một thời gian đáng kể phải dùng để tìm ra thị trường, người bán, và các khối hệ hỗ trợ quyết định theo nhóm. Các tài liệu bên ngoài về sản phẩm thích hợp cho các câu hỏi phát triển dần dần tới các thông tin cần thiết. Sau khi kiến thức về thị trường đã được thu thập thì phỏng vấn và họp nhóm thích hợp cho xác định các yêu cầu đặc trưng và để xem xét lại với người dùng hệ hỗ trợ quyết định theo nhóm có thể làm và không làm cái gì. Các phương pháp khác có thể có giá trị hữu hạn. Ví dụ, quan sát có thể giúp kỹ sư phần mềm biết các công cụ sẽ làm việc thế nào. Các tài liệu bên trong cung cấp các thông tin về họp nhóm mà hệ hỗ trợ quyết định theo nhóm được chờ đợi sẽ cung cấp cũng có thể được dùng.

- Hệ thông tin điều hành tương tự hệ hỗ trợ quyết định theo nhóm về sự khan hiếm và thiếu tổng quát về các thông tin - tri thức - liên quan. Hệ thông tin điều hành thường được xây dựng cho một nhóm người dùng tương đối ít nên không phù hợp với việc đặt câu hỏi. Hệ thông tin điều hành là môi trường khá chuyên biệt nên các tài liệu và phần mềm cũ không có giá trị nhiều. Quan sát không có tác dụng nhiều bởi vì việc thực hiện không thích hợp cho quan sát. Công việc tạm thời không dùng được vì ta không thể làm việc được chỉ trong một hoặc hai tuần. Do vậy, các phương pháp phỏng vấn, họp nhóm, xem tài liệu là các kỹ thuật phù hợp nhất.

Tính phù hợp của các kỹ thuật thu thập dữ liệu đối với các kiểu ứng dụng được chỉ ra ở bảng sau:

Bảng 2-5 Tính phù hợp của các kỹ thuật ứng dụng với các kiểu ứng dụng

Loại ứng dụng Kỹ thuật	TPS	CSDL	DSS	GDSS	EIS	ES
Phỏng vấn	Tốt	Tốt	Tốt	Tốt	Tốt	Tốt
Họp nhóm	Tốt	Tốt	Tốt	Tốt	Tốt	Tốt
Quan sát	Tốt	Tốt	Tốt	Hạn chế	Hạn chế	
Câu hỏi	Tốt	Tốt	Tốt			

Loại ứng dụng Kỹ thuật	TPS	CSDL	DSS	GDSS	EIS	ES
Công việc tạm thời	Tốt	Tốt	Tốt			
Xem tài liệu	Tốt	Tốt	Tốt			
Xem xét phần mềm	Tốt	Tốt	Tốt	Hạn chế	Hạn chế	Hạn chế

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

Câu 1. Hãy chỉ rõ vai trò quan trọng của việc xác định đúng đắn yêu cầu của bài toán.

Câu 2. Hãy phân tích để chỉ rõ các sai lầm thường gặp trong việc xác định yêu cầu.

Câu 3. Các công việc phải làm để có được một tài liệu yêu cầu chính xác.

Câu 4. Hãy nêu và phân tích các đặc tính dữ liệu của yêu cầu.

Câu 5. Các kỹ thuật để thu thập dữ liệu cho bài toán. Khi xây dựng một ứng dụng, bạn đã sử dụng những kỹ thuật nào, vì sao?

Câu hỏi trắc nghiệm:

Câu 1. Khi đặc tả yêu cầu, người xây dựng hệ thống không cần quan tâm tới yêu cầu nào sau đây?

- Đầu ra của hệ thống là cái gì.
- Hệ thống sẽ phải làm cái gì để có kết quả mong muốn, nghĩa là phải xử lý những cái gì.
- Các phương án sẽ tiếp cận hệ thống để hoàn tất sản phẩm.
- Hệ thống sẽ phải làm cái gì để có kết quả mong muốn, nghĩa là phải xử lý những cái gì.

Câu 2. Câu trả lời nào không phải là nhiệm vụ của giai đoạn phân tích yêu cầu?

- Đầu vào, đầu ra của hệ thống là gì.
- Những quá trình cần xử lý trong hệ thống, hay hệ thống phần mềm sẽ phải xử lý những cái gì.
- Những công cụ sẽ sử dụng trợ giúp trong quá trình xây dựng chương trình.
- Những ràng buộc trong hệ thống, chủ yếu là mối quan hệ giữa đầu vào và đầu ra như thế nào.

Câu 3. Trong các kỹ thuật sau, đâu không phải là kỹ thuật để thu thập dữ liệu?

- Phỏng vấn
- Họp nhóm

-
- c. Quan sát
 - d. Tiếp xúc

Câu 4. Thu thập dữ liệu bằng cách "tập hợp một số lượng ít người - thường với một hoặc hai người - cho một thời gian cố định với một mục đích cụ thể..." là sử dụng kỹ thuật nào?

- a. Điều tra qua câu hỏi
- b. Quan sát
- c. Họp nhóm
- d. Phỏng vấn

Câu 5: “Tập hợp ba hoặc nhiều hơn một số người cho một thời hạn nhất định để thảo luận một số chủ đề” là kỹ thuật thu thập dữ liệu nào?

- a. Phỏng vấn
- b. Họp nhóm
- c. Quan sát
- d. Xem xét tài liệu

Bài tập vận dụng:

Bài 1: Bạn được giao nhiệm vụ xác định yêu cầu, phân tích, đặc tả phần mềm “Quản lý sinh viên của một trường Đại học”, bạn hãy phân tích, lựa chọn phương pháp phù hợp và thực hiện nhiệm vụ này?

Bài 2: Bạn được giao nhiệm vụ xác định yêu cầu, phân tích, đặc tả phần mềm “Game 3D”, bạn hãy phân tích, lựa chọn phương pháp phù hợp và thực hiện nhiệm vụ này?

Bài 3: Bạn được giao nhiệm vụ xác định yêu cầu, phân tích, đặc tả phần mềm “Website quản lý bán hàng online của 1 hệ thống bán quần áo”, bạn hãy phân tích, lựa chọn phương pháp phù hợp và thực hiện nhiệm vụ này?

Bài 4: Bạn được giao nhiệm vụ xác định yêu cầu, phân tích, đặc tả ứng dụng dạng “Diệt virus”, bạn hãy phân tích, lựa chọn phương pháp phù hợp và thực hiện nhiệm vụ này?

Bài 5: Hãy chọn một phần mềm mà bạn từng tham gia phát triển hoặc bạn có định hướng tìm hiểu, bạn hãy phân tích, lựa chọn phương pháp phù hợp và thực hiện nhiệm vụ xác định yêu cầu, phân tích, đặc tả phần mềm này?

CHƯƠNG 3. THIẾT KẾ PHẦN MỀM

Mục tiêu của chương:

Mục tiêu của chương này là giới thiệu một số phương pháp và kỹ thuật chính trong thiết kế phần mềm, đối với việc triển khai một hệ thống thành nhiều hệ thống con và hệ thống con thành nhiều thành phần (components), và quản lý những vấn đề liên quan đến cấu trúc nội tại của những thành phần hệ thống. Đầu tiên chúng ta xem qua vài kỹ thuật thiết kế. Kế đến chúng ta sẽ xét qua một vài kỹ thuật thiết kế và phương pháp nền tảng một cách chi tiết. Thêm vào đó, chúng ta bàn qua những khía cạnh thiết kế như thiết kế giao diện người dùng và mô đun hóa.

3.1 Khái niệm thiết kế phần mềm

Thiết kế phần mềm là một quá trình được lặp đi lặp lại các yêu cầu qua đó chuyển thành “bản thiết kế” để xây dựng phần mềm. Ban đầu, bản thiết kế mô tả một cái nhìn tổng thể về phần mềm. Nghĩa là, thiết kế được thể hiện ở mức trừu tượng cao - mức có thể được truy tìm trực tiếp đến mục tiêu hệ thống cụ thể và các yêu cầu về dữ liệu, chức năng và hành vi chi tiết hơn. Khi các lần lặp lại của thiết kế xảy ra thì việc chỉnh sửa tiếp theo dẫn đến các biểu diễn thiết kế ở mức độ trừu tượng thấp hơn nhiều. Những điều này vẫn có thể được truy tìm từ các yêu cầu, nhưng kết nối là tinh tế hơn [1].

3.2 Đặc điểm và nguyên tắc thiết kế phần mềm

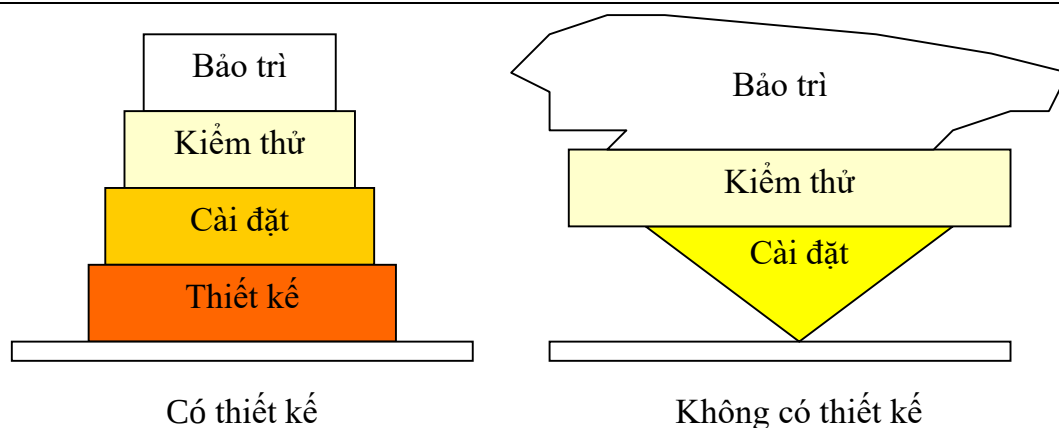
3.2.1. Đặc điểm

Ba đặc điểm đóng vai trò là hướng dẫn cho việc đánh giá một thiết kế phần mềm tốt:

- Thiết kế phải thực hiện tất cả các yêu cầu rõ ràng có trong mô hình phân tích và nó phải đáp ứng tất cả các yêu cầu mà khách hàng mong muốn.
- Thiết kế phải là một bản hướng dẫn dễ đọc, dễ hiểu cho những người lập trình và cho những người kiểm thử và sau đó hỗ trợ phần mềm.
- Thiết kế phải cung cấp một bức tranh toàn cảnh về phần mềm, giải quyết các miền dữ liệu, chức năng và hành vi từ góc độ triển khai.

Tầm quan trọng của thiết kế:

- Không có thiết kế, ta có nguy cơ dựng lên một hệ thống không ổn định - một hệ thống sẽ thất bại khi có một thay đổi nhỏ; một hệ thống khó có thể mà kiểm thử được; một hệ thống không thể nào xác định được chất lượng chừng nào chưa đến cuối tiến trình kiểm thử, khi thời gian còn rất ngắn mà không ít tiền đã phải chi ra.
- Thiết kế tốt là chìa khoá cho công trình hữu hiệu, không thể hình thức hoá quá trình thiết kế trong bất kỳ một công trình nào.



Hình 3.1 Mô hình có thiết kế và không có thiết kế

Như vậy, thiết kế là một thực tế về một quyết định chọn lựa, xây dựng một đặc tả về hành vi nhìn thấy được từ bên ngoài và bổ sung các chi tiết cần thiết cho việc cài đặt trên hệ thống máy tính bao gồm cả chi tiết về tổ chức quản lý dữ liệu, công việc và tương tác với con người. Thiết kế phải nhờ vào các kinh nghiệm và phải học tập những cái có sẵn từ các hệ thống khác; không thể chỉ đọc sách là đủ. Bản thiết kế tốt là chìa khóa cho sự thành công của hệ thống.

3.2.2. Nguyên tắc

Để thiết kế phần mềm tốt chúng ta cùng tìm các nguyên tắc thiết kế dưới đây.

Nguyên tắc 1: Một thiết kế nên thể hiện một cấu trúc, kiến trúc đã được tạo ra bằng cách sử dụng các mẫu thiết kế dễ nhận biết, bao gồm các thành phần thể hiện các đặc điểm thiết kế tốt và có thể được thực hiện theo cách tiến hóa, do đó tạo điều kiện thuận lợi cho việc triển khai và thử nghiệm.

Nguyên tắc 2: Một thiết kế nên được mô-đun hóa; nghĩa là, phần mềm nên được phân vùng một cách hợp lý thành các phần tử thực hiện các chức năng và chức năng con cụ thể.

Nguyên tắc 3: Một thiết kế phải chứa các đại diện riêng biệt về dữ liệu, kiến trúc, giao diện và các thành phần (mô-đun).

Nguyên tắc 4: Một thiết kế phải dẫn đến cấu trúc dữ liệu thích hợp cho các đối tượng được thực hiện và được rút ra từ các mẫu dữ liệu dễ nhận biết.

Nguyên tắc 5: Một thiết kế nên dẫn đến các thành phần thể hiện các đặc tính chức năng độc lập.

Nguyên tắc 6: Một thiết kế nên dẫn đến các giao diện làm giảm sự phức tạp của các kết nối giữa các mô-đun và với môi trường bên ngoài.

Nguyên tắc 7: Một thiết kế nên được tạo ra bằng cách sử dụng một phương pháp lặp lại được điều khiển bởi thông tin thu được trong quá trình phân tích yêu cầu phần mềm [1].

Quá trình thiết kế phần mềm khuyến khích thiết kế tốt thông qua việc áp dụng các nguyên tắc thiết kế cơ bản trên, phương pháp luận có hệ thống và xem xét kỹ lưỡng.

3.3 Các hoạt động của quá trình thiết kế phần mềm

- Thiết kế là một quá trình sáng tạo, đòi hỏi kinh nghiệm và sự tinh nhanh của người thiết kế.

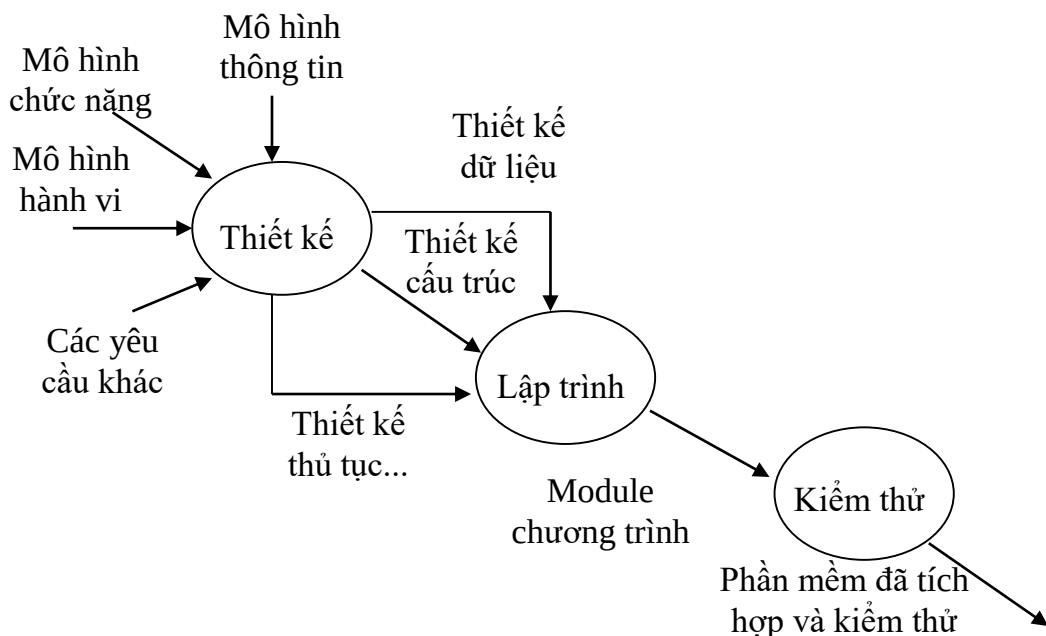
- Thiết kế phải được thực hành và học bằng kinh nghiệm, bằng khảo sát các hệ thống đang tồn tại, không thể học bằng sách vở (nói đúng ra là không đủ).

- Thiết kế phần mềm là một quá trình chuyển hoá các yêu cầu thành một biểu diễn phần mềm. Bước đầu, biểu diễn mô tả toàn bộ về phần mềm. Việc làm mịn tiếp theo sau dẫn tới một biểu diễn thiết kế gần với chương trình gốc.

- Thiết kế phần mềm nằm ở trung tâm kỹ thuật của tiến trình kỹ nghệ phần mềm và được áp dụng bất kể khuôn cảnh kỹ nghệ được sử dụng (thác nước, xoắn ốc, bản mẫu, thể hệ thứ 4 - 4GT, ...).

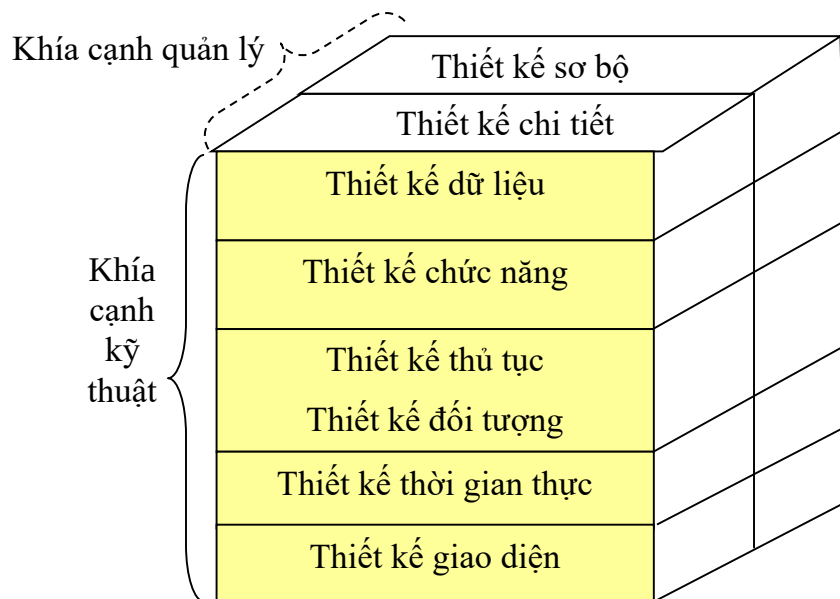
Một khi các yêu cầu về phần mềm đã được phân tích và đặc tả thì thiết kế phần mềm là một trong ba hoạt động kỹ thuật - *thiết kế, lập trình, kiểm thử* - những hoạt động cần để xây dựng và kiểm chứng phần mềm. Từng hoạt động này biến đổi thông tin theo cách cuối cùng tạo ra phần mềm máy tính hợp lệ.

Luồng thông tin trong giai đoạn kỹ thuật này của tiến trình kỹ nghệ phần mềm được minh hoạ trong sơ đồ sau:



Hình 3.2 Môi liên hệ thiết kế phần mềm và Công nghệ phần mềm

Thiết kế phần mềm là hoạt động được xác lập dựa trên hai mặt: quản lý và kỹ thuật, chúng đan xen với nhau. Môi quan hệ giữa hai khía cạnh kỹ thuật và quản lý được thể hiện qua sơ đồ:



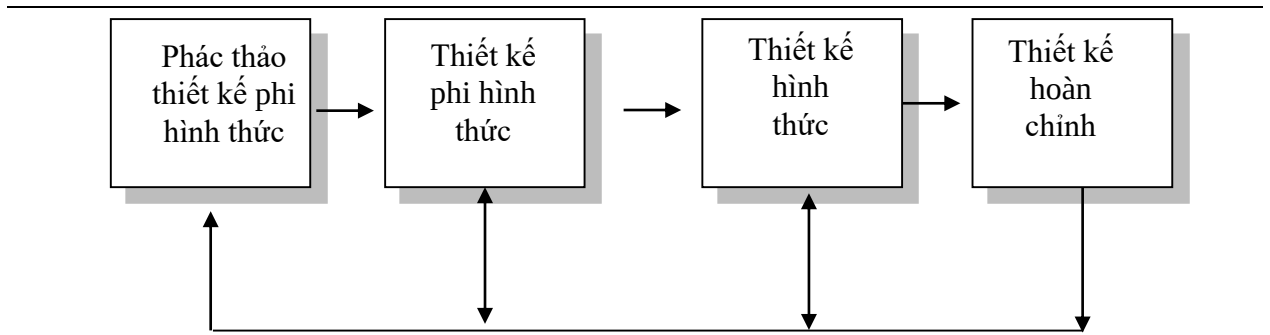
Hình 3.3 Mối quan hệ giữa khía cạnh quản lý và khía cạnh kỹ thuật

- Trong quan điểm quản lý: thiết kế phần mềm được tiến hành 2 bước:
 - + Thiết kế sơ bộ: Quan tâm đến việc chuyển hóa các yêu cầu thành các kiến trúc dữ liệu và các thành phần phần mềm.
 - + Thiết kế chi tiết: Tập trung vào việc làm mịn biểu diễn kiến trúc để dẫn đến cấu trúc dữ liệu chi tiết và biểu diễn thuật toán cho phần mềm.

Trong phạm vi thiết kế sơ bộ và chi tiết, có xuất hiện một số hoạt động thiết kế khác nhau. Bên cạnh việc thiết kế dữ liệu, kiến trúc và thủ tục, nhiều ứng dụng hiện đại có hoạt động thiết kế giao diện phân biệt. Thiết kế giao diện lập ra cách bố trí và cơ chế tương tác người-máy (HCI – human computer interface). Mối quan hệ giữa các khía cạnh kỹ thuật và quản lý của thiết kế được minh họa như hình 3.3.

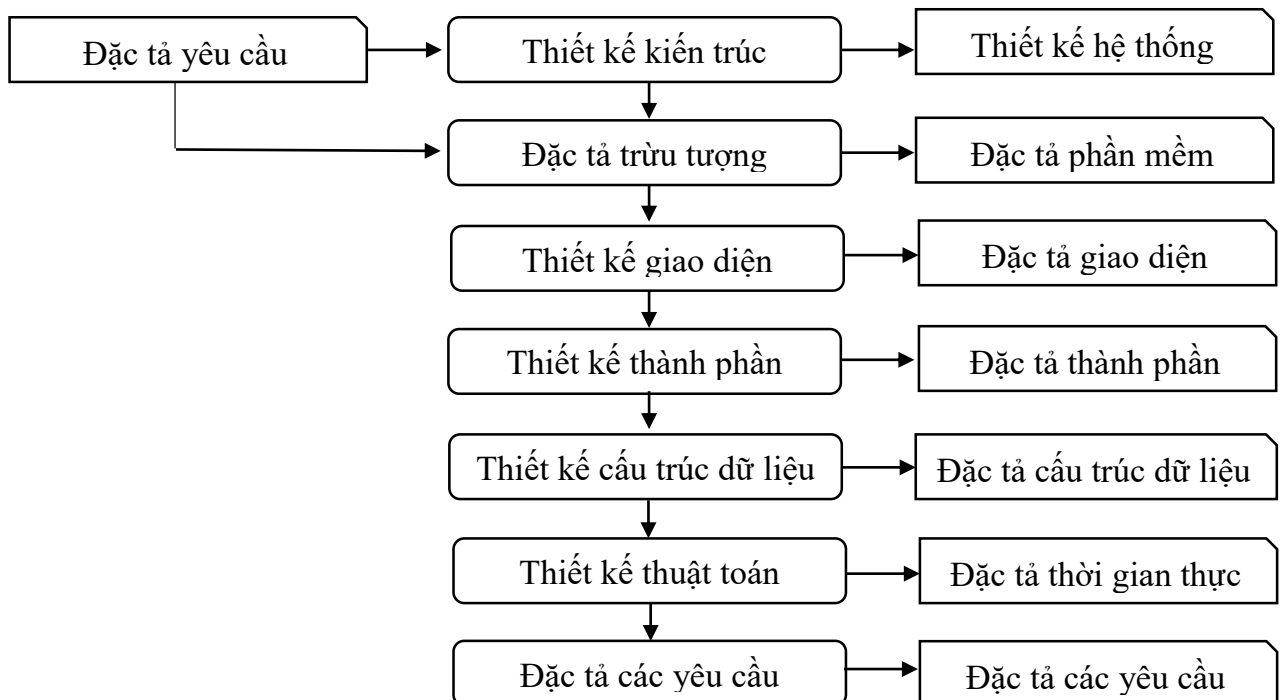
- Đối với khía cạnh kỹ thuật, xuất hiện một số hoạt động thiết kế như:
 - + Thiết kế dữ liệu
 - + Thiết kế chức năng
 - + Thiết kế thủ tục
 - + Thiết kế đối tượng
 - + Thiết kế thời gian thực
 - + Thiết kế giao diện

Quá trình thiết kế là quá trình tăng cường hình thức hoá trong sự tiến triển của thiết kế và phải luôn quay trở lại các thiết kế đúng đắn ít hình thức (từ “hình thức” ở đây có nghĩa là mang tính mô tả được hệ thống trong thực tế) có trước đây của quá trình đó. Nhà thiết kế phải bắt đầu với một bản phác thảo phi hình thức rồi sau đó tinh chế nó, thêm vào đó các thông tin để là cho thiết kế trở nên hình thức hơn. Quá trình thiết kế thể hiện như sau:



Hình 3.4 Quan hệ giữa thiết kế và đặc tả

Quan hệ giữa thiết kế và đặc tả là rất chặt chẽ. Mặc dù quá trình đưa ra một đặc tả yêu cầu được xem như là một phần tử cơ bản của hợp đồng là một hoạt động riêng biệt, song việc hình thức hoá đặc tả yêu cầu hẳn là một phần của quá trình thiết kế. Thực tế, người làm thiết kế sẽ lặp đi lặp lại giữa đặc tả và thiết kế.



Hình 3.5 Các hoạt động thiết kế và sản phẩm của thiết kế [3]

Quá trình thiết kế liên quan mật thiết đến việc mô tả hệ thống ở một số mức trừu tượng khác nhau. Khi một thiết kế được phân chia thành nhiều thành phần thì người ta thường phát hiện ra được những sai sót ở giai đoạn trước. Do đó phải quay trở lại để tinh chỉnh. Thông thường thì người ta bắt đầu giai đoạn sau ngay trước khi giai đoạn trước kết thúc đơn giản là để lui quá trình tinh chỉnh. Hình 3.5 nêu các hoạt động của quá trình thiết kế và các sản phẩm của nó. Các giai đoạn là khá tùy ý nhưng nó làm cho quá trình thiết kế trở nên nhìn thấy được và từ đó dễ quản lý được.

Thành quả của mỗi hoạt động thiết kế là một bản đặc tả. Đặc tả này có thể là một đặc tả trừu tượng, hình thức và được tạo ra để làm rõ các yêu cầu, nó cũng có thể là một

đặc tả về một thành phần nào đó của hệ thống phải được thực hiện như thế nào. Khi quá trình thiết kế tiến triển thì các yêu cầu ngày càng được bổ sung vào bản đặc tả đó. Các kết quả cuối cùng là các đặc tả về thuật toán và các cấu trúc dữ liệu được dùng làm cơ sở cho việc thực hiện hệ thống.

Thực tế, các hoạt động thiết kế diễn ra song song với các sản phẩm thiết kế khác nhau. Các sản phẩm này lại được triển khai ở các mức chi tiết khác nhau trong diễn biến của quá trình thiết kế.

Các hoạt động cốt yếu trong việc thiết kế một hệ thống phần mềm lớn

1. Thiết kế kiến trúc: Các hệ con tạo nên hệ tổng thể và các quan hệ của chúng là được phân hoạch rõ ràng và ghi thành tài liệu.

2. Đặc tả trừu tượng: Đối với mỗi hệ con, một đặc tả trừu tượng các dịch vụ mà nó cung cấp và các ràng buộc phải tuân theo cũng được hỗ trợ.

3. Thiết kế giao diện: Ở đây chúng ta không nên hiểu “giao diện” chỉ là những gì hiển thị trên màn hình, mà phải hiểu rằng đó có thể là tương tác giữa các thành phần trong hệ thống với nhau.

Giao diện với từng hệ con khác cũng được thiết kế và ghi thành tài liệu. Đặc tả giao diện không được mơ hồ và cho phép sử dụng hệ con đó mà không cần biết đến những gì được diễn ra bên trong của hệ con đó (theo kiểu “hộp đen”).

4. Thiết kế các thành phần: Các dịch vụ được cung cấp bởi hệ con được phân chia thành các thành phần hợp thành của hệ con đó.

5. Thiết kế cấu trúc dữ liệu: Các cấu trúc dữ liệu được dùng trong việc thực hiện hệ thống được thiết kế chi tiết và được đặc tả ở đây.

6. Thiết kế thuật toán: Các cách thức (phương pháp xử lý) được dùng để cung cấp cho các dịch vụ được thiết kế chi tiết và được đặc tả.

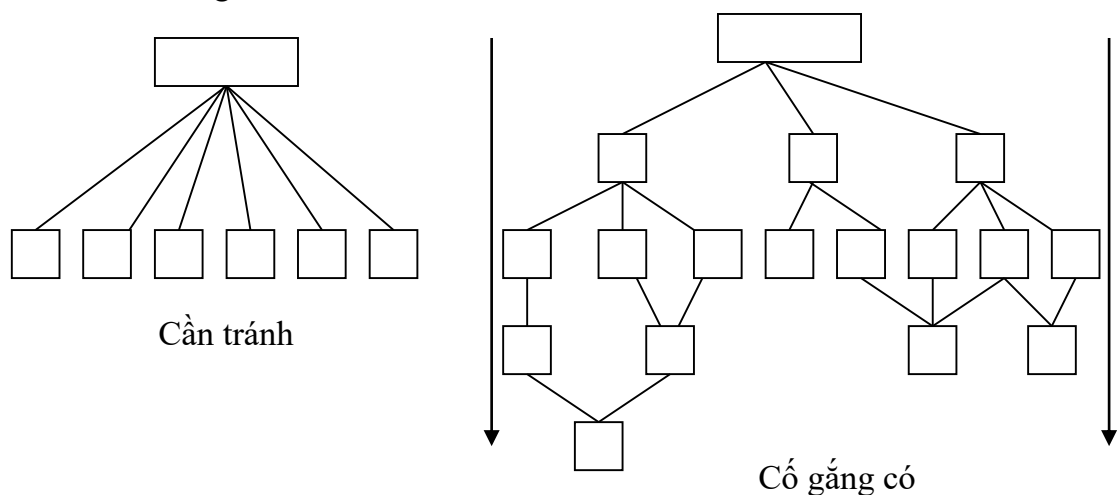
Quá trình này được lặp lại cho mỗi hệ con sao cho đến khi các thành phần hợp thành được xác định một cách rõ ràng và đều có thể chuyển đổi (ánh xạ) một cách trực tiếp vào các thành phần của ngôn ngữ lập trình, chẳng hạn như các gói (packets), các thủ tục (procedures) và các hàm (functions).

Phương pháp tiếp cận thường xuyên được khuyến khích sử dụng là phương pháp tiếp cận từ trên xuống (top down): Vấn đề lớn được phân chia một cách đệ quy thành các vấn đề con cho đến khi các vấn đề dễ giải quyết được xác định rõ ràng. Trong quá trình này người thiết kế không nhất thiết phải phân rã tất cả các thành phần trừu tượng (nghĩa là vấn đề này còn phức tạp mà cách giải quyết là chưa xác định rõ) khi mà bằng kinh nghiệm họ đã biết chắc chắn rằng có thể hoàn toàn xây dựng được. Do đó họ có thể tập trung sức lực vào các thành phần đáng xét nhất.

Chú ý rằng khi mà phương pháp hướng đối tượng được chấp nhận thì phương pháp từ trên xuống sẽ ít hiệu quả. Khi đó người thiết kế sử dụng các đối tượng sẵn có để làm khung thiết kế.

3.4 Một số yêu cầu thiết kế

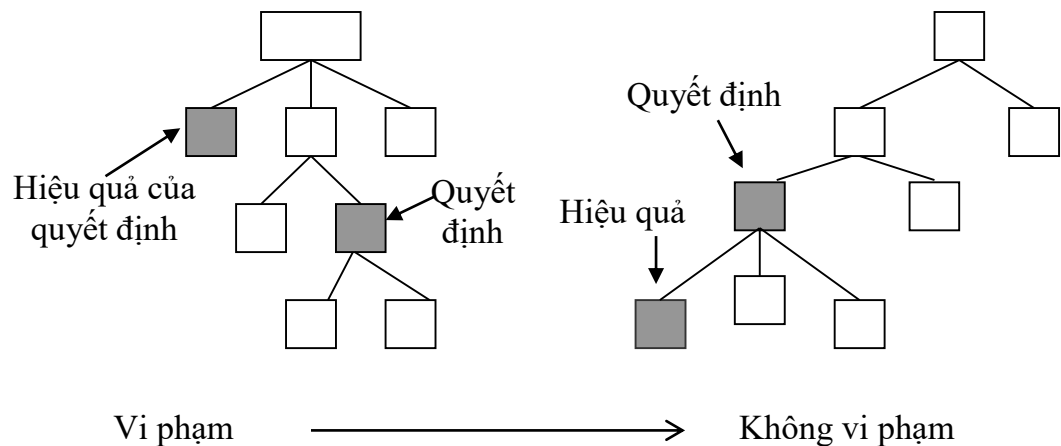
- Linh hoạt đối với những yêu cầu thay đổi không định trước.
- Dễ thử nghiệm.
- Tính sáng sửa, dễ đọc
- Kích thước module nhỏ
- Tính độc lập module (tính mở/đóng giữa các module), sử dụng yếu tố "hộp đen"
- Phải quan hệ chặt chẽ giữa thiết kế và yêu cầu.
- Mỗi module hoàn toàn độc lập, nó thực hiện một chức năng duy nhất và thực hiện trọn vẹn chức năng đó.
- Mọi thứ trong module ràng buộc với nhau qua việc xử lý nối tiếp nhau trên cùng một dòng dữ liệu.
- Mọi thứ trong module được điều khiển bởi cùng một dữ liệu vào, hay cùng một phức hợp thiết bị, hay cùng thực hiện từng phần của cùng một kết xuất.
- Module có thể hiểu được hoàn toàn dựa vào những tham biến truyền cho nó và nhận từ nó.
- Khi thiết kế cố gắng giảm thiểu cấu trúc bằng việc tản ra nhiều và cố gắng co cụm khi chiều sâu tăng thêm.



Hình 3.6 Thiết kế nên theo chiều sâu

- Giữa phạm vi hiệu quả của một module bên trong phạm vi kiểm soát của module đó.
- + Phạm vi hiệu quả của module m được định nghĩa là tất cả các module khác bị ảnh hưởng bởi một quyết định thực hiện trong module m.

+ Phạm vi kiểm soát của module m là tất cả các module và mọi module thuộc cấp và thuộc cấp cuối cùng đối với module m.



Hình 3.7 Hiệu quả phạm vi module

- Ước lượng giao diện module để giảm độ phức tạp, dư thừa và tăng tính nhất quán.
- Xác định các module có chức năng dự kiến được.
- Cố gắng giữ các module một đầu vào và một đầu ra, tránh các "mối nối bị lỗi".

3.5 Kỹ thuật thiết kế

Do các hệ phần mềm lớn là phức tạp nên người ta thường dùng các phương pháp tiếp cận khác nhau trong việc thiết kế các phần khác nhau của một hệ thống. Chẳng có một chiến lược tốt nhất nào cho các dự án. Hai chiến lược thiết kế hiện đang được dùng rộng rãi trong việc phát triển phần mềm đó là thiết kế hướng chức năng và thiết kế hướng đối tượng. Mỗi chiến lược thiết kế đều có ưu và nhược điểm riêng phụ thuộc vào ứng dụng phát triển và nhóm phát triển phần mềm.

Cách tiếp cận hướng chức năng hay hướng đối tượng là bổ sung và hỗ trợ cho nhau chứ không phải là đối kháng nhau. Kỹ sư phần mềm sẽ chọn cách tiếp cận thích hợp nhất cho từng giai đoạn thiết kế. Ngoài ra, chúng ta cùng tìm hiểu thêm các kỹ thuật thiết kế khác.

3.5.1. Thiết kế hướng chức năng

3.5.1.1 Cách tiếp cận hướng chức năng

Thiết kế hướng chức năng là một cách tiếp cận thiết kế phần mềm trong đó bản thiết kế được phân giải thành một bộ các đơn thể tác động lẫn nhau, mà mỗi đơn thể có một chức năng được xác định rõ ràng. Các chức năng có các trạng thái cục bộ nhưng chúng chia sẻ với nhau trạng thái hệ thống, trạng thái này là tập trung và mọi chức năng đều có thể truy cập được. Nhiều tổ chức đã phát triển các chuẩn và các phương pháp dựa trên sự phân giải chức năng. Nhiều phương pháp thiết kế kết hợp với công cụ CASE đều là hướng

chức năng. Vô khối các hệ thống đã được phát triển bằng cách sử dụng phương pháp tiếp cận hướng chức năng. Các hệ thống đó sẽ được bảo trì cho một tương lai xa xôi. Bởi vậy thiết kế hướng chức năng vẫn sẽ còn được tiếp tục sử dụng rộng rãi.

Trong thiết kế hướng chức năng, người ta dùng các biểu đồ luồng dữ liệu (mô tả việc xử lý dữ liệu), các lược đồ cấu trúc (nó chỉ ra cấu trúc của phần mềm), và các mô tả thiết kế chi tiết. Thiết kế hướng chức năng gắn với các chi tiết của một thuật toán của chức năng đó nhưng các thông tin trạng thái hệ thống là không bị che dấu. Việc thay đổi một chức năng và cách nó sử dụng trạng thái của hệ thống có thể gây ra những tương tác bất ngờ đối với các chức năng khác. Cách tiếp cận chức năng để thiết kế là tốt nhất khi mà khối lượng thông tin trạng thái hệ thống được làm nhỏ nhất và thông tin dùng chung nhau là rõ ràng.

3.5.1.2 Biểu đồ luồng dữ liệu

Biểu đồ luồng dữ liệu chỉ ra cách thức biến đổi dữ liệu vào thành dữ liệu ra thông qua một dãy các phép biến đổi. Bước đầu của thiết kế hướng chức năng là phát triển một biểu đồ luồng dữ liệu hệ thống. Biểu đồ này không nhất thiết bao gồm các thông tin điều khiển nhưng nên lập tư liệu các phép biến đổi dữ liệu. Biểu đồ luồng dữ liệu là một phần hợp nhất của một số các phương pháp thiết kế và các công cụ CASE thường trợ giúp cho việc tạo ra biểu đồ luồng dữ liệu.

3.5.1.3 Lược đồ cấu trúc

Lược đồ cấu trúc chỉ ra cấu trúc các thành phần theo thứ bậc của hệ thống. Nó chỉ ra rằng các phần tử của một biểu đồ luồng dữ liệu có thể được thực hiện như thế nào với tư cách là một thứ bậc của các đơn vị chương trình. Lược đồ cấu trúc có thể được dùng như là một mô tả chương trình nhìn thấy được với các thông tin xác định các sự lựa chọn và các vòng lặp. Lược đồ cấu trúc được dùng để trình bày một tổ chức tĩnh của thiết kế.

3.5.1.4 Các từ điển dữ liệu

Từ điển dữ liệu vừa có ích cho việc bảo trì hệ thống vừa có ích trong quá trình thiết kế. Với mỗi khái niệm thiết kế, cần có một từ khóa mô tả ứng với từ khóa (entry) của từ điển dữ liệu cung cấp thông tin về khái niệm đó (kiểu, chức năng của dữ liệu...). Đôi khi người ta gọi cái này là một mô tả ngắn của chức năng thành phần.

Các từ điển dữ liệu dùng để nối các mô tả thiết kế kiểu biểu đồ và các mô tả thiết kế kiểu văn bản. Một vài bộ công cụ CASE cung cấp một phép nối tự động biểu đồ luồng dữ liệu và từ điển dữ liệu.

3.5.2. Thiết kế hướng đối tượng

Hệ thống được nhìn nhận như một bộ các đối tượng (chứ không phải là bộ các chức năng). Hệ thống được phân tán, mỗi đối tượng có những thông tin trạng thái riêng của nó. Đối tượng là một bộ các thuộc tính xác định trạng thái của đối tượng đó và các phép toán

của nó. Nó được thừa kế từ một vài lớp đối tượng lớp cao hơn, sao cho dễ định nghĩa nó chỉ cần nêu đủ các khác nhau giữa nó và các lớp cao hơn nó.

3.5.2.1 Cách tiếp cận hướng đối tượng

Thiết kế hướng đối tượng dựa trên chiến lược che dấu thông tin cấu trúc vào bên trong các thành phần. Cái đó ngầm hiểu rằng việc kết hợp điều khiển logic và cấu trúc dữ liệu được thực hiện trong thiết kế càng chậm càng tốt. Liên lạc thông qua các thông tin trạng thái dùng chung (các biến tổng thể) là ít nhất, nhờ vậy khả năng hiểu được nâng lên. Thiết kế là tương đối dễ thay đổi vì sự thay đổi cấu trúc một thành phần có thể không cần quan tâm tới các hiệu ứng phụ trên các thành phần khác.

Việc che dấu thông tin trong thiết kế hướng đối tượng là dựa trên sự nhìn hệ phân mềm như là một bộ các đối tượng tương tác với nhau chứ không phải là bộ các chức năng như cách tiếp cận chức năng. Các đối tượng có một trạng thái riêng được che dấu và các phép toán trên trạng thái đó. Thiết kế biểu thị các dịch vụ được yêu cầu cùng với những hỗ trợ mà các đối tượng có tương tác với nó cung cấp.

3.5.2.2 Ba đặc trưng của thiết kế hướng đối tượng

Thiết kế hướng đối tượng bao gồm các đặc trưng chính sau:

- *Không có vùng dữ liệu dùng chung.* Các đối tượng liên lạc với nhau bằng cách trao đổi thông báo.

- *Các đối tượng là các thực thể độc lập,* dễ thay đổi vì rằng tất cả các trạng thái và các thông tin biểu diễn chỉ ảnh hưởng trong phạm vi chính đối tượng đó thôi. Các thay đổi về biểu diễn thông tin có thể được thực hiện không cần sự tham khảo tới các đối tượng khác.

- *Các đối tượng có thể phân tán và có thể hoạt động tuần tự hoặc song song.* Đây là một trong những lý do khiến cho thiết kế hướng đối tượng được sử dụng rộng rãi trong các hệ thống nhúng.

3.5.2.3 Cơ sở của thiết kế hướng đối tượng

Cơ sở của thiết kế hướng đối tượng là các lớp. Lớp là một trừu tượng mô tả cho một nhóm sự vật. Đối tượng của một lớp là một thực thể (cụ thể hóa) của lớp đó.

Thiết kế của một lớp bao gồm:

- Cấu trúc dữ liệu (thuộc tính)
- Hàm, thủ tục (chức năng)
- Giao diện (cung cấp khả năng trao đổi dữ liệu đối với các lớp khác, về bản chất là các chức năng của đối tượng)

Việc cài đặt các giao diện là một yếu tố quan trọng để đảm bảo che dấu cấu trúc dữ liệu. Tức là thiết kế nội tại của đối tượng độc lập với giao diện do đó chúng ta có thể sửa đổi thiết kế mà không sợ ảnh hưởng tới các đối tượng khác.

Các đối tượng trao đổi với nhau bằng cách truyền các thông báo. Tức là một đối

tương yêu cầu một đối tượng khác thực hiện một chức năng nào đó. Thông báo bao gồm: tên đối tượng, tên phương thức, và các tham số.

Vòng đời của một đối tượng khi hệ thống hoạt động như sau:

- Khởi tạo: hệ thống tạo ra đối tượng bằng cách xác lập vùng dữ liệu đồng thời tự động thực hiện các chức năng liên quan đến khởi tạo đối tượng.
- Hoạt động: đối tượng nhận các thông báo và thực hiện các chức năng được yêu cầu.
- Hàm hủy: hệ thống giải phóng vùng nhớ đã được cấp phát sau khi thực hiện tự động các thao tác cần thiết để hủy đối tượng.

Nhờ có chức năng khởi tạo và hàm hủy được gọi tự động chúng ta có thể tự động hóa được một số công việc và tránh được nhiều sai sót trong lập trình như quên khởi tạo dữ liệu, quên cấp phát hay quên giải phóng vùng nhớ động.

- Sự kế thừa: kế thừa là một khái niệm quan trọng trong thiết kế hướng đối tượng. Một lớp có thể được định nghĩa dựa trên sự kế thừa một hoặc nhiều lớp đã được định nghĩa. Kế thừa ở đây bao gồm:

- + Kế thừa cấu trúc dữ liệu
- + Kế thừa chức năng

Khả năng kế thừa giúp cho rút gọn được chương trình và nâng cao tính tái sử dụng. Một chiến lược chung là trước tiên tạo ra các lớp trừu tượng (để có thể dùng chung) và đối với các bài toán cụ thể tạo ra các lớp kế thừa bằng cách thêm các thông tin đặc thù.

3.5.2.4 Các bước thiết kế

Thiết kế hướng đối tượng bao gồm các bước chính sau:

Bước 1: Xác định lớp đối tượng.

Bước 2: Xác định thuộc tính cho lớp: các biến của lớp

Bước 3: Xác định hành vi (chức năng): các hàm

Bước 4: Xác định tương tác giữa các lớp đối tượng: giao diện (thông báo).

Bước 5: Áp dụng tính kế thừa: xây dựng các lớp trừu tượng có các thuộc tính chung, đây là một khâu đặc trưng của thiết kế hướng đối tượng.

Ví dụ, giả sử cần xây dựng các lớp hình tròn, elip và đa giác. Có thể thấy elip và hình tròn có một số các thuộc tính chung như tọa độ tâm, chúng ta có thể xây dựng lớp hình nón chứa các thuộc tính chung này. Giữa hình nón và đa giác lại có thể tìm ra các thuộc tính chung như màu nền, màu biên..., và có thể xây dựng lớp trừu tượng hình học chứa các thuộc tính này.

Phương pháp xác định đối tượng: xác định đối tượng là một trong những công đoạn thiết kế quan trọng, phụ thuộc nhiều vào kinh nghiệm và bài toán cụ thể. Có một số phương pháp được đề xuất, một trong các phương pháp này là phân tích từ vựng của “câu yêu cầu”. Cụ thể là danh từ trong câu yêu cầu sẽ được coi là đối tượng và động từ sẽ được

coi là chức năng. Ví dụ: Với yêu cầu Phần mềm Email cung cấp cho user khả năng gửi thư, đọc thư và soạn thảo thư điện tử, chúng ta có thể sơ bộ tạo ra các đối tượng phần mềm, user, email và các chức năng gửi, nhận, soạn thảo.

3.5.2.5 Ưu nhược điểm

Thiết kế hướng đối tượng có các ưu điểm chính sau:

- Dễ bảo trì vì các đối tượng là độc lập. Các đối tượng có thể hiểu và cải biên như là một thực thể độc lập. Thay đổi trong thực hiện một đối tượng hoặc thêm các dịch vụ sẽ không làm ảnh hưởng tới các đối tượng hệ thống khác.
- Các đối tượng là các thành phần dùng lại được thích hợp do tính độc lập của chúng và khả năng kế thừa cao.
- Có một vài lớp hệ thống thể hiện phản ánh quan hệ rõ ràng giữa các thực thể có thực (chẳng hạn như các thành phần phần cứng) với các đối tượng điều khiển nó trong hệ thống. Điều này đạt được tính dễ hiểu của thiết kế.

3.5.3. Thiết kế có cấu trúc

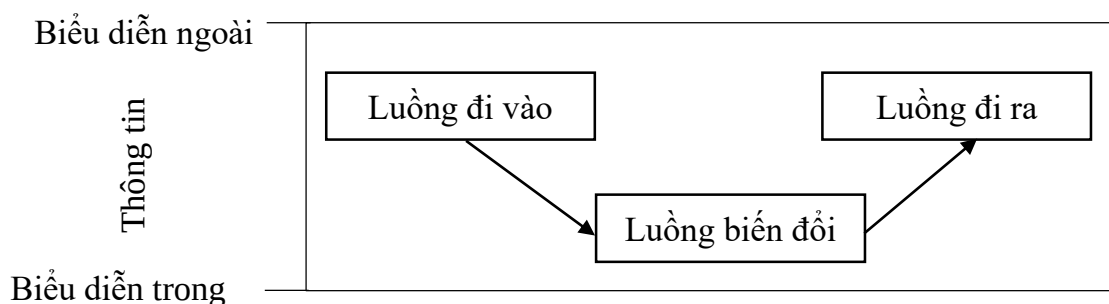
Thiết kế có cấu trúc cho phép biến đổi thuận tiện từ các biểu diễn thông tin như biểu đồ luồng dữ liệu có trong các bản đặc tả yêu cầu phần mềm sang mô tả về cấu trúc chương trình. Việc biến đổi từ luồng thông tin sang cấu trúc thông tin được thực hiện như một phần của tiến trình 5 bước:

- Thiết lập kiểu luồng thông tin
- Chỉ ra biên giới luồng
- Ánh xạ DFD vào cấu trúc chương trình
- Xác định các cấp bậc điều khiển theo cách tạo nhân tố.
- Cấu trúc kết quả được làm mịn bằng cách dùng các phương tiện thiết kế và trực cảm.

Kiểu luồng thông tin là bộ dẫn lối cho cách tiếp cận ánh xạ ở bước 3. Tiếp theo, chúng ta sẽ xem xét hai kiểu luồng.

Luồng biến đổi

Những dữ liệu bên ngoài đều được chuyển thành dạng bên trong để xử lý.

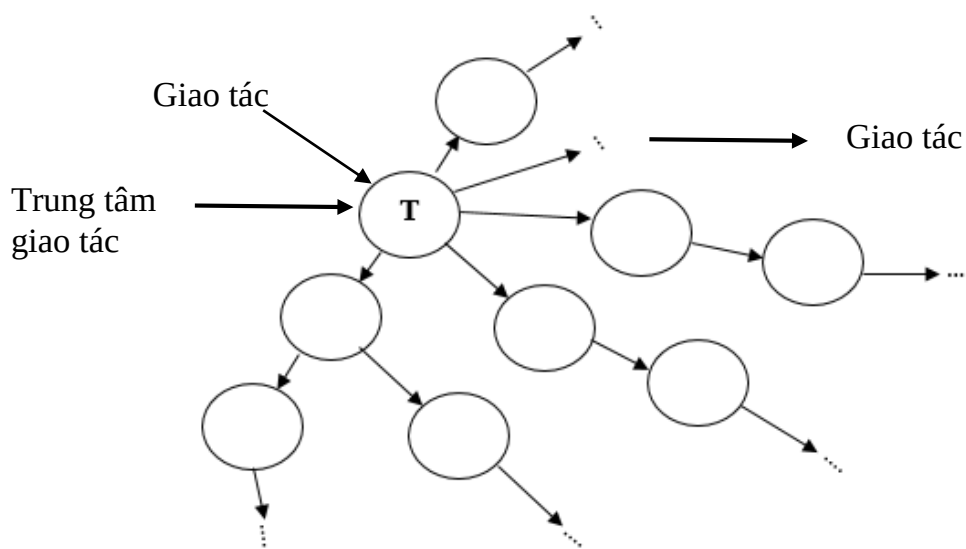


Hình 3.8 Luồng thông tin

Thông tin đi vào hệ thống theo con đường biến đổi dữ liệu ngoài thành dạng bên trong và được coi như luồng đi vào. Trong hạt nhân của phần mềm, việc chuyển đổi xuất hiện. Dữ liệu tới truyền qua một trung tâm biến đổi và bắt đầu chuyển dọc theo những con đường dẫn tới đầu ra của phần mềm. Dữ liệu chuyển dọc theo những con đường này gọi là luồng đi ra. Toàn bộ luồng dữ liệu xuất hiện một cách tuần tự đi theo một hay một vài đường trực tiếp. Khi một đoạn của biểu đồ luồng dữ liệu có những đặc trưng này thì đó là luồng biến đổi.

Luồng giao tác

Luồng thông tin được đặc trưng bởi một khoản mục dữ liệu riêng biệt, gọi là giao tác, gây ra luồng dữ liệu khác đi theo một trong nhiều đường dẫn. Khi một DFD có dạng như trong hình 3.9, luồng giao tác xuất hiện.



Hình 3.9 Luồng giao tác

Luồng giao tác được đặc trưng bởi dữ liệu chuyển theo đường đi vào (còn gọi là đường nhận) đổi thông tin thế giới ngoài thành một giao tác. Giao tác được tính toán và dựa trên giá trị của nó, luồng thông tin mà từ đó nhiều đường hành động đi ra được gọi là trung tâm giao tác [2].

3.5.4. Thiết kế thời gian thực

Thiết kế các hệ thống tính toán thời gian thực là một nhiệm vụ phức tạp và có tính thách thức nhất mà kỹ sư phần mềm cần tiến hành. Bởi chính bản chất của nó, phần mềm cho hệ thống thời gian thực yêu cầu các kỹ thuật phân tích, thiết kế và kiểm thử mà chưa được biết đến trong các lĩnh vực khác.

Phần mềm thời gian thực liên hệ chặt chẽ với thế giới bên ngoài, đáp ứng vấn đề (thế giới thực) trong khuôn khổ thời gian do lĩnh vực vấn đề chi phối. Vì phần mềm thời gian thực phải vận hành dưới những ràng buộc hiệu năng chặt chẽ nên việc thiết kế phần

mềm thường bị chi phối bởi phần cứng cũng như kiến trúc phần mềm, các đặc trưng hệ điều hành cũng như các yêu cầu áp dụng, tính chất thường của ngôn ngữ lập trình cũng như các vấn đề thiết kế [2].

3.5.4.1 Các xem xét hệ thống

Giống như bất cứ hệ thống máy tính nào, hệ thống thời gian thực phải tích hợp các yếu tố phần cứng, phần mềm, con người và cơ sở dữ liệu để thu được một tập các yêu cầu chức năng và hoàn thiện. Kỹ sư hệ thống phải cấp phát chức năng và sự hoàn thiện trong các phần tử hệ thống. Vấn đề đối với hệ thống thời gian thực là phải cấp phát đúng. Sự hoàn thiện thời gian thực cũng quan trọng như chức năng, nhưng các quyết định cấp phát thường khó đảm bảo. Liệu một thuật toán xử lý có đáp ứng được các ràng buộc thời gian khắt khe, hay chúng ta phải xây dựng phần cứng đặc biệt để làm việc đó.

3.5.4.2 Hệ thống thời gian thực

Hệ thống thời gian thực sinh ra một hành động nào đó để đáp ứng các sự kiện bên ngoài. Để hoàn thành chức năng này, chúng thực hiện việc thu thập và kiểm soát dữ liệu tốc độ cao trong những ràng buộc thời gian và độ tin cậy nghiêm ngặt. Vì những ràng buộc này là nghiêm ngặt nên các hệ thống thời gian thực thường chuyên dụng cho một ứng dụng.

Nhiều năm, khách hàng chính của các hệ thống thời gian thực là giới quân sự. Tuy nhiên, ngày nay giá phần cứng giảm đáng kể đã làm cho các hệ thống này được ứng dụng ngày càng nhiều hơn, đa dạng hơn trong nhiều lĩnh vực như trong điều khiển tiến trình, tự động hóa công nghiệp, nghiên cứu y học và khoa học, đồ họa máy tính, truyền thông mạng cục bộ và mạng diện rộng, hệ thống hàng không, chứng khoán, ngân hàng, kiểm thử có máy tính trợ giúp và rất nhiều thiết bị công nghiệp.

3.5.4.3 Cơ sở dữ liệu thời gian thực

Giống như nhiều hệ thống xử lý dữ liệu, hệ thống thời gian thực thường đi đôi với chức năng quản trị cơ sở dữ liệu. Tuy nhiên, cơ sở dữ liệu phân tán dường như là cách tiếp cận được ưa thích nhất trong hệ thống thời gian thực vì đa nhiệm thông dụng và dữ liệu thường được xử lý song song. Nếu cơ sở dữ liệu phân tán thì từng nhiệm vụ có thể thâm nhập vào dữ liệu của chúng nhanh hơn và tin cậy hơn, ít tắc nghẽn hơn cơ sở dữ liệu tập trung. Việc dùng cơ sở dữ liệu phân tán cho các ứng dụng thời gian thực phân chia lưu thông vào/ra và làm ngắn đi hàng nhiệm vụ đang chờ để thâm nhập vào cơ sở dữ liệu. Một sai sót của một cơ sở dữ liệu sẽ hiếm khi gây ra hỏng hóc toàn bộ hệ thống nếu có sự dư thừa được xây dựng sẵn.

Tính hiệu quả hiệu năng đạt được thông qua việc dùng cơ sở dữ liệu phân tán phải tương ứng với vấn đề tiềm năng liên quan đến phân hoạch và tái tạo. Mặc dù việc dư thừa dữ liệu làm tăng thời gian đáp ứng do cung cấp nhiều nguồn thông tin, nhưng các yêu cầu tái tạo cho các tệp phân tán cũng tạo ra các vấn đề tổng chi phí và các nguồn vì tất cả các bản sao tệp đều phải được cập nhật. Bên cạnh đó, việc dùng các cơ sở dữ liệu phân tán sẽ

đưa đến vấn đề điều khiển tương tranh. Điều khiển tương tranh bao gồm việc đồng bộ hóa các cơ sở dữ liệu sao cho mọi bản sao đều có thông tin thống nhất, đúng đắn, tự do cho việc truy cập.

Cách tiếp cận quy ước dựa trên yếu tố đã được biết như khóa và dấu thời gian. Theo những khoảng đều đặn, các nhiệm vụ sau đây được khởi đầu:

- Cơ sở dữ liệu được hãm lại để việc điều khiển tương tranh được đảm bảo không vào, ra nào được thực hiện.
- Việc cập nhật xuất hiện như yêu cầu.
- Cơ sở dữ liệu được mở lại
- Các tệp được làm hợp lệ để đảm bảo rằng tất cả mọi cập nhật đều đã được thực hiện đúng.

- Việc cập nhật phải được hoàn tất.

Mọi việc chính đều được điều phối bởi đồng hồ thời gian. Vấn đề trễ bao gồm trong những thủ tục này, cũng như vấn đề tránh cập nhật không nhất quán và bế tắc là giảm bớt việc dùng rộng rãi cơ sở dữ liệu phân tán.

Tuy nhiên, một số kỹ thuật đã được phát triển để tăng tốc việc cập nhật và để giải quyết vấn đề tương tranh. Một trong những kỹ thuật này đó là loại trừ nơi ghi, duy trì tính nhất quán của các tệp tái tạo bằng cách cho phép chỉ riêng một nhiệm vụ ghi là được cập nhật tệp. Do đó, nó khử bớt tổng chi phí cao của thủ tục khóa hay dấu thời gian.

3.5.5. Thiết kế dữ liệu

3.5.5.1 Giới thiệu

Mục tiêu chính của thiết kế dữ liệu là mô tả cách thức tổ chức lưu trữ các dữ liệu của phần mềm. Có ba dạng lưu trữ chính mà người thiết kế cần phải cân nhắc và lựa chọn.

- Lưu trữ dưới dạng tập tin
- Lưu trữ dưới dạng cơ sở dữ liệu
- Lưu trữ dạng điện toán đám mây

Lưu trữ dưới dạng tập tin thường chỉ thích hợp với một số phần mềm đặc thù (cờ tướng, trò chơi, v.v.) đặc điểm chung của các phần mềm này là chú trọng rất nhiều vào xử lý, hình thức giao diện và không chú trọng nhiều đến việc lưu trữ lại các thông tin được tiếp nhận trong quá trình sử dụng phần mềm (thông thường các thông tin này được tiếp nhận và xử lý ngay).

Cách tiếp cận dùng cơ sở dữ liệu rất thông dụng và tài liệu này sẽ giới hạn trình bày chi tiết các phương pháp kỹ thuật liên quan đến việc tổ chức lưu trữ dữ liệu dùng cơ sở dữ liệu quan hệ.

Lưu trữ dạng điện toán đám mây là dịch vụ được các công ty tạo ra nhằm giúp người dùng có thể lưu trữ, quản lý, chia sẻ các dữ liệu như hình ảnh, video, tập tin, ...

3.5.5.2 Kết quả thiết kế

Cách thức tổ chức lưu trữ dữ liệu của phần mềm được mô tả thông qua 2 loại thông tin sau:

Thông tin tổng quát: Cung cấp góc nhìn tổng quan về các thành phần lưu trữ.
Danh sách các bảng dữ liệu: Việc lưu trữ cần sử dụng bao nhiêu bảng dữ liệu và đó là các bảng nào?

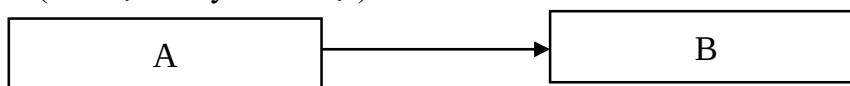
Danh sách các liên kết: Các bảng dữ liệu có quan hệ, có mối liên kết giữa chúng ra sao?

Thông tin chi tiết: Danh sách các thuộc tính của từng thành phần: Các thông tin cần lưu trữ của thành phần?

Danh sách các miền giá trị toàn vẹn: Các qui định về tính hợp lệ của các thông tin được lưu trữ. Có nhiều phương pháp, nhiều đề nghị khác nhau về việc mô tả các thông tin trên. Tài liệu này chọn sơ đồ logic để biểu diễn các thông tin tổng quát và bảng thuộc tính. Miền giá trị để mô tả chi tiết các thành phần trong sơ đồ logic. Sơ đồ logic là sơ đồ cho phép thể hiện hệ thống các bảng dữ liệu cùng với quan hệ mối nối liên kết giữa chúng. Các ký hiệu được dùng trong sơ đồ rất đơn giản như sau:

Bảng: hình chữ nhật

Liên kết: (xác định duy nhất một): Mũi tên



Mũi tên hình trên có ngữ nghĩa: 1 phần tử A sẽ xác định duy nhất 1 phần tử B, ngược lại 1 phần tử B có thể tương ứng với nhiều phần tử A.

Ví dụ: Với phần mềm quản lý thư viện có sơ đồ logic sau:



Theo sơ đồ này việc lưu trữ các dữ liệu của phần mềm quản lý thư viện được tổ chức 3 bảng (DOCGIA, MUONSACH, SACH) vùng với 2 liên kết giữa chúng. Tất nhiên sơ đồ trên chỉ là một trong các cách thức tổ chức lưu trữ dữ liệu còn nhiều cách khác có thể có. Chi tiết các cách này sẽ được trình bày trong phương pháp thiết kế cơ sở dữ liệu. Bảng thuộc tính cho phép mô tả chi tiết thành phần trong sơ đồ logic theo dạng như sau:

Stt	Thuộc tính	Kiểu	Miền giá trị	Ý nghĩa	Ghi chú
1					
2					
...					

Bảng thuộc tính cho phép mô tả chi tiết thành phần cần lưu trữ và sẽ được dùng trong báo cáo về thiết kế dữ liệu của phần mềm. Tuy nhiên cách mô tả trên khá dài dòng, trong tài liệu này sẽ sử dụng một dạng trình bày cô đọng hơn theo dạng lược đồ quan hệ. Với dạng trình bày này gồm tên bảng và thuộc tính đi kèm, các thuộc tính khóa được gạch chân.

Ví dụ:

DOCGIA(MDG, Hoten, Loaidg, Ngsinh, Nglapthe, Diachi)

SACH(MSACH, Tensach, Theloai, NgNhap, Tacgia, Nhaxb, Namxb)

MUONSACH(MDG, MSACH, NgMuon, Ngtra)

3.5.5.3 Quá trình thiết kế

Có 2 cách tiếp cận chính để thiết kế dữ liệu:

Phương pháp trực tiếp:

Từ các yêu cầu đã xác định, tạo lập trực tiếp sơ đồ logic cùng với bảng thuộc tính, bảng miền giá trị. Các tiếp cận này rất khó thực hiện đối với sơ đồ logic phức tạp.

Phương pháp gián tiếp:

Từ các yêu cầu đã xác định, tạo lập mô hình quan niệm dữ liệu, và sau đó đưa vào mô hình này sẽ tạo lập sơ đồ logic, bảng thuộc tính, bảng miền giá trị. Các tiếp cận này dễ thực hiện hơn vì mô hình quan niệm dữ liệu thường đơn giản (chứa các thành phần dữ liệu bản chất nhất của phần mềm). Khái niệm chi tiết về mô hình quan niệm dữ liệu cùng với các bước cụ thể sẽ được trình bày chi tiết trong phần sau.

Tương ứng với 3 yêu cầu của phần mềm, quá trình thiết kế dữ liệu bao gồm 3 bước lớn:

- Thiết kế với tính đúng đắn
- Thiết kế với yêu cầu chất lượng
- Thiết kế với yêu cầu hệ thống

Thiết kế với tính đúng đắn

- Đảm bảo đầy đủ và chính xác về mặt ngữ nghĩa các thông tin liên quan đến các công việc trong yêu cầu.

- Các thông tin phục vụ cho các yêu cầu chất lượng sẽ không được xét đến trong bước thiết kế này.

Thiết kế với yêu cầu chất lượng

- Vẫn đảm bảo tính đúng đắn nhưng thỏa mãn thêm các yêu cầu chất lượng khác (tiến hóa, tốc độ nhanh, lưu trữ tối ưu).

- Cần chú ý bảo đảm tính đúng đắn khi cải tiến sơ đồ logic.

Thiết kế với yêu cầu hệ thống

Vẫn đảm bảo tính đúng đắn và các yêu cầu chất lượng khác nhưng thỏa mãn thêm các yêu cầu hệ thống (phân quyền, cấu hình phần cứng, môi trường phần mềm, v.v)

3.5.6. Thiết kế thủ tục

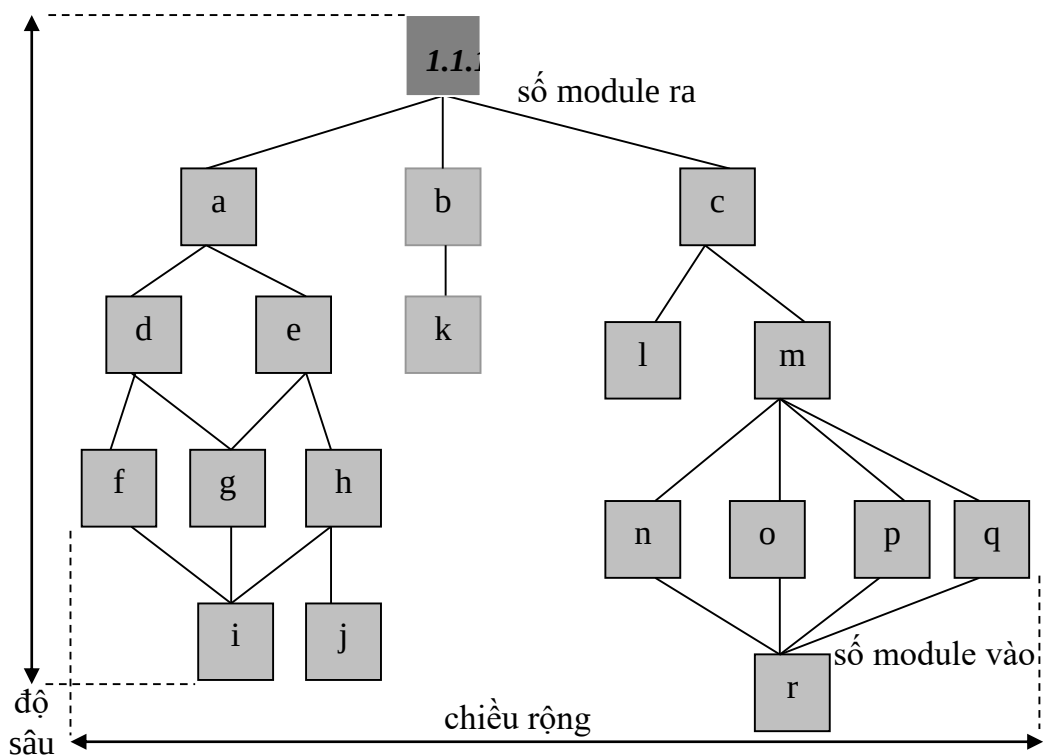
Sự trừu tượng hóa và che dấu thông tin được dùng để thiết kế thủ tục và biểu diễn bằng các module. Bên trong cấu trúc chương trình, các module có thể được phân loại:

- Module tuần tự: được tham khảo và thực hiện không bị ngắt
- Module tăng trưởng: có thể bị ngắt trước khi hoàn tất và sau đó chạy lại tại thời điểm ngắt.

- Module song song: thực hiện đồng thời với module khác.

Với mỗi module cần có:

- Tính độc lập chức năng,
- Tính cố kết,
- Tính gắn nối, ...



Hình 3.10 Cấu trúc chương trình và các chỉ số

3.5.7. Thiết kế giao diện người sử dụng

Thiết kế hệ thống máy tính bao gồm một phổ rất rộng các công việc từ thiết kế phần cứng cho đến thiết kế giao diện người sử dụng. Giao diện của hệ thống thường là tiêu chuẩn so sánh để phán xét về hệ thống. Giao diện được thiết kế kém sẽ gây ra những nhầm lẫn cho người sử dụng, khiến cho họ không sử dụng được các chức năng cần thiết và trong trường hợp xấu có thể thực hiện các thao tác nguy hiểm như phá hủy thông tin cần thiết.

Tầm quan trọng của giao diện còn được xem xét trên hai yếu tố:

- Khía cạnh nghiệp vụ: người dùng thông qua giao diện để tương tác với hệ thống, đây là khâu nghiệp vụ thủ công duy nhất do đó nếu được thiết kế tốt sẽ nâng cao tốc độ xử lý công việc và dẫn tới hiệu quả kinh tế cao.
- Khía cạnh thương mại: đối với các sản phẩm bán hàng loạt, giao diện được thiết kế tốt (dễ sử dụng, đẹp) sẽ gây ấn tượng với khách hàng và là yếu tố chính khi khách hàng chọn mua sản phẩm.

Ngoài các yếu tố hiệu quả công việc, đẹp, dễ học dễ sử dụng, một thiết kế giao diện hiện đại nên có tính độc lập cao với khối chương trình xử lý dữ liệu. Đối với nhiều hệ thống, giao diện là bộ phận có tầm quan trọng chủ chốt và có yêu cầu sửa đổi thường xuyên. Do đó, để tiện cho việc sửa đổi, giao diện nên được thiết kế có tính mô đun hóa cao và nên có độ độc lập tối đa với khối chương trình xử lý dữ liệu. Điều này cũng dẫn đến khả năng chúng ta có thể xây dựng nhiều giao diện khác nhau cho các đối tượng sử dụng khác nhau hay chạy trên các hệ thống khác nhau.

Có hai dòng giao diện chính là:

- *Giao diện dòng lệnh*: là loại giao diện đơn giản nhất, thường được thiết kế gắn chặt với chương trình và có tính di chuyển cao (tương đương với chương trình). Giao diện dòng lệnh phù hợp với các ứng dụng thuần túy xử lý dữ liệu, nhất là đối với các chương trình mà đầu ra là đầu vào của chương trình khác. Giao diện dòng lệnh gọn nhẹ, dễ xây dựng nhưng thường khó học, khó sử dụng và chỉ phù hợp với người dùng chuyên nghiệp trong các ứng dụng đặc thù.

- *Giao diện đồ họa*: sử dụng các cửa sổ, menu, icon... cho phép người dùng có thể truy cập song song đến nhiều thông tin khác nhau; người dùng thường tương tác bằng cách phối hợp cả bàn phím và con chuột; giao diện đồ họa dễ học, dễ sử dụng và trở nên rất thông dụng và có độ chuẩn hóa cao.

Nhìn trên khía cạnh độc lập với khối chương trình xử lý, có một số cách thức xây dựng giao diện khác nhau:

- *Giao diện đồ họa (GUI) truyền thống*: là giao diện đồ họa được thiết kế có độ liên kết cao với chương trình (được xây dựng cùng ngôn ngữ, cùng bộ công cụ...), hầu hết các chương trình trên máy tính cá nhân sử dụng loại giao diện này.

- X protocol: giao diện đồ họa sử dụng giao thức X protocol, phổ biến trên các máy Unix/Linux. Loại giao diện này có ưu điểm là có thể hoạt động độc lập với khối chương trình còn lại, tức là ta có thể chạy giao diện trên một máy tính trong khi đó phần xử lý bên trong lại hoạt động trên một máy khác. Đáng tiếc là phương thức này vẫn chưa phổ biến trên các máy tính cá nhân (chạy hệ điều hành MS Windows).

- Client/server: một cách tiếp cận để hướng tới tính độc lập và khả chuyển của giao diện là xây dựng giao diện như là một chương trình client, tương tác với khối chương trình xử lý (server) thông qua các giao thức trao đổi thông tin trên mạng (TCP/IP).

- Web based: một trong các cách thức xây dựng giao diện phổ biến hiện nay là dựa trên nền web, sử dụng các trình duyệt web để trao đổi thông tin với server. Tuy có một số nhược điểm về an toàn thông tin và tốc độ nhưng với tính độc lập hoàn toàn với phần xử lý, độ chuẩn hóa cao và khả năng sẵn có trên hầu hết các thiết bị nối mạng, phương thức này đang được ứng dụng rộng rãi.

Thiết kế giao diện khác với thiết kế các chức năng khác của phần mềm ở điểm hướng tới người sử dụng, cần người sử dụng đánh giá. Các công đoạn thiết kế khác như thiết kế dữ liệu, thiết kế thuật toán che dấu hoạt động kỹ thuật chi tiết khỏi khách hàng. Ngược lại, khách hàng (người dùng tiềm ẩn) nên tham gia vào quá trình thiết kế giao diện. Kinh nghiệm và khả năng của họ cần phải được tính đến khi thiết kế giao diện.

3.5.7.1 Một số vấn đề thiết kế giao diện

Trong thiết kế giao diện, cần chú ý tới một số vấn đề sau:

a. Thời gian phản hồi.

Chúng ta cần quan tâm tới hai loại thời gian là

- Thời gian đáp ứng trung bình: là thời gian trung bình mà hệ thống phản hồi đối với một yêu cầu của người dùng. Thời gian để sinh ra “kết quả thực sự” của yêu cầu sẽ phụ thuộc vào bản chất yêu cầu, thuật toán, tốc độ của máy tính, tuy nhiên chúng ta cần quan tâm khía cạnh tâm lý là nếu người dùng đợi quá lâu mà không nhận được thông tin gì thì họ sẽ nghĩ là có vấn đề và có thể sẽ tiến hành các thao tác ngoài mong đợi như lặp lại thao tác hay dừng hệ thống.

- Độ biến thiên của thời gian: độ biến thiên của thời gian cũng là đại lượng cần quan tâm. Nếu độ biến thiên lớn, ví dụ một thao tác thường được đáp ứng trong 1 giây mà có trường hợp phải mất 5 giây mới hoàn thành thì cũng có thể làm cho người dùng đưa ra các thao tác sai.

b. Các tiện ích.

Một giao diện tốt cần có các tiện ích để trợ giúp người sử dụng. Có các loại tiện ích sau

- Tích hợp: là tiện ích được tích hợp vào giao diện như nút Help cung cấp các thuyết minh về thao tác.

-
- Phụ thêm: là các tiện ích phụ thêm như các tài liệu trực tuyến.
 - Macro: một số chương trình còn cho phép người dùng tự động hóa một số thao tác bằng các lệnh kiểu macro.

c. Thông báo.

Các thông báo do hệ thống đưa ra cần:

- Có nghĩa: mọi thông báo cần có nghĩa đối với người dùng.
- Ngắn gọn: các thông báo cần ngắn gọn đi vào bản chất vấn đề, đặc biệt là đối với kiểu giao diện dòng lệnh.
- Có tính xây dựng: thông báo nên có tính xây dựng như đưa ra các nguyên nhân và các hướng khắc phục.

3.5.7.2 Một số hướng dẫn thiết kế

Dưới đây là một số yếu tố mà giao diện tốt nên có:

- Hướng người dùng: đối tượng người dùng phải rõ ràng, giao diện nên được thiết kế có tính đến năng lực, thói quen... của loại đối tượng đó.
- Có khả năng tùy biến cao: giao diện nên có khả năng tùy biến cao để phục vụ cho các cá nhân có cách sử dụng khác nhau, các môi trường hoạt động khác nhau.

Các phần mềm trên hệ UNIX với giao diện theo chuẩn X protocol thường được thiết kế có độ tùy biến rất cao.

- Nhất quán: các biểu tượng, thông báo, cách thức nhập dữ liệu phải nhất quán và nên tuân theo các chuẩn thông thường.
- An toàn: nên có chế độ xác nhận lại đối với các thao tác nguy hiểm (như xóa dữ liệu) và nên có khả năng phục hồi trạng thái cũ (undo).
- Dễ học, dễ sử dụng: giao diện luôn cần được thiết kế hướng tới tính dễ học, dễ sử dụng, tức là không đòi hỏi người dùng phải có các năng lực đặc biệt. Ví dụ như không cần nhớ nhiều thao tác, không đòi hỏi phải thao tác nhanh, các thông tin trên màn hình dễ đọc... Một cách tốt nhất để xây dựng giao diện dễ học dễ sử dụng là tuân theo các chuẩn giao diện thông dụng.

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

- Câu 1. Tại sao phải có thiết kế ứng dụng?
- Câu 2. Nêu các hoạt động của quá trình thiết kế phần mềm.
- Câu 3. Thế nào là một thiết kế phần mềm tốt.
- Câu 4. Trình bày các nguyên tắc thiết kế phần mềm?
- Câu 5. Vì sao phải thiết kế giao diện ứng dụng? Chỉ rõ các vấn đề quan trọng khi thiết kế giao diện ứng dụng.

Câu hỏi trắc nghiệm:

Câu 1. Trong ba giai đoạn: thiết kế, cài đặt và bảo trì thì giai đoạn nào là giai đoạn quan trọng nhất, chịu trách nhiệm đến 80% đối với sự thành công của một sản phẩm:

- a. Giai đoạn thiết kế
- b. Giai đoạn cài đặt
- c. Giai đoạn bảo trì
- d. Không có giai đoạn nào

Câu 2. Nhiệm vụ của giai đoạn nào trong các giai đoạn thiết kế, cài đặt, kiểm thử, bảo trì là chuyển đổi những yêu cầu của hệ thống (kết quả của quá trình phân tích) sang dạng biểu diễn của hệ thống phần mềm:

- a. Giai đoạn thiết kế
- b. Giai đoạn cài đặt
- c. Giai đoạn bảo trì
- d. Giai đoạn kiểm thử

Câu 3. Trong quan điểm quản lý: thiết kế phần mềm được tiến hành gồm những bước nào:

- a. Thiết kế sơ bộ: quan tâm đến việc dịch các yêu cầu thành các kiến trúc dữ liệu và phần mềm.
- b. Thiết kế chi tiết: tập trung vào việc làm mịn biểu diễn kiến trúc để dẫn đến cấu trúc dữ liệu chi tiết và biểu diễn thuật toán cho phần mềm.
- c. Cả thiết kế sơ bộ và thiết kế chi tiết
- d. Không cần bước nào.

Câu 4. Tiến trình thiết kế được thực hiện theo thứ tự nào:

- a. Thiết kế hình thức, Phác thảo thiết kế phi hình thức, Thiết kế phi hình thức, Thiết kế hoàn chỉnh
- b. Phác thảo thiết kế phi hình thức, Thiết kế hình thức, Thiết kế phi hình thức, Thiết kế hoàn chỉnh.
- c. Thiết kế hoàn chỉnh, Phác thảo thiết kế phi hình thức, Thiết kế phi hình thức, Thiết kế hình thức.
- d. Phác thảo thiết kế phi hình thức, Thiết kế phi hình thức, Thiết kế hình thức, Thiết kế hoàn chỉnh.

Câu 5. Kết quả của việc thiết kế phần mềm là các bản thiết kế về:

- a. Thành phần Giao diện
- b. Thành phần Xử lý
- c. Thành phần Dữ liệu
- d. Cả 3 thành phần trên

Bài tập vận dụng:

Bài 1: Sau khi đã vận dụng những kiến thức ở chương 2 để khảo sát và phân tích, đặc tả phần mềm “Quản lý sinh viên của một trường Đại học”. Yêu cầu tiếp theo hãy thiết kế hệ thống phần mềm, thiết kế bộ cơ sở dữ liệu, thiết kế giao diện phần mềm trên.

Bài 2: Để xây dựng và phát triển phần mềm “Game 3D”. Yêu cầu hãy thiết kế hệ thống phần mềm này, thiết kế các cửa sổ giao diện ứng dụng tương ứng như giao diện ban đầu, cửa sổ thiết lập thuộc tính Game, giao diện mức (level) tiếp theo, giao diện kết thúc Game, giao diện bảng xếp hạng người chơi.

Bài 3: Để xây dựng và phát triển phần mềm “Website quản lý bán hàng online của một sản phẩm thời trang”. Yêu cầu hãy thiết kế hệ thống phần mềm này, đưa ra bộ cơ sở dữ liệu của phần mềm. Hãy thiết kế giao diện trang chủ tương tác với người dùng, giao diện trang quản trị tương tác với người quản trị.

Bài 4: Bạn muốn xây dựng một phần mềm “Diệt virus” mới. Bạn hãy thiết kế hệ thống phần mềm này, hãy mô tả và thiết kế giao diện ứng dụng tương ứng.

Bài 5: Hãy chọn một phần mềm mà bạn đã từng tham gia phát triển hoặc sử dụng bạn hãy mô tả lại bản thiết kế hệ thống, bộ cơ sở dữ liệu, giao diện ứng dụng của phần mềm đó.

CHƯƠNG 4. CÀI ĐẶT PHẦN MỀM

Mục tiêu của chương:

Người học sẽ hiểu quy trình cài đặt và vận dụng để cài đặt được các phần mềm tương ứng. Cài đặt là một công đoạn trong việc phát triển phần mềm và nó được xem là một hệ quả tất yếu của thiết kế. Tuy vậy, phong cách lập trình và các đặc trưng của ngôn ngữ lập trình có ảnh hưởng lớn đến chất lượng của phần mềm. Một chương trình được cài đặt tốt đem lại cho ta thuận lợi trong việc bảo trì sau này.

4.1 Nền tảng của ngôn ngữ lập trình

4.1.1. Kiểu dữ liệu, định nghĩa kiểu dữ liệu và kiểm tra kiểu dữ liệu

Kiểu dữ liệu là loại dữ liệu được định nghĩa từ trước của ngôn ngữ và mỗi ngôn ngữ hỗ trợ một số kiểu dữ liệu. Tất cả các ngôn ngữ đều hỗ trợ biến, hằng số dùng trong dữ liệu số và dữ liệu ký tự. Kiểu dữ liệu được hỗ trợ chung là: số nguyên, số thực và xâu ký tự.

Một số ít ngôn ngữ hỗ trợ các kiểu dữ liệu khác như: Logical, Boolean, Pointer, Object, Bit, Date, ... hoặc kiểu dữ liệu tự định nghĩa.

Kiểu Boolean sinh ra giá trị nhị phân True, False dựa trên so sánh logic. Pointer là địa chỉ của chương trình khác hoặc cấu trúc dữ liệu mà được dùng để tham chiếu đến trong chương trình. Object được xây dựng để đóng gói dữ liệu và phương thức. Kiểu dữ liệu Date định nghĩa ngày tháng năm trong một khuôn dạng hợp lệ - thay cho việc phải viết các chương trình để xử lý kiểu Date, ta có thể sử dụng các thủ tục có sẵn của ngôn ngữ.

Các cấu trúc dữ liệu như mảng, bảng, danh sách tuyến tính, ... là loại thứ ba của cấu trúc dữ liệu của ngôn ngữ. Các ngôn ngữ có thể hỗ trợ hoặc không hỗ trợ kiểu này. Tuy nhiên, các kiểu dữ liệu đơn giản như mảng, danh sách tuyến tính, ... thường được hầu hết các ngôn ngữ hỗ trợ.

Cuối cùng, kiểu dữ liệu tự định nghĩa là kiểu dữ liệu do lập trình viên định nghĩa và chỉ có giá trị trong một chương trình hoặc ứng dụng nhất định. Kiểu dữ liệu tự định nghĩa có thể dùng để định nghĩa các kiểu dữ liệu khi ngôn ngữ không hỗ trợ kiểu dữ liệu đó.

Kiểm tra kiểu dữ liệu là việc ngôn ngữ kiểm tra sự phù hợp của kiểu dữ liệu được định nghĩa trong các phép toán học và các toán tử logic. Có bốn mức kiểm tra kiểu, từ không kiểm tra kiểu đến kiểm tra chặt chẽ, mức độ chặt chẽ của kiểm tra phụ thuộc vào dạng ứng dụng. Nói chung các tiến trình càng cần sự chính xác, nhất quán và ổn định thì càng đòi hỏi mức độ kiểm tra kiểu chặt chẽ hơn. Trong lập trình hướng đối tượng, kiểm tra kiểu càng quan trọng bởi tính đa hình cho phép nhiều module thực hiện cùng chức năng trên nhiều kiểu dữ liệu khác nhau, cho nên kiểm tra kiểu chặt chẽ sẽ làm giảm khả năng chương trình gặp lỗi.

- Không kiểm tra kiểu (typeless checking) nghĩa là không tiến hành sự kiểm tra kiểu một cách tường minh.

Ví dụ: Trong các ngôn ngữ không kiểu như Basic hoặc Cobol, các kí tự được phép gán bởi integer, nhưng có thể gây ra lỗi nếu trường này được tham chiếu như là một số nguyên.

Không có gì bảo đảm việc không gặp lỗi khi ta thao tác trên các trường không kiểu. Các ngôn ngữ hoặc chương trình dịch có cách xử lý trường không kiểu không thống nhất.

- Mức kiểm tra kiểu tiếp theo là ép kiểu tự động (automatic type coercion), trong đó nhiều kiểu dữ liệu được phép dùng chung, nhưng không phải tất cả và có thể dẫn đến lỗi chuyển đổi các kiểu không tương thích. Mức kiểm tra kiểu này còn có tên kiểm tra kiểu dạng hỗn hợp (mixed mode type checking), những kiểu dữ liệu khác nhau nhưng thuộc cùng một phân loại được chuyển sang một kiểu đích đối với toán tử kiểu hỗn hợp.

- Ví dụ, trong Fortran, trộn lẫn số thực và số nguyên trong toán tử toán học dẫn đến các kết quả không thể dự đoán được bởi vì kiểu đích (target type) được quyết định bởi việc định nghĩa trường kết quả. Nếu trường kết quả được định nghĩa là thực, kết quả tính toán là số thực. Nếu trường kết quả được định nghĩa là integer, tiến trình sẽ làm tròn câu trả lời (số thực) và đưa ra kết quả là integer [1].

- Kiểm tra kiểu giả chặt chẽ (Pseudostrong type checking) là mức thứ ba của kiểm tra kiểu, nó cho phép thao tác các đối tượng dữ liệu thuộc cùng một kiểu dữ liệu, nhưng phép kiểm tra kiểu này chỉ áp dụng khi chúng được định nghĩa trong cùng một module. Pascal là ngôn ngữ có kiểm tra kiểu giả chặt, nó hỗ trợ kiểm tra kiểu chặt chẽ trong module, nhưng không hỗ trợ chéo giữa các module. Cho nên, dữ liệu truyền từ một module sang module khác có thể chuyển sang kiểu dữ liệu khác mà không bị bắt lỗi.

- Ở mức cao nhất của kiểm tra kiểu của ngôn ngữ, kiểm tra kiểu chặt chẽ chỉ cho phép thao tác trên những đối tượng dữ liệu có cùng kiểu đã xác định từ trước, bất kể nó nằm trong cùng hoặc khác module. Nếu trong module có kiểu dữ liệu không hợp lệ, ứng dụng sẽ dừng và đưa ra một thông báo lỗi. Ada là ngôn ngữ cung cấp kiểm tra kiểu chặt chẽ.

4.1.2. Chương trình con

Sự tinh tế của ngôn ngữ thể hiện ở mức độ hỗ trợ module hoá và quản lý bộ nhớ. Module hoá là cách thức tạo ra chương trình con và hàm. Các ngôn ngữ khác nhau ở cách hỗ trợ chương trình con và dữ liệu của nó. Trước hết, khả năng định nghĩa chương trình con, hàm là quan trọng để có được các đặc trưng chương trình mong muốn. Thứ hai, dữ liệu trong các module được quản lý như thế nào? Dữ liệu có thể là cục bộ hoặc tổng thể. Khả năng có được dữ liệu cục bộ là quan trọng trong việc che giấu thông tin và giảm thiểu việc liên kết. Phạm vi dữ liệu tổng thể cần được giới hạn để đảm bảo chất lượng của chương trình trong việc giấu thông tin và sự liên kết.

Trong các ngôn ngữ, chương trình con được gọi thông qua tên của nó. Tùy chọn cho xử lý việc gọi bao gồm cả việc truyền dữ liệu bằng biến, bằng tên, bằng địa chỉ, hoặc bằng giá trị. Truyền giá trị đòi hỏi sự định nghĩa dữ liệu cục bộ trong khi truyền dữ liệu bằng tên hoặc bằng địa chỉ được sử dụng với hoặc dữ liệu cục bộ hoặc dữ liệu tổng thể [2].

Nói chung, khi sử dụng chương trình con, module chính gọi chương trình con làm những việc của nó và trả lại kết quả cho module chính. Khả năng hỗ trợ xử lý chương trình con đòi hỏi một hoặc nhiều hơn một mục vào hoặc điểm thoát. Xử lý Exit và Return cũng quan trọng khi chuyển quyền điều khiển giữa các module. Trong các trường hợp, càng nhiều cơ hội để vào và thoát khỏi module đã xác định trước, thì lập trình viên càng cần sự thành thạo, đảm bảo khả năng xử lý thành thạo, đảm bảo khả năng xử lý hoàn hảo. Theo các nhà lập trình cấu trúc, một module được thiết kế tốt nên có một điểm vào và một điểm ra. Module một vào và một ra ít gây lỗi hơn so với các module có nhiều mục vào, điểm ra.

4.1.3. Cấu trúc điều khiển

Về bản chất, một chương trình máy tính là một bản mã hoá thuật toán. Ở đây, các đối tượng chịu thao tác được mô tả và kiến trúc thông qua cấu trúc dữ liệu còn các thao tác được mô tả thông qua các cấu trúc điều khiển. Như vậy, cấu trúc điều khiển của ngôn ngữ là yếu tố quyết định thao tác gì và thao tác như thế nào trên dữ liệu đã mô tả. Chúng cung cấp các khả năng xử lý: tuần tự, lặp và cách thức lựa chọn các cấu trúc dữ liệu.

Sự tuần tự có hai dạng: giữa các dòng lệnh và trong dòng lệnh. Lập trình viên điều khiển sự tuần tự giữa các dòng lệnh (between-command sequencing) như là một trật tự của các lệnh, còn sự tuần tự trong dòng lệnh đó chính là thứ tự ưu tiên của các phép toán - operator precedence- dùng trong thao tác dữ liệu, nó được các ngôn ngữ quy định sẵn. Với hai khối lệnh A, B tuân theo phương thức xử lý tuần tự thì với R là số lần thực hiện của khối lệnh ta có $R_A = R_B = 1$. Cấu trúc tuần tự trong các ngôn ngữ lập trình thường tuân theo trật tự từ trái sang phải và từ trên xuống dưới.

Cấu trúc lựa chọn trong ngôn ngữ lập trình thường được mô tả dưới các từ khoá If hoặc Case. Với biểu thức điều kiện lựa chọn E và các khối lệnh lựa chọn $A_1, A_2, \dots, A_n$, theo ký hiệu trên ta có $1 = R_E \Rightarrow R_{A1} + \dots + R_{An}$.

Cấu trúc lặp trong ngôn ngữ lập trình được hỗ trợ bởi các dạng: lặp biết trước số lần lặp (For), lặp với kiểm tra điều kiện lặp trước - lính canh đặt trước (While.....do), và lặp với kiểm tra điều kiện lặp sau (Do.....while).

Lặp biết trước số lần lặp được đánh dấu bởi các biểu thức đếm được đầu (D) đến cuối (C). Với khối lệnh A trong thân vòng lặp, ta có $R_C = R_D = 1$ và $R_A = C - D + 1$ nếu $C \geq D$, ngược lại thì $R_A = 0$ nếu $C < D$.

Lặp với kiểm tra điều kiện lặp trước ứng với biểu thức điều kiện lặp E thì lúc này, khối lệnh A trong thân vòng lặp tuân theo: $1 \leq R_E = R_A + 1$.

Còn lặp với kiểm tra điều kiện lặp sau ứng với biểu thức điều kiện lặp E thì khối lệnh A trong thân vòng lặp tuân theo: $1 \leq R_E = R_A$.

Sự tương đương của các chương trình trong việc mã hoá bởi các cấu trúc điều khiển đã được chỉ ra ở định lý Boehm&Jaccopini như sau: *Mọi chương trình P được thể hiện bằng sơ đồ khối đều tồn tại một chương trình Q tương đương mạnh với nó nhưng chỉ dùng hai cấu trúc điều khiển để mô tả đó là cấu trúc tuần tự và cấu trúc lặp với điều kiện lặp xét trước.*

Ngoài việc cung cấp các cấu trúc điều khiển, các ngôn ngữ còn hỗ trợ các phương thức như: *Exits, Return, Fail, ...* để thoát khỏi module hiện tại trở về module gọi hoặc tới module khác.

Bên cạnh các cấu trúc điều khiển đã đề cập ở trên, đệ quy là một thuộc tính của module. Chúng xuất hiện khi module gọi chính chúng hoặc các module gọi lẫn nhau. Trong một số ngôn ngữ lập trình, sự đệ quy không được hỗ trợ một cách tường minh, nhưng nó lại được coi là sức mạnh chính của một số ngôn ngữ khác- ví dụ như ngôn ngữ Prolog. Ở các chương trình sử dụng đệ quy, đòi hỏi khả năng duy trì hàng đợi hoặc stack của chương trình.

4.1.4. Vào và ra dữ liệu

Có bốn dạng thông tin vào/ra (I/O) là: lệnh vào/ra cụ thể, hướng bản ghi, hướng tập hợp, và hướng mảng.

Vào/ra hướng bản ghi đọc hoặc ghi các bản ghi vật lý, bản ghi này có thể chứa đựng một hoặc nhiều bản ghi logic. Các bản ghi (hoặc là bộ trong đại số quan hệ) sẽ nhóm các trường dữ liệu có quan hệ với nhau. Vào/ra hướng bản ghi đòi hỏi đóng mở file, đọc ghi các bản ghi và quản lý người sử dụng tất cả các công việc xử lý file. Ví dụ: Cobol, Fortrans, Assembler, Ada là các ngôn ngữ hướng bản ghi.

Hướng tập hợp giả sử rằng tất cả các bản ghi (hoặc các bộ) được coi như nhau. Ngôn ngữ điều khiển mọi file và mọi tiến trình đọc ghi theo sự lựa chọn mà người sử dụng định nghĩa. Ở cuối thủ tục, tập các bản ghi (là kết quả của thủ tục) được lưu trữ trong bộ nhớ phục vụ cho việc in ấn, hiển thị. Ví dụ SQL là ngôn ngữ hướng tập hợp.

Vào/ra hướng mảng là đọc và ghi chuỗi các trường được giả thiết là kiểu mảng, người sử dụng có nhiệm vụ định nghĩa và thao tác kiểu dữ liệu của mảng. Ngôn ngữ chỉ đơn giản đọc và ghi cho đến cuối mảng dữ liệu. Pascal là ngôn ngữ hướng mảng. Vào/ra trực tiếp danh sách (list-directed I/O) là một biến thể của vào/ra hướng mảng. Fortrans sử dụng vào/ra trực tiếp danh sách để định nghĩa danh sách các tên biến, mỗi tên biến được truy cập trực tiếp khi chúng được đọc. Nó đọc cho đến khi danh sách đầy rồi xử lý cho đến khi lệnh đọc được thực hiện lại. Các mục dữ liệu không được định dạng cụ thể, mà khuôn dạng ngầm chỉ trong tên biến.

4.1.5. Quản lý bộ nhớ

Sự tinh tế của ngôn ngữ còn thể hiện ở mức độ lập trình viên kiểm soát điều khiển việc quản lý bộ nhớ. Quản lý bộ nhớ là khả năng chương trình phân bổ bộ nhớ máy tính khi cần. Đây là tùy chọn nhưng chúng được sử dụng nhiều khi xử lý danh sách biến và các ứng dụng thời gian thực quản lý tài nguyên nhiều người sử dụng. Các ngôn ngữ có độ tinh tế thấp sử dụng bộ nhớ tĩnh: chương trình nhận lượng bộ nhớ lớn nhất tại thời điểm khởi tạo. Nếu chương trình cần nhiều bộ nhớ hơn lượng được cấp phát thì chương trình sẽ bị treo, ngôn ngữ điều khiển nhiệm vụ (job control language) sẽ cấp phát lượng bộ nhớ thiếu đó để chương trình chạy lại. Các ngôn ngữ tinh tế hơn sử dụng khả năng cấp phát bộ nhớ động, tức là chỉ cấp phát bộ nhớ khi nào cần thiết.

4.1.6. Quản lý lỗi

Quản lý lỗi là mức chương trình được cài đặt để phát hiện và quản lý lỗi mà không phải dừng chương trình. Khả năng này sẽ làm tăng độ phức tạp và mở rộng phạm vi hữu ích của ngôn ngữ. Ví dụ Cobol cho phép ta chặn đứng lỗi dữ liệu như tràn, chia cho 0, nhưng lại không chặn được lỗi như định nghĩa dữ liệu không hợp lệ, đọc quá cuối file, ... Ngược lại Smalltalk cho phép chặn được bất kỳ lỗi nào.

Tóm lại, ngôn ngữ lập trình khác nhau ở mức độ chúng hỗ trợ các cách khác nhau cho điều khiển dữ liệu, xử lý vào/ra, thao tác toán học, chương trình con, và quản lý bộ nhớ. Ngôn ngữ hỗ trợ ít là ngôn ngữ đơn giản. Cấu trúc ngôn ngữ càng phức tạp thì phạm vi bao quát của nó càng lớn.

4.2 Phong cách cài đặt phần mềm

Phong cách lập trình bao hàm một triết lý về lập trình nhấn mạnh tới tính dễ hiểu của chương trình nguồn. Các yếu tố của phong cách bao gồm: tài liệu bên trong chương trình, phương pháp khai báo dữ liệu, cách xây dựng câu lệnh và các kỹ thuật vào/ra.

4.2.1. Tài liệu chương trình

Tài liệu bên trong của chương trình gốc bắt đầu với việc chọn lựa các tên gọi định danh (biến và nhãn), tiếp tục với vị trí và thành phần của việc chú thích, và kết luận với cách tổ chức trực quan của chương trình. Việc lựa chọn các tên gọi định danh có nghĩa là điều chủ chốt cho việc hiểu chương trình. Những ngôn ngữ giới hạn độ dài tên biến hay nhãn làm các tên mang nghĩa mơ hồ. Cho dù một chương trình nhỏ thì một tên gọi có nghĩa cũng làm tăng tính dễ hiểu. Theo ngôn từ của mô hình cú pháp/ngữ nghĩa tên có ý nghĩa làm “đơn giản hóa việc chuyển đổi từ cú pháp chương trình sang cấu trúc ngữ nghĩa bên trong”.

Một điều rõ ràng là: phần mềm phải chứa tài liệu bên trong. Lời chú thích cung cấp cho người phát triển một ý nghĩa truyền thông với các độc giả khác về chương trình gốc. Lời chú thích có thể cung cấp một hướng dẫn rõ rệt để hiểu trong pha cuối cùng của công nghệ phần mềm - bảo trì. Có nhiều hướng dẫn đã được đề nghị cho việc viết lời chú

thích. Các chú thích mở đầu và chú thích chức năng là hai phạm trù đòi hỏi cách tiếp cận có hơi khác. Lời chú thích mở đầu nên xuất hiện ở ngay đầu của mọi mô đun. Định dạng cho lời chú thích như thế là:

1. Một phát biểu về mục đích chỉ rõ chức năng mô đun.

2. Mô tả giao diện bao gồm:

- Một mẫu cách gọi
- Mô tả về dữ liệu
- Danh sách tất cả các mô đun thuộc cấp

3. Thảo luận về dữ liệu thích hợp (như các biến quan trọng và những hạn chế, giới hạn về cách dùng chúng) và các thông tin quan trọng khác.

4. Lịch sử phát triển bao gồm:

- Tên người thiết kế mô đun (tác giả).
- Tên người xét duyệt và ngày tháng.
- Ngày tháng sửa đổi và mô tả sửa đổi.

Các chú thích mô tả được nhúng vào bên trong thân của chương trình gốc và được dùng để mô tả cho các hàm xử lý. Lời chú thích nên đưa ra một điều gì đó phụ trợ, không chỉ là lời diễn giải chương trình. Bên cạnh đó, lời chú thích mô tả nên:

- Mô tả các khối chương trình, thay vì chú thích cho từng dòng.
- Dùng dòng trống hay thụt cấp để cho lời chú thích có thể được phân biệt với chương trình
- Phải đúng đắn; một lời chú thích không đúng hay gây ra hiểu sai thì còn tồi tệ hơn là không có chú thích nào cả.

Với những tên gọi tượng trưng đúng đắn và việc chú thích tốt, việc làm tài liệu bên trong thích hợp sẽ được đảm bảo.

Khi một thiết kế thủ tục chi tiết được biểu diễn bằng cách dùng một ngôn ngữ thiết kế chương trình thì tài liệu thiết kế có thể được nhúng trực tiếp vào trong văn bản chương trình gốc như những câu chú thích. Kỹ thuật này đặc biệt có ích khi việc làm tài liệu được thực hiện trong hợp ngữ và giúp đảm bảo rằng cả chương trình và thiết kế sẽ được bảo trì khi những thay đổi được thực hiện cho cả hai.

4.2.2. Khai báo dữ liệu

Độ phức tạp và việc tổ chức cấu trúc dữ liệu được xác định trong bước thiết kế nhưng phong cách khai báo dữ liệu thì được thiết lập khi chương trình được sinh ra. Thứ tự khai báo dữ liệu nên được chuẩn hoá cho dù ngôn ngữ lập trình không có yêu cầu bắt buộc. Điều này tạo điều kiện thuận lợi cho việc kiểm thử, gỡ rối và bảo trì. Thậm chí, khi có nhiều định danh được khai báo trong câu lệnh thì việc sắp xếp theo trật tự chữ cái cho các tên gọi đó cũng có giá trị.

Nếu thiết kế có mô tả trước cấu trúc dữ liệu phức tạp thì nên dùng chú thích để giải thích các điểm đặc thù trong cài đặt ở ngôn ngữ lập trình.

4.2.3. Xây dựng câu lệnh

Việc xây dựng luồng logic phần mềm được thiết lập trong khi thiết kế. Việc xây dựng từng câu lệnh tuy nhiên lại là một phần của bước lập trình. Việc xây dựng câu lệnh nên tuân theo một qui tắc quan trọng hơn cả: mỗi câu lệnh nên đơn giản và trực tiếp. Nhiều ngôn ngữ lập trình cho phép nhiều câu lệnh trên một dòng. Khía cạnh tiết kiệm không gian của tính năng này khó mà biện minh bởi tính khó đọc nảy sinh. Cấu trúc chu trình và các phép toán điều kiện được chứa trong đoạn trên đều bị che lấp bởi cách xây dựng nhiều câu lệnh trên một dòng.

Cách xây dựng câu lệnh đơn và việc thật lẻ minh họa cho các đặc trưng logic và chức năng của đoạn này. Các câu lệnh chương trình gốc riêng lẻ có thể được đơn giản hóa bởi:

- Tránh dùng các phép kiểm tra điều kiện phức tạp
- Khử bỏ các phép kiểm tra điều kiện phủ định
- Tránh lồng nhau nhiều giữa các điều kiện hay chu trình
- Dùng dấu ngoặc để làm sáng tỏ các biểu thức logic hay số học
- Dùng dấu cách và/hoặc các ký hiệu dễ đọc để làm sáng tỏ nội dung câu lệnh
- Chỉ dùng các tính năng chuẩn của ngôn ngữ

Để hướng tới chương trình dễ hiểu luôn nên đặt ra câu hỏi: Liệu có thể hiểu được điều này nếu ta không là người lập trình cho nó không?

4.2.4. Vào/ra

Phong cách vào/ra được thiết lập khi phân tích và thiết kế phần mềm nhưng cách thức cài đặt vào/ra lại ảnh hưởng lớn đến người sử dụng hệ thống. Phong cách vào/ra sẽ thay đổi theo mức độ tương tác con người.

Với vào/ra theo lô thì cách tổ chức cái vào logic, kiểm tra lỗi vào/ra có nghĩa, phục hồi lỗi vào/ra tốt và định dạng báo cáo ra hợp lý là những đặc trưng mong muốn. Với vào/ra tương tác thì một sơ đồ đưa vào có hướng dẫn, đơn giản, việc kiểm tra lỗi kỹ lưỡng và có thể phục hồi, sự nhất quán định dạng vào/ra lại là các mối quan tâm chủ yếu.

Khi cài đặt vào/ra, cần thoả mãn các tiêu chí cơ bản sau:

- Làm hợp lệ mọi cái vào,
- Kiểm tra sự tin cậy của các tổ hợp dữ liệu vào quan trọng,
- Giữ cho định dạng dữ liệu vào đơn giản,
- Dùng các chỉ báo cuối dữ liệu thay vì yêu cầu người sử dụng xác định số các khoản mục vào,

-
- Đặt nhãn cho các dữ liệu vào,
 - Giữ các định dạng dữ liệu vào thống nhất, ...

4.3 Các đặc trưng của ngôn ngữ cài đặt

Các đặc trưng được đánh giá ở đây gồm: đồng nhất (uniformity), sáng sủa (ambiguity), cô đọng (compactness), địa phương – cục bộ (locality), tuyến tính (linearity), dễ lập trình, dịch hiệu quả, khả chuyển. Tính sẵn có của công cụ trợ giúp, các bộ sinh mã và tính sẵn dùng của công cụ trợ giúp kiểm tra cũng được thêm vào nhằm làm tăng tính hấp dẫn của ngôn ngữ.

Tính đồng nhất là cách sử dụng ký hiệu nhất quán trong cả ngôn ngữ. Một ví dụ của sự không nhất quán trong Focus là việc sử dụng dấu ngoặc đơn cho tiêu đề bản báo cáo do người sử dụng tạo ra và dấu ngoặc kép của trang bản báo cáo. Ngôn ngữ không nhất quán cản trở người sử dụng học và dễ gây lỗi.

Tính sáng sủa đề cập đến mức độ con người và chương trình dịch bất đồng trong việc dịch các câu lệnh của ngôn ngữ. Lý tưởng nhất là ý nghĩa của con người tương tự với sự biên dịch của trình dịch và chương trình dịch ra giống sự nhận thức của con người. Thật không may, tính sáng sủa có những vấn đề cố hữu của mình, như các ứng dụng trí tuệ nhân tạo (ứng dụng suy luận trong cả tiến trình), khi thêm luật, cơ chế mới vào, sự thông dịch của dữ liệu, luật đó có lẽ cũng thay đổi.

Tính cô đọng của ngôn ngữ nằm ở sự ngắn gọn. Các đặc trưng của chương trình bao gồm sự kết cấu có cấu trúc, từ khoá và viết tắt, hàm có sẵn, đã đơn giản hoá việc lập trình. Tương phản với hai ngôn ngữ thế hệ bốn SQL và Focus là Cobol, ngôn ngữ thế hệ ba. Thực tế cho thấy 3 đến 5 dòng lệnh 4GLs tương đương với 50 đến 150 dòng lệnh trong ngôn ngữ Cobol. Thời gian học Focus ngắn hơn Cobol một phần là bởi tính cô đọng của ngôn ngữ.

Tính cô đọng bao hàm tính cục bộ trong việc cung cấp sự phân đoạn tự nhiên của mã lệnh, làm đơn giản hoá việc học, trực quan hoá từng phần của vấn đề và có thể mô phỏng các giải pháp. Tính cục bộ được cung cấp thông qua khối case, hoặc những cơ chế phân đoạn (chunks). Sự phân đoạn có lẽ được thực hiện thông qua thực thi đoạn mã trong ngôn ngữ Cobol, cấu trúc case trong ngôn ngữ Focus, hoặc định nghĩa đối tượng trong ngôn ngữ Smalltalk.

Tính tuyến tính đề cập đến mức độ có thể đọc mã một cách liên tiếp (tuần tự). Ngôn ngữ càng tuyến tính (tuần tự) thì càng dễ phân đoạn và hiểu đoạn mã. Tính tuyến tính đơn giản hoá việc hiểu và bảo trì. Trong ví dụ đoạn mã Cobol được chắt thành các đoạn và thực hiện.

Trong lựa chọn ngôn ngữ độ khó khi biên dịch cũng đóng một vai trò quan trọng. Nói chung, nhiều ngôn ngữ mô tả, ví dụ như SQL, đang được xem xét, cân nhắc trên cơ sở dễ dàng hơn khi dịch ra mã ngữ so với các ngôn ngữ thủ tục như Fortran. Mặc dù vậy,

Prolog và các ngôn ngữ suy diễn khác tuy đơn giản trong việc mô tả và phát triển các luật đơn nhưng không tầm thường trong việc quyết định kết hợp các luật để tạo ra các tri thức đúng mới.

Tính hiệu quả của trình biên dịch nằm ở tính hiệu quả của mã assembler nhận được sau khi dịch. Tính hiệu quả đó thay đổi tùy theo ngôn ngữ và nhà sản xuất. Tính hiệu quả của trình biên dịch đặc biệt quan trọng khi lập trình một hệ thống máy bay hay các ứng dụng thường trú tương tác với các thành phần hệ thống như là một phần của hệ thống lớn.

Cùng với tính hiệu quả, tính khả chuyển của mã cũng rất quan trọng. Tính khả chuyển là khả năng đáp ứng của mã trên các cơ sở thực hiện khác nhau. Các cơ sở thực hiện bao gồm cả phần cứng, hệ điều hành, hay môi trường thực hiện phần mềm. Khi các ứng dụng dùng chung và phân tán càng phổ biến thì sự cần thiết đối với tính khả chuyển của ngôn ngữ sẽ càng tăng. Lý tưởng nhất, chương trình sẽ thực hiện được ở bất cứ nơi nào, trên bất cứ phần cứng hay hệ điều hành nào [3].

Tóm lại, nền tảng không đóng vai trò chính để phân biệt ngôn ngữ thì những tính đặc trưng của ngôn ngữ sẽ trở nên quan trọng trong việc lựa chọn ngôn ngữ.

4.4 Phân lớp và đánh giá về ngôn ngữ cài đặt

4.4.1. Các lớp ngôn ngữ

Hiện nay có hàng trăm ngôn ngữ lập trình, tuy nhiên theo đánh giá thì người ta chia nó ra làm bốn thế hệ - từ thế hệ thứ nhất đến thế hệ thứ bốn.

- **Các ngôn ngữ thế hệ thứ nhất:** là các chương trình được viết theo mã máy hoặc hợp ngữ. Các ngôn ngữ này phụ thuộc vào máy và có mức độ trừu tượng thấp. Ta chỉ nên dùng các ngôn ngữ này khi các ngôn ngữ cấp cao không thể đáp ứng được hay không hỗ trợ yêu cầu của ứng dụng.

- **Các ngôn ngữ thế hệ thứ hai:** được phát triển từ cuối những năm 1950 đến đầu những năm 1960, như FORTRAN, COBOL, ALGOL, BASIC, ... Nó được xem là nền tảng cho mọi ngôn ngữ lập trình hiện đại - thế hệ thứ ba. Các ngôn ngữ thế hệ thứ hai được đặt trung bởi việc sử dụng rộng rãi thư viện phần mềm không lồ và nó cũng đã được chấp nhận rộng rãi.

- **Các ngôn ngữ thế hệ thứ ba:** còn được gọi là ngôn ngữ lập trình hiện đại hay có cấu trúc. Nó được đặc trưng bởi khả năng cấu trúc dữ liệu và thủ tục mạnh. Các ngôn ngữ thuộc thế hệ này như: PASCAL, C, ADA, MODULA-2, C++, C-OBJECTIVE, ...

- **Các ngôn ngữ thế hệ thứ tư:** Trọng tâm của ngôn ngữ thế hệ thứ tư là nâng mức độ trừu tượng của chương trình lên cao. Các ngôn ngữ này cũng giống như mọi ngôn ngữ nhân tạo khác đều chứa một cú pháp phân biệt để biểu diễn điều khiển và cấu trúc dữ liệu, tuy nhiên nó biểu thị các cấu trúc này ở mức độ trừu tượng cao hơn bằng cách xóa bỏ yêu cầu xác định chi tiết thuật toán. Một số ngôn ngữ thuộc thế hệ thứ tư như ngôn ngữ văn đáp, ngôn ngữ hỗ trợ quyết định, ngôn ngữ làm bản mẫu, ...

4.4.2. So sánh, đánh giá về một số ngôn ngữ cài đặt

Ở đây, chúng ta đánh giá một số ngôn ngữ phổ biến được dùng trong các tổ chức kinh doanh ngày nay như: SQL, Focus, Basic, Cobol, Fortran, C, C++, Ada, Prolog, Java, và Smalltalk. Những ngôn ngữ này đại diện cho những kiểu lập trình chủ yếu đã xét ở trên gồm: lập trình thủ tục (Basic, Cobol, Fortran, Pascal), hướng đối tượng (Smalltalk, Ada, C++), xử lý khai báo (SQL, Prolog), các ngôn ngữ thể hệ thứ tư (Focus), và hệ chuyên gia (Prolog), lập trình trực quan (Visual Studio .NET)

1. SQL- *Structured Query Language*

Được xem là chuẩn American National Standards Institute đối với ngôn ngữ hỏi đáp cơ sở dữ liệu, SQL là một ngôn ngữ khá thành công. Ưu điểm của SQL hầu hết không mang tính kỹ thuật: dễ dàng sử dụng, gọn gàng, đồng nhất, cục bộ, tuyến tính, tính khả chuyên và khả năng tự động của các công cụ. Sự đơn giản của ngôn ngữ được thể hiện ở thời gian học ngôn ngữ nhanh đối với những người lần đầu sử dụng ngôn ngữ - người mới học có thể viết câu hỏi trong vòng ít phút. Và thời gian để trở thành thành thạo ít hơn so với các ngôn ngữ cơ sở dữ liệu khác.

Nhiều môi trường hỗ trợ phân tích và thiết kế trên hệ cơ sở dữ liệu logic thông qua các quá trình chuẩn hoá. Các sản phẩm này cũng sinh ra lệnh SQL định nghĩa cơ sở dữ liệu như là kết quả thiết kế logic cơ sở dữ liệu.

2. Focus

Là ngôn ngữ thể hệ bốn bao gồm một Database Engine cùng ngôn ngữ hỏi đáp tương thích với SQL, bộ hiển thị, hệ hỗ trợ đồ hoạ, thiết kế, bảo trì và các tiến trình xử lý thông minh. Focus DB hỗ trợ các mô hình quan hệ, mô hình phân cấp và mô hình mạng, cung cấp một giao diện với nhiều khuôn dạng. Cũng như SQL, mặt mạnh chủ yếu của Focus liên quan tới những đặc trưng phi kỹ thuật của ngôn ngữ, đó là tính cô đọng, tính cục bộ, tính tuyến tính, không bị ràng buộc bởi mã chuyển đổi, tính khả chuyển và tính sẵn dùng của công cụ CASE cho việc phân tích thiết kế dữ liệu. Đôi khi Focus có thể nhập nhằng trong việc biên dịch sự phân cấp dữ liệu hay đa kết nối dữ liệu. Hàng loạt các version của Focus hỗ trợ các khả năng đa người sử dụng. Focus là một ngôn ngữ đã được ngầm định là không hỗ trợ những định nghĩa của người dùng hoặc những tài nguyên khác của người sử dụng.

3. Basic - *Beginners All purpose Symbolic Interchange Code*

Được đánh giá một ngôn ngữ mạnh, cơ bản, trong ngôn ngữ không có những kỹ thuật phức tạp nhưng có toàn bộ các thành phần sơ đẳng. Basic là một ngôn ngữ dễ học, dễ viết, có tính thống nhất, chặt chẽ và các hệ thống trợ giúp kiểm tra tự động tốt. Các đặc trưng ngôn ngữ còn lại thay đổi tùy thuộc vào các phiên bản Basic khác nhau. Khả năng khả chuyển của Basic kém bởi các lệnh vào ra thường phải thay đổi để phù hợp với môi trường.

Basic hỗ trợ các thao tác lập trình chuẩn với một số giới hạn, cùng một số kiểu dữ liệu nhưng không có chức năng kiểm tra kiểu. Cấu trúc ngôn ngữ bao gồm các phép lặp, điều kiện và xử lý mảng, đọc/viết các file.

4. Cobol- Common Business Oriented Language

Là một ngôn ngữ được sử dụng nhiều trong lịch sử máy tính. Cobol được ví như một chiếc xe bus, lập trình Cobol mất nhiều thời gian, nhưng nó lại phù hợp với một số vấn đề thương mại. Như một ngôn ngữ đa mục đích, Cobol cung cấp tất cả các chức năng cơ bản.

Các tiến trình vào/ra của Cobol rất hiệu quả, có tính thống nhất cao và hỗ trợ hầu hết các loại dữ liệu. Ngôn ngữ Cobol không phù hợp cho những ứng dụng thời gian thực hay các ứng dụng đệ quy.

Trong các đặc trưng phi kỹ thuật, Cobol có tính sẵn dùng cao của công cụ trợ giúp, bộ sinh mã, và các chương trình kiểm tra. Như hầu hết các ngôn ngữ thông dụng khác Cobol là ngôn ngữ đầu tiên được phát triển hỗ trợ tự động. Đây là ngôn ngữ có tính tự động cao và được hỗ trợ bởi nhiều trình biên dịch. Trong các đặc trưng phi kỹ thuật khác, Cobol thường kém hơn SQL và Focus nhưng cũng tốt hơn nhiều các ngôn ngữ thủ tục khác.

5. Fortran - Formula Tranlastion

Là một ngôn ngữ của những năm 60. Điểm yếu của Fortran là trong lĩnh vực xử lý dữ liệu và hỗ trợ cấu trúc file. Fortran không được tích hợp với các phần mềm DBMS các giới hạn về tuần tự... Vì thế các quá trình vào ra của Fortran thường bị giới hạn nhiều so với các ngôn ngữ khác.

Điểm mạnh của Fortran là tính hiệu quả trong giải thuật sinh mã để thực hiện quá trình xử lý số. Chương trình dịch của Fortran thường được hỗ trợ bởi một thư viện các chương trình con chứa nhiều thuật toán ngắn được sử dụng thường xuyên, các quá trình thiết kế và xử lý toán học. Các chương trình con này được thiết kế để dễ dàng định nghĩa và sử dụng các biến tổng thể và các biến cục bộ. Sự xáo trộn các dạng dữ liệu trong Fortran là rất quan trọng bởi vì quá trình xử lý số sẽ cho kết quả khác nhau tùy thuộc vào định nghĩa những trường dữ liệu được xử lý.

6. C

C là một ngôn ngữ cấp cao được phát triển để thực hiện các xử lý cấp thấp. Một chương trình viết bằng C là một dãy các hàm và chúng được truy cập đến bởi một tên của chúng trong mã của chương trình.

C là một ngôn ngữ ngắn gọn, súc tích và khó hiểu vì thế nó chỉ thực sự hiệu quả cho những người lập trình có nhiều kỹ năng và kinh nghiệm về lập trình và có thể sẽ không mang lại hiệu quả cao cho những người lập trình viên kém.

7. C++

C++ là ngôn ngữ lập trình được phát triển trên nền tảng của ngôn ngữ lập trình C. Do đó, C++ có song song cả 2 phong cách(style) lập trình hướng cấu trúc giống C và có thêm phong cách hướng đối tượng. Trong nhiều trường hợp, C++ sử dụng kết hợp cả 2 style trên. Do đó, nó được xem là một ngôn ngữ “lai tạo”.

8. Prolog - Programming in Logic

Là một ngôn ngữ được phát triển riêng cho lĩnh vực trí tuệ nhân tạo. Prolog được phát triển bởi một trường đại học ở Marseiller từ rất sớm (những năm 70) nhưng được phát triển rộng rãi ở Mỹ bởi David Warren. Prolog là một ngôn ngữ hướng mục đích, một ngôn ngữ đặc tả với cấu trúc là những mệnh đề và các luật.

Prolog mệnh đề là những thành phần cụ thể của thông tin thực. Prolog luật được định nghĩa từ mệnh đề được giả định để tạo thông tin.

9. Smalltalk

Smalltalk được phát triển như là môi trường điều hành và ngôn ngữ lập trình vào những năm 70 tại trung tâm nghiên cứu Xerox Palo Alto bởi nhóm Learning Research. Đó là một ngôn ngữ hướng đối tượng, coi mọi thứ như là đối tượng, thậm chí đối với thể hiện, các số nguyên. Smalltalk được tối ưu ở mức cao và do vậy, được sử dụng để thiết kế các ứng dụng có hiệu quả.

Smalltalk có đầy đủ các chức năng, là ngôn ngữ lập trình có thể làm được mọi việc không hạn chế. Điểm yếu chủ yếu của Smalltalk là không hỗ trợ các đối tượng liên tục như là file. Nhưng nếu file được coi là một đối tượng, thì nó có thể được xử lý trong Smalltalk.

Điểm mạnh của Smalltalk là nó được sử dụng trong các quá trình xử lý hướng sự kiện như trong điều khiển tiến trình, việc điều khiển hệ thống điều hoà nhiệt độ, hoặc là sự thông báo kịp thời nhu cầu sản xuất. Các ứng dụng loại này sử dụng các thông điệp không liên tục từ môi trường bên ngoài để điều khiển quá trình xử lý thực hiện bởi ứng dụng.

10. Ada

Ada, ngôn ngữ lập trình chính thức của Bộ Quốc phòng Mỹ với hàng trăm nghìn người sử dụng, có một lối tư duy khác về cách lập trình so với các ngôn ngữ khác.

Ada được thiết kế bởi một hội đồng và không tạo thành một ngôn ngữ hoàn thiện nhưng nó lại tốt hơn tất cả. Phiên bản hiện hành của Ada là dựa trên đối tượng hơn là hướng đối tượng. Trong các ứng dụng dựa trên các đối tượng, các chương trình cùng hoạt động trên một tập hợp các đối tượng, mỗi tập hợp đại diện một thể hiện của một vài kiểu đối tượng. Tất cả các kiểu đối tượng là thành phần của mô hình phân cấp các kiểu mà chúng được kết nối thông qua quá trình xử lý hơn là việc kế thừa các quan hệ. Các lớp thường khó phân biệt với các kiểu bởi vì không có các đối tượng nhất quán như là file và nó không hỗ trợ kế thừa.

Khái niệm file trong Ada giống như trong Smalltalk được định nghĩa là một kiểu trong cấu trúc của ngôn ngữ và mọi quá trình xử lý hiện trên các kiểu. Giống như Smalltalk,

sức mạnh của Ada là khả năng của nó trong việc hỗ trợ xử lý hướng sự kiện như tên lửa dẫn đường trong hệ thống phòng thủ quốc gia.

Phiên bản tương lai của Ada có thể đáp ứng cấu trúc thừa kế và xử lý đa lớp và sự liên kết động các đối tượng, xử lý thông điệp thực nhất quán và các đối tượng nhất quán cung cấp các cấu trúc dữ liệu đa dạng. Với sự mở rộng này ngôn ngữ Ada thích hợp với các ứng dụng ảo. Một sự lưu ý tương tự về sự khác nhau trong tư duy hướng đối tượng như đã chỉ ra trong phần Smalltalk: thiết kế hướng đối tượng và phát triển chương trình khác nhau về cơ bản hơn là sự phát triển các ứng dụng thủ tục thông thường với các ngôn ngữ như là Cobol.

11. Java

Java là một ngôn ngữ lập trình hiện đại, bậc cao, hướng đối tượng, bảo mật và mạnh mẽ. và là một Platform.

Platform: Bất cứ môi trường phần cứng hoặc phần mềm nào mà trong đó có một chương trình chạy, thì được hiểu như là một Platform. Với môi trường runtime riêng cho mình (JRE) và API, Java được gọi là Platform.

Ngôn ngữ lập trình Java ban đầu được phát triển bởi Sun Microsystems do James Gosling khởi xướng và phát hành vào năm 1995. Phiên bản mới nhất của Java Standard Edition là Java SE 8. Với sự tiến bộ của Java và sự phổ biến rộng rãi của nó, nhiều cấu hình đã được xây dựng để phù hợp với nhiều loại nền tảng khác nhau. Ví dụ: J2EE cho các ứng dụng doanh nghiệp, J2ME cho các ứng dụng di động.

Các phiên bản J2 mới đã được đổi tên thành Java SE, Java EE và Java ME. Phương châm của java là "Write Once, Run Anywhere" - viết một lần chạy nhiều nơi, nghĩa là bạn chỉ cần viết một lần trên window chẳng hạn, sau đó vẫn chương trình đó bạn có thể chạy trên Linux, Android, các thiết bị J2ME...

12. C#

C# (hay C sharp) là một ngôn ngữ lập trình đơn giản, được phát triển bởi đội ngũ kỹ sư của Microsoft vào năm 2000, trong đó người dẫn đầu là Anders Hejlsberg và Scott Wiltamuth.

C# là ngôn ngữ lập trình hiện đại, hướng đối tượng và nó được xây dựng trên nền tảng của hai ngôn ngữ mạnh nhất là C++ và Java.

C# được thiết kế cho Common Language Infrastructure (CLI), mà gồm Executable Code và Runtime Environment, cho phép chúng ta sử dụng các ngôn ngữ high-level đa dạng trên các nền tảng và cấu trúc máy tính khác nhau.

4.4.3. Chọn ngôn ngữ cho ứng dụng

Khi đã làm việc trên ứng dụng ta không có sự lựa chọn về ngôn ngữ. Nhưng nếu ban đầu chọn sai ngôn ngữ thì chúng ta phải liên tục sửa đổi yêu cầu để phù hợp với những giới hạn của ngôn ngữ. Việc lựa chọn ngôn ngữ lập trình phải xem ngôn ngữ lập trình đó

có phù hợp với kiểu ứng dụng hay không và xem nó có phù hợp với việc dùng để phát triển ứng dụng.

Dựa vào các đánh giá ở 4.4.2, ta có bảng thống kê về việc lựa chọn ngôn ngữ lập trình dựa trên các tiêu chí của ứng dụng như sau:

Kiểu ứng dụng	SQL	Focus	Basic	Cobol	Fortran	C	C++	Prolog	Ada	Smalltalk	Java	C#
Lô	X	X	X	X							X	X
Trực tuyến	X	X	X	X		X	X		X	X	X	X
Thời gian thực						X	X		X	X	X	X
Hỏi đáp CSDL	X	X				X	X		X	X	X	X
Hỗ trợ quyết định	X	X			X	X	X	X	X	X	X	X
Hệ chuyên gia								X			X	X
EIS		X				X			X	X	X	X

4.5 Hiệu quả của chương trình và tầm quan trọng của ngôn ngữ cài đặt

Là khả năng đáp ứng hiệu năng một cách thích hợp nhằm tiết kiệm tài nguyên, tăng hiệu suất ứng dụng và khả năng mở rộng của phần mềm. Tính toán nguồn lực, mã nguồn đảm bảo hiệu năng cao khi phần mềm chạy trên hệ thống như khi xử lý các thuật toán, giao dịch, khả năng mở rộng, ... Việc phân tích này cung cấp những rủi ro tiềm ẩn, nguy hại do sự trì trệ của phần mềm theo thời gian.

Ví dụ như các phần mềm đòi hỏi tốc độ xử lý cao để xử lý các thuật toán phức tạp hoặc lượng dữ liệu lớn.

Đánh giá tính hiệu quả của phần mềm dưới những điều kiện sau:

- Cấu trúc ứng dụng
- Độ tương tác thích hợp với các nguồn tài nguyên
- Hiệu suất, thời gian truy cập và quản lý dữ liệu
- Quản lý bộ nhớ, mạng và không gian đĩa
- Quy trình mã hóa, lập trình

Có hai lý do cơ bản để nghiên cứu về việc tăng hiệu suất của chương trình. Thứ nhất là tầm quan trọng mang tính bản chất của nó trong nhiều ứng dụng: chi phí về thời gian và bộ nhớ. Lý do thứ hai là vấn đề giáo dục - đây là một mảnh đất tốt để đào tạo.

Trong giới hạn ở đây, chúng ta chỉ bàn luận về các vấn đề cho việc tăng tính hiệu quả thực tế - có thể đo được - của chương trình.

4.6 Một số vấn đề trong cải tiến hiệu suất

Như đã đề cập ở trên, ta sẽ quan tâm đến các vấn đề để giảm thời gian chạy và chi phí bộ nhớ cho chương trình.

Để minh họa cho các luận cứ được nêu về các vấn đề trên, ở đây, ta sẽ chỉ ra các vấn đề quan tâm thông qua các bài toán ví dụ minh họa.

4.6.1. Tốc độ xử lý

Trong hầu hết các trường hợp, tốc độ của chương trình là quan trọng như các ứng dụng thời gian thực, ứng dụng về xử lý trên các cơ sở dữ liệu lớn, ... Để một ứng dụng có tốc độ nhanh, người lập trình chúng phải quan tâm đến nhiều yếu tố như: thuật toán sử dụng, lựa chọn cấu trúc dữ liệu, tinh chế mã cho chương trình, ...

4.6.1.1 Thuật toán sử dụng

1. Xác định lại bài toán

Yêu cầu: Trước khi bắt tay vào giải bài toán, hãy tìm hiểu kỹ các yêu cầu mà bài toán đặt ra và tận dụng mọi điều đã biết từ bài toán.

Bài toán minh họa: Cho một mảng số nguyên gồm 1.000.000 phần tử; các giá trị nằm trong khoảng từ 0...10 một cách ngẫu nhiên. Hãy sắp xếp để được một mảng có thứ tự giảm dần.

- Giải bài toán tổng quát: là một bài toán sắp xếp; dùng một đoạn chương trình sắp xếp có sẵn của hệ thống hay sử dụng một thuật toán sắp xếp có sẵn như Insert - Sort hay Quick - Sort chẳng hạn. Chi phí về độ phức tạp là $O(n^2)$ hay $O(n \log n)$.

- Tuy nhiên, ta đã bỏ qua một tính chất của bài toán đó là các giá trị chỉ nằm trong khoảng 0...10. Sau khi nghiên cứu bài toán ta quyết định sử dụng thuật toán đếm cho việc sắp xếp bài toán.

- + Khởi tạo 10 biến nguyên với giá trị 0.
 - + Với mỗi giá trị i trong mảng, tăng biến thứ i lên một đơn vị
 - + Thực hiện rải giá trị cho mảng ứng với số lần là giá trị của biến thứ i ;
- Như thế, chi phí về độ phức tạp của bài toán là $O(n)$.

2. Sức mạnh của thuật toán

Yêu cầu: Việc nghiên cứu thuật toán giúp ích rất nhiều cho các nhà lập trình. Các thuật toán có ảnh hưởng quan trọng đến các hệ thống phần mềm và đặc biệt chúng tăng nhanh tốc độ vận hành.

Bài toán minh họa: Quay mảng một chiều chứa N phần tử về bên trái I một vị trí. Với $N = 8$; $I = 3$ ta được mảng ABCDEFGH sẽ quay thành DEFGHABC.

- Thuật toán 1: Ta có thể giải bài toán bằng cách sao I phần tử đầu tiên của mảng sang một mảng đoạn; dịch chuyển $N - I$ phần tử còn lại của mảng về bên trái I vị trí; sau đó sao I phần tử đầu tiên từ mảng tạm về cuối mảng. Trong trường hợp N và I lớn, như thế việc cần mảng tạm khá tốn bộ nhớ; xét trong trường hợp bộ nhớ của máy không dồi dào thì giải quyết như thế nào?

- Thuật toán 2: Ta cũng có thể viết 1 thủ tục con quay mảng X sang bên trái một vị trí (như thế sẽ giải quyết được vấn đề tốn bộ nhớ vì chỉ cần dùng một biến phụ) và sau đó thực hiện thủ tục trên I lần. Tuy nhiên, thủ tục trên tốn thời gian tỉ lệ với N và như thế chương trình sẽ tỉ lệ với thời gian $I \cdot N$; do vậy khi N và I lớn thì đây là điều không thể thực hiện được.

- Thuật toán 3: Đề ý rằng: khi quay mảng X gồm N phần tử về I vị trí, (giả sử quay sang trái), lúc này phần tử $X[i+1]$ sẽ là phần tử $X'[1]$ (ký hiệu $X'[i]$ là phần tử thứ i của mảng X sau khi quay); $X[2i+1]$ sẽ là phần tử $X'[i+1]$, ... và cứ thế tiếp tục. Do đó ta có thuật toán sau:

- + Dịch chuyển $X[1]$ đến 1 biến tạm T
- + Dịch chuyển $X[i+1]$ và $X[1]$;
- + Dịch chuyển $X[2i+1]$ và $X[i+1]$, ...
-
- + Quá trình cứ thế tiếp tục; chỉ số được tính theo module của N tức khi vượt quá N sẽ chia lấy dư cho N.
- + Quá trình lặp cho đến khi gặp phần tử $X[1]$ và lúc này dùng giá trị từ biến T và quá trình chấm dứt.

Ví dụ: Với trường hợp $N = 8, I = 3 \Rightarrow$ mảng $X = ABCDEFGH$ ta có như sau:

$X = ABCDEFGH; N = 8; I = 3; T = A$

- + Dịch chuyển $X[i+1]$: $X[4] \rightarrow X[1]$
- + Dịch chuyển $X[2i+1]$: $X[7] \rightarrow X[4]$
- + Dịch chuyển $X[3i+1]$: $X[10] \Rightarrow X[2] \rightarrow X[7]$
- + Dịch chuyển $X[4i+1]$: $X[13] \Rightarrow X[5] \rightarrow X[10] \Rightarrow X[2]$
-

Quá trình được tóm tắt như sau:

$T = A; N = 8; I = 3; X = ABCDEFGH$

$4 \rightarrow 1$ DBCDEFGH

$7 \rightarrow 4$ DBCGEFGH

$(10)2 \rightarrow 7$ DBCGEFBH

$5 \rightarrow 2$ DECGEFBH

$8 \rightarrow 5$ DECGHFBH

$(11)3 \rightarrow 8$ DECGHFBC

$6 \rightarrow 3(11) \dots \text{DEFGHFBC}$

$(9)1 \rightarrow 6 \dots \text{DEFGHABC}$

T => Dừng

Đây là thuật toán có chi phí vùng nhớ không lớn và thời gian chạy chấp nhận được.

3. Các kỹ thuật thiết kế thuật toán và tình chế thuật toán.

Yêu cầu: Thực hiện theo các nguyên tắc sau:

- Lưu trữ các trạng thái cần thiết để tránh tính lại,
- Tiền xử lý thông tin để đưa vào các cấu trúc dữ liệu,
- Sử dụng các thuật toán thích hợp,
- Chỉ ra được cận dưới của thuật toán,
- Sử dụng các kết quả được tích lũy, ...

Bài toán minh họa: Cho vector X chứa N số thực $X[1], X[2], \dots, X[N]$. Gọi vector con của X là vector mà phần tử của nó là các phần tử liên tiếp trong X . Tổng của một vector được tính là tổng các phần tử của vector đó. Tính tổng lớn nhất trong các vector con, tức tìm $L, U \in 1..N$ để tổng $X[i], i \in L..U$ là lớn nhất.

Để đơn giản, vector con có tổng lớn nhất được gọi tắt là vector lớn nhất. Ví dụ, nếu vector đầu vào có dạng:

31	-41	59	26	53	58	97	-93	3	84
		↑				↑			
		3				7			

Lúc này, kết quả của bài toán là tổng của vector $X[3..7]$. Bài toán rất đơn giản khi tất cả các số là số dương - khi đó kết quả chính là bản thân vector X . Vấn đề sẽ phức tạp hơn khi có thêm các số âm. Chúng ta nhận xét rằng nếu tất cả đều là số âm thì kết quả là bằng 0 (đó chính là tổng của vector rỗng).

- Thuật toán 1: Một chương trình có thể viết ngay được là xét tất cả các cặp số nguyên L và U thỏa mãn $1 \leq L \leq U \leq N$; đối với mỗi cặp như vậy ta tính tổng của vector con $X[L..U]$ và so sánh tổng này với giá trị lớn nhất hiện có. Ta có mã giả cho thuật toán 1 như sau:

```
MaxSoFar = 0;
for(int L=1; L<=N; L++)
    for(int U=L; U<=N; U++)
    {
        Sum = 0;
        for(int I=L; I<=U; I++)
```

```

Sum = Sum + X[I];
    /* Sum chứa tổng của X[L..U] */
MaxSoFar = Max(MaxSoFar, Sum);
}

```

Chương trình này ngắn và dễ hiểu, tuy nhiên điều không may là nó chạy rất chậm. Độ phức tạp của chương trình là $O(n^3)$.

- Thuật toán 2: Đối với thuật toán 1, đa số người lập trình cho rằng có thể viết chương trình chạy nhanh hơn. Có hai cách như vậy. Các cách này đều có độ phức tạp $O(n^2)$. Thuật toán thứ nhất tính nhanh các tổng của vector con bằng cách sử dụng hệ thức: Tổng của $X[L..U] = \text{Tổng của } X[L..U-1] + X[U]$; Ta có thuật toán 2 như sau:

```

MaxSoFar = 0.0;
for(int L=1; L<=N; L++)
{
    Sum = 0.0;
    for(int U=L; U<=N; U++)
    {
        Sum = Sum + X[U];
        /* Sum chứa tổng của X[L..U] */
        MaxSoFar = Max(MaxSoFar, Sum);
    }
}

```

Các lệnh trong vòng lặp thứ nhất thực hiện N lần. Với mỗi lần thực hiện các lệnh trong vòng lặp thứ nhất, các lệnh trong vòng lặp thứ hai thực hiện nhiều nhất là N lần. Vậy ta có độ phức tạp là $O(n^2)$.

- Thuật toán 3: Thuật toán chia để trị: "Để giải bài toán kích thước N, chúng ta giải một cách đệ quy hai bài toán con kích thước khoảng $N/2$, kết hợp lời giải của chúng để tạo ra lời giải của toàn bộ bài toán".

Trong trường hợp này bài toán của ta là xử lý vector độ dài N, do đó một cách tự nhiên là chia vector này thành hai vector con có độ dài gần bằng nhau. Chúng ta gọi hai vector này là A và B.

A	B
---	---

Sau đó, bằng đệ quy chúng ta tìm vector con lớn nhất trong A và B gọi là M_A và M_B .

	M_A			M_B	
--	-------	--	--	-------	--

Đề ý rằng kết quả bài toán là giá trị lớn nhất trong hai tổng của vector M_A và M_B . Kết quả của bài toán có thể là tổng của vector M_C chứa đồng thời các thành phần của A và B. Ta gọi là vector con như vậy là vector vượt biên.

	M_C	
--	-------	--

Như vậy thuật toán chỉ để trị sẽ tính M_A , M_B bằng đệ quy và tính M_C bằng phương pháp khác, kết quả bài toán này là giá trị lớn nhất trong ba tổng của ba vector này. Các mô tả trên là gần đủ để viết chương trình. Chúng ta còn phải mô tả cách quản lý các vector nhỏ và cách tính vector M_C . Phần đầu tiên rất dễ: Đối với vector chỉ chứa một phần tử, vector con lớn nhất hoặc là chính nó hoặc là vector rỗng trong trường hợp phần tử của vector đó là số âm, và vector con lớn nhất của vector rỗng cũng là vector rỗng. Để tính M_C , chúng ta nhận xét rằng thành phần của vector M_C nằm trong vector A là vector con lớn nhất trong tất cả các vector con của vector A, bắt đầu từ biên của A và B. Tương tự như thế đối với thành phần của vector M_C nằm trong vector B. Kết hợp tất cả các yếu tố này, chúng ta có thuật toán 3, được gọi bởi lệnh:

```

Answer = MaxSum(1,N);
int MaxSum(L,U)
{
    if L > U
        return 0; /* vector rỗng */
    if L = U
        return Max(0.0, X[L]); /* vector một phần tử */
    M = (L+U) / 2;
    /* A là vector X[L..M], B là vector X[M+1..U] */
    /* Tìm giá trị lớn nhất của tổng các thành phần bên trái (trong vector A) của vector vượt biên */
    Sum = 0;
    MaxToLeft = 0;
    for(int I=M; I>=L; I--)
    {
        Sum = Sum + X[I];
        MaxToLeft = Max(MaxToLeft, Sum);
    }
    /* Tìm giá trị lớn nhất của tổng các thành phần bên phải (trong vector B) của vector vượt biên */
    Sum = 0;
    MaxToRight = 0;
    for(int I=M+1; I<=U; I++)

```

```

    {
        Sum = Sum + X[I];
        MaxToRight = Max(MaxToRight, Sum);
    }
    MaxCrossing = MaxToLeft + MaxToRight;
    MaxInA = MaxSum(L,M);
    MaxInB = MaxSum(M+1,U);
    return Max(MaxCrossing, MaxInA, MaxInB);
}

```

Thuật toán thực hiện $O(n)$ công việc trong mỗi mức đệ quy, và có tất cả là $O(\log n)$ mức đệ quy. Nên chương trình này giải quyết bài toán với độ phức tạp $O(n \log n)$.

- Thuật toán 4: Thuật toán quét: Giả sử rằng chúng ta đã giải bài toán cho vector $X[1..I-1]$; làm thế nào để mở rộng kết quả này cho bài toán với vector $X[1..I]$? Lý luận tương tự như trong thuật toán "chia để trị": tổng lớn nhất trong vector $X[1..I-1]$ (gọi là MaxSoFar), hoặc tổng lớn nhất trong tất cả các tổng của vector con kết thúc tại I (gọi là MaxEndingHere).

	MaxSoFar		MaxEndingHere
--	----------	--	---------------

Nếu chúng ta tính MaxEndingHere bằng cách tương tự như trong thuật toán 3, thì ta chỉ có một thuật toán bình phương (có độ phức tạp $O(n^2)$). Để làm nhanh hơn, chúng ta nhận xét điều như sau: vector con lớn nhất kết thúc tại vị trí I là vector con lớn nhất kết thúc tại vị trí I-1 được bổ sung thêm phần tử $X[I]$ ở cuối hoặc là vector rỗng trong trường hợp tổng của vector nhận được là số âm. Ta có thuật toán 4 như sau:

```

MaxSoFar = 0;
MaxEndingHere = 0;
for(int I=1; I<=N; I++)
{
    /* Bất biến: MaxEndingHere và MaxSoFar là đúng đối với X[1..I-1] */
    MaxEndingHere = Max(MaxEndingHere + X[I], 0);
    MaxSoFar = Max(MaxSoFar, MaxEndingHere);
}

```

Chương trình này có thời gian chạy là $O(n)$. Vì vậy thuật toán này được gọi là thuật toán tuyến tính.

Như vậy, khi xây dựng ứng dụng, việc sử dụng các thuật toán phù hợp làm giảm thời gian chạy chương trình một cách đáng kể.

4.6.1.2 Lựa chọn cấu trúc dữ liệu

Song song với thuật toán, việc chọn lựa cấu trúc dữ liệu ảnh hưởng lớn đến hiệu suất chương trình và nó tác động đến bản thân thuật toán bởi cấu trúc dữ liệu gắn bó mật thiết với thuật toán.

Việc chọn đúng đắn cấu trúc dữ liệu làm giảm không gian bộ nhớ, giảm thời gian chạy, tăng tính chuyên đặc và dễ bảo trì, đặc biệt là các cấu trúc dữ liệu cao cấp, mặc dầu chúng không thường được dùng nhưng khi cần thiết thì không thể thiếu chúng được [1].

Ta sẽ gặp lại việc chọn lựa cấu trúc dữ liệu trong phần 4.6.2. ở sau, trong phần xét về không gian bộ nhớ chương trình.

4.6.1.3 Tinh chế mã

Thông thường, để tăng tính hiệu quả của chương trình, người ta thường bàn về các tiếp cận bậc cao như: định nghĩa bài toán, cấu trúc hệ thống, thiết kế thuật toán và chọn cấu trúc dữ liệu.

Tuy nhiên, các tiếp cận bậc thấp như tinh chế mã mà nó thường được thực hiện ở những phần tốn kém của chương trình để cải tiến hiệu suất. Mặc dù đây là phương pháp không phải lúc nào cũng cần thiết nhưng đôi lúc nó tạo ra khác biệt lớn trong hiệu suất của chương trình.

Các phương pháp thường dùng của tinh chế mã.

- Tính trước các giá trị,
- Thay tương đương,
- Dùng biến trung gian thích hợp, không tính lại các hằng trong vòng lặp.

Bài toán: Cho một chuỗi gồm 1 triệu ký tự. Hãy phân loại mỗi ký tự theo 4 kiểu sau: kiểu chữ in, kiểu chữ hoa, kiểu số hay là các kiểu "khác".

- Lờ giải mà ta thường làm: là thực hiện các so sánh đối với mỗi ký tự. Như vậy, trong bảng mã ASCII, để xác định mỗi ký tự thuộc loại nào phải mất rất nhiều lần so sánh; và đây chính là điểm "nóng" của chương trình.

- Tinh chế mã: Ở đây, nếu ta xem mỗi ký tự như là một chỉ số của mảng mà thành phần của nó là các kiểu ký tự. Như vậy, kiểu ký tự C là mảng [C] và để xác định kiểu của một ký tự, ta chỉ cần truy cập đến một mảng đơn giản thay vì phải thực hiện các chuỗi so sánh phức tạp.

Như vậy, khi thực hiện tinh chế mã, cần xác định ở đâu là điểm "nóng" của chương trình và hãy tập trung vào điểm nóng. Hơn nữa, ta đã biết rất nhiều phương pháp để cải tiến hiệu suất của chương trình và hãy dùng đến phương pháp này sau cùng và đôi khi, phương pháp này còn được dùng để làm giảm không gian chiếm bởi chương trình.

4.6.2. Không gian bộ nhớ

Trong những ngày đầu của kỹ thuật máy tính, các nhà lập trình bị hạn chế bởi những bộ nhớ nhỏ; Ngày nay vấn đề này không còn là điểm "nóng" nữa. Tuy vậy, khi thiết kế chương trình không phải lúc nào ta cũng có đủ bộ nhớ để sử dụng bởi nhiều lý do khác nhau.

4.6.2.1 Không gian dữ liệu

Nguyên tắc để làm giảm không gian lưu trữ dữ liệu.

- Đảm bảo tính đơn giản,
 - Trong một số trường hợp dùng lưu trữ, hãy tính lại khi cần thiết,
 - Đặc biệt, việc nghiên cứu kỹ các cấu trúc dữ liệu, (thường là cấu trúc dữ liệu thưa thớt) sẽ làm giảm nhiều không gian cần thiết để lưu trữ các thông tin cho trước,
 - Nén dữ liệu sau đó giải nén khi dùng,
 - Sử dụng các nguyên tắc cấp phát bộ nhớ: chẳng hạn như cấp phát bộ nhớ động,
- ...

Xét bài toán: Trên một bản đồ chứa 2.000 điểm (bảng đồ quân sự) được đánh số từ 1 đến 2.000. Một vị trí trên bảng đồ được xác định bằng cặp tọa độ (x,y) với x là số nguyên nằm trong khoảng 1...200; y là số nguyên nằm trong khoảng 1...150. Chương trình dùng cặp (x,y) để xác định điểm nào (nếu có) được chọn trong 2000 điểm đã cho.

- Không gian lưu trữ 1: Một cách hiển nhiên để lưu trữ bản đồ trên là dùng mảng 2 chiều 200 x 150 số nguyên; ứng với mỗi x.y sẽ chứa một giá trị trong khoảng từ 1...2.000 hay sẽ chứa giá trị 0.

Bảng 4-1 Minh họa lưu trữ mảng 2 chiều 200x150

...									
6	538								
5				965					
4						1121			
3	17								
2		98						162	
1	1	2	3	4	5	6	7	8	...

Việc dùng bảng này, thời gian truy cập nhanh; nhưng nó chiếm đến $200 \times 150 = 30.000$ ô nhớ; và giả sử để lưu trữ dữ liệu (ở đây là 1...2.000) cần 2 byte thì ta cần 60.000B bộ nhớ.

Tuy nhiên, trong mảng trên thì đa số là giá trị 0 (giá trị không dùng). Do vậy, nếu ta dùng cái nào chỉ lưu trữ các giá trị cần thiết (ở đây là 2.000 giá trị) thì việc chiếm bộ nhớ sẽ giảm đáng kể.

- Không gian lưu trữ 2: Thay vì dùng một mảng 2 chiều ở trên, ta sử dụng 3 mảng 1 chiều như sau:

Value	17	538	1080	98	15	1800	...	437	832
-------	----	-----	------	----	----	------	-----	-----	-----

Row	3	6	127	2	139	12	...	112	68
-----	---	---	-----	---	-----	----	-----	-----	----

FirstCol	1	4	6	6	...	1999	2001
----------	---	---	---	---	-----	------	------

1
2
3
4
199
200
201

Mảng value dùng để chứa các giá trị (ở đây là 1...2.000) theo từng cột, như vậy cần 2.000 ô nhớ; Mảng Row là mảng để chứa hàng tương ứng với giá trị ở mảng value; như thế cần 2.000 ô nhớ. Mảng FirstCol là mảng để chứa số cột hiện có - tương ứng với giá trị ở mảng value; như thế cần 200 ô nhớ; thêm 1 ô nhớ đầu tiên để đánh dấu, nên tổng cộng cần 201 ô.

Các điểm trong cột I được biểu diễn bằng các phần tử trong mảng Row và Value giữa các vị trí FirstinCol [I] và FirstinCol [I+1]-1. Ở đây, FirstinCol [201] được xác định (mặc dù chúng ta chỉ có 200 cột) để biểu thức I+1 hợp lệ. Theo hình trên, chúng ta có 3 điểm trong cột thứ nhất điểm 17 ở vị trí (1,3), điểm 538 ở vị trí (1,6), điểm 1053 ở vị trí (1,127). Có hai điểm trong cột 2 (điểm 98 ở vị trí (2,2), điểm 15 ở vị trí (2,139)), cột 3 không có điểm nào, và có 2 điểm trong cột 200. Để xác định điểm nào (trong số 2000 điểm) được lưu giữ tại vị trí (I,J), chúng ta dùng giải mã sau:

```
for(int K= FirstinCol[I]; K<= FirstinCol[I+1]; K++)
```

```
    If (Row[K] = J)
```

```
        /* Tìm thấy ở vị trí */
```

```
        return Value[K]
```

```
        /* Không có điểm nào tại vị trí (I,J) */
```

```
    return 0;
```

Phương pháp này dùng ít không gian hơn nhiều so với phương pháp trước đó. Ở đây chúng ta dùng hai mảng 2000 phần tử và một mảng 201 phần tử (như vậy tất cả là 4201 từ 16 bit thay vì 30000 từ trong phương pháp trước đó). Mặc dù nó hơi chậm hơn (một lần truy nhập phải mất 150 lần so sánh trong trường hợp tồi nhất, nhưng trung bình chỉ cần 6 lần), nhưng chương trình chạy tốt và người dùng không gặp phải vấn đề gì.

Lời giải này minh họa một số điểm tổng quát về cấu trúc dữ liệu. Vấn đề ở đây rất cổ điển: việc biểu diễn thừa thớt (tức là mảng trong đó hầu hết các phần tử có cùng một giá trị, thường là giá trị 0). Lời giải trên có ý tưởng rất đơn giản và dễ cài đặt bằng mảng. Chú ý rằng ở đây không có mảng LastinCol đi cùng với mảng FirstinCol, bởi vì chúng ta sử dụng một điều là điểm cuối cùng trong một cột chính là điểm đứng trước điểm đầu tiên của

cột tiếp theo. Đây là một ví dụ tầm thường của nguyên tắc tính lại thay vì lưu trữ. Tương tự, chúng ta không có mảng Col đi cùng với mảng Row vì chúng ta chỉ truy nhập mảng Row thông qua mảng FirstInCol, do đó chúng ta luôn biết cột hiện thời là cột nào.

Nhiều kỹ thuật sử dụng cấu trúc dữ liệu khác cũng có thể làm giảm không gian. Trong thực tế, chúng ta tiết kiệm không gian bằng cách thay thế mảng 3 chiều thành mảng 2 chiều. Nếu chúng ta sử dụng một khoá được lưu trữ như là chỉ số của mảng, thì chúng ta không cần phải lưu trữ bản thân khoá này; thay vào đó, chúng ta chỉ cần lưu trữ các thuộc tính thích hợp của nó, chẳng hạn như số lần xuất hiện của nó. Thêm vào đó, các ứng dụng của kỹ thuật đánh chỉ số bằng khoá đã được vận dụng. Trong ví dụ về ma trận thừa thớt trên đây, việc đánh chỉ số bằng khoá thông qua mảng FirstInCol cho phép chúng ta giải quyết vấn đề mà không cần dùng đến mảng Col.

4.6.2.2 Không gian chương trình

Trong một số chương trình, đôi lúc thì kích thước của chính bản thân nó là vấn đề. Hãy định nghĩa các chương trình con hay sử dụng các bộ thông dịch chuyên dụng để làm cho chương trình đơn giản, trong sáng hơn làm cho nó rõ ràng hơn và dễ bảo trì.

4.6.3. Lựa chọn hệ thống và phần cứng

Nên lựa chọn các ngôn ngữ lập trình phù hợp với ứng dụng của bạn. Đôi lúc cần hãy thay thế các chương trình con viết trên ngôn ngữ khác để có tốc độ lớn hơn.

Trong xu thế phát triển của phần cứng hiện nay, cần phải tận dụng thế mạnh của phần cứng để có hiệu suất của chương trình cao, mặc dù điều này làm hạn chế tính phổ cập của nó nhưng hiện nay yêu cầu về phần cứng cao là chấp nhận được.

4.7 Công cụ trợ giúp và phân loại

Mọi người đều biết sự vất vả và các khó khăn của việc phát triển phần mềm. Việc cung cấp cho các kỹ sư phần mềm các công cụ trợ giúp, nó cung cấp khả năng tự động phát triển chương trình là rất hiệu quả về mặt chi phí. Trong thời đại nhiều máy móc như hiện nay, những trợ giúp phát triển thường có sẵn: môi trường CASE, sinh mã và những trợ giúp kiểm tra như gỡ rối, các chương trình biên dịch tăng, môi trường thực hiện theo kiểu cửa sổ và cả tốc độ phát triển mã hoạt động. Bất kỳ ngôn ngữ nào đều có sự trợ giúp tự động,

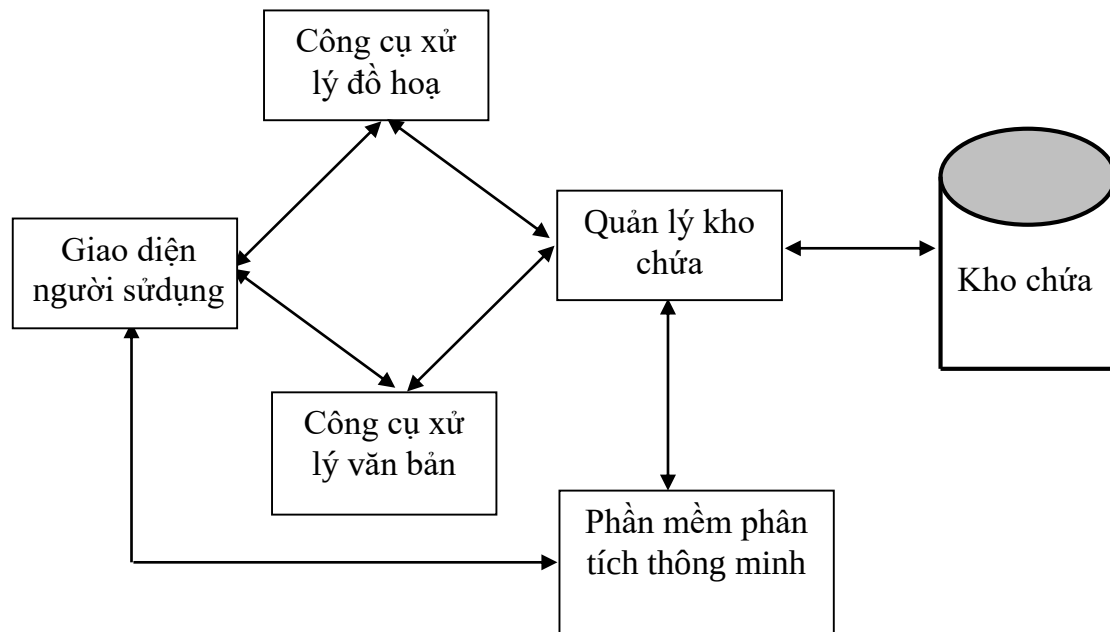
Như thế, một môi trường phát triển phần mềm là một bộ các công cụ phần cứng và phần mềm chúng được kết lại để trợ giúp cho việc sản sinh ra một hệ thống phần mềm trong một miền ứng dụng chuyên biệt.

4.7.1. Công cụ CASE

CASE có các loại ICASE, Upper CASE và Lower CASE. ICASE nghĩa là "Integrated" CASE hay là CASE tích hợp, "Upper" nghĩa là công cụ ý tưởng hay chỉ là thiết kế logic, "Lower" nghĩa là công cụ chỉ hỗ trợ lập trình. Trong những công cụ trên, tài liệu này tập trung vào các công cụ CASE hỗ trợ tối thiểu một giai đoạn phân tích và có thể

hỗ trợ các giai đoạn khác; chúng được gọi chung là "CASE". Chúng ta cũng xác định những giai đoạn hiện nay đang được hỗ trợ (dĩ nhiên là các giai đoạn này có thể thay đổi khi phát hành).

Môi trường CASE chuẩn bao gồm một kho chứa, các công cụ đồ hoạ, phần mềm soạn thảo văn bản, phần mềm giao diện kho chứa, phần mềm đánh giá, và giao diện người sử dụng.



Hình 4.1 Môi trường CASE

Một kho chứa là một từ điển dữ liệu hỗ trợ định nghĩa về các kiểu đối tượng khác nhau và quan hệ giữa các đối tượng đó. Các công cụ đồ hoạ hỗ trợ cho việc phát triển dạng sơ đồ và đánh giá sự hoàn chỉnh của sơ đồ dựa vào các luật đã được xác định trước. Phần mềm văn bản cho phép định dạng tên, nội dung, và chi tiết các phần tử trong kho chứa. Phần mềm giao diện là bộ biên dịch xác định dạng dữ liệu được dùng (đồ hoạ hoặc văn bản). Phần mềm đánh giá là trí tuệ của CASE. Phần mềm này phân tích các đầu vào của sơ đồ hoặc kho chứa và xác định xem chúng có cú pháp hoàn chỉnh hay không (ví dụ có thoả mãn các định nghĩa của kiểu dữ liệu thành phần không), và chúng có tương thích với các đối tượng đang tồn tại khác trong ứng dụng hay không. Giao diện người dùng cung cấp các màn hình và các báo cáo để xử lý tương tác và gián tiếp.

CASE lý tưởng phải cung cấp đầy đủ hỗ trợ tự động cho toàn bộ chu kỳ tồn tại của dự án, bắt đầu bằng việc phân tích ở mức xí nghiệp, duy trì công việc và kết thúc. CASE lý tưởng do đó trở thành tiêu điểm cho mọi công việc nằm trong công nghệ phần mềm, và công việc của kỹ sư hệ thống tập trung vào khía cạnh logic của thiết kế. Công cụ CASE lý tưởng cũng có thể cung cấp về mặt kỹ thuật, dữ liệu và thực hiện thiết kế của tổ chức, lập dự án và kiểm tra nhóm làm việc trong các ứng dụng, chuẩn hoá dữ liệu, sơ đồ xây dựng

cơ sở dữ liệu, xây dựng mã đơn giản bằng ngôn ngữ người sử dụng lựa chọn, tự động kiểm tra mã sinh ra để phòng các lỗi logic và đánh giá sự hoàn chỉnh và đúng đắn trong quá trình làm việc một cách thông minh. Công cụ CASE thực hiện sự tiên tiến phải nhận ra các thành phần đã có trong kho chứa sử dụng lại việc phân tích, thiết kế và mã nguồn.

Kho chứa của CASE xác định đồng thời cái gì được hỗ trợ và trong phạm vi nào đó, có thể là hỗ trợ được bao nhiêu. Kho chứa là một siêu từ điển có thể nhập và lưu trữ siêu dữ liệu. Chẳng hạn, một phần tử dữ liệu trong một ứng dụng là dữ liệu, và các thuộc tính của nó tạo thành dữ liệu trao đổi có thể lưu trữ trong từ điển. Các thuộc tính của một phần tử bao gồm kiểu dữ liệu, kích thước, dung lượng, tần suất thay đổi và tiêu chuẩn soạn thảo. Một kho chứa CASE hoạt động như là một hệ quản trị cơ sở dữ liệu phục vụ cho hoạt động kỹ sư, cung cấp khả năng mở rộng lưu giữ dữ liệu trao đổi, và duy trì tất cả các thành phần và mối liên hệ qua lại giữa chúng.

Sự thông minh trong CASE thể hiện ở hai dạng chính: thông minh của giao diện và thông minh của bản thân sản phẩm CASE. Giao diện phải cung cấp cả kiểu thao tác cho người mới tập sự và cho người đã có kinh nghiệm. Nó phải cho phép khả năng một công việc được lưu giữ và tiếp tục lại. Công cụ này có thể được thay đổi theo yêu cầu cá nhân của người sử dụng. Ví dụ, nếu tôi muốn in màu vàng trên nền xanh, tôi sẽ gọi một lược đồ dòng dữ liệu, và tôi phải được phép thay đổi giá trị ngầm định để dùng các thuật ngữ và dạng của mình.

Các dạng biến đổi các giá trị đưa vào phải được phản ánh thông qua các tập sơ đồ. Điều này có nghĩa là nếu người dùng đưa các thực thể và các thuộc tính vào một kho chứa, khi anh ta di chuyển để triển khai một lược đồ đồ hoạ quan hệ thực thể, thông tin trong kho chứa phải được phản ánh trên lược đồ.

Trí tuệ của sản phẩm CASE bao gồm việc phân tích bên trong và giữa các kiểu lược đồ và đầu vào kho chứa. Trong trường hợp lý tưởng, yêu cầu của ứng dụng A nếu xung đột với mục tiêu của xí nghiệp Z, sẽ phải được bật cờ báo để xem ý kiến của người quản lý.

CASE lý tưởng phải cho phép người sử dụng phân tách và tích hợp các ứng dụng khác nhau một cách dễ dàng. Ví dụ, một công ty có thể muốn tư liệu hoá tất cả ứng dụng đã sử dụng và bắt đầu quản lý điện tử chúng. Khi người sử dụng xác định một ứng dụng mới có thể muốn tích hợp nó với một ứng dụng cũ, họ phải được phép để tạo ra một định nghĩa thứ ba để làm sáng tỏ sự chồng chéo, dư thừa, không chắc chắn và các vấn đề khác mà cặp tích hợp gặp phải.

Theo quy luật 40-20-40 của việc triển khai hệ thống, 40% thời gian của dự án được dùng để phân tích và thiết kế, 20% dành cho lập trình và 40% còn lại để chạy thử. Hướng hiện tại của các nhà bán sản phẩm là giảm bớt mã, do đó giảm đi 20% thời gian phát triển. Nhưng CASE lý tưởng sẽ còn có thể giảm được 40% thời gian chạy thử. Các công cụ kiểm

tra CASE không cần thiết bằng các vấn đề khác (chẳng hạn cho sản phẩm làm việc tự do lỗi).

Trong một số sản phẩm ở thập kỷ 90, các nhà sản xuất sẽ cung cấp hỗ trợ kiểm tra trong các môi trường CASE của họ. Một cách lý tưởng, những hỗ trợ như vậy sẽ bao gồm cả thử nghiệm hộp đen và hộp trắng cùng với can thiệp của con người, nhưng không bắt buộc. Thử nghiệm hộp đen dùng cho kiểm tra sự kiểm tra chính xác của đầu ra dựa trên đầu vào; trong khi đó thử nghiệm hộp trắng dùng cho các sơ đồ logic trong chương trình. Phần mềm thông minh sẽ phân tích kiểu thực hiện và xác định chiến lược thử nghiệm thích hợp nhất. Thêm vào đó, nó sẽ phát triển dữ liệu thử nghiệm dựa trên các yêu cầu logic, dẫn dắt thử nghiệm và duy trì các kết quả thử nghiệm. Các kết quả thử nghiệm sẽ được tổng hợp thông qua chạy thử, các giai đoạn thử nghiệm, phiên bản phần mềm, và ngay cả các môi trường phân cứng. Khi lỗi được tìm ra, phần mềm này sẽ quay lại chỉnh tìm nguồn gốc lỗi - có khả năng là thông qua các module.

Khi phần mềm tạo nên lỗi phải có khả năng sửa chúng; nhưng nếu gây ra lỗi là một người hay là một lỗi logic, phần mềm phải thông báo cho kỹ sư hệ thống yêu cầu sửa lỗi.

Các sản phẩm trong tương lai này sẽ thực sự khắc phục 40% thời gian của dự án do chúng đủ thông minh để xác định các thành phần có thể sử dụng lại được của ứng dụng. Việc sử dụng lại thiết kế sẽ tiết kiệm được nhiều nhất nhưng cũng là việc khó khăn nhất. Đầu tiên, các module mã sẽ được sử dụng lại, sau đó đến lượt thiết kế, cuối cùng là phân tích logic cũng được dùng lại. Việc tìm ra mã có thể sử dụng lại được các công cụ CASE cung cấp vào giữa những năm 90, các phần khác sẽ được cung cấp vào thế kỷ này.

Việc miêu tả tính chất của CASE lý tưởng tập trung vào việc xem CASE có thể làm gì tốt hơn hiện tại. Với vấn đề này, chúng ta sẽ nghiên cứu CASE mỗi khi nó hỗ trợ một phương pháp luận và phát triển trong các chương sau. Mặc dù CASE và trí tuệ nhân tạo đang còn rất non trẻ, nhưng sự phát triển được mô tả ở trên hiện tại đã có thể thực hiện được với hiện trạng kỹ thuật công nghệ thế giới. Kho chứa CASE sẽ trở thành một trung tâm cho mọi công việc hoạt động trong các tổ chức hệ thống thông tin. Trí tuệ của CASE sẽ là chính trí tuệ của con người.

Các cấp độ tổ hợp CASE có thể phân loại. Tại mức thấp nhất của phổ tích hợp là các công cụ đơn. Khi các công cụ riêng lẻ cung cấp các tiện ích về truyền dữ liệu mức độ tích hợp đã có đôi chút cải thiện. Các công cụ như vậy sản xuất các đầu ra theo dạng chuẩn có thể tương thích với các công cụ khác có thể đọc format này. Trong một số trường hợp, các người xây dựng các công cụ CASE hoàn chỉnh làm việc với nhau thông qua "cầu nối" giữa các tools. Single- source intergration xuất hiện khi các nhà sản xuất công cụ CASE đơn lẻ tích hợp một số lượng công cụ khác nhau và bán chúng như là một gói phần mềm.

Mức độ tích hợp cao nhất có (IPSE) Intergrated Project Support Environment. Các chuẩn cho mỗi khối cấu thành được mô tả trên được tạo. Các nhà sản xuất công cụ CASE

sử dụng các chuẩn IPSE này để xây dựng các công cụ tương thích với các nhà sản xuất khác theo chuẩn.

4.7.2. Phân loại các công cụ CASE

4.7.2.1 Các công cụ lập kế hoạch hệ thống tác nghiệp

Bằng cách mô hình hoá các yêu cầu thông tin chiến lược của tổ chức, công cụ lập kế hoạch hệ thống tác nghiệp (Business System Planning Tools) cung cấp một siêu mô hình mà từ đó hệ thống thông tin đặc trưng sẽ được suy ra. Các thông tin tác nghiệp được mô hình hoá khi nó chuyển từ các thực thể được tổ chức khác nhau trong công ty. Mục đích chính của các công cụ trong phân loại là giúp hiểu biết được thông tin di chuyển giữa các đơn vị tổ chức như thế nào.

Các công cụ như vậy cung cấp nội dung quan trọng khi các chiến lược hệ thống thông tin được cấu trúc và khi các hệ thống và phương pháp hiện tại không hợp với yêu cầu của tổ chức.

4.7.2.2 Các công cụ quản lý dự án

Nhiều nhà quản lý dự án phần mềm đang tiếp tục đánh giá, điều khiển và theo dõi các dự án phần mềm theo cách trước đây đã làm từ 1950. Mĩa mai thay, có một dãy rộng các công cụ quản lý dự án CASE có tác động sâu sắc lên chất lượng của quản lý dự án cho các cố gắng phát triển phần mềm cỡ lớn và nhỏ.

Hiện nay, phần lớn các công cụ quản lý dự án CASE định hướng vào một phần đặc trưng của quản lý dự án hơn là cung cấp hỗ trợ toàn bộ cho hoạt động quản lý. Bằng cách sử dụng một tập hợp công cụ CASE có chọn lọc, quản trị dự án có thể tạo ra các đánh giá hiệu quả về giá thành, nguồn lực, và thời gian của dự án phần mềm, xác định các cấu trúc công việc và thời biểu làm việc đồng thời theo dõi dự án. Hơn nữa người quản lý có thể sử dụng các công cụ để thu thập các metrics mà cuối cùng cung cấp các chỉ định về chất lượng và hiệu quả phát triển phần mềm.

Công cụ lập kế hoạch dự án: Các công cụ trong lớp này tập trung vào hai mảng chính: định lượng giá, nguồn lực dự án và lập biểu dự án. Các công cụ định lượng giá thành cho phép quản trị dự án ước lượng cỡ của dự án bằng cách dùng các độ đo gián tiếp (số dòng mã và số các chức năng) và mô tả toàn bộ các đặc tính dự án (ví dụ độ phức tạp, kinh nghiệm của đội ngũ). Các công cụ này tiếp theo ước lượng nguồn lực, thời gian dự án và gợi ý số lượng người. Nhiều công cụ trong số đó cho phép mô phỏng tình huống để quản trị có thể đã định thời gian hoàn thành và kiểm tra giá thành và khả năng thực hiện.

Các công cụ lập biểu dự án cho phép nhà quản lý xác định mọi nhiệm vụ, tạo mạng các công việc, biểu diễn sự phụ thuộc công việc. Phần lớn các công cụ sử dụng phương pháp lập đoạn găng để xác định thời lượng hoàn thành dự án.

Các công cụ theo dõi các yêu cầu: Khi hệ thống được phát triển lớn dần thì rất có nguy cơ rơi vào tình trạng đồ vớ, hệ thống đã hoàn thiện không đáp ứng hoàn toàn các yêu

cầu của khách hàng. Mục đích của các công cụ theo dõi yêu cầu là cung cấp một cách tiếp cận hệ thống để phân tách các yêu cầu, bắt đầu với các quy trình gọi thầu -RFP (Request For Proposal)- của khách hàng hoặc các đặc tả.

Công cụ theo dõi yêu cầu đặc trưng bao gồm các định lượng text giao tác người-máy, cùng với hệ thống quản lý cơ sở dữ liệu lưu trữ và phân loại các yêu cầu hệ thống mà chuyển đến từ các RFP hoặc các đặc tả. Phân tích viên phân loại các yêu cầu được biểu diễn bởi các câu và đưa chúng vào cơ sở dữ liệu.

Các công cụ quản lý và độ đo: Các độ đo phần mềm cải thiện khả năng của nhà quản lý để điều khiển và phối hợp quá trình xử lý sản xuất phần mềm và khả năng của các cộng tác viên để cải thiện chất lượng phần mềm. Các công cụ đánh giá hoặc độ đo hiện nay tập trung vào các đặc trưng xử lý và chế tạo. Các công cụ định hướng quản lý thu thập các độ đo đặc trưng dự án. Các công cụ định lượng kỹ thuật xác định các độ đo kỹ thuật. Cung cấp các điểm quan trọng nhất về chất lượng thiết kế hoặc mã.

Các công cụ quản lý hỗ trợ cho nhà quản lý hệ thống thông tin cho phép ưu tiên các dự án có sự cạnh tranh về các nguồn tài nguyên hữu hạn. Do sử dụng các yêu cầu, độ ưu tiên, các ràng buộc được đặt trong các tổ chức và các lỗi, rủi ro của kỹ thuật và nghiệp vụ, những công cụ này sử dụng các kiến thức chuyên gia để đưa ra các gợi ý quyết định cho nhà quản lý.

4.7.2.3 Các công cụ hỗ trợ

Các công cụ tư liệu hoá cho phép cán bộ phát triển ứng dụng tự động hoá cập nhật tài liệu và in các báo cáo về ứng dụng.

4.7.2.4 Các công cụ phân tích và thiết kế

Các công cụ phân tích và thiết kế cho phép các kỹ sư phần mềm tạo các mô hình của hệ thống. Nó bao gồm biểu diễn cho dữ liệu, luồng điều khiển, nội dung dữ liệu (thông qua các định nghĩa từ điển các yêu cầu), quá trình xử lý, các đặc tả điều khiển, và các biểu diễn mô hình hoá khác.

Các công cụ phân tích và thiết kế hỗ trợ cả việc tạo mô hình cũng như đánh giá chất lượng. Bằng quá trình kiểm tra tính chắc chắn và giá trị của mô hình, các công cụ phân tích và thiết kế cung cấp cho công nghệ phần mềm khả năng giảm tối thiểu các lỗi có khả năng lan truyền tới chương trình ứng dụng.

Công cụ SA/SD: Phần lớn các công cụ phân tích và thiết kế sử dụng phương pháp phân tích và thiết kế cấu trúc. Nó cho phép tạo các mô hình của hệ thống phức tạp dần, bắt đầu từ mức độ yêu cầu và kết thúc với sơ đồ kiến trúc.

Công cụ PRO/SIM: Các công ty tạo mẫu và mô phỏng (Prototyping and simulation) cho khả năng dự đoán trước đáng điệu của hệ thống. Mặt khác, nó cho phép khách hàng trong thời gian ngắn nhất có thể quan sát được mô hình. Nhiều công cụ dạng này cho phép sản sinh mã.

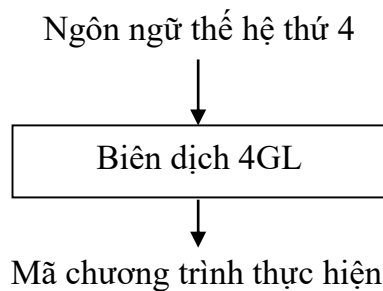
Các công cụ phát triển và thiết kế giao diện: Các nghiên cứu công nghiệp cho thấy rằng 50-80% mã của các ứng dụng tương tác là dành cho quản lý giao diện. Các công cụ phát triển và thiết kế giao diện thực sự là tập hợp các công cụ tạo các đơn vị chương trình như menu, button, windows, ...Tuy nhiên, các công cụ trên đang được thay thế bởi các công cụ tạo mẫu giao diện cho phép tạo các màn hình theo chuẩn một cách nhanh chóng.

4.7.2.5 Các công cụ lập trình

Các công cụ lập trình bao gồm bộ dịch, soạn thảo, gỡ lỗi cho phép dùng phần lớn các ngôn ngữ lập trình truyền thống. Các công cụ này có liên quan nhiều tới môi trường lập trình hướng đối tượng, ngôn ngữ thế hệ bốn, sản sinh chương trình.

Các công cụ mã hoá truyền thống: Đã có thời các công cụ mã hoá quy ước là: Compiler, Editor, Debugger. Pressman đã nói về vấn đề này như sau: "Khi trong tay bạn chỉ có cái búa, thì mọi vấn đề sẽ giống như cái đinh". Hầu như suốt 30 năm, lập trình viên chỉ có các công cụ trên trong tay nên mọi vấn đề về công nghệ phần mềm đều quy về vấn đề mã hoá.

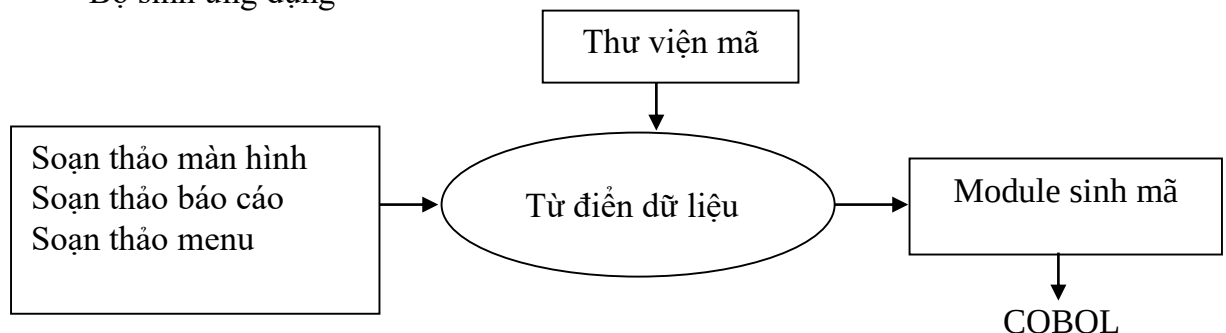
Các công cụ mã hoá thế hệ 4: Hệ thống hỏi đáp cơ sở dữ liệu, sinh mã và ngôn ngữ thế hệ 4 đã làm thay đổi cách lập trình



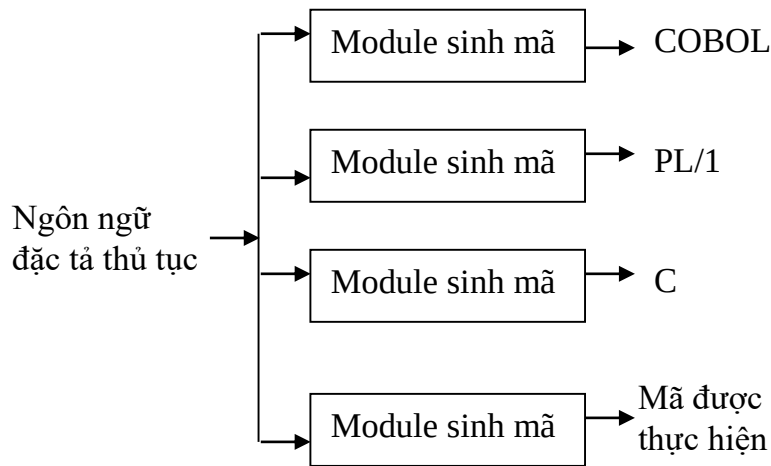
Hình 4.2 Biên dịch ngôn ngữ thế hệ thứ tư

Các công cụ lúc này được phân làm một số loại, ví dụ như:

Bộ sinh ứng dụng



Hình 4.3 Bộ sinh ứng dụng



Hình 4.4 Bộ sinh mã

Các công cụ lập trình hướng đối tượng: Lập trình hướng đối tượng là một trong các công nghệ "nóng nhất" trong công nghệ phần mềm. Vì lý do này, các nhà chế tạo CASE đang cung cấp ồ ạt các công cụ phát triển phần mềm hướng đối tượng tới thị trường.

Các ngôn ngữ phổ biến hiện nay là C++, Eiffel, Smalltalk, Objective-C. Môi trường hướng đối tượng đặc trưng thường kết hợp với các cung cấp giao diện thể hệ 3 (menu, mouse, multitasking, ...) cùng với các chức năng đặc biệt như "browser" - một chức năng cho phép kỹ sư phần mềm kiểm tra tất cả các đối tượng được chứa trong thư viện.

4.7.2.6 Các công cụ tích hợp và kiểm tra

Có ba hạng công cụ kiểm tra được sử dụng nhiều nhất. Nhiều công cụ phải trải hết 2 hoặc 3 phân loại nói trên.

Các công cụ phân tích tĩnh: Các công cụ kiểm tra tĩnh giúp các kỹ sư phần mềm trong việc rút ra các trường hợp kiểm tra. Ba kiểu khác nhau của các công cụ kiểm tra tĩnh được dùng trong công nghiệp: các công cụ kiểm tra dựa trên mã, các ngôn ngữ kiểm tra đặc tả và các công cụ kiểm tra dựa trên yêu cầu.

Code-Based testing tools nhận mã nguồn như là đầu vào và tiến hành một số phân tích. Căn cứ vào mô tả của đầu vào chương trình và thiết kế thủ tục như là chỉ dẫn các công cụ kiểm tra tĩnh suy ra các trường hợp kiểm tra sử dụng đường dẫn, các kiểm tra điều kiện và các tiêu chuẩn luồng dữ liệu.

Requirements-based testing tools phân tách các yêu cầu người dùng và khuyên một số trường hợp kiểm tra (hoặc lớp kiểm tra) mà sẽ thử các yêu cầu. Để thực hiện tốt thì các công cụ trong phân hạng này phải truy nhập tới các đặc tả hình thức của phần mềm.

Specialized testing languages cho phép các kỹ sư phần mềm viết các đặc tả kiểm tra chi tiết mà mô tả mỗi trường hợp kiểm tra và logic cho sự thực hiện nó. Trong phần lớn

các trường hợp, các công cụ kiểm tra tĩnh sẽ tự động hoá và phân loại các phép kiểm tra. Và nó sẽ so sánh kết quả thực với kết quả dự tính.

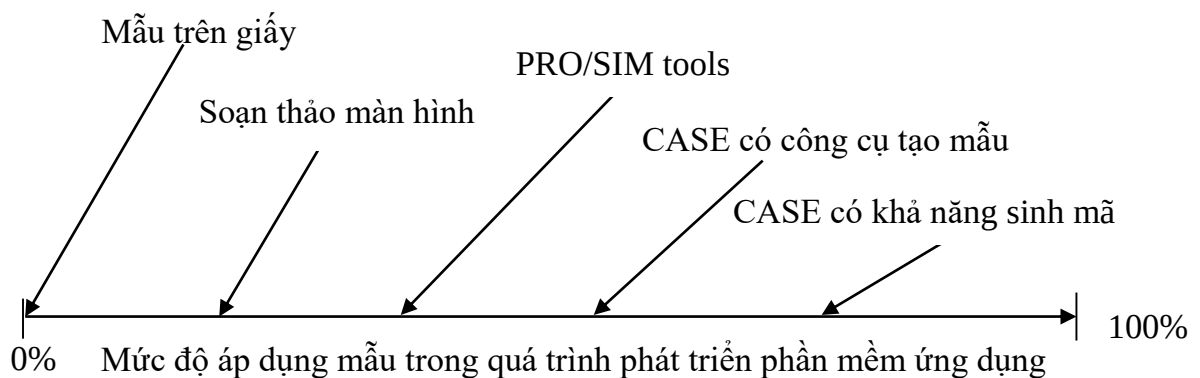
Các công cụ phân tích động: Các công cụ kiểm tra động tương tác với quá trình thực hiện chương trình, kiểm tra đường thử, kiểm tra xác nhận về giá trị các biến, các công cụ động có thể là loại can thiệp hoặc không can thiệp. Công cụ can thiệp (intrusive) thay đổi phần mềm để kiểm tra. Công cụ không can thiệp (nointrusive) sử dụng một xử lý phân cứng tách rời chạy song song với xử lý chưa có chương trình đang kiểm tra. Phần lớn các công cụ thuộc phân loại phân tích động tạo các báo cáo chỉ rõ số lượng các khối, câu lệnh đã được thực hiện và các thời gian thực hiện trung bình cho các khối lệnh. Công cụ kiểm tra động có thể được dùng để nối tiếp với công cụ kiểm tra tĩnh. Các kiểm tra tĩnh tạo ra các trường hợp test sau đó được quản lý bởi các công cụ động.

Công cụ quản lý test: Công cụ quản lý test dùng để điều khiển và phối hợp các kiểm tra phần mềm cho mỗi bước kiểm tra chính. Các công cụ trong phân loại này quản lý và phối hợp các kiểm tra regression. Tiến hành các so sánh output thực và biểu kiến. Nhiều công cụ quản lý test cũng phục vụ như là các bộ điều khiển test.

Một bộ điều khiển test đọc một hoặc nhiều trường hợp test từ file test, định dạng các dữ liệu test để phù hợp nhu cầu phần mềm, sau đó gọi phần mềm để test. Cuối cùng, nhà quản lý test đôi khi làm việc với công cụ theo dõi yêu cầu để cung cấp các phân tích coverage yêu cầu cho kiểm tra.

4.7.2.7 Các công cụ tạo mẫu

Tạo mẫu được dùng rộng rãi như là sự tiến hoá của công nghệ phần mềm. Các công cụ tạo mẫu được phân bố theo hình sau:



Hình 4.5 Các công cụ mẫu

Mức thấp nhất, các công cụ tạo "mẫu trên giấy". Công cụ chạy trên máy có thể tạo các hình ảnh thực dùng để minh hoạ các chức năng, đánh điều hệ thống. Các hình ảnh này không thể thực hiện được. Trong một số trường hợp, bộ vẽ màn hình có thể sinh ra các mã để tạo màn hình. Các công cụ CASE phức tạp hơn cho phép tạo các thiết kế dữ liệu, cả màn hình hiển thị và báo biểu.

4.7.2.8 Các công cụ bảo trì

Các công cụ bảo trì có thể được phân lại theo các chức năng sau:

- Thiết kế ngược với các công cụ đặc trưng.
- Các công cụ phân tích và cấu trúc lại mã.
- Các công cụ kiến tạo lại hệ thống trực tuyến.

Các công cụ thiết kế ngược: Các công cụ thiết kế ngược tiến hành tạo lại các phân tích ban đầu trên cơ sở các chương trình đã tồn tại. Các công cụ này cũng có thể phân ra loại tĩnh và động.

Một công cụ thiết kế ngược dùng mã nguồn như đầu vào và phân tích lấy ra kiến trúc chương trình, cấu trúc điều khiển, luồng logic, cấu trúc dữ liệu, luồng dữ liệu. Các công cụ thiết kế phụ thuộc (Dependency analysis tools) tiến hành phần lớn các chức năng trên, ngoài ra nó còn xây dựng sơ đồ graphic biểu diễn sự phụ thuộc chỉ sự liên kết giữa các cấu trúc dữ liệu, khối chương trình, và các đặc tính khác của chương trình.

Các công cụ thiết kế ngược quan sát phần mềm khi nó chạy và sử dụng các thông tin nhận được để xây dựng mô hình đáng điệu của chương trình. Mặc dù chúng ít được dùng nhưng cần thiết cho việc bảo trì các chương trình thời gian thực hoặc được nhúng trong hệ thống khác.

Các công cụ thiết kế: Các công cụ thiết kế lại có thể chia làm hai nhóm: công cụ cấu trúc lại mã, công cụ thiết kế lại dữ liệu.

Các công cụ cấu trúc mã lại nhận mã nguồn phi cấu trúc như là đầu vào, tạo thiết kế ngược, sau đó cấu trúc lại mã mới. Mặc dù các công cụ như vậy có thể hữu ích, chúng chỉ tập trung trong việc thiết kế thủ tục của chương trình.

Các công cụ thiết kế lại dữ liệu làm việc tại cực kia của thiết kế. Các công cụ như vậy truy nhập định nghĩa dữ liệu hoặc cơ sở dữ liệu được mô tả trong ngôn ngữ lập trình hoặc ngôn ngữ mô tả cơ sở dữ liệu. Sau đó nó chuyển sang dạng biểu diễn graphic có thể phân tích được bởi kỹ sư phần mềm.

Tương tác với các công cụ thiết kế lại, kỹ sư phần mềm có thể thay đổi cấu trúc của cơ sở dữ liệu, chuẩn hoá dữ liệu, sau đó tự động sinh mã mới. Các công cụ có thể sử dụng hệ chuyên gia.

4.7.3. Một số công cụ được cung cấp tự động cho việc sinh mã

Các tiện ích CASE được xây dựng trên việc sinh mã hay các giao diện phục vụ cho sinh mã, cho phép trộn lẫn các mã khác nhau để phù hợp với môi trường phát triển và ngôn ngữ sinh ra.

Một số CASE thông dụng hiện nay như:

Bảng 4-2 Một số CASE thông dụng

Sản phẩm	Kỹ thuật
ADW-Construction Workbench của Knowledgeware Inc., Atlanta, CA	Xây dựng các mã giả bằng các modul, nó dùng cho việc sinh mã cho MsDOS, MVS
Cdevelopment Environment, OOSD/C++ của S/Cubed Inc. Stamford, CT	Sinh mã Cobol cho các Mainframe, AS/400, OS/2. Sinh mã C cho MsDOS, OS/2
IEW của Texas Instruments Dallas, TX	Sinh mã Cobol gắn với SQL, sinh mã C Cho MVS, MsDOS, OS/2. Giao diện cho Telon và sinh ra các mã khác.
Sản phẩm	Kỹ thuật
NeXTStep 3.0 của NeXT Computer Redwood City, CA	Môi trường cơ sở dữ liệu hướng đối tượng
ObjectMaker của Mark V Systems	Sinh mã C hay C++ cho MsDOS, VMS, Unix, AIX
Software Through Pictures của Intergrated Development	Sinh mã C hay C++ cho Unix, AIX
Systems Architect của Popkin Software & System Inc. New York, NY	Sinh mã C cho MsDOS, OS/2
Teamwork,Ensemble của Cadre Technologies Providence, RI.	Sinh mã C hay C++ cho Unix, OS/2, AIX
Visible Analyst Workbench của Visible Systems Corp. Newton, MA.	Sinh mã C cho MsDOS

Những tiện ích trên cung cấp một cách tự động bao gồm các tiện ích sinh mã, chương trình biên dịch lớn dần và môi trường tạo ra chương trình. Toàn bộ chúng chính là tiện ích Lower CASE hay Back-End CASE.

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

- Câu 1. Thế nào là một phong cách cài đặt chương trình tốt.
- Câu 2. Tài liệu trong của chương trình đem lại những lợi ích gì?
- Câu 3. Các nền tảng của một ngôn ngữ có thể lập trình được? Hãy chỉ rõ sự tương đương giữa bản thiết kế và cài đặt để đảm bảo được sự đúng đắn của chương trình.
- Câu 4. Với hồ sơ thiết kế hệ thống trong chương 3, bạn chọn công cụ cài đặt thế nào, vì sao?

Câu 5. Trong cài đặt ứng dụng, để nâng cao hiệu suất của hệ thống, bạn đã làm gì? Hãy nêu và phân tích sự cải tiến đó.

Câu hỏi trắc nghiệm:

Câu 1. Phong cách lập trình và các đặc trưng của ngôn ngữ lập trình có ảnh hưởng đến chất lượng của phần mềm không?

- a. Ảnh hưởng lớn đến chất lượng phần mềm
- b. Ảnh hưởng không đáng kể đến chất lượng phần mềm
- c. Không ảnh hưởng gì lắm đến chất lượng phần mềm
- d. Không ảnh hưởng gì đến chất lượng phần mềm

Câu 2. Phong cách lập trình nào được xem là tốt?

- a. Sử dụng nhiều kỹ thuật nâng cao của các ngôn ngữ lập trình hiện đại nhất
- b. Sử dụng các câu lệnh theo thói quen của bản thân không cần quan tâm đến cấu trúc đơn giản hay phức tạp miễn sao giải quyết được công việc là được
- c. Sử dụng các câu lệnh đơn giản, ngắn gọn, dễ hiểu và hiệu quả.
- d. Sử dụng các câu lệnh phức tạp chứng tỏ được khả năng am hiểu ngôn ngữ lập trình của lập trình viên.

Câu 3. Lời chú thích trong chương trình nên tạo như thế nào:

- a. Phải đúng đắn và nên mô tả các khối chương trình
- b. Nên chú thích cho từng dòng
- c. Không cần phải quan tâm xem đúng hay sai vì nó không ảnh hưởng đến kết quả chương trình.
- d. Tất cả đều đúng

Câu 4. Việc xây dựng câu lệnh nằm ở bước nào:

- a. Bảo trì
- b. Thiết kế
- c. Kiểm thử
- d. Lập trình

Câu 5. Ngôn ngữ cài đặt SQL là thuộc loại:

- a. Ngôn ngữ lập trình theo phong cách cấu trúc
- b. Ngôn ngữ hỏi đáp trong lĩnh vực cơ sở dữ liệu
- c. Ngôn ngữ lập trình theo phong cách hướng đối tượng
- d. Ngôn ngữ lập trình theo phong cách logic

Bài tập vận dụng:

Bài 1: Bạn được giao nhiệm vụ cài đặt phần mềm “Quản lý sinh viên của một trường Đại học”, bạn hãy phân tích, lựa chọn phương pháp, ngôn ngữ lập trình phù hợp và thực hiện nhiệm vụ này?

Bài 2: Bạn được giao nhiệm vụ cài đặt phần mềm “Game 3D”, bạn hãy phân tích, lựa chọn phương pháp, ngôn ngữ lập trình phù hợp và thực hiện nhiệm vụ này?

Bài 3: Bạn được giao nhiệm vụ cài đặt phần mềm “Website quản lý bán hàng online của 1 hệ thống bán quần áo”, bạn hãy phân tích, lựa chọn phương pháp, ngôn ngữ lập trình phù hợp và thực hiện nhiệm vụ này?

Bài 4: Bạn được giao nhiệm vụ cài đặt ứng dụng dạng “Diệt virus”, bạn hãy phân tích, lựa chọn phương pháp, ngôn ngữ lập trình phù hợp và thực hiện nhiệm vụ này?

Bài 5: Hãy chọn một phần mềm mà bạn từng tham gia phát triển hoặc bạn có định hướng tìm hiểu, bạn hãy phân tích, lựa chọn phương pháp, ngôn ngữ lập trình phù hợp và thực hiện nhiệm vụ cài đặt phần mềm này?

CHƯƠNG 5: KIỂM THỬ PHẦN MỀM

Mục tiêu của chương:

Sản phẩm phần mềm được gọi là đúng nếu nó thực hiện được chính xác những tiêu chuẩn mà người thiết kế đã đặt ra. Để có một đánh giá chính xác về cấp độ đúng của phần mềm, ta phải kiểm tra chất lượng phần mềm. Như thế, kiểm tra là quá trình tìm lỗi và nó là một đánh giá cuối cùng về các đặc tả, thiết kế và mã hoá. Mục đích của kiểm tra là đảm bảo rằng tất cả các thành phần của ứng dụng ăn khớp, vận hành như mong đợi và phù hợp các tiêu chuẩn thiết kế.

Trong chương này, chúng ta thảo luận các chiến lược kiểm tra phần mềm và các kỹ thuật, phương pháp hiệu quả cho mỗi mức độ kiểm tra. Cuối cùng, các công cụ hỗ trợ kiểm tra tự động và các công cụ hỗ trợ kiểm tra độc lập được trình bày để hỗ trợ cho quá trình kiểm tra.

5.1 Độ tin cậy của phần mềm

5.1.1. Chất lượng phần mềm và việc đảm bảo chất lượng phần mềm

5.1.1.1 *Chất lượng phần mềm:*

Chất lượng nói chung được hiểu là tốt hay không tốt, dùng được lâu hay nhanh hỏng. Chất lượng là mức độ đáp ứng các yêu cầu của người dùng. Phần mềm cũng là một sản phẩm, có thể đánh giá về chất lượng thông qua các tiêu chí khác nhau.

Có rất nhiều định nghĩa về chất lượng phần mềm được đưa ra bởi các tổ chức, cá nhân khác nhau. Sau đây là một số định nghĩa tiêu biểu:

- Định nghĩa theo Viện kỹ sư Điện và Điện tử IEEE (Institute of Electrical and Electronics Engineers) (1991) [4]:

- + Định nghĩa 1: Chất lượng phần mềm là một mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu đã được đặc tả.

- + Định nghĩa 2: Chất lượng phần mềm là mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu và sự mong đợi của khách hàng hay người sử dụng.

Phân tích hai quan điểm của IEEE:

Theo quan điểm thứ nhất của IEEE: chúng ta sẽ bị phụ thuộc quá nhiều vào tài liệu đặc tả của yêu cầu, dẫn đến nếu việc xác định yêu cầu bị sai, thiếu thì một phần mềm được làm đúng với đặc tả chưa chắc đã là một phần mềm có chất lượng.

Theo quan điểm thứ hai của IEEE: khách hàng đôi khi không có kiến thức về công nghệ, họ có thể đưa ra những mong muốn hết sức vô lý và có thể thay đổi yêu cầu với phần mềm nhiều lần, thậm chí thay đổi ngay trong giai đoạn cuối. Điều này gây nhiều khó khăn cho việc phát triển phần mềm.

- Định nghĩa theo Pressman: Chất lượng phần mềm là sự phù hợp của các yêu cầu cụ thể về hiệu năng và chức năng, các tiêu chuẩn phát triển phần mềm được ghi lại rõ ràng

bằng tài liệu với các đặc tính ngầm định của tất cả các phần mềm được phát triển chuyên nghiệp.

Định nghĩa của Pressman đề xuất ba yêu cầu với chất lượng phần mềm phải được đáp ứng khi phát triển phần mềm [4]:

- + Các yêu cầu chức năng rõ ràng là nhân tố chính quyết định chất lượng đầu ra của phần mềm.
- + Các tiêu chuẩn chất lượng phần mềm sẽ được nói đến trong hợp đồng.
- + Các đặc tính ngầm định cần được đáp ứng trong quá trình phát triển cho dù không được nói đến rõ ràng trong hợp đồng.

5.1.1.2 Đảm bảo chất lượng phần mềm

Ban đầu đảm bảo chất lượng phần mềm có mục tiêu đạt được các yêu cầu đề ra, tuy nhiên thực tế phát triển phần mềm tồn tại rất nhiều ràng buộc đòi hỏi người phát triển phải tối ưu hóa công tác quản lý.

Theo Daniel Galin: Đảm bảo chất lượng phần mềm (Software Quality Assure) là một tập hợp các hành động cần thiết được lên kế hoạch một cách hệ thống để cung cấp đầy đủ niềm tin rằng quá trình phát triển phần mềm phù hợp với các yêu cầu chức năng kỹ thuật cũng như các yêu cầu quản lý theo lịch trình và hoạt động trong giới hạn ngân sách.

Kiểm tra chất lượng phần mềm là một hoạt động khó được chấp nhận đối với việc quản lý vì nó tốn kém, mất thời gian và hiếm khi phát hiện được lỗi. Tuy vậy, chất lượng phần mềm cao là một mục tiêu quan trọng của nhóm phát triển phần mềm. Do vậy, cần và phải đảm bảo các tiêu chuẩn của phần mềm. Đảm bảo chất lượng phần mềm là một hoạt động có hệ thống và kế hoạch. Nó bao gồm nhiều nhiệm vụ liên kết với các hoạt động chính sau [4]:

- + Áp dụng các phương pháp kỹ thuật,
- + Tiến hành các cuộc xét duyệt kỹ thuật chính thức,
- + Kiểm thử phần mềm,
- + Buộc tôn trọng các chuẩn,
- + Kiểm soát thay đổi,
- + Đo chất lượng,
- + Báo cáo, lưu giữ kết quả.

Theo chuẩn ANSI/IEEE, kế hoạch đảm bảo chất lượng phần mềm như sau:

- I. Mục đích của kế hoạch
- II. Tham khảo
- III. Quản lý
 - A. Tổ chức
 - B. Nhiệm vụ
 - C. Trách nhiệm

IV. Tài liệu

- A. Mục đích
- B. Tài liệu công nghệ phần mềm cần thiết
- C. Các tài liệu khác

V. Chuẩn, thực hành và quy ước

- A. Mục đích
- B. Quy ước

VI. Xét duyệt và kiểm toán

- A. Mục đích
- B. Các yêu cầu xét duyệt
 1. Xét duyệt yêu cầu phần mềm
 2. Xét duyệt thiết kế
 3. Kiểm chứng phần mềm và xét duyệt hợp lệ
 4. Kiểm toán chức năng
 5. Kiểm toán vật lý
 6. Kiểm toán trong tiến trình
 7. Xét duyệt quản lý

VII. Quản lý cấu hình phần mềm

VIII. Báo cáo vấn đề và cách sửa chữa

IX. Công cụ, kỹ thuật và phương pháp luận

X. Kiểm soát mã

XI. Kiểm soát phương tiện

XII. Kiểm soát người cung cấp

XIII. Thu thập bảo trì và ghi nhớ báo cáo

Việc đảm bảo chất lượng phần mềm là một hoạt động bản chất cho bất kỳ nhóm phát triển phần mềm nào sản xuất ra phần mềm cho người sử dụng.

5.1.2. Độ tin cậy của phần mềm

5.1.2.1 Lỗi phần mềm

Có rất nhiều định nghĩa về lỗi phần mềm nhưng có thể hiểu và phát biểu một cách đơn giản thì "Lỗi phần mềm là sự không khớp giữa chương trình và đặc tả của nó".

Dựa vào định nghĩa, ta có thể phân loại lỗi phần mềm thành 3 dạng:

- Lỗi sai: Sản phẩm phần mềm được xây dựng khác với đặc tả.
- Lỗi thiếu: Các yêu cầu của sản phẩm phần mềm đã có trong đặc tả nhưng lại không có trong sản phẩm thực tế.

-
- Lỗi thừa: Sản phẩm thực tế có những tính năng không có trong tài liệu đặc tả.

5.1.2.2 Các nguyên nhân gây lỗi phần mềm

Lỗi phần mềm có thể đến từ nhiều nguyên nhân khác nhau, trong đó có cả các nguyên nhân chủ quan và các nguyên nhân khách quan. Dưới đây là chín nguyên nhân chủ yếu gây ra lỗi phần mềm:

- Việc định nghĩa các yêu cầu bị lỗi: Những lỗi trong việc xác định yêu cầu thường nằm ở phía khách hàng. Một số lỗi thường gặp là: định nghĩa sai yêu cầu, lỗi không hoàn chỉnh, thiếu các yêu cầu quan trọng hoặc là quá chú trọng các yêu cầu không thật sự cần thiết.

- Các lỗi trong giao tiếp giữa khách hàng và nhà phát triển: Hiểu lầm trong giao tiếp giữa khách hàng và nhà phát triển cũng là nguyên nhân gây lỗi. Những lỗi này thường xuất hiện trong giai đoạn đầu của dự án. Một số lỗi hay gặp phải: hiểu sai chỉ dẫn trong tài liệu yêu cầu, hiểu sai thay đổi khi khách hàng trình bày bằng lời nói và tài liệu, hiểu sai về phản hồi và thiếu quan tâm đến những đề cập của khách hàng.

Giải pháp khắc phục: Cần có ủy ban liên kết giữa khách hàng và nhà cung cấp để tránh lỗi trong giao tiếp. Ủy ban do quản trị dự án đứng đầu và khách hàng phải giới thiệu những người hiểu về mặt nghiệp vụ vào ủy ban đó.

- Sai lệch có chủ ý với các yêu cầu phần mềm: Trong một số trường hợp các nhà phát triển cố tình làm sai lệch các yêu cầu trong tài liệu đặc tả. Nguyên nhân của việc này đến từ các áp lực thời gian, ngân sách, hay cố tình sử dụng lại các mô-đun từ các dự án trước mà chưa phân tích đầy đủ những thay đổi để thích nghi với các yêu cầu mới.

Giải pháp khắc phục: Dựa trên những thống kê để quyết định xem giải pháp như thế nào, sắp xếp ưu tiên xem bỏ được yêu cầu nào hay sử dụng lại được mô-đun nào.

- Các lỗi thiết kế logic: Lỗi phần mềm xảy ra trong quá trình các chuyên gia thiết kế hệ thống, các kiến trúc sư hệ thống, kỹ sư phần mềm, các nhà phân tích xây dựng phần mềm theo yêu cầu. Các lỗi điển hình bao gồm:

- + Định nghĩa các yêu cầu phần mềm bằng các thuật toán sai.
- + Quy trình định nghĩa có chứa trình tự lỗi.
- + Sai sót trong các định nghĩa biên như > 3 hay ≥ 3 .
- + Thiếu sót các trạng thái hệ thống phần mềm được yêu cầu.

- Các lỗi lập trình: Có rất nhiều lý do dẫn đến việc các lập trình viên gây ra các lỗi lập trình. Những lý do này bao gồm: sự hiểu sai các tài liệu thiết kế, ngôn ngữ; sai sót trong ngôn ngữ lập trình; sai sót trong việc áp dụng các công cụ phát triển; sai sót trong lựa chọn dữ liệu...

- Không tuân thủ theo các tài liệu hướng dẫn và tiêu chuẩn lập trình: Các lỗi phần mềm có thể đến từ việc không tuân thủ các tài liệu và tiêu chuẩn lập trình của các tổ chức phát triển phần mềm.

- Thiếu sót trong quá trình kiểm thử: Lỗi phần mềm có thể đến từ chính quá trình kiểm thử khi mà người kiểm thử để lọt lỗi. Những lỗi này đến từ các nguyên nhân sau đây:

- + Kế hoạch kiểm thử chưa hoàn chỉnh, để sót yêu cầu cần kiểm thử.
- + Lỗi trong tài liệu và báo cáo kiểm thử.
- + Việc sửa chữa các lỗi được phát hiện không hoàn chỉnh do áp lực thời gian hay do thiếu cẩn thận.

Giải pháp: Lên kế hoạch kiểm thử cụ thể tại giai đoạn đầu của dự án.

- Các lỗi thủ tục: Các thủ tục hướng dẫn cho người sử dụng tại từng bước của tiến trình. Chúng có tầm quan trọng đặc biệt trong các hệ thống phần mềm phức tạp mà các tiến trình được thực bằng nhiều bước, mỗi bước có thể có nhiều kiểu dữ liệu và cho phép kiểm tra các kết quả trung gian. Các lỗi có thể đến từ việc viết các thủ tục.

- Các lỗi về tài liệu: Các lỗi về tài liệu là vấn đề của các đội phát triển và bảo trì khi có những sai sót trong các tài liệu liên quan. Những lỗi này có thể là nguyên nhân gây ra lỗi trong giai đoạn phát triển kế tiếp và giai đoạn bảo trì.

5.1.2.3 Chi phí cho việc sửa lỗi phần mềm

Việc kiểm thử và sửa lỗi phần mềm có thể thực hiện trong bất cứ giai đoạn nào của vòng đời phần mềm. Tuy nhiên công việc này nên được thực hiện càng sớm càng tốt vì càng về giai đoạn sau của vòng đời phần mềm, chi phí cho việc tìm và sửa lỗi càng tăng, đặc biệt là đến giai đoạn đã triển khai phần mềm thì chi phí cho sửa lỗi sẽ trở nên rất lớn và ảnh hưởng trực tiếp đến uy tín của tổ chức phát triển phần mềm.

5.1.2.4 Một số đánh giá vì độ tin cậy

- Đặc tả độ tin cậy phần mềm: gồm các bước

- + Phân tích hệ quả của các thất bại.
- + Chia các thất bại thành các nhóm khác nhau.
- + Thiết lập các yêu cầu về độ tin cậy bằng cách sử dụng các độ đo thích hợp cho từng loại.

- Đo độ tin cậy: theo một vài cách đo như sau

- + Xác suất thất bại tính theo đòi hỏi.
- + Tỷ lệ xuất hiện thất bại
- + Thời gian trung bình giữa hai thất bại kế tiếp nhau.
- + Độ đo mức sẵn sàng hoạt động của hệ thống.

- Thử nghiệm tĩnh: Mục tiêu chủ yếu của thử nghiệm tĩnh là xác định độ tin cậy của phần mềm chứ không phải là xác định các hư hỏng phần mềm.

Quá trình thử nghiệm tĩnh liên quan đến 4 bước sau:

- + Xác định độ đo thao tác phần mềm. Độ đo thao tác là một mẫu sử dụng phần mềm và xác định mẫu đó liên quan đến việc phát hiện các lớp thông tin vào của chương trình và ước tính xác suất của chúng.

- + Chọn ra hoặc sinh ra một tập các dữ liệu thử tương ứng với độ đo đó.
- + Áp dụng các trường hợp thử chương trình, ghi lại độ dài thời gian thi hành giữa mỗi cặp thất bại quan sát được. Thích hợp hơn là dùng thời gian thô, với đơn vị thời gian thích hợp cho độ đo mức tin cậy.

- + Tính toán độ đo mức tin cậy sau một số đáng kể (về mặt thống kê) các thất bại đã quan sát được.

- An toàn phần mềm: Có những hệ thống mà thất bại của nó có thể gây ra một mối đe dọa tính mạng con người. Thí dụ về hệ thống an toàn sinh mệnh như vậy là hệ thống điều khiển máy bay.

Có hai lớp phần mềm an toàn sinh mệnh.

- + Các phần mềm an toàn sinh mệnh sơ cấp: các phần mềm lỏng nhúng trong một hệ phần cứng dùng để điều khiển quá trình khác mà sự làm việc sai sót của nó có thể trực tiếp gây ra thương vong hoặc phá hủy môi trường sống của con người.

- + Các phần mềm an toàn sinh mệnh thứ cấp: các phần mềm có thể gián tiếp gây ra thương vong. Thí dụ hệ thống phần mềm trợ giúp thiết kế kỹ thuật, hệ thống cơ sở dữ liệu y tế liên quan đến các chất độc bảng A.

- Thử nghiệm khiếm khuyết: Thử nghiệm chương trình có hai mục đích: thứ nhất là chỉ ra rằng hệ thống là phù hợp với các đặc tả của nó, thứ hai là thực hành hệ thống theo một cách sao cho các khuyết tật được phơi ra. Các thử nghiệm với mục đích thứ nhất chính là các thử nghiệm để chấp nhận. Các thử nghiệm cho mục đích thứ hai lại khác hẳn: thử nghiệm thành công nhất là thử nghiệm phơi ra được nhiều khuyết tật nhất.

Các thử nghiệm có thể được phát triển song song với việc thiết kế và thực hiện bởi những người không dính dáng tới việc thiết kế [4].

5.1.2.5 Lập trình vì độ tin cậy

Lập trình là một công đoạn phụ thuộc nhiều vào kỹ xảo cá nhân, sự chú ý đến các chi tiết và kiến thức về việc làm như thế nào để sử dụng các công cụ sẵn có theo cách thức tốt nhất. Nhu cầu các hệ thống đáng tin là đang tăng lên vì vậy cần có các kỹ thuật chuyên biệt nhằm đạt được một hệ thống tin cậy được. Hiện nay, có hai kỹ thuật để tăng độ tin cậy phần mềm khi viết các chương trình ứng dụng là: tránh lỗi và tha thứ lỗi.

- Tránh lỗi

Tất cả các kỹ sư phần mềm hẳn đều muốn sản xuất ra các phần mềm không có lỗi. Một quá trình phát triển dựa vào việc phát hiện lỗi và trừ khử lỗi chứ không phải là tránh lỗi là một quá trình kém cỏi và lỗi thời.

Phần mềm không có lỗi ở đây là phần mềm tuân theo đúng đặc tả. Nói chung, nó có thể có lỗi trong đặc tả hoặc có thể không phản ánh đúng các nhu cầu của người sử dụng vậy là phần mềm không có lỗi không nhất thiết là các phần mềm luôn luôn hành xử như người dùng dự đoán.

Việc phát triển phần mềm không có lỗi là một việc rất đắt đỏ, và khi mà một số lỗi đã được tháo khỏi chương trình thì giá cả cho việc tìm và tháo lỗi còn lại có xu hướng tăng theo hàm số mũ. Do đó một tổ chức có thể quyết định chấp nhận một vài lỗi còn lưu lại. Tính về mặt giá cả thì thà rằng chịu tiền chi trả cho các thất bại của hệ thống do các lỗi đó gây ra còn hơn là đi phát hiện tháo gỡ các lỗi đó trước khi phân phối.

Tránh lỗi và phát triển phần mềm vô lỗi dựa trên:

- + Sản phẩm của một đặc tả hệ thống chính xác.
- + Chấp nhận một cách tiếp cận thiết kế phần mềm dựa trên việc che dấu thông tin và bao gói thông tin.
- + Tăng cường việc duyệt lại trong quá trình phát triển và thẩm định hệ phần mềm.
- + Chấp nhận triết lý chất lượng tổ chức: chất lượng là bánh lái của quá trình phát triển phần mềm.
- + Việc lập kế hoạch cẩn thận cho việc thử nghiệm hệ thống để trưng ra các lỗi mà các lỗi này chưa được phát hiện trong quá trình duyệt lại và để định lượng độ tin cậy của hệ thống.

- Thứ lỗi

Ngay với một hệ thống không có lỗi thì vẫn cần một tiện ích thứ lỗi: đó là vì có thể có các lỗi đặc tả. Một tiện ích thứ lỗi là cần thiết cho một hệ thống đáng tin cậy.

Có bốn hoạt động cần phải tiến hành nếu hệ thống phải là thứ lỗi:

- + Phát hiện lỗi
- + Định ra mức độ thiệt hại
- + Hồi phục sau khi gặp lỗi. Hệ thống phải hồi phục về trạng thái mà nó biết là an toàn. Cũng có thể là chỉnh lý trạng thái bị hủy hoại (hồi phục tiến), cũng có thể là lui về một trạng thái trước mà an toàn (hồi phục lùi).
- + Chữa lỗi. Cải tiến hệ thống để cho lỗi đó không xuất hiện nữa. Trong nhiều trường hợp sự thất bại của phần mềm là tàng hình và gây ra bởi một tổ hợp đặc biệt của thông tin vào.

- Xử lý bất thường

Một lỗi sai loại nào đó hoặc một sự cố bất ngờ xuất hiện thì ta gọi chúng là một bất thường. Các bất thường có thể do phần cứng cũng có thể do phần mềm. Khi mà một bất thường không được dự đoán thì bộ điều khiển sẽ chuyển cho cơ chế xử lý bất thường hệ thống. Nếu một bất thường đã được dự đoán thì mã phải bao gồm cả việc phát hiện và việc xử lý bất thường đó.

Hầu hết các ngôn ngữ lập trình là không có các tiện ích tự động để phát hiện và xử lý bất thường về mặt giải thuật.

Các bất thường có thể được ghi lại bằng cách dùng một biến Logic nhằm chỉ ra rằng có một bất thường đã xuất hiện.

- Lập trình phòng thủ

Lập trình phòng thủ là cách phát triển chương trình mà người lập trình giả định rằng các mâu thuẫn hoặc các lỗi chưa được phát hiện có thể tồn tại trong chương trình. Mã sẽ có phần kiểm tra trạng thái hệ thống sau khi biến đổi và phải đảm bảo rằng sự biến đổi trạng thái là kiên định. Nếu phát hiện một mâu thuẫn thì việc biến đổi trạng thái là phải rút lại và trạng thái phải trở về trạng thái đúng đắn trước đó.

Lập trình phòng thủ là một cách thứ lỗi, được tiến hành không cần bộ điều khiển thứ lỗi. Về cơ bản quá trình vẫn là: phát hiện lỗi, đánh giá lỗi, và phục hồi sau lỗi. Nói chung một lỗi gây ra một sự sụp đổ trạng thái: các biến trạng thái được gán các trị không hợp luật. Ngôn ngữ lập trình như Ada cho phép phát hiện các lỗi đó ngay trong thời gian biên dịch. Tuy nhiên việc kiểm tra biên dịch chỉ hạn chế cho các giá trị tĩnh và một vài phép kiểm tra thời gian thực là không thể tránh được.

Một cách để phát hiện lỗi trong chương trình Ada là dùng cơ chế xử lý bất thường kết hợp với đặc tả miền trị.

Hồi phục lỗi là một quá trình cải biên không gian trạng thái của hệ thống sao cho hiệu ứng của lỗi là nhỏ nhất và hệ thống có thể tiếp tục vận hành, có lẽ là trong một mức suy giảm. Hồi phục tiến liên quan đến việc cố gắng chỉnh lại trạng thái hệ thống. Hồi phục lùi liên quan đến việc lưu trạng thái của hệ thống ở một trạng thái đúng đắn đã biết.

Hồi phục tiến thường là một chuyên biệt ứng dụng, mà kiến thức miền được sử dụng để tính ra các sự chỉnh lý có thể. Tuy nhiên có hai tình thế chung mà khi đó hồi phục tiến có thể thành công:

- 1) Khi dữ liệu mã gặp sự cố. Việc sử dụng mã hóa thích hợp bằng cách thêm các dữ liệu dư thừa vào dữ liệu cho phép sửa sai khi phát hiện lỗi.

- 2) Khi cấu trúc nối gặp sự cố. Nếu các con trỏ tiến và lùi đã có trong cấu trúc dữ liệu thì cấu trúc đó có thể tái tạo nếu như còn đủ các con trỏ chưa bị sụp. Kỹ thuật này thường được dùng cho việc sửa chữa hệ thống tệp và cơ sở dữ liệu.

Hồi phục lùi là một kỹ thuật đơn giản liên quan đến việc duy trì các chi tiết của trạng thái an toàn và cất giữ trạng thái đó khi mà một sai đã bị phát hiện. Hầu hết các hệ quản trị cơ sở dữ liệu đều có bộ phục hồi lỗi. Cơ sở dữ liệu chỉ cập nhật dữ liệu một khi giao dịch đã hoàn tất và không phát hiện được vấn đề gì. Nếu giao dịch thất bại thì cơ sở dữ liệu không được cập nhật.

Một kỹ thuật khác là thiết lập các điểm kiểm tra thường kỳ mà chúng là các bản sao của trạng thái hệ thống. Khi mà một lỗi được phát hiện thì trạng thái an toàn đó được tái lưu kho từ điểm kiểm tra gần nhất.

Không may là khi hệ thống dính líu tới nhiều quá trình hợp tác thì dãy các thao tác có thể là các điểm kiểm tra của các quá trình đó là không đồng bộ và để phục hồi thì mỗi quá trình phải lần trở lại trạng thái ban đầu của nó.

5.2 Kiểm thử và các chiến lược kiểm thử phần mềm

5.2.1. Kiểm thử phần mềm, khái niệm, vai trò và lợi ích của kiểm thử phần mềm.

5.2.1.1 Khái niệm Kiểm thử phần mềm:

Kiểm thử phần mềm là quá trình thực thi một chương trình với mục đích tìm ra lỗi. Đó là phương pháp kiểm tra xem sản phẩm phần mềm trên thực tế có phù hợp với các yêu cầu đã đặt ra hay không, và đảm bảo rằng không có lỗi hay khiếm khuyết. Nó bao gồm việc kiểm tra, phân tích, quan sát và đánh giá các khía cạnh khác nhau của sản phẩm. Người kiểm thử phần mềm (Tester) sử dụng kết hợp các công cụ thủ công và tự động. Sau khi tiến hành kiểm thử, Tester báo cáo kết quả cho nhóm phát triển. Mục đích là xác định các lỗi, khiếm khuyết hoặc các yêu cầu còn thiếu so với yêu cầu thực tế.

Kiểm thử rất quan trọng đối với mỗi công ty phát triển phần mềm. Bởi vì, với kiểm thử phần mềm, nếu có bất kỳ lỗi nào, nó có thể được xác định sớm và giải quyết trước khi giao sản phẩm.

Một số công ty phát triển phần mềm thường bỏ qua bước này vì ngân sách eo hẹp và cho rằng nó sẽ không dẫn đến hậu quả lớn. Nhưng để tạo những trải nghiệm tốt nhất cho khách hàng, chất lượng sản phẩm cần phải được đặt lên hàng đầu. Vì vậy, việc kiểm thử sản phẩm để tìm lỗi là điều gần như bắt buộc. Doanh nghiệp chỉ có thể mang đến giá trị cho khách hàng khi sản phẩm cung cấp được coi là lý tưởng. Và để đạt được điều đó, các công ty phải đảm bảo rằng người dùng không gặp phải bất kỳ vấn đề nào khi sử dụng sản phẩm của mình. Cách tốt nhất để làm điều đó là tạo ra sản phẩm không có lỗi [4].

5.2.1.2 Vai trò của kiểm thử phần mềm:

Kiểm thử phần mềm có vai trò rất quan trọng trong quá trình phát triển. Thêm nữa, khi khách hàng sử dụng sản phẩm, họ rất có thể phải tiết lộ một số thông tin cá nhân. Để ngăn chặn tin tặc nắm được dữ liệu này, việc kiểm tra bảo mật là điều bắt buộc trước khi phần mềm đến tay người dùng. Sản phẩm phần mềm được kiểm thử kỹ càng qua quy trình phù hợp sẽ đảm bảo độ tin cậy, bảo mật, giúp tiết kiệm thời gian, chi phí, mang đến sự hài lòng cho khách hàng.

Một lý do nữa khiến việc kiểm thử ngày càng trở nên quan trọng đó là phát hiện khả năng tương thích với các thiết bị và nền tảng khác nhau. Giả sử khi phát triển một trang web, Tester phải kiểm tra xem trang web có chạy trên độ phân giải thiết bị khác nhau, các trình duyệt khác nhau hay không? Những gì hoạt động tốt trên Chrome có thể không chạy tốt trên Safari hoặc Microsoft Edge. Điều này làm phát sinh nhu cầu kiểm tra trình duyệt chéo, bao gồm kiểm tra tính tương thích của ứng dụng trên các trình duyệt khác nhau.

5.2.1.3 Lợi ích của Kiểm thử phần mềm:

- Hiệu quả về chi phí: Đây là một trong những lợi ích quan trọng của kiểm thử phần mềm. Thực tế cho thấy rằng các lỗi thiết kế khó có thể được loại trừ hoàn toàn đối với bất kỳ hệ thống nào. Đó không phải là lỗi bất cần của nhà phát triển phần mềm

(Developer) mà đôi khi do sự phức tạp của hệ thống. Nếu các vấn đề về thiết kế không được phát hiện, thì việc tìm ra và sửa các lỗi, khiếm khuyết sẽ trở nên khó khăn và tốn kém hơn. Kiểm thử bất kỳ dự án IT (Information technology – Công nghệ thông tin) nào cũng sẽ giúp công ty tiết kiệm, việc xác định lỗi trong giai đoạn đầu sẽ giúp quá trình sửa chữa tốn ít chi phí hơn.

- **Bảo mật:** Đây là điểm nhạy cảm và dễ bị tấn công nhất của kiểm thử phần mềm. Kiểm thử giúp loại bỏ các rủi ro và vấn đề trong sản phẩm. Cùng với đó, tất cả khách hàng đều đang tìm kiếm những sản phẩm đáng tin cậy.

- **Chất lượng sản phẩm:** Đây là yêu cầu thiết yếu của bất kỳ sản phẩm phần mềm nào. Kiểm thử phần mềm giống như việc củng cố danh tiếng công ty bằng cách cung cấp các sản phẩm chất lượng cho khách hàng.

- **Sự hài lòng của khách hàng:** Trong bất kỳ hoạt động kinh doanh sản phẩm nào, mục tiêu cuối cùng đều là mang đến cho khách hàng trải nghiệm tốt nhất. Sự hài lòng của khách hàng rất quan trọng trong quá trình hợp tác lâu dài.

5.2.2. Các chiến lược kiểm thử phần mềm.

Kiểm thử phần mềm không phải là một việc đơn lẻ. Nó có nhiều hình thức khác nhau và được phân loại theo một số tiêu chí. Về cơ bản, kiểm thử phần mềm được chia làm 4 loại: Kiểm thử chức năng, Kiểm thử phi chức năng, Kiểm thử cấu trúc và Kiểm thử liên quan đến các thay đổi.

5.2.2.1 Kiểm thử chức năng (Functional testing)

Kiểm thử chức năng là xác minh hệ thống hoạt động theo đúng theo các yêu cầu nghiệp vụ. Hình thức kiểm thử này có thể được thực hiện từ hai khía cạnh: dựa trên yêu cầu (requirements-based) và dựa trên quy trình nghiệp vụ (business – process – based).

Trong kiểm thử dựa trên yêu cầu, các yêu cầu được ưu tiên tùy thuộc vào tiêu chí rủi ro. Điều này sẽ đảm bảo những phần quan trọng nhất sẽ được kiểm tra (test) đầy đủ. Mặt khác, kiểm thử dựa trên quy trình nghiệp vụ sẽ sử dụng những kiến thức tương ứng. Quy trình nghiệp vụ mô tả các việc liên quan đến nghiệp vụ hằng ngày của hệ thống [4].

Kiểm thử chức năng bao gồm 5 bước:

- Xác định các chức năng mà phần mềm sẽ thực hiện.
- Tạo các dữ liệu đầu vào dựa trên các tài liệu đặc tả kỹ thuật của các chức năng.
- Xác định các kết quả đầu ra dựa trên các tài liệu đặc tả kỹ thuật của các chức năng.
- Thực hiện các trường hợp kiểm thử.
- So sánh kết quả thực tế và kết quả mong muốn.

Trong đó, kiểm thử chức năng còn được chia nhỏ ra thành các loại:

- **Kiểm thử đơn vị (Unit testing):** Kiểm thử đơn vị là hoạt động kiểm thử nhỏ nhất. Kiểm thử thực hiện trên các hàm hay thành phần riêng lẻ. Đây là 1 công việc mà để thực

hiện được nó thì người kiểm thử sẽ phải hiểu biết về code, về chương trình, các hàm, ... Mục đích của việc thực hiện kiểm thử đơn vị là cô lập từng thành phần của chương trình và chứng minh các bộ phận riêng lẻ chính xác về các yêu cầu chức năng

- Kiểm thử khói (Smoke Testing): Smoke test là một loại Kiểm thử phần mềm được thực hiện sau khi xây dựng phần mềm để xác định rằng các chức năng quan trọng của chương trình đang hoạt động tốt. Nó được thực hiện "trước" bất kỳ kiểm tra chức năng hoặc hồi quy chi tiết nào được thực thi trên bản dựng phần mềm. Mục đích là để từ chối một ứng dụng bị hỏng nặng, vì vậy mà nhóm QA sẽ không mất nhiều thời gian cài đặt và thử nghiệm ứng dụng phần mềm. Trong Smoke test, các trường hợp thử nghiệm được chọn bao gồm chức năng quan trọng nhất hoặc thành phần của hệ thống. Mục tiêu không phải là để thực hiện kiểm tra toàn diện, nhưng để xác minh rằng các chức năng quan trọng của hệ thống đang hoạt động tốt.

Ví dụ một Smoke test điển hình sẽ là - Xác minh rằng ứng dụng chạy thành công, kiểm tra xem GUI (Giao diện đồ họa người dùng) có đáp ứng không v.v...

- Kiểm thử tình trạng (Sanity Testing): Sanity testing thường được thực hiện khi bất kỳ lỗi nhỏ được khắc phục, hoặc khi có một thay đổi nhỏ trong chức năng. Nó là một loại kiểm thử phần mềm được thực hiện bởi người kiểm thử (tester) để đảm bảo rằng chức năng hoạt động đúng như mong đợi. Sanity testing hẹp và sâu. Không giống Smoke testing, Sanity testing tập trung vào một hoặc hai chức năng trong khi đó smoke testing được thực hiện để đảm bảo rằng tất cả các chức năng chính của dự án (project) đều hoạt động đúng. Sau thay đổi hoặc sửa lỗi được thực hiện trong code, bản xây dựng là sẵn sàng cho người kiểm thử (tester). Tester sẽ thực hiện Sanity testing trên những chức năng thay đổi thay vì thực hiện kiểm thử hồi quy (regression testing) để tiết kiệm thời gian. Nếu lỗi và chức năng thay đổi không hoạt động đúng như mong đợi, tester sẽ từ chối bản xây dựng. Thất bại này được phát hiện sớm trong Sanity testing.

- Kiểm thử giao diện (Interface testing):

Khi phát triển ứng dụng, phần mềm hay một trang web, chúng ta cần phải có các thành phần cấu tạo nên nó. Ví dụ: server, database v.v... Việc kết nối giữa các thành phần này được gọi là interface.

Với một máy tính, interface có thể là các API, web service, ... Sự giao tiếp giữa các thành phần khác nhau của một phần mềm, một ứng dụng hoặc một website v.v... có thể ảnh hưởng đến hiệu suất tổng thể. Do đó interface cũng cần được kiểm thử và xác nhận.

Việc kiểm thử được thực hiện để kiểm chứng các chức năng giao tiếp được gọi là interface testing.

- Kiểm thử tích hợp (Integration testing):

Kiểm thử tích hợp (Integration testing) hay còn gọi là tích hợp và kiểm thử (integration and testing, viết tắt: I&T) là một giai đoạn trong kiểm thử phần mềm. Mỗi mô-đun phần mềm riêng biệt được kết hợp lại và kiểm thử theo nhóm.

Kiểm thử tích hợp xảy ra sau kiểm thử đơn vị (Unit Test) và trước kiểm thử xác nhận. Kiểm thử tích hợp nhận các mô đun đầu vào đã được kiểm thử đơn vị, nhóm chúng vào các tập hợp lớn hơn, áp dụng các ca kiểm thử đã được định nghĩa trong kế hoạch kiểm thử tích hợp vào tập hợp đó, và cung cấp đầu ra cho hệ thống tích hợp.

- Kiểm thử hệ thống (System testing):

Kiểm thử hệ thống là một phương pháp theo dõi và đánh giá hành vi của sản phẩm hoặc hệ thống phần mềm hoàn chỉnh và đã được tích hợp đầy đủ, dựa vào đặc tả và các yêu cầu chức năng đã được xác định trước. Đó là giải pháp cho câu hỏi "Liệu hệ thống hoàn chỉnh có hoạt động đúng với yêu cầu hay không?"

System test được thử nghiệm trong hộp đen, tức là chỉ có các tính năng làm việc bên ngoài của phần mềm được đánh giá trong quá trình thử nghiệm này. Nó không đòi hỏi bất kỳ kiến thức nội bộ nào về code, lập trình, thiết kế, v.v. và hoàn toàn dựa trên quan điểm của người dùng.

- Kiểm thử hồi quy (Regression testing):

Kiểm thử hồi quy là một hình thức kiểm thử phần mềm để xem các chức năng cũ và mới của nó có còn hoạt động đúng sau khi thay đổi hệ thống hay không.

Ví dụ, một ứng dụng phần mềm có thể cho phép giáo viên thêm, lưu, xóa và làm mới trong một công cụ học tập trực tuyến. Sau đó, các Developer triển khai 1 version mới với chức năng bổ sung để cập nhật. Chức năng mới được kiểm tra để xác nhận rằng bản cập nhật hoạt động như mong đợi. Trong trường hợp này, kiểm tra hồi quy có thể cải thiện chất lượng tổng thể của sản phẩm.

- Kiểm thử chấp nhận (Acceptance testing):

Kiểm thử chấp nhận của người dùng được định nghĩa là một loại kiểm thử thực hiện bởi khách hàng để xác nhận hệ thống đã làm việc đúng như mong đợi và thỏa mãn yêu cầu của người dùng. Hoạt động này được thực hiện ở giai đoạn kiểm thử cuối cùng trước khi phần mềm được đưa vào hoạt động chính thức. Mục đích của thử nghiệm này là kiểm tra lại sản phẩm để đưa sản phẩm đến chuyển giao. Thử nghiệm này được thực hiện trong một môi trường thử nghiệm riêng biệt với môi trường phát triển. Đây là một loại thử nghiệm hộp đen trong đó hai hoặc nhiều người dùng cuối sẽ tham gia.

Ưu điểm của kiểm thử chức năng:

- Hình thức kiểm thử này mô phỏng việc sử dụng hệ thống thực tế
- Được thực hiện trong các điều kiện gần với điều kiện của khách hàng
- Không có giả định nào về cấu trúc hệ thống được đưa ra trong khi kiểm thử chức năng
- Rất dễ dàng để thực hiện kiểm thử thủ công

Ngược lại, kiểm thử chức năng có những giới hạn sau:

- Khả năng cao xảy ra tình trạng kiểm thử dư thừa

- Các lỗi logic trong phần mềm có thể bị bỏ sót trong khi kiểm thử chức năng

5.2.2.2 Kiểm thử phi chức năng (Non-functional testing)

Kiểm thử phi chức năng là kiểm tra các đặc tính chất lượng của hệ thống. Ví dụ, kiểm tra xem bao nhiêu người có thể đăng nhập đồng thời vào một phần mềm. Kiểm tra phi chức năng cũng quan trọng không kém như kiểm tra chức năng và ảnh hưởng đến sự hài lòng của khách hàng [4].

Mục tiêu của kiểm thử phi chức năng

- Kiểm thử phi chức năng tăng khả năng sử dụng, hiệu quả, khả năng bảo trì và tính di động của sản phẩm.
- Giúp giảm rủi ro sản xuất và chi phí liên quan đến các khía cạnh phi chức năng của sản phẩm.
- Tối ưu hóa cách cài đặt sản phẩm, thiết lập, thực thi, quản lý và giám sát.
- Thu thập và tạo các phép đo, số liệu cho nghiên cứu và phát triển nội bộ.
- Cải thiện và nâng cao kiến thức về hành vi và công nghệ sản phẩm đang sử dụng.

Đặc điểm của kiểm thử phi chức năng

- Kiểm thử phi chức năng nên được đo lường, vì vậy không có chỗ cho đặc tính chủ quan như tốt, tốt hơn, tốt nhất, v.v...
- Số chính xác không có khả năng được biết khi bắt đầu quá trình yêu cầu.
- Quan trọng để ưu tiên các yêu cầu.
- Đảm bảo rằng các thuộc tính chất lượng được xác định chính xác trong Kỹ thuật phần mềm.

Các thông số của kiểm thử phi chức năng

Security	Availability	Efficiency	Integrity
Reliability	Survivability	Usability	Flexibility
Scalability	Reusability	Interoperability	Portability

Hình 5.1 Các thông số của Kiểm thử phi chức năng

1. Security (Bảo mật): Tham số xác định cách hệ thống được bảo vệ an toàn trước các cuộc tấn công có chủ ý và đột ngột từ các nguồn bên trong và bên ngoài.

2. Reliability (Độ tin cậy): Mức độ mà bất kỳ hệ thống phần mềm nào liên tục thực hiện các chức năng được chỉ định mà không gặp sự cố.

3. Survivability (Khả năng sống sót): Tham số kiểm tra rằng hệ thống phần mềm tiếp tục hoạt động và tự phục hồi trong trường hợp lỗi hệ thống.

4. Availability (Tính sẵn có): Tham số xác định mức độ mà người dùng có thể phụ thuộc vào hệ thống trong quá trình hoạt động.

5. Usability (Khả năng sử dụng): Người dùng có thể dễ dàng học hỏi, vận hành, chuẩn bị đầu vào và đầu ra thông qua tương tác với một hệ thống.

6. Scalability (Khả năng mở rộng): Thuật ngữ này đề cập đến mức độ mà bất kỳ ứng dụng phần mềm nào cũng có thể mở rộng khả năng xử lý của nó để đáp ứng nhu cầu gia tăng.

7. Interoperability (Khả năng tương tác): Tham số phi chức năng này kiểm tra giao diện hệ thống phần mềm với các hệ thống phần mềm khác.

8. Efficiency (Tính hiệu quả): Mức độ mà bất kỳ hệ thống phần mềm nào cũng có thể xử lý dung lượng, số lượng và thời gian đáp ứng.

9. Flexibility (Tính linh hoạt): Thuật ngữ này đề cập đến sự dễ dàng mà ứng dụng có thể hoạt động trong các cấu hình phần cứng và phần mềm khác nhau. Giống như RAM tối thiểu, yêu cầu CPU.

10. Portability (Tính di động): Tính linh hoạt của phần mềm để chuyển từ môi trường phần cứng hoặc phần mềm hiện tại của nó.

11. Reusability (Tái sử dụng): Nó đề cập đến một phần của hệ thống phần mềm có thể được chuyển đổi để sử dụng trong một ứng dụng khác.

12. Integrity (Tính toàn vẹn): Tính toàn vẹn chủ yếu nói đến tính toàn vẹn dữ liệu. Toàn vẹn dữ liệu là dữ liệu hay thông tin không bị thay đổi, mất mát trong khi lưu trữ hay truyền tải. Nói cách khác tính toàn vẹn là tính không bị hiệu chỉnh của dữ liệu.

Các loại kiểm thử phi chức năng

- Performance Testing (Kiểm thử hiệu năng)
- Load Testing (Kiểm thử tải)
- Failover Testing (Kiểm thử chuyển đổi dự phòng)
- Compatibility Testing (Kiểm thử tương thích)
- Usability Testing (Kiểm thử khả năng sử dụng)
- Stress Testing (Kiểm thử về áp lực)
- Maintainability Testing (Kiểm thử bảo trì)
- Scalability Testing (Kiểm thử khả năng mở rộng)
- Volume Testing (Kiểm thử khối lượng)
- Security Testing (Kiểm thử bảo mật)

-
- Disaster Recovery Testing (Kiểm thử khắc phục thảm họa)
 - Compliance Testing (Kiểm thử tuân thủ)
 - Portability Testing (Kiểm thử tính di động)
 - Efficiency Testing (Kiểm thử hiệu quả)
 - Reliability Testing (Kiểm thử độ tin cậy)
 - Baseline Testing (Kiểm thử cơ bản)
 - Endurance Testing (Kiểm thử độ bền)
 - Documentation Testing (Kiểm thử tài liệu)
 - Recovery Testing (Kiểm thử phục hồi)
 - Internationalization Testing (Kiểm thử quốc tế hóa)
 - Localization Testing (Kiểm thử nội địa hóa)

5.2.2.3 Kiểm thử cấu trúc (Structural testing)

Kiểm thử cấu trúc thường được gọi là “hộp trắng” hoặc “hộp thủy tinh” bởi vì phương pháp này quan tâm đến việc tìm kiếm những gì đang xảy ra bên trong, kiểm tra dựa trên phân tích cấu trúc bên trong của thành phần hoặc hệ thống. Nó thường được sử dụng như một cách đo lường của kiểm thử, thông qua độ bao phủ của một tập hợp các yếu tố cấu trúc. Kiểm thử cấu trúc chủ yếu được áp dụng ở kiểm thử thành phần, kiểm thử tích hợp.

Các mục tiêu chính của kiểm thử cấu trúc bao gồm:

- Nhận ra những điểm bất cập
- Test chức năng bổ sung
- Xác định những phần bị thiếu trong bộ kiểm thử

Ưu điểm của kiểm thử cấu trúc:

- Loại bỏ code chết
- Có khả năng tìm ra lỗi ở giai đoạn đầu
- Đảm bảo kiểm tra phần mềm kỹ lưỡng hơn
- Tiết kiệm thời gian

Bên cạnh đó, nhược điểm của kiểm thử cấu trúc:

- Kiểm tra kết cấu khá tốn kém
- Yêu cầu kiến thức về code
- Đòi hỏi kiến thức vững chắc về công cụ được sử dụng để test

5.2.2.4 Kiểm thử liên quan đến các thay đổi (Change related testing)

Kiểm thử liên quan đến các thay đổi gồm Kiểm thử xác nhận và Kiểm thử hồi quy:

- Kiểm thử xác nhận (Confirmation testing)

Khi kiểm thử gặp lỗi, Tester phải xác định nguyên nhân lỗi là do lỗi phần mềm. Sau khi Tester phát hiện lỗi và báo cho nhà phát triển (Developer) để sửa thì phần mềm sau đó sẽ cập nhật phiên bản vá lỗi. Cuối cùng, Tester cần thực hiện kiểm tra thêm một lần nữa để xác định rằng lỗi thực sự đã được giải quyết.

Khi thực hiện kiểm tra xác nhận, điều quan trọng nhất là phải đảm bảo rằng các trường hợp kiểm thử phải được thực hiện chính xác giống như lần đầu tiên, sử dụng cùng một đầu vào, dữ liệu và môi trường kiểm thử để đảm bảo rằng các lỗi đã được sửa. Tester cần phải biết rằng trong lần kiểm thử sau khi vá lỗi khả năng sinh ra lỗi khác trong phần mềm là điều hoàn toàn có thể xảy ra. Vì vậy kiểm thử chính xác ở phiên bản hiện tại của phần mềm là chưa đủ. Cách phát hiện các điểm ngoài ý muốn của việc kiểm lỗi là thực hiện kiểm thử hồi quy.

- Kiểm thử hồi quy (Regression testing)

Tương tự như kiểm thử xác nhận thì kiểm thử hồi quy liên quan đến việc lặp lại các trường hợp kiểm thử đã được thực hiện trước đó. Kiểm thử hồi quy được thực hiện khi phần mềm thay đổi do sửa lỗi, chức năng mới.

Mục đích của kiểm thử hồi quy để xác minh rằng các sửa đổi trong phần mềm hoặc môi trường không gây ra bất lợi ngoài ý muốn, ảnh hưởng hoặc làm hư các chức năng và hệ thống vẫn đáp ứng các yêu cầu của phần mềm. Tất cả các trường hợp trong quá trình kiểm thử hồi quy sẽ được thực hiện mỗi khi một phiên bản vá lỗi của phần mềm được công nhận (release), và điều này khiến chúng trở nên lý tưởng cho tự động hóa.

5.2.2.5 Kiểm thử đơn vị (Unit Testing)

Kiểm thử đơn vị là cấp độ kiểm thử đầu tiên và thường được thực hiện bởi chính các nhà phát triển. Đây là quá trình đảm bảo các thành phần riêng lẻ của một phần mềm ở cấp độ mã có chức năng và hoạt động như chúng được thiết kế. Các nhà phát triển trong một môi trường hướng tới thử nghiệm thường sẽ viết và chạy các thử nghiệm trước khi phần mềm hoặc tính năng được chuyển cho nhóm thử nghiệm. Kiểm tra đơn vị có thể được tiến hành theo cách thủ công, nhưng tự động hóa quy trình sẽ tăng tốc chu kỳ phân phối và mở rộng phạm vi kiểm tra. Kiểm thử đơn vị cũng sẽ giúp việc gỡ lỗi dễ dàng hơn vì việc phát hiện các vấn đề sớm hơn có nghĩa là chúng mất ít thời gian hơn để khắc phục so với nếu chúng được phát hiện sau đó trong quá trình kiểm tra. TestLeft là một công cụ cho phép người kiểm tra và nhà phát triển nâng cao chuyển sang trái bằng công cụ tự động hóa kiểm tra nhanh nhất được nhúng trong bất kỳ IDE nào.

5.2.2.6 Kiểm thử sự tích hợp (Integration Testing)

Sau khi mỗi đơn vị được kiểm tra kỹ lưỡng, nó được tích hợp với các đơn vị khác để tạo ra các mô-đun hoặc thành phần được thiết kế để thực hiện các nhiệm vụ hoặc hoạt động cụ thể. Sau đó, chúng được kiểm tra theo nhóm thông qua kiểm tra tích hợp để đảm bảo toàn bộ các phân đoạn của ứng dụng hoạt động như mong đợi (tức là các tương tác giữa các đơn vị là liền mạch). Các bài kiểm tra này thường được đóng khung theo các tình

huống của người dùng, chẳng hạn như đăng nhập vào ứng dụng hoặc mở tệp. Các bài kiểm tra tích hợp có thể được tiến hành bởi các nhà phát triển hoặc những người kiểm tra độc lập và thường bao gồm sự kết hợp của các bài kiểm tra chức năng tự động và thủ công.

5.2.2.7 Kiểm thử hệ thống (System Testing)

Kiểm thử hệ thống là một phương pháp kiểm tra hộp đen được sử dụng để đánh giá tổng thể hệ thống đã hoàn thiện và tích hợp, nhằm đảm bảo nó đáp ứng các yêu cầu đã định. Chức năng của phần mềm được kiểm tra từ đầu đến cuối và thường được tiến hành bởi một nhóm kiểm thử riêng biệt hơn nhóm phát triển trước khi sản phẩm được đưa vào sản xuất.

5.2.2.8 Kiểm thử chấp nhận (Acceptance Testing)

Kiểm thử chấp nhận là giai đoạn cuối cùng của kiểm thử chức năng và được sử dụng để đánh giá xem liệu phần mềm cuối cùng đã sẵn sàng để phân phối hay chưa. Nó liên quan đến việc đảm bảo rằng sản phẩm tuân thủ tất cả các tiêu chí kinh doanh ban đầu và đáp ứng nhu cầu của người dùng cuối. Điều này yêu cầu sản phẩm được thử nghiệm cả bên trong và bên ngoài, có nghĩa là bạn cần đưa sản phẩm đến tay người dùng cuối để thử nghiệm beta cùng với những người trong nhóm đảm bảo chất lượng (QA) của bạn. Thử nghiệm beta là chìa khóa để nhận được phản hồi thực sự từ khách hàng tiềm năng và có thể giải quyết bất kỳ mối lo ngại nào về khả năng sử dụng cuối cùng [4].

5.2.2.9 Kiểm thử hiệu suất (Performance Testing)

Kiểm thử hiệu suất là một kỹ thuật kiểm tra phi chức năng được sử dụng để xác định cách một ứng dụng sẽ hoạt động trong các điều kiện khác nhau. Mục đích là để kiểm tra khả năng phản hồi và độ ổn định của nó trong các tình huống người dùng thực tế. Kiểm tra hiệu suất có thể được chia thành bốn loại:

- Kiểm tra tải là quá trình đưa lượng nhu cầu mô phỏng ngày càng tăng lên phần mềm, ứng dụng hoặc trang web của bạn để xác minh xem phần mềm, ứng dụng hoặc trang web của bạn có thể xử lý những gì được thiết kế để xử lý hay không.
- Kiểm tra căng thẳng tiến thêm một bước nữa và được sử dụng để đánh giá cách phần mềm của bạn sẽ phản hồi ở mức hoặc vượt quá tải cao nhất của nó. Mục tiêu của kiểm tra căng thẳng là cố ý làm quá tải ứng dụng cho đến khi nó bị hỏng bằng cách áp dụng cả kịch bản tải thực tế và phi thực tế. Với kiểm tra căng thẳng, bạn sẽ có thể tìm ra điểm lỗi của phần mềm của mình.
- Kiểm tra độ bền, còn được gọi là kiểm tra ngâm, được sử dụng để phân tích hành vi của ứng dụng dưới một lượng tải mô phỏng cụ thể trong khoảng thời gian dài hơn. Mục đích là để hiểu hệ thống của bạn sẽ hoạt động như thế nào trong điều kiện sử dụng lâu dài, khiến nó trở thành một quá trình lâu hơn so với thử nghiệm tải hoặc căng thẳng (được thiết kế để kết thúc sau vài giờ). Một phần quan trọng của thử nghiệm độ bền là nó giúp phát hiện ra các lỗi hỏng bộ nhớ.

- Kiểm tra tăng tốc là một loại kiểm tra tải được sử dụng để xác định cách phần mềm của bạn sẽ phản ứng với các đợt bùng nổ lớn hơn đáng kể của hoạt động hệ thống hoặc người dùng đồng thời trong những khoảng thời gian khác nhau. Tốt nhất, điều này sẽ giúp bạn hiểu điều gì sẽ xảy ra khi tải tăng đột ngột và mạnh mẽ.

5.2.2.10 Kiểm thử bảo mật (Security Testing)

Với sự gia tăng của các nền tảng thử nghiệm dựa trên đám mây và các cuộc tấn công mạng, ngày càng có nhiều mối quan tâm và nhu cầu về bảo mật của dữ liệu được sử dụng và lưu trữ trong phần mềm. Kiểm thử bảo mật là một kỹ thuật kiểm tra phần mềm phi chức năng được sử dụng để xác định xem thông tin và dữ liệu trong hệ thống có được bảo vệ hay không. Mục đích là nhằm mục đích tìm ra các lỗ hổng và rủi ro bảo mật trong hệ thống có thể dẫn đến việc truy cập trái phép hoặc mất thông tin bằng cách thăm dò các điểm yếu của ứng dụng. Có nhiều loại phương pháp kiểm tra này, mỗi loại nhằm xác minh sáu nguyên tắc bảo mật cơ bản:

- Chính trực
- Bảo mật
- Xác thực
- Ủy quyền
- Khả dụng
- Không bác bỏ

5.2.2.11 Kiểm thử khả năng sử dụng (Usability Testing)

Kiểm tra khả năng sử dụng là một phương pháp kiểm tra đo lường mức độ dễ sử dụng của ứng dụng từ góc độ người dùng cuối và thường được thực hiện trong các giai đoạn kiểm tra hệ thống hoặc kiểm tra chấp nhận. Mục đích là để xác định xem thiết kế và tính thẩm mỹ có thể nhìn thấy của một ứng dụng có đáp ứng quy trình công việc dự kiến cho các quy trình khác nhau hay không, chẳng hạn như đăng nhập vào một ứng dụng. Kiểm tra khả năng sử dụng là một cách tốt nhất để các nhóm xem xét các chức năng riêng biệt hoặc toàn bộ hệ thống, sử dụng một cách trực quan.

5.2.2.12 Kiểm thử khả năng tương thích (Compatibility Testing)

Kiểm tra tính tương thích được sử dụng để đánh giá cách một ứng dụng hoặc phần mềm sẽ hoạt động trong các môi trường khác nhau. Nó được sử dụng để kiểm tra xem sản phẩm của bạn có tương thích với nhiều hệ điều hành, nền tảng, trình duyệt hoặc cấu hình độ phân giải hay không. Mục đích là để đảm bảo rằng chức năng phần mềm của bạn được hỗ trợ nhất quán trên bất kỳ môi trường nào mà bạn mong đợi người dùng cuối của mình đang sử dụng.

5.2.2.13 Kiểm thử với TestComplete (Testing With TestComplete)

TestComplete là công cụ kiểm tra GUI tự động mạnh mẽ, vượt trội trong kiểm tra khả năng tương thích và tích hợp. Nó giúp các nhóm QA tạo và chạy các bài kiểm tra trên

máy tính để bàn, thiết bị di động và ứng dụng web - cho phép các chuyên gia kiểm tra tăng tốc chu kỳ phân phối và cải thiện chất lượng phần mềm. Testcomplete đi kèm với hỗ trợ tích hợp cho các môi trường thử nghiệm khác nhau, tích hợp với các công cụ kiểm tra hiệu suất, cũng như hỗ trợ cho các nhà quản lý chuỗi cung ứng (SCM) thân thiện với nhà phát triển, cho phép bạn tích hợp liền mạch vào quy trình phát triển của mình. Sử dụng TestComplete sẽ cho phép bạn xây dựng một khuôn khổ kiểm tra mạnh mẽ sử dụng nhiều phương pháp kiểm thử phần mềm có sẵn.

5.3 Kỹ thuật kiểm thử phần mềm

5.3.1. Kỹ thuật kiểm thử phần mềm là gì? Phân loại?

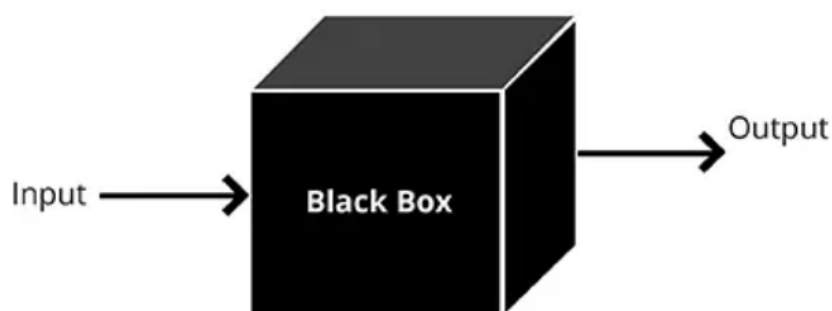
Kiểm thử một sản phẩm phần mềm là xây dựng một cách có chủ đích những tập dữ liệu và dãy thao tác nhằm đánh giá một số hoặc toàn bộ các tiêu chuẩn của sản phẩm phần mềm đó.

Thử nghiệm có hai mục đích: chỉ ra hệ thống phù hợp với đặc tả và phơi ra được các khuyết tật của hệ thống.

Kỹ thuật kiểm thử phần mềm giúp bạn thiết kế các trường hợp kiểm thử tốt hơn. Vì kiểm thử toàn diện là không thể nên kỹ thuật kiểm tra thủ công sẽ giúp giảm số lượng các trường hợp kiểm thử được thực thi trong khi tăng phạm vi kiểm thử. Chúng giúp xác định các điều kiện kiểm tra khó nhận biết.

Trong kiểm thử phần mềm, có rất nhiều kỹ thuật kiểm thử. Tuy nhiên dựa vào đặc điểm có thể kể đến Kỹ thuật kiểm thử hộp đen (Black Box test) và Kỹ thuật kiểm thử hộp trắng (White box test)

- Kỹ thuật kiểm thử hộp đen (Black Box test): Là một phương pháp kiểm thử phần mềm mà việc kiểm tra các chức năng của một ứng dụng không cần quan tâm vào cấu trúc nội bộ hoặc hoạt động của nó mà chỉ quan tâm đến dữ liệu đầu vào và kết quả chương trình có như mong đợi hay không.

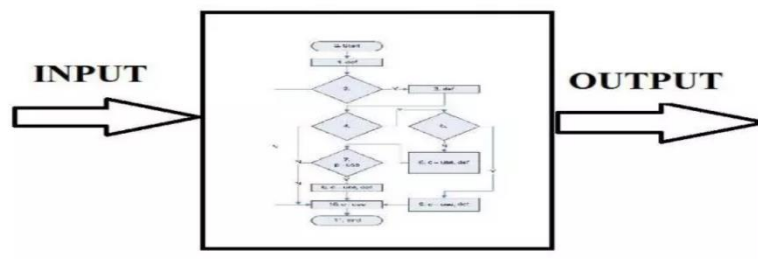


Hình 5.2 Kỹ thuật kiểm thử hộp đen

Một số kỹ thuật phổ biến trong kiểm thử hộp đen gồm:

- + Kỹ thuật phân lớp tương đương (Equivalence Class Partitioning).
- + Kỹ thuật phân tích các giá trị biên (Boundary value analysis).
- + Kỹ thuật dùng các bảng quyết định (Decision Tables)

- + Kỹ thuật kiểm thử các bộ n thân kỳ (Pairwise)
- + Kỹ thuật dùng bảng chuyển trạng thái (State Transition)
- + Kỹ thuật phân tích vùng miền (domain analysis)
- + Kỹ thuật dựa trên đặc tả Use Case (Use case)
- + Kỹ thuật dùng lược đồ quan hệ nhân quả (Cause-Effect Diagram)
- Kiểm thử hộp trắng (White box test): Là phương pháp thử nghiệm phần mềm, trong đó các thiết kế, cấu trúc giải thuật bên trong, và việc thực hiện các công việc đều được biết đến.



Hình 5.3 Kỹ thuật kiểm thử hộp trắng

Kỹ thuật White box test thích hợp dùng để kiểm thử đơn vị (Unit test)

Ngoài 2 kỹ thuật Black box test và White box test, thì có kỹ thuật Gray box test (Kiểm thử hộp xám) là sự kết hợp giữa Black box test và White box test. Gray Box Test là một phương pháp kiểm thử phần mềm được kết hợp giữa phương pháp kiểm thử Black box (hộp đen) và White box (hộp trắng). Trong Kiểm thử Hộp xám, cấu trúc bên trong sản phẩm chỉ được biết một phần.

Và dưới đây là các loại kỹ thuật kiểm thử cụ thể thông dụng:

- Phân vùng tương đương (Equivalence Class Partitioning)
- Phân tích giá trị biên (Boundary Value Analysis (BVA))
- Bảng quyết định (Decision Table based testing)
- Đoán lỗi (Error Guessing)

5.3.2. Kỹ thuật Phân vùng tương đương

Phân vùng lớp tương đương cho phép phân chia tập hợp các điều kiện kiểm tra thành một phân vùng nên được coi là giống nhau.

Phương pháp kiểm thử phần mềm này chia miền đầu vào của chương trình thành các lớp dữ liệu mà từ đó các trường hợp kiểm thử nên được thiết kế.

Với các giá trị đầu vào chia thành các vùng tương đương:

- Vùng tương đương hợp lệ: tập hợp các giá trị kiểm thử thỏa mãn điều kiện của hệ thống
- Vùng tương đương không hợp lệ: Tập hợp các giá trị kiểm thử mô tả trạng thái khác của hệ thống: sai, thiếu, không đúng, ...

- Mục đích: Giảm đáng kể số lượng trường hợp kiểm tra (test case) cần phải thiết kế vì với mỗi lớp tương đương ta chỉ cần test trên các phân tử đại diện.

- Thiết kế test case bằng phân lớp tương đương tiến hành theo 2 bước:
- Xác định các lớp tương đương
- Xác định các ca kiểm thử
- Nguyên tắc:
- 1 lớp các giá trị lớn hơn
- 1 lớp các giá trị nhỏ hơn
- n lớp các giá trị hợp lệ

Ví dụ: Thiết kế test case cho ô text chỉ cho nhập số nguyên với độ dài ký tự thuộc [1-10] hoặc [20-30]

Với yêu cầu trên ta có các vùng:

- nhỏ hơn 1: vùng không hợp lệ
- [1-10]: vùng hợp lệ
- lớn hơn 10 và nhỏ hơn 20: vùng không hợp lệ
- [20-30]: vùng hợp lệ
- lớn hơn 30: vùng không hợp lệ
- Nhập các ký tự không phải số nguyên: vùng không hợp lệ

Vì vậy có các case:

- Case hợp lệ:
 - + Nhập 5 ký tự
 - + Nhập 25 ký tự số
- Case không hợp lệ:
 - + Không nhập vào trường
 - + Nhập 15 ký tự
 - + Nhập số thập phân
 - + Nhập 35 ký tự
 - + Nhập ký tự chữ: Tiếng Việt, Tiếng Anh, Full-size, Half-size
 - + Nhập ký tự đặc biệt, dấu cách, ký tự Enter
 - + Nhập câu lệnh SQL injection, HTML, XSS

5.3.3. Kỹ thuật Phân tích giá trị biên

Phân tích giá trị biên dựa trên việc kiểm thử tại các ranh giới giữa các phân vùng, Chúng ta sẽ tập trung vào các giá trị biên chứ không kiểm tra toàn bộ dữ liệu. Thay vì chọn nhiều giá trị trong lớp đương tương để làm đại diện, phân tích giá trị biên yêu cầu chọn một hoặc vài giá trị là các cạnh của lớp tương đương để làm điều kiện kiểm tra.

Chúng ta thường thấy rằng một số lượng lớn lỗi xảy ra tại các ranh giới của các giá trị đầu vào được xác định thay vì các giá trị giữa, còn được gọi là các giá trị biên. Từ đó đưa ra lựa chọn các test case thực hiện giá trị đầu vào các giá trị biên.

Kỹ thuật thiết kế test case này bổ sung cho phân vùng tương đương. Kỹ thuật kiểm thử phần mềm này dựa trên nguyên tắc: Nếu một hệ thống hoạt động tốt với các giá trị biên thì nó sẽ hoạt động tốt cho tất cả các giá trị nằm giữa hai giá trị biên.

Phân tích giá trị biên sẽ chọn các giá trị:

- Giá trị ngay dưới giá trị nhỏ nhất
- Giá trị nhỏ nhất
- Giá trị ngay trên giá trị nhỏ nhất
- Giá trị ngay dưới giá trị lớn nhất
- Giá trị lớn nhất
- Giá trị ngay trên giá trị lớn nhất

Ví dụ: Với ví dụ trên ta có các case:

- Không nhập ký tự nào (Giá trị ngay dưới giá trị nhỏ nhất vùng 1)
- Nhập 1 ký tự (giá trị nhỏ nhất vùng 1)
- Nhập 2 ký tự (giá trị ngay trên giá trị nhỏ nhất vùng 1)
- Nhập 9 ký tự (giá trị ngay dưới giá trị lớn nhất vùng 1)
- Nhập 10 ký tự(giá trị lớn nhất vùng 1)
- Nhập 11 ký tự(giá trị ngay trên giá trị lớn nhất vùng 1)
- Nhập 19 ký tự (giá trị ngay dưới giá trị nhỏ nhất vùng 2)
- Nhập 20 ký tự(giá trị nhỏ nhất vùng 2)
- Nhập 21 ký tự (giá trị ngay trên giá trị nhỏ nhất vùng 2)
- Nhập 29 ký tự(giá trị ngay dưới giá trị lớn nhất vùng 2)
- Nhập 30 ký tự(giá trị lớn nhất vùng 2)
- Nhập 31 ký tự(giá trị ngay trên giá trị lớn nhất vùng 2)

=> Kết hợp kỹ thuật phân vùng tương đương với phân tích giá trị biên ta có các case:

- Không nhập ký tự nào
- Nhập 1 ký tự
- Nhập 5 ký tự
- Nhập 10 ký tự
- Nhập 11 ký tự
- Nhập 19 ký tự
- Nhập 20 ký tự

-
- Nhập 21 ký tự
 - Nhập 25 ký tự
 - Nhập 30 ký tự
 - Nhập 31 ký tự
 - Nhập số thập phân
 - Nhập ký tự chữ: Tiếng Việt, Tiếng Anh, Full-size, Half-size
 - Nhập ký tự đặc biệt, phím cách, kí tự Enter
 - Nhập câu lệnh SQL injection, HTML, XSS

5.3.4. Bảng quyết định

Bảng quyết định còn được gọi là bảng Nguyên nhân – Kết quả (Cause-Effect).

Kỹ thuật kiểm thử phần mềm này được sử dụng cho các chức năng cần sự kết hợp của các yếu tố đầu vào các biến.

Ví dụ: Nút Submit phải được chọn (enable) nếu người dùng đã nhập tất cả các trường bắt buộc.

Đầu tiên là xác định đầu ra của các chức năng có phụ thuộc vào sự kết hợp của các đầu vào. Nếu có tập hợp kết hợp đầu vào lớn, thì hãy chia nó thành các tập hợp nhỏ hơn hữu ích cho việc quản lý bảng quyết định.

Đối với mọi chức năng, cần tạo một bảng và liệt kê tất cả các loại kết hợp đầu vào và đầu ra tương ứng. Điều này giúp xác định các điều kiện bị tester bỏ qua.

Các bước để tạo bảng quyết định:

- Nhập đầu vào theo hàng
- Nhập tất cả các quy tắc trong cột
- Điền vào bảng với sự kết hợp của đầu vào
- Trong hàng cuối cùng, ghi chú đầu ra so với kết hợp đầu vào.

5.3.5. Đoán lỗi

Đoán lỗi là một kỹ thuật kiểm thử phần mềm dựa trên việc đoán lỗi có thể chiếm ưu thế trong code. Đây là một kỹ thuật dựa trên kinh nghiệm, trong đó nhà phân tích kiểm thử sử dụng kinh nghiệm của mình để đoán phần có vấn đề hoặc có lỗi của ứng dụng kiểm thử.

Kỹ thuật xác định danh sách các lỗi có thể xảy ra hoặc các tình huống dễ xảy ra lỗi. Sau đó, người kiểm thử viết test cases để tìm kiếm những lỗi đó. Để thiết kế các test case dựa trên kỹ thuật kiểm thử phần mềm này, nhà phân tích có thể sử dụng các kinh nghiệm trong quá khứ để xác định các điều kiện.

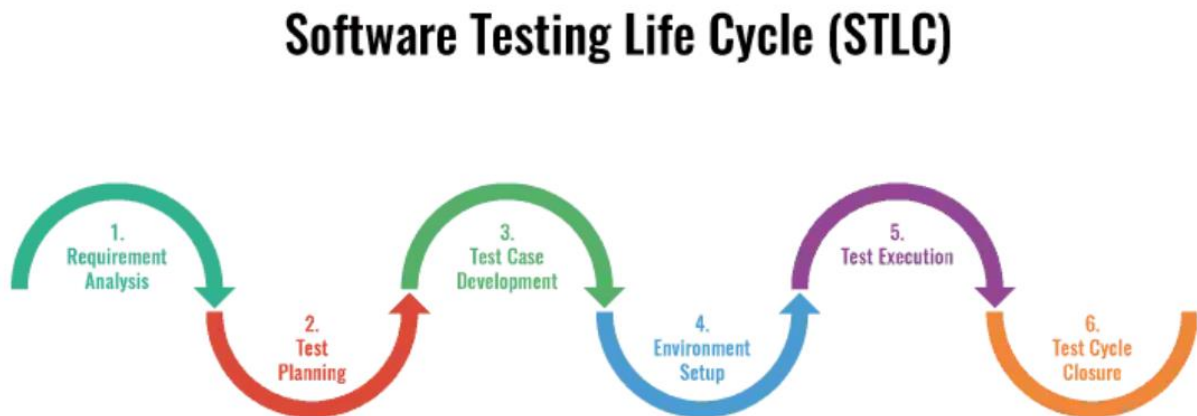
Cách đoán lỗi:

- Tester nên sử dụng kinh nghiệm trước đây để kiểm thử các ứng dụng tương tự
- Hiểu biết về hệ thống đang kiểm thử

- Kiến thức về các lỗi thực hiện điển hình
- Nhớ những chức năng phức tạp trước đây
- Đánh giá lịch sử dữ liệu và kết quả kiểm thử

5.4 Quy trình kiểm thử phần mềm

Quy trình kiểm thử phần mềm (Software testing life circle – STLC) xác định các giai đoạn trong kiểm thử phần mềm. Tuy nhiên, không có STLC tiêu chuẩn cố định nào trên thế giới, nhưng về cơ bản quy trình kiểm thử bao gồm những giai đoạn sau [5]:



Hình 5.4 Quy trình kiểm thử

1. Requirement analysis - Phân tích yêu cầu
2. Test planning - Lập kế hoạch kiểm thử
3. Test case development - Thiết kế kịch bản kiểm thử
4. Test environment set up - Thiết lập môi trường kiểm thử
5. Test execution - Thực hiện kiểm thử
6. Test cycle closure - Đóng chu trình kiểm thử

Các giai đoạn kiểm thử được thực hiện một cách tuần tự. Mỗi giai đoạn sẽ có những mục tiêu khác nhau, đầu vào và kết quả đầu ra khác nhau nhưng mục đích cuối cùng vẫn là đảm bảo chất lượng sản phẩm phần mềm tốt nhất. Sau đây, chúng ta sẽ tìm hiểu chi tiết thông tin về các hoạt động, ai là người thực hiện, đầu vào, đầu ra của từng giai đoạn trong quy trình kiểm thử phần mềm:

5.4.1. Requirement analysis - Phân tích yêu cầu

Đầu vào

Đầu vào của giai đoạn phân tích yêu cầu bao gồm các tài liệu như: tài liệu đặc tả yêu cầu, tài liệu thiết kế hệ thống, tài liệu khách hàng yêu cầu về các tiêu chí chấp nhận của sản phẩm, bản nguyên mẫu (prototype) yêu cầu của khách hàng (nếu có), ...

Hoạt động

- Phân tích yêu cầu là giai đoạn đầu tiên trong quy trình kiểm thử phần mềm.

- Nhóm đảm bảo chất lượng (QA team) sẽ thực hiện đọc hiểu, nghiên cứu và phân tích cụ thể các yêu cầu trong tài liệu đặc tả của dự án hoặc tài liệu khách hàng. Qua hoạt động này, QA team sẽ nắm bắt được các yêu cầu mà dự án đưa ra bao gồm yêu cầu kiểm thử chức năng, phi chức năng nào.

- Ngoài ra, trong quá trình phân tích, nghiên cứu tài liệu, nếu có câu hỏi phát sinh hay đề xuất giải quyết, QA team sẽ đưa ra câu hỏi với các bên liên quan như phân tích kinh doanh (Business Analysis - BA), quản lý dự án (Project Manager - PM), trưởng nhóm (team leader), khách hàng để hiểu chính xác hơn về yêu cầu của sản phẩm. Những câu hỏi này sẽ được lưu trữ vào file câu hỏi và trả lời (Question and Answer - Q&A). Các câu hỏi nên được đưa ra dưới dạng Yes, No question hoặc các lựa chọn để tiết kiệm thời gian trả lời cũng như hỗ trợ đưa ra những gợi ý hay đề xây dựng sản phẩm ngay từ đầu. Như vậy, chúng ta không nên nêu ra những câu hỏi dạng là gì, như thế nào, tại sao, ... Những câu hỏi như thế thường mất thời gian để giải thích và cũng khó có thể giải thích một cách chi tiết nhất có thể. Hơn nữa, đối với khách hàng không có sự hiểu biết về lĩnh vực phần mềm mà họ yêu cầu thì càng không thể trả lời những câu hỏi mang tính chuyên môn cao. Chính chúng ta sẽ là người hỗ trợ và đưa ra giải pháp thích hợp cho khách hàng lựa chọn.

Đầu ra

Đầu ra của giai đoạn phân tích yêu cầu bao gồm tài liệu chứa các câu hỏi và câu trả lời liên quan đến nghiệp vụ của hệ thống, tài liệu báo cáo tính khả thi, phân tích rủi ro của việc kiểm thử phần mềm.

5.4.2. Test planning - Lập kế hoạch kiểm thử

Đầu vào

Đầu vào của giai đoạn lập kế hoạch kiểm thử là các tài liệu đặc tả đã được cập nhật thông qua các câu hỏi và trả lời được đưa ra trong giai đoạn phân tích yêu cầu, tài liệu báo cáo tính khả thi, phân tích rủi ro của việc kiểm thử phần mềm.

Hoạt động

Dựa vào các tài liệu được cung cấp và cập nhật mới nhất, thông thường, quản lý kiểm thử (test manager) hoặc trưởng nhóm kiểm thử (test leader) sẽ là người lập kế hoạch kiểm thử cho cả QA team. Lập kế hoạch kiểm thử nhằm xác định một số yếu tố quan trọng sau [5]:

- Xác định phạm vi (Scope) dự án: Dự án thực hiện trong thời gian bao lâu? Bao gồm những công việc gì cho từng khoảng thời gian xác định? Từ đó đưa ra lịch trình thực hiện cho từng công việc nhỏ sao cho phù hợp với toàn bộ đội dự án.

- Xác định phương pháp tiếp cận: Nói về cách tiếp cận để kiểm thử cho một đối tượng nào đó, thì phải dựa vào nhiều thứ, ví dụ như: Thời gian cho phép test có phù hợp với con số ước lượng, nhiều hay ít, yêu cầu chất lượng từ phía khách hàng thế nào? Cao, thấp hay khắt khe hay không có yêu cầu nào? Công nghệ, kỹ thuật sử dụng để phát triển ứng dụng này là gì? Lĩnh vực của hệ thống, sản phẩm đang được test là gì?... Từ đó, người

quản lý kiểm thử có thể đưa ra những phương pháp và kế hoạch phù hợp nhất cho cả quá trình thực hiện dự án sao cho đúng với các tiêu chí chấp nhận của sản phẩm và kịp tiến độ với các mốc thời gian bàn giao, phát hành.

- Xác định các nguồn lực
 - + Con người: Bao nhiêu người tham gia dự án, ai sẽ test phần nào, bao nhiêu tester tham gia? Tester và nhóm phát triển có kinh nghiệm về lĩnh vực này không?
 - + Thiết bị: số lượng server, version, máy tính, mobile để thực hiện test là bao nhiêu.
- Lên kế hoạch thiết kế công việc test: Bản kế hoạch kiểm thử sẽ bao gồm các nội dung:
 - + Liệt kê các chức năng cần kiểm thử.
 - + Để thực hiện test chức năng này thì cần làm những công việc gì, trong thời gian bao lâu, cái nào thực hiện trước, cái nào thực hiện sau, ai là người thực hiện.
 - + Xác định điều kiện bắt đầu: xác định những điều kiện tối thiểu để bắt đầu hoạt động kiểm thử cho từng chức năng.
 - + Xác định điều kiện kết thúc: khi có những điều kiện nào thì sẽ kết thúc việc kiểm thử.

Đầu ra

Đầu ra của giai đoạn lập kế hoạch bao gồm các tài liệu như kế hoạch kiểm thử (test plan), đánh giá kiểm thử (test estimation), lịch trình kiểm thử (test schedule).

5.4.3. Test case development - Thiết kế kịch bản kiểm thử

Đầu vào

Đầu vào của giai đoạn thiết kế kịch bản kiểm thử là test plan, test estimation, test schedule, các tài liệu đặc tả đã được cập nhật.

Hoạt động

- Kiểm tra lại tài liệu: Đầu tiên, các kiểm thử viên cần kiểm tra lại tất cả các tài liệu để xác định công việc cần làm, các công việc có khác gì so với dự án trước khách hàng đưa cho, chức năng nào cần test, chức năng nào không cần test lại nữa. Từ đó, vừa có thể tiết kiệm thời gian mà vẫn đưa ra được một kịch bản kiểm thử đầy đủ và hiệu quả.
- Viết test case, check list: Sau đó, tester bắt tay vào việc viết test case chi tiết dựa vào kế hoạch đã đưa ra và vận dụng các kỹ thuật thiết kế kịch bản kiểm thử. Test case cần bao phủ được tất cả các trường hợp kiểm thử có thể xảy ra cũng như đáp ứng đầy đủ các tiêu chí của sản phẩm. Đồng thời tester cũng cần đánh giá mức độ ưu tiên cho từng test case.
- Chuẩn bị dữ liệu kiểm thử: Cùng với việc tạo ra các test case chi tiết, đội kiểm thử cũng cần chuẩn bị trước các dữ liệu kiểm thử cho các trường hợp cần thiết như dữ liệu kiểm tra (test data), kịch bản kiểm tra (test script).

- Kiểm tra lại các test case, check list: Sau khi hoàn thành, các thành viên trong đội kiểm thử hoặc test leader cũng cần kiểm tra lại các test case đã tạo để có thể bổ sung, hỗ trợ lẫn nhau nhằm tránh những sai sót trong thiết kế test case và rủi ro về sau.

Đầu ra

Sau khi hoàn thành thiết kế kịch bản kiểm thử, đội kiểm thử sẽ có các tài liệu bao gồm: bản thiết kế kiểm thử (test design), test case, check list, test data, kịch bản kiểm thử tự động (test automation script).

5.4.4. Test environment set up - Thiết lập môi trường kiểm thử

Đầu vào

Đầu vào của giai đoạn cài đặt môi trường kiểm thử là test plan, smoke test case, test data.

Hoạt động

- Việc cài đặt môi trường kiểm thử là giai đoạn cũng rất quan trọng trong vòng đời phát triển phần mềm. Môi trường kiểm thử sẽ được quyết định dựa trên những yêu cầu của khách hàng, hay đặc thù của sản phẩm ví dụ như server, client, network, ...
- Tester cần chuẩn bị một vài test case để kiểm tra xem môi trường cài đặt đã sẵn sàng cho việc kiểm thử hay chưa. Đây chính là việc thực thi các smoke test case.

Đầu ra

Đầu ra của giai đoạn này là môi trường đã được cài đặt đúng theo yêu cầu, sẵn sàng cho việc kiểm thử và kết quả của smoke test case.

5.4.5. Test execution - Thực hiện kiểm thử

Đầu vào

Tài liệu đầu vào của giai đoạn này là test plan, test design, test case, check list, test data, test automation script.

Hoạt động

- Thực hiện các test case như thiết kế và mức độ ưu tiên đã đưa ra trên môi trường đã được cài đặt.
- So sánh với kết quả mong đợi sau báo cáo các lỗi xảy ra lên công cụ quản lý lỗi và theo dõi trạng thái của lỗi đến khi được sửa thành công.
- Thực hiện kiểm tra lại để kiểm chứng các lỗi đã được khắc phục và kiểm tra hồi quy (regression tes) khi có sự thay đổi liên quan.
- Trong quá trình thực hiện kiểm thử, kiểm thử viên cũng có thể hỗ trợ, đề xuất cho cả đội dự án để có giải pháp hợp lý và kết hợp công việc hiệu quả.
- Đo và phân tích tiến độ: kiểm thử viên cũng cần kiểm soát chặt chẽ tiến độ công việc của mình bằng cách so sánh tiến độ thực tế với kế hoạch, nếu chậm cần phải điều chỉnh sao cho kịp tiến độ dự án, nếu nhanh cũng cần điều chỉnh vì có thể test lead lên kế

hoạch chưa sát với thực tế dự án. Từ đó có thể sửa chữa test plan cần điều chỉnh để phù hợp với tiến độ dự án đưa ra.

- Báo cáo thường xuyên cho quản lý dự án và khách hàng về tình hình thực hiện dự án: Cung cấp thông tin trong quá trình kiểm thử đã làm được những chức năng nào, còn chức năng nào, hoàn thành được bao nhiêu phần trăm công việc, báo cáo các trường hợp phát sinh sớm, tránh ảnh hưởng tiến độ công việc của cả ngày.

Đầu ra

Đầu ra của giai đoạn này là kết quả kiểm thử (test results), danh sách các lỗi tìm được (defect reports).

5.4.6. Test cycle closure - Đóng chu trình kiểm thử

Đầu vào

Đầu vào của giai đoạn đóng chu trình kiểm thử là bao gồm tất cả những tài liệu liên quan đã được tổng hợp, ghi chép và hoàn thiện đầy đủ trong suốt quy trình kiểm thử của dự án: tài liệu phân tích đặc tả yêu cầu, kế hoạch kiểm tra (test plan), kết quả kiểm tra (test results), báo cáo lỗi (defect reports), tài liệu câu hỏi và trả lời (Q&A), ... [5].

Hoạt động

- Đây là giai đoạn cuối cùng trong quy trình kiểm thử phần mềm.
- Ở giai đoạn này, nhóm đảm bảo chất lượng (QA team) thực hiện tổng kết, báo cáo kết quả về việc thực thi test case, bao nhiêu trường hợp đã vượt qua (case pass), thất bại (fail), bao nhiêu case đã được khắc phục, mức độ nghiêm trọng của lỗi, bao nhiêu lỗi cao, thấp, lỗi còn nhiều ở chức năng nào, nhóm phát triển nào nhiều lỗi. Chức năng nào đã hoàn thành test, chưa hoàn thành test, trễ tiến độ bàn giao.
- Đánh giá các tiêu chí hoàn thành như phạm vi kiểm tra, chất lượng, chi phí, thời gian, mục tiêu kinh doanh quan trọng.
- Ngoài ra, giai đoạn này cũng thảo luận tất cả những điểm tốt, điểm chưa tốt và rút ra bài học kinh nghiệm cho những dự án sau, giúp cải thiện quy trình kiểm thử.

Đầu ra

Đầu ra của giai đoạn này bao gồm các tài liệu: Báo cáo kiểm thử (test report), kết quả kiểm thử cuối cùng (test results final).

Như vậy, quy trình kiểm thử phần mềm gồm những thông tin chính sau đây:

Bảng 5-1 Quy trình kiểm thử phần mềm

Giai đoạn	Đầu vào	Hoạt động	Đầu ra	Người thực hiện
Requirement analysis	* Tài liệu đặc tả yêu cầu * Tài liệu	Nghiên cứu, phân tích yêu cầu dự án	* Q&A document * Tài liệu báo cáo tính khả thi,	QA team

Giai đoạn	Đầu vào	Hoạt động	Đầu ra	Người thực hiện
	khách hàng * Tài liệu thiết kế hệ thống		phân tích rủi ro của việc kiểm thử phần mềm	
Test planning	* Tài liệu đặc tả yêu cầu (đã được cập nhật) * Tài liệu báo cáo tính khả thi, phân tích rủi ro của việc kiểm thử phần mềm	* Xác định phạm vi dự án * Xác định phương pháp tiếp cận * Xác định nguồn lực * Lên kế hoạch thiết kế công việc test	* Test Plan * Test Estimation * Test Schedule	Test manager, Test leader
Test case development	* Tài liệu đặc tả yêu cầu (đã được cập nhật) * Test plan * Test estimation * Test schedule	* Review tài liệu * Viết test case, check list * Chuẩn bị dữ liệu kiểm thử * Review test case, check list	* Test design * Test case, check list * Test data * Test automation script	Tester
Test environment setup	* Test plan * Smoke test case * Test data	* Thiết lập môi trường kiểm thử (server, client, network, ...) * Kiểm tra môi trường kiểm thử bằng các smoke test case	* Môi trường kiểm thử * Kết quả của smoke test case	Tester

Giai đoạn	Đầu vào	Hoạt động	Đầu ra	Người thực hiện
Test execution	* Test plan * Test design, test case, check list * Test data * Test automation script	* Thực hiện kiểm thử phần mềm * So sánh với kết quả mong đợi và báo cáo các bug xảy ra lên tool quản lý lỗi * Thực hiện re-test để verify các bug đã được fix và regression test khi có sự thay đổi liên quan * Đo và phân tích tiến độ * Điều chỉnh, sửa chữa tài liệu tiến độ dự án theo tình hình thực tế * Report thường xuyên cho Project Manager và khách hàng về tình hình thực hiện dự án	* Test results * Defect reports	Tester
Test cycle closure	* Tài liệu đặc tả yêu cầu * Test plan * Test results * Defect reports * Tài liệu Q&A	* Tổng kết, báo cáo kết quả về việc thực thi test case * Đánh giá các tiêu chí hoàn thành như phạm vi kiểm tra, chất lượng, chi phí, thời gian, mục tiêu kinh doanh quan trọng * Thảo luận và rút ra bài học kinh nghiệm	* Test report * Test results (final)	QA team

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

Câu 1: Độ tin cậy của phần mềm là gì? Vì sao phải đảm bảo chất lượng phần mềm?

Câu 2: Có những chiến lược kiểm thử nào? Phân tích các chiến lược kiểm thử đó?

Câu 3: Thế nào là kiểm thử phần mềm? Các đặc điểm của việc kiểm thử phần mềm?

Câu 4: Thế nào là kiểm tra tính đúng đắn của văn bản chương trình?

Câu 5: Đánh giá các công cụ hỗ trợ kiểm thử phần mềm?

Câu hỏi trắc nghiệm:

Câu 1: Trong giai đoạn phát triển hệ thống, loại kiểm thử nào sau đây không được chấp nhận?

- a. Kiểm thử đơn vị (unit testing)
- b. Kiểm thử từ dưới lên (bottom up testing)
- c. Kiểm thử tích hợp (integration testing)
- d. Kiểm thử chấp nhận (acceptance testing)

Câu 2: Loại lỗi (error) nào sau đây bị bỏ qua trong kiểm thử hộp đen nhưng có thể được phát hiện trong kiểm thử hộp trắng?

- a. Lỗi luận lý (logic error)
- b. Lỗi thực thi và lỗi chính tả (performance error)
- c. Lỗi chính tả
- d. Lỗi luận lý và lỗi chính tả

Câu 3: Ai là người thích hợp nhất cho việc kiểm thử đơn vị (unit testing)?

- a. Người quản lý cấu hình
- b. Người kiểm thử độc lập
- c. Nhóm kiểm tra chất lượng
- d. Người phát triển

Câu 4: Nếu 1 hàm có n biến thì số trường hợp kiểm thử giá trị biên của các biến sẽ là:

- a. $4n + 3$
- b. $4n + 1$
- c. $4n$
- d. $n + 4$

Câu 5: Kiểm thử sự chấp nhận (Acceptance tests) thường được thực hiện bởi:

- a. Nhà phát triển
- b. Người dùng cuối

-
- c. Đội kiểm thử
 - d. Kỹ sư hệ thống

Bài tập vận dụng:

Bài 1: Bạn được giao nhiệm vụ kiểm thử phần mềm “Quản lý sinh viên của một trường Đại học”, bạn hãy phân tích các kỹ thuật, phương pháp để thực hiện nhiệm vụ này?

Bài 2: Bạn được giao nhiệm vụ kiểm thử phần mềm “Game 3D”, bạn hãy phân tích các kỹ thuật, phương pháp để thực hiện nhiệm vụ này?

Bài 3: Bạn được giao nhiệm vụ kiểm thử phần mềm “Website quản lý bán hàng online của 1 hệ thống bán quần áo”, bạn hãy phân tích các kỹ thuật, phương pháp để thực hiện nhiệm vụ này?

Bài 4: Bạn được giao nhiệm vụ kiểm thử ứng dụng dạng “Diệt virus”, bạn hãy phân tích các kỹ thuật, phương pháp để thực hiện nhiệm vụ này?

Bài 5: Hãy chọn một phần mềm mà bạn từng tham gia phát triển hoặc bạn có định hướng tìm hiểu, hãy phân tích các kỹ thuật, các phương pháp để thực hiện nhiệm vụ kiểm thử phần mềm này?

CHƯƠNG 6: BẢO TRÌ PHẦN MỀM VÀ QUẢN LÝ THAY ĐỔI PHẦN MỀM

Mục tiêu của chương:

Bảo trì là giai đoạn cuối cùng của một chu trình phát triển phần mềm. Các chương trình máy tính luôn thay đổi - phải mở rộng, sửa lỗi, tối ưu hoá, ... và theo thống kê thì bảo trì chiếm từ 40% đến 80% toàn bộ chi phí của một dự án phần mềm. Do vậy, bảo trì là một hoạt động phức tạp nhưng nó lại là rất cần thiết trong chu trình sống của sản phẩm phần mềm để đảm bảo cho phần mềm phù hợp với người sử dụng.

Do nhu cầu phát triển của các hệ thống thông tin, rất hiếm hay không muốn nói là không thể có một hệ thống thông tin không có sự thay đổi trong suốt chu trình sống của nó. Để duy trì tính đúng đắn, trật tự trong giai đoạn bảo trì thì quản lý sự thay đổi phần mềm là một hoạt động cần thiết song song.

Trong chương này, sẽ trình bày các vấn đề liên quan đến bảo trì phần mềm, bao gồm: bảo trì phần mềm là gì, đặc điểm, các hình thức, các công việc của bảo trì phần mềm và quản lý thay đổi phần mềm.

6.1 Định nghĩa bảo trì phần mềm và phân loại

6.1.1. Định nghĩa bảo trì phần mềm:

Bảo trì phần mềm đó là điều chỉnh các lỗi mà chưa được phát hiện trong giai đoạn xây dựng hoặc trong quá trình sử dụng phần mềm có nhiều thay đổi, nâng cấp tính năng sử dụng và an toàn vận hành của phần mềm. Việc bảo trì phần mềm định kỳ chiếm tới 65-75% công sức trong chu kỳ sống của một phần mềm.

Quá trình phát triển phần mềm bao gồm nhiều giai đoạn: xác định yêu cầu, phân tích, thiết kế, cài đặt, kiểm thử, triển khai và bảo trì phần mềm. Nhiệm vụ của giai đoạn bảo trì phần mềm là giữ cho phần mềm được cập nhật khi môi trường thay đổi và yêu cầu người sử dụng thay đổi.

Theo IEEE (1993), thì bảo trì phần mềm được định nghĩa là việc sửa đổi một phần mềm sau khi đã bàn giao để chỉnh lại các lỗi phát sinh, cải thiện hiệu năng của phần mềm hoặc các thuộc tính khác, hoặc làm cho phần mềm thích ứng trong một môi trường đã bị thay đổi.

Bảo trì phần mềm là một phần của Vòng đời phát triển phần mềm. Mục đích chính của nó là sửa đổi và cập nhật ứng dụng phần mềm sau khi giao hàng để sửa lỗi và cải thiện hiệu suất. Phần mềm là một mô hình của thế giới thực. Khi thế giới thực thay đổi, phần mềm yêu cầu thay đổi bất cứ khi nào có thể [1].

6.1.2. Phân loại bảo trì phần mềm:

Bảo trì phần mềm là phức tạp và chúng ta có thể chia hoạt động bảo trì ra làm bốn hoạt động như sau:

-
- Bảo trì hiệu chỉnh
 - Bảo trì thích hợp
 - Bảo trì hoàn thiện
 - Bảo trì phòng ngừa

6.1.2.1 Bảo trì hiệu chỉnh

Công việc bảo trì đầu tiên cần phải thực hiện là do việc kiểm tra chương trình không thể tránh được mọi lỗi ẩn chứa bên trong một hệ phần mềm lớn. Trong khi sử dụng bất kỳ một chương trình lớn nào, các lỗi sẽ được báo về lại cho người phát triển.

Bảo trì hiệu chỉnh chính là quá trình phân tích và hiệu chỉnh một hay nhiều lỗi của chương trình .

6.1.2.2 Bảo trì thích hợp

Hoạt động thứ hai diễn ra bởi sự thay đổi thường xuyên môi trường. Những thể hệ phần cứng mới dường như được công bố theo chu trình 24 tháng một lần. Những hệ điều hành mới hay phiên bản mới của các hệ cũ đều đặn xuất hiện; thiết bị ngoại vi và các thành phần hệ thống khác liên tục được nâng cấp và thay đổi. Thời gian hữu dụng của một phần mềm ứng dụng mặt khác lại dễ dàng vượt qua thời hạn mười năm, lâu hơn môi trường hệ thống đã phát triển nó đầu tiên.

Bảo trì thích hợp là hoạt động sửa đổi phần mềm để thích ứng được với những thay đổi của môi trường.

6.1.2.3 Bảo trì hoàn thiện

Hoạt động thứ ba diễn ra khi một phần mềm đã được hoàn tất thành công. Hoạt động này chiếm hầu hết các công sức tiêu tốn cho việc bảo trì phần mềm. Lúc sử dụng, các yêu cầu về những khả năng mới, các thay đổi những chức năng đã có, và các mở rộng tổng quát được người dùng gửi đến.

Để thỏa mãn những yêu cầu phát triển của người sử dụng, ta tiến hành bảo trì hoàn thiện.

6.1.2.4 Bảo trì phòng ngừa

Bảo trì phòng ngừa là hoạt động bảo trì diễn ra khi phần mềm được thay đổi để cải thiện tính năng bảo trì hay độ tin cậy trong tương lai hoặc để cung cấp một nền tảng tốt hơn cho những mở rộng sau này.

Bảo trì phòng ngừa, hoạt động này vẫn còn nhiều xa lạ trong thế giới phần mềm hiện nay.

Các thuật ngữ dùng để mô tả ba hoạt động bảo trì đầu tiên là do Swanson đề xướng. Thuật ngữ thứ tư thường được dùng trong việc bảo trì phần cứng hay các hệ thống vật lý khác. Tuy nhiên cần chú ý rằng những điểm tương tự giữa bảo trì phần mềm và bảo trì phần cứng có thể gây nhầm lẫn. Phần mềm khác với phần cứng, không thể tận dụng được,

vì vậy hoạt động bảo trì phần cứng chủ yếu - thay thế các bộ phận bị hỏng hóc hay gãy vỡ - không được kể đến.

Trong thực tế của hoạt động bảo trì, các nhiệm vụ được làm như một phần của bảo trì tiếp hợp và bảo trì hoàn thiện cũng giống như các nhiệm vụ cần làm trong giai đoạn phát triển của một chu trình phần mềm. Để tiếp hợp hay hoàn thiện, chúng ta đều phải xác định yêu cầu, thiết kế lại, tạo mã và kiểm tra phần mềm có được. Thông thường các nhiệm vụ đó đã được gọi là bảo trì rồi [1].

6.2 Đặc điểm của bảo trì phần mềm

Bảo trì phần mềm cho tới gần đây vẫn còn là một giai đoạn bị coi nhẹ của chu trình phần mềm. Kiến thức về bảo trì có được rất ít khi so sánh với các giai đoạn hoạch định và phát triển. Có rất ít các số liệu nghiên cứu và chế tạo tập trung vào đề tài này, và rất ít các phương pháp kỹ thuật được đưa ra. Để hiểu được điểm bản chất của bảo trì, chúng ta sẽ xem xét các vấn đề từ ba góc độ khác nhau:

- Các hoạt động cần thiết để hoàn thành giai đoạn bảo trì và tính toàn vẹn của một cách tiếp cận theo công nghệ phần mềm đối với hiệu quả của những hoạt động đó, hay sự thiếu hụt nó.
- Chi phí kèm theo giai đoạn bảo trì.
- Các vấn đề thường gặp phải khi tiến hành bảo trì phần mềm.

6.2.1. Bảo trì có cấu trúc đối với bảo trì không cấu trúc.

Nếu thành phần có được duy nhất của một cấu hình phần mềm là mã nguồn, hoạt động bảo trì bắt đầu với việc đánh giá chi tiết mã nguồn thường là khá phức tạp với những tài liệu nghèo nàn bên trong. Những đặc điểm tế nhị như cấu trúc phần mềm, các cấu trúc dữ liệu toàn cục, giao diện hệ thống, hoạt động và các ràng buộc thiết kế thường rất khó sáng tỏ và hay bị hiểu lầm. Các thay đổi lặt vặt thường xuyên làm cho các mã rất khó đánh giá. Các kiểm tra hồi quy (lặp lại các kiểm tra trước kia để đảm bảo rằng những thay đổi không tạo ra lỗi trong phần mềm đã hoạt động) là không thể thực hiện được bởi không hề có các bản lưu về các kiểm tra đó. Chúng ta đang tiến hành phép bảo trì không cấu trúc và đang phải trả giá (khi lãng phí công sức và gây tâm trạng thất vọng). Sự trả giá này luôn đi kèm với các phần mềm đã không được phát triển theo những phương pháp đúng đắn.

Nếu có một cấu hình phần mềm hoàn thiện, nhiệm vụ bảo trì bắt đầu bằng việc đánh giá các tài liệu thiết kế. Sau đó là xác định các đặc điểm thuộc cấu trúc quan trọng, các đặc điểm hoạt động và giao diện. Tính toàn vẹn của những sửa đổi và hiệu chỉnh cần thiết sẽ được đánh giá và một kế hoạch sửa đổi sẽ được thiết lập. Thiết kế được thay đổi (sử dụng những kỹ thuật phù hợp với những điều đã bàn luận ở các chương trước) rồi nhận xét đánh giá. Mã nguồn được phát triển, sau đó tiến hành các kiểm tra hồi quy sử dụng thông tin chứa trong phần "đặc tả kiểm tra" và rồi phần mềm lại được phát hành [2].

Các mô tả trên đây là phép bảo trì cấu trúc và được tiến hành như là kết quả của những ứng dụng trước đây trong khoa học về công nghệ phần mềm. Mặc dù sự có mặt của

một cấu hình phần mềm không đảm bảo được các vấn đề bảo trì nảy sinh, nhưng khối lượng công việc đã được giảm bớt và chất lượng chung của những thay đổi và hiệu chỉnh đã được cải thiện.

6.2.2. Giá thành bảo trì

Theo thống kê, giá thành cho việc bảo trì các phần mềm tăng lên một cách đều đặn trong suốt 20 năm qua. Trong những năm 1970, bảo trì chiếm đến 35 đến 40 phần trăm kinh phí phần mềm dành cho tổ chức hệ thống thông tin. Tỷ lệ này đã nhảy tới con số 60 vào giữa những năm 1980. Và nhiều công ty đã chi 80% kinh phí cho việc bảo trì vào giữa những năm 1990.

Chi phí cho việc bảo trì là rõ ràng nhất. Tuy nhiên những chi phí khác khó thấy hơn có thể gây ra mối quan tâm lớn hơn:

- Một chi phí khó xác định của việc bảo trì phần mềm là các cơ hội phát triển bị bỏ qua vì các tài nguyên có sẵn đều dành cho nhiệm vụ bảo trì.
- Sự không hài lòng của người dùng khi các yêu cầu có vẻ hợp lý cho việc sửa chữa hay sửa đổi không được chú ý một cách hợp lý.
- Việc suy giảm chất lượng nói chung do lỗi tạo ra bởi sự thay đổi trong các phần mềm được bảo trì.
- Một yêu cầu bất chợt làm ngắt quãng quá trình phát triển của một nhân viên buộc anh ta tiến hành công việc bảo trì.

Chi phí cuối cùng cho việc bảo trì là sự giảm sút kinh khủng về năng suất lao động (được đo theo số dòng lệnh -LOC- của một người trong một tháng hay số tiền chi phí cho dòng lệnh). Sự giảm sút này xuất hiện khi tiến hành bảo trì đối với các phần mềm cũ. Người ta đã ghi nhận sự giảm sút năng suất lao động theo tỷ lệ 40:1, có nghĩa là chi phí cho việc phát triển trị giá \$25.00 trên một dòng lệnh sẽ có thể trị giá tới \$1000.00 cho việc bảo trì mỗi dòng lệnh [1].

Công sức cho việc bảo trì có thể được phân chia thành các thao tác làm việc: phân tích, ước lượng, thay đổi thiết kế, và sửa chữa chương trình nguồn và các thao tác lặp lại: việc cố gắng hiểu chương trình nguồn làm gì, cố gắng sáng tỏ cấu trúc dữ liệu, các thuộc tính giao diện, giới hạn của việc thực hiện, ... Biểu thức dưới đây cung cấp một mô hình cho công việc bảo trì:

$$M = p + K * \exp(c-d),$$

với M = toàn bộ các công việc cho việc bảo trì;

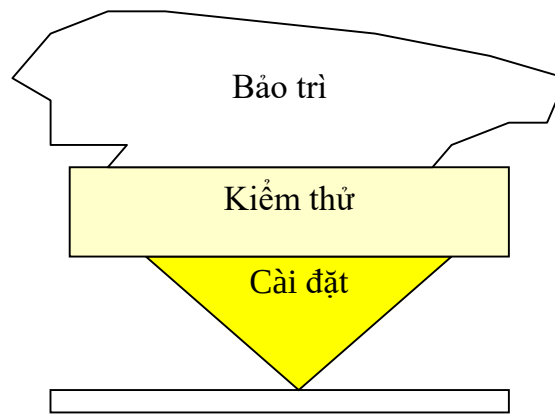
p = công việc làm (như miêu tả ở trên);

K = hằng số kinh nghiệm;

c = đánh giá mức độ phức tạp được tính cho việc thiếu thiết kế về cấu trúc và dữ liệu;

d = đánh giá mức độ hiểu biết về phần mềm.

Mô hình trên đây cho thấy công việc và giá thành có thể tăng lên theo cấp số mũ nếu một phương pháp phát triển phần mềm kém cỏi được sử dụng - tức là thiếu sót của công nghệ phần mềm, và nếu một người hay một nhóm dùng các phương pháp không có giá trị để bảo trì phần mềm. Chi phí cho bảo trì khi phần mềm được phát triển không đúng phương pháp được minh họa ở hình sau:



Hình 6.1 Chi phí của việc phát triển phần mềm không có phương pháp

6.2.3. Một số vấn đề khác

Hầu hết các vấn đề liên quan tới việc bảo trì phần mềm đều liên quan tới các sai lệch trong cách xây dựng và phát triển phần mềm. Sự thiếu sót trong việc điều khiển và tổ chức trong hai giai đoạn đầu tiên của một chu trình phần mềm gần như luôn luôn tạo ra các vấn đề giai đoạn cuối.

Nhiều vấn đề kinh điển có thể liên quan tới việc bảo trì phần mềm được trình bày dưới đây:

- Rất khó hoặc không thể theo dõi sự tiến hóa của phần mềm qua các phiên bản. Các thay đổi không được tư liệu hóa.
- Khó theo dõi được các quá trình xử lý được tạo bởi các phần mềm.
- Thường xuyên gặp rất nhiều khó khăn trong việc tìm hiểu chương trình của người khác viết. Những khó khăn này tăng lên khi số thành phần các cấu hình của phần mềm giảm đi. Nếu chỉ có các chương trình nguồn không có tài liệu hướng dẫn thì không nên tìm hiểu phần mềm đó.
- Những người viết phần mềm thường không có mặt để giải thích. Chúng ta không thể trông cậy vào những giải thích cá nhân của các nhà phát triển phần mềm khi việc bảo trì được yêu cầu.
- Các tài liệu chính xác không có hay thiếu trầm trọng. Phải thừa nhận rằng phải có tài liệu về phần mềm là bước đầu tiên, nhưng tài liệu phải hiểu được và phù hợp với chương trình lại là chuyện khác.

- Phần lớn các phần mềm không thiết kế để thay đổi, trừ phi sử dụng phương pháp thiết kế dùng các khái niệm về phân tách chương trình thành các module độc lập. Việc thay đổi phần mềm sẽ rất khó khăn và dẫn đến xu hướng sai.

- Việc bảo trì phần mềm không được coi là một công việc dễ dàng mà công việc bảo trì phần mềm luôn liên quan tới các sai lệch ở mức độ cao.

6.3 Công việc bảo trì phần mềm và một số hiệu ứng

6.3.1. Khả năng bảo trì

Khả năng bảo trì của phần mềm có thể coi như các khả năng hiểu, hiệu chỉnh, tiếp hợp hoặc có thể thêm vào khả năng phát triển. Khả năng bảo trì là chìa khóa để dẫn đến các phương pháp thiết kế xây dựng phần mềm.

6.3.1.1 Yếu tố kiểm soát

Khả năng bảo trì cơ bản bao gồm nhiều yếu tố. Sự thiếu cẩn trọng trong việc thiết kế, viết chương trình nguồn, kiểm tra có ảnh hưởng tiêu cực đến việc bảo trì có kết quả một phần mềm. Cấu hình yếu kém cũng có các tác động tương tự, thậm chí cả khi từng bước của kỹ thuật xây dựng phần mềm được xem xét một cách cẩn thận. Thêm vào đó nhiều yếu tố khác liên quan tới phương pháp phát triển phần mềm, như:

- Chất lượng hiệu quả của đội ngũ phần mềm.
- Cấu trúc của hệ thống dễ hiểu.
- Dễ dàng kiểm soát hệ thống.
- Dùng các ngôn ngữ lập trình chuẩn.
- Dùng các hệ điều hành chuẩn.
- Dùng các cấu trúc chuẩn tài liệu.
- Dùng được các tài liệu kiểm tra.
- Phương tiện gỡ rối đi kèm.
- Dùng được các máy tính tốt để thực hiện việc bảo trì.

Các yếu tố được đưa ra ở trên đã phản ánh những đặc điểm về phần cứng cũng như chương trình nguồn được dùng trong việc phát triển phần mềm. Những yếu tố khác chỉ ra sự cần thiết để có được phương pháp chuẩn, chương trình nguồn chuẩn. Có thể, yếu tố quan trọng nhất tác động tới khả năng bảo trì là kế hoạch cho khả năng bảo trì. Nếu coi phần mềm như là một hệ thống các thành phần sẽ phải chịu những thay đổi không tránh được, thì cơ hội tạo những phần mềm có khả năng bảo trì sẽ tăng thực sự.

6.3.1.2 Đánh giá định lượng

Khả năng bảo trì, như chất lượng hay độ tin cậy là hết sức khó xác định.

Tuy nhiên chúng ta có thể đánh giá khả năng bảo trì gián tiếp bằng việc xem xét các thuộc tính của các công việc bảo trì có thể đánh giá được:

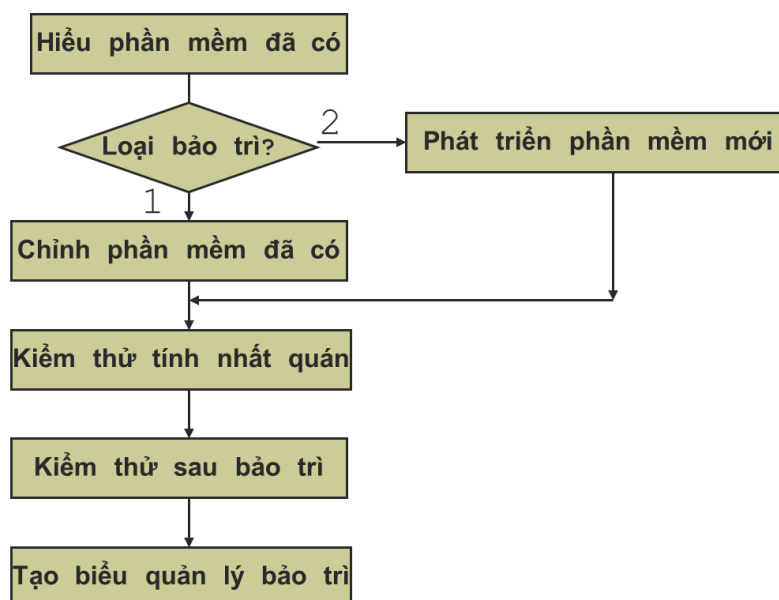
- Thời gian nhận biết vấn đề.

- Thời gian trễ do các công việc hành chính.
- Thời gian lựa chọn công cụ bảo trì.
- Thời gian phân tích vấn đề.
- Thời gian xác định sự thay đổi.
- Thời gian hiệu chỉnh (hay sửa đổi) thực sự.
- Thời gian chạy thử cục bộ.
- Thời gian chạy thử tổng thể.
- Thời gian tổng kết bảo trì.
- Tổng thời gian của công việc bảo trì.

Mỗi đánh giá trên thực tế là các dữ liệu đó có thể cung cấp cho người quản lý cùng với chỉ số về hiệu quả của công việc.

6.3.2. Các công việc bảo trì

Bảo trì cũng là một quy trình, và có thể mô tả quy trình bảo trì theo sơ đồ dưới đây:



Hình 6.2 Các công việc chính của quá trình bảo trì

Những công việc liên quan tới việc bảo trì như xác định loại bảo trì, phát triển phần mềm mới, kiểm thử, ... đã được tìm hiểu ở các nội dung trước. Trong phần này, chúng ta sẽ bàn đến các nhiệm vụ gồm: các tổ chức bảo trì được thiết lập; các thủ tục ghi nhận và đánh giá được miêu tả; và một loạt thứ tự chuẩn của các bước cho mỗi yêu cầu bảo trì phải được định nghĩa. Thêm vào đó, một thủ tục lưu trữ các hồ sơ cho các hoạt động bảo trì được thiết lập và bản tổng kết những tiêu chuẩn đánh giá được vạch rõ.

6.3.2.1 Cơ cấu bảo trì

Mặc dù những tổ chức bảo trì chuẩn không cần được thiết lập, nhưng sự ủy thác trách nhiệm rất là cần thiết kể cả cho các tổ chức phát triển phần mềm nhỏ. Những yêu cầu

bảo trì được chuyển qua cho người kiểm soát công việc bảo trì và từ đây chuyển tiếp yêu cầu tới người quản lý hệ thống để đánh giá. người quản lý hệ thống là thành viên của nhóm nhân viên kỹ thuật. Những nhân viên này có trách nhiệm về một phần nhỏ của chương trình sản phẩm. Khi một yêu cầu được đánh giá, người được ủy quyền quản lý việc thay đổi phải quyết định hành động nào được thực hiện tiếp.

Cơ cấu được miêu tả ở trên phục vụ cho việc thiết lập phạm vi trách nhiệm đối với công việc bảo trì. Người kiểm soát và người ủy quyền quản lý việc thay đổi có thể là một người hay là một nhóm quản lý và chuyên gia kỹ thuật cao cấp.

6.3.2.2 Báo cáo

Tất cả các yêu cầu về việc bảo trì phần mềm cần được trình bày theo một tiêu chuẩn. Người phát triển phần mềm thường cung cấp một đơn yêu cầu bảo trì còn được gọi là báo cáo các lỗi phần mềm. Báo cáo này được người sử dụng điền vào khi yêu cầu công việc bảo trì. Nếu xuất hiện một lỗi, bản mô tả đầy đủ tình huống dẫn đến lỗi bao gồm dữ liệu, đoạn chương trình và các yêu cầu khác phải được điền đầy đủ vào bản báo cáo. Nếu yêu cầu bảo trì là bảo trì tiếp hợp hay bảo trì hoàn thiện thì một yêu cầu chi tiết sẽ được thảo ra. Đơn yêu cầu bảo trì sẽ được người kiểm soát bảo trì và người quản lý hệ thống xem xét như phần trước đã nêu.

Đơn yêu cầu bảo trì được thiết lập từ bên ngoài và được coi như một cơ sở để đề ra kế hoạch của công việc bảo trì. Ngoài ra trong nội bộ của cơ quan, một báo cáo thay đổi phần mềm cũng được tạo ra. Nó chỉ ra: các công sức đòi hỏi để thỏa mãn một đơn yêu cầu bảo trì, trạng thái ban đầu của yêu cầu sửa đổi, mức ưu tiên của yêu cầu, các dữ liệu cần cho việc sửa đổi, ...

6.3.2.3 Lưu giữ các hồ sơ

Thường chúng ta không có đầy đủ các hồ sơ cho tất cả các giai đoạn trong chu kỳ sống của một phần mềm. Thêm nữa không có các hồ sơ về việc bảo trì phần mềm. Do đó chúng ta thường khó có thể tiến hành các công việc bảo trì có hiệu quả, không có khả năng xác định tính chất của các chương trình và khó xác định được giá bảo trì phần mềm.

Các thông tin cần phải lưu giữ trong hồ sơ bảo trì thường gồm:

- Dấu hiệu nhận biết chương trình.
- Số lượng các câu lệnh trong chương trình nguồn.
- Số lượng các lệnh mã máy.
- Ngôn ngữ lập trình được sử dụng.
- Ngày cài đặt chương trình.
- Số các chương trình chạy từ khi cài đặt.
- Số các lỗi xử lý xảy ra.
- Mức và dấu hiệu thay đổi chương trình.
- Số các câu lệnh được thêm vào chương trình nguồn khi chương trình thay đổi.

-
- Số các câu lệnh được xóa khỏi chương trình nguồn khi chương trình thay đổi.
 - Số giờ mỗi người sử dụng cho mỗi lần sửa đổi.
 - Ngày thay đổi chương trình.
 - Dấu hiệu của kỹ sư phần mềm.
 - Dấu hiệu của đơn yêu cầu bảo trì.
 - Kiểu bảo trì.
 - Ngày bắt đầu và kết thúc bảo trì.
 - Tổng số giờ của mỗi người dùng cho việc bảo trì.

6.3.2.4 Xác định giá bảo trì

Việc xác định giá trị bảo trì thường phức tạp do thiếu thông tin. Nếu tiến hành việc lưu giữ các hồ sơ có thể tiến hành một số cách đánh giá về việc thực hiện bảo trì. Theo các chuyên gia thì đánh giá về việc thực hiện bảo trì dựa vào:

- Số lượng trung bình các lỗi xử lý cho một lần chạy chương trình.
- Tổng số giờ của mỗi người dùng cho mỗi loại bảo trì.
- Số lượng trung bình các thay đổi theo chương trình, theo ngôn ngữ lập trình, theo kiểu bảo trì.
- Số giờ trung bình của mỗi người cho một dòng lệnh được thêm vào và xóa đi.
- Số giờ trung bình của mỗi người cho một ngôn ngữ lập trình.
- Thời gian trung bình cho việc bảo trì một đơn yêu cầu bảo trì.
- Tỷ lệ phần trăm của mỗi kiểu bảo trì.

6.3.3. Một số hiệu ứng lề của công việc bảo trì

Sửa đổi phần mềm là công việc nguy hiểm, ta thường gặp ba loại chính của hiệu ứng lề như sau:

6.3.3.1 Hiệu ứng lề của việc thay đổi mã nguồn

Một thay đổi đơn giản tới một câu lệnh đơn cũng có thể đem lại một hậu quả thảm khốc. Mặc dù không phải các ảnh hưởng đều là tiêu cực, nhưng việc sửa lỗi luôn dẫn đến các vấn đề phức tạp.

Mặc dù tất cả các thay đổi mã lệnh chương trình đều có thể tạo ra lỗi, nhưng tập hợp các thay đổi sau có thể gây ra nhiều lỗi hơn.

- Một chương trình con bị xóa hay thay đổi.
- Một dòng nhãn bị xóa hay thay đổi.
- Một biến bị xóa hay thay đổi.
- Các thay đổi để tăng khả năng thực hiện.
- Việc mở và đóng file bị thay đổi.
- Các phép toán logic bị thay đổi.

-
- Việc thay đổi thiết kế chuyển thành các thay đổi lớn về chương trình.
 - Các thay đổi ảnh hưởng đến việc chạy thử các trường hợp biên.

6.3.3.2 Hiệu ứng lề của việc thay đổi dữ liệu

Trong quá trình bảo trì, việc sửa đổi thường được tiến hành đối với các phần tử riêng rẽ của cấu trúc dữ liệu. Khi dữ liệu thay đổi, việc thiết kế phần mềm sẽ không còn phù hợp với dữ liệu và lỗi có khả năng xảy ra.

Hiệu ứng lề của dữ liệu xảy ra như là kết quả của việc thay đổi cấu trúc dữ liệu. Các thay đổi dữ liệu sau đây thường gây ra lỗi:

- Định nghĩa lại các hằng số cục bộ và hằng số địa phương.
- Định nghĩa lại cấu trúc bản ghi hay cấu trúc file.
- Tăng hoặc giảm kích thước một mảng.
- Thay đổi dữ liệu tổng thể.
- Định nghĩa lại các cờ điều khiển và các con trỏ.
- Xếp lại các tham số vào ra hay tham số của chương trình con.

Hiệu ứng lề dữ liệu có thể được hạn chế bằng tài liệu thiết kế mô tả cấu trúc dữ liệu và cung cấp một lời chỉ dẫn tham khảo đến từng phần tử dữ liệu, các bản ghi, các file và các cấu trúc khác của các module phần mềm.

6.3.3.3 Hiệu ứng lề của việc thay đổi tài liệu

Việc bảo trì thường tập trung vào cấu hình phần mềm và không tập trung riêng vào việc sửa đổi mã. Sự ảnh hưởng của tài liệu xảy ra khi thay đổi chương trình nguồn mà không thay đổi tài liệu thiết kế và tài liệu hướng dẫn sử dụng.

Bất cứ lúc nào có thay đổi về luồng dữ liệu, cấu trúc phần mềm, các thủ tục hay bất cứ cái gì có liên quan, tài liệu kỹ thuật phải được cập nhật. Tài liệu về thiết kế phản ánh không đúng trạng thái hiện tại của phần mềm có lẽ còn tồi tệ hơn không có tài liệu. Hiệu ứng lề xảy ra trong các lần bảo trì sau đó, khi việc nghiên cứu không kỹ các tài liệu kỹ thuật, dẫn tới sự đánh giá sai về các đặc tính của phần mềm. Đối với người sử dụng, phần mềm tốt chỉ khi có tài liệu hướng dẫn sử dụng chúng.

Các hiệu ứng lề trong tài liệu có thể được giảm về căn bản nếu toàn bộ cấu hình được xem xét trước khi phát hành phiên bản phần mềm tiếp sau. Thực tế một vài yêu cầu bảo hành có thể đòi hỏi không được thay đổi thiết kế của phần mềm hoặc mã chương trình, mà chỉ cần chỉ ra sự thiếu rõ ràng trong tài liệu của người sử dụng. Trong những trường hợp như vậy nỗ lực bảo trì tập trung vào tài liệu.

6.4 Một số hình thức bảo trì phần mềm

6.4.1. Bảo trì mã chương trình xa lạ

Các chương trình được gọi là mã chương trình xa lạ nếu:

-
- Không một thành viên nào trong phòng kỹ thuật tiếp tục phát triển chương trình đó nữa;
 - Không tiếp tục áp dụng lý thuyết phát triển, và vì vậy tồn tại vấn đề thiết kế nghèo nàn và ít tài liệu (theo tiêu chuẩn hiện tại);
 - Cấu trúc khối chưa được thiết kế theo tiêu chuẩn, và các khái niệm về thiết kế có cấu trúc chưa được áp dụng.
 - Yourdon đưa ra một số đề nghị hữu dụng cho người quản trị hệ thống phải bảo trì các chương trình xa lạ như sau:
 - Nghiên cứu chương trình trước khi bạn bị đặt vào "chế độ khẩn cấp". Cố gắng thu nhận được càng nhiều thông tin cơ sở càng tốt...
 - Cố gắng làm quen với toàn bộ các luồng điều khiển của chương trình; trước hết bỏ qua các chi tiết về mã chương trình. Sẽ rất có ích khi vẽ cho riêng bạn sơ đồ cấu trúc và sơ đồ luồng hoạt động mức cao, nếu chưa có bản nào tồn tại.
 - Đánh giá tình hình hợp lý của tài liệu hiện có; bổ sung thêm các lời chú thích của bản thân vào chương trình nguồn nếu thấy cần thiết.
 - Sử dụng tốt các danh sách chỉ dẫn tham khảo, các bảng ký hiệu, và các trợ giúp khác thông thường được chương trình dịch hoặc assembler cung cấp.
 - Thực hiện sửa đổi chương trình với sự chú ý lớn nhất. Lưu ý tới kiểu và dạng của chương trình tại tất cả các chỗ có thể. Đánh dấu trên chương trình những lệnh bạn đã sửa.
 - Đừng loại bỏ chương trình trừ khi bạn chắc chắn nó không được sử dụng nữa.
 - Đừng cố sử dụng chung các biến tạm thời và vùng nhớ làm việc mà đã có sẵn trong chương trình. Thêm các biến của riêng bạn để tránh các rắc rối.
 - Giữ các bản ghi chép chi tiết (về hoạt động bảo trì và các kết quả).
 - Tránh sự nóng vội vô lý bỏ chương trình cũ đi và viết lại nó.
 - Thực hiện các kiểm tra lỗi.

6.4.2. Công nghệ phản hồi và công nghệ tái sử dụng

Công nghệ phản hồi - Reverse engineering - đối với phần mềm là đơn giản. Trong nhiều trường hợp, chương trình được tổ chức ngược không phải thuộc nhà cạnh tranh mà thuộc bản thân công ty. Bí mật cần khám phá là do không còn giữ được các đặc tả. Do đó tổ chức ngược đối với phần mềm là quá trình phân tích chương trình trong cố gắng biểu diễn lại các chương trình ở mức độ trừu tượng cao hơn mã nguồn. Tổ chức lại là quá trình khám phá thiết kế. Các công cụ của công nghệ phản hồi lấy ra dữ liệu, kiến trúc, các thông tin thiết kế thủ tục từ chương trình đã tồn tại.

Công nghệ tái sử dụng, Re-engineering, không đơn thuần phát hiện các thông tin thiết kế mà còn dùng các thông tin này để biến đổi hoặc tổ chức lại hệ thống đã tồn tại với mục đích cải thiện chất lượng. Trong nhiều trường hợp, phần mềm ứng dụng lại các chức

năng của hệ thống tồn tại. Nhưng trong cùng thời điểm, nhà phát triển phần mềm cũng phải thêm các chức năng mới và cải thiện các xử lý.

6.4.3. Bảo trì phòng ngừa

Bảo trì phòng ngừa các phần mềm máy tính là một vấn đề khá mới và còn đang được tranh cãi. Thay vì đợi cho đến khi nhận được yêu cầu bảo trì, các tổ chức phát triển hay bảo trì chọn một chương trình mà:

- Sẽ được sử dụng trong một số năm định trước;
- Hiện đang được sử dụng tốt;
- Dễ bị thay đổi hoặc nâng cấp trong tương lai gần.

Đầu tiên ý kiến đề nghị phát triển lại một chương trình lớn khi một phiên bản đang làm việc đã tồn tại ta thấy dường như quá lãng phí. Nhưng chúng ta hãy xem xét các điểm sau:

- Chi phí để bảo trì một dòng mã lệnh có thể lớn hơn 20 tới 40 lần chi phí cho phát triển ban đầu dòng lệnh đó.
- Thiết kế lại cấu trúc của phần mềm, với sự sử dụng các khái niệm thiết kế hiện tại có thể làm cho việc bảo hành tương lai dễ dàng hơn.
- Bởi vì khuôn mẫu phần mềm đã tồn tại, năng suất phát triển chương trình chắc sẽ cao hơn mức trung bình nhiều.
- Người sử dụng bây giờ đã làm quen với phần mềm. Vì vậy, các đòi hỏi mới và hướng thay đổi có thể tìm ra dễ dàng hơn nhiều.
- Các công cụ CASE dành cho kỹ thuật đảo ngược (reverse engineering) và tái thiết kế (re-engineering) sẽ thực hiện tự động một số phần của công việc.
- Một cấu hình phần mềm sẽ tồn tại trên sự hoàn thành của bảo trì phòng ngừa.

Khi một tổ chức phát triển phần mềm bán phần mềm như là một sản phẩm, bảo trì phòng ngừa được xem như "phiên bản mới" của chương trình. Nhiều hãng phát triển phần mềm lớn có thể có từ 500 tới 2000 sản phẩm chương trình trong phạm vi trách nhiệm của nó. Các chương trình như vậy có thể được xếp theo thứ tự ưu tiên và xem xét lại như các ứng cử cho bảo trì phòng ngừa.

6.4.4. Chiến lược phần mềm thành phần

Như đặc tính cổ điển của bảo hành phần cứng là tháo bỏ phần hỏng và thay thế bằng phụ tùng mới. Một khái niệm được gọi là nguyên mẫu phần mềm có thể dẫn tới việc phát triển các phụ tùng cho các chương trình.

Nguyên mẫu phần mềm là một quá trình mô hình hóa yêu cầu người dùng trong một hay nhiều mức chi tiết, bao gồm cả các mô hình làm việc. Các tài nguyên của dự án được xếp đặt làm sao để sản xuất các phiên bản phần mềm được mô tả theo yêu cầu phải nhỏ đi. Phiên bản nguyên mẫu làm cho người dùng, người thiết kế và quản trị, ... có thể xem lại

được phần mềm. Quá trình đó sẽ tiếp tục khi được đề nghị, với phiên bản đang chạy chuẩn bị sẵn sàng phát hành sau vài lần làm lại.

Nếu các mức nguyên mẫu khác nhau được phát triển, nó có thể có một bộ các phụ tùng phần mềm có thể được sử dụng khi nhận được yêu cầu bảo trì hiệu chỉnh. Ví dụ một module phân tích có thể được thiết kế và thực hiện theo hai cách khác nhau nhưng có cùng giao diện bên ngoài. Một phiên bản của module có được sử dụng trong phần mềm làm việc. Nếu module đó hỏng, một phụ tùng có thể được lắp thay ngay.

Mặc dù chiến lược phụ tùng thay thế cho phần mềm có vẻ khác thường một chút, nhưng không có bằng chứng gì chứng tỏ nó tốn kém, khi chúng ta tính đến chi phí cho tất cả chu kỳ sống của phần mềm.

6.5 Quản lý thay đổi phần mềm

Quản lý thay đổi phần mềm đề cập đến việc chuyển đổi từ trạng thái hiện có của sản phẩm phần mềm sang trạng thái cải tiến khác của sản phẩm. Nó kiểm soát, hỗ trợ và quản lý các thay đổi đối với tạo tác, chẳng hạn như thay đổi mã, thay đổi quy trình hoặc thay đổi tài liệu

Các ứng dụng thường xuyên phải thiết kế lại do sự phân công của một nhóm quản lý mới, dự án vượt quá ngân sách, ứng dụng chậm và có nhiều lỗi, và sự thiếu tin tưởng của chủ sử dụng về việc các kỹ sư phần mềm hiểu rõ các yêu cầu của mình, ...

Các thay đổi có thể là các yêu cầu, thiết kế, chương trình, giao diện, phần cứng hoặc phần mềm phải mua. Phần lớn các thay đổi bắt nguồn từ bên trong tổ chức phát triển ứng dụng, nhưng cũng có thể được kích hoạt từ các tác nhân bên ngoài, ví dụ như thay đổi về luật.

Việc quản lý thay đổi ứng dụng giúp cho nhóm triển khai bỏ qua những ý thích chợt nảy ra của người sử dụng trong khi vẫn cho phép thực hiện các yêu cầu hợp lý.

6.5.1. Các thủ tục quản lý thay đổi

Quản lý điều khiển thay đổi có hiệu lực từ khi sản phẩm đầu tiên được chấp nhận là hoàn thiện cho đến khi dự án kết thúc. Trước tiên, các sản phẩm công việc cơ sở được tạo lập để đưa vào quản lý. Một sản phẩm công việc cơ sở là một sản phẩm được coi là hoàn thiện và là cơ sở cho các công việc hiện tại khác của nhóm triển khai dự án. Ví dụ như, một tài liệu cơ sở là bản quy định yêu cầu chức năng sau khi nó đã được chấp nhận bởi người sử dụng.

Dưới đây là ví dụ một quá trình của các thao tác yêu cầu thay đổi của một đặc tả chức năng:

- Tạo yêu cầu mở.
- Khai báo file tác động.
- Phê chuẩn file về thời gian và chi phí do chủ, người sử dụng ký.
- Hoàn thiện danh sách và kiểm soát về thay đổi của người điều hành dự án.

- File tài liệu liên quan đến thay đổi. Nếu tài liệu hoặc chương trình bị thay đổi, thì xác định ngày và các mục cập nhật đã hoàn thiện. Nếu các thủ tục hoặc thử nghiệm bị thay đổi, xác định các ngày mà việc sửa đổi xảy ra.

- Mẫu yêu cầu đóng file được chủ hoặc người sử dụng thông qua.
- Tóm tắt các ngày tháng, quá trình và chi phí.

Trước tiên, tài liệu cơ sở được giữ nguyên, sau đó thêm vào các yêu cầu thay đổi. Khi quy định chức năng được cập nhật để điều tiết thay đổi, nó được đóng băng lại và công việc lại tiếp tục. Ba yêu cầu trước có thể được thêm vào ứng dụng nếu chúng không làm thay đổi ứng dụng nhiều. Chúng cũng có thể bị bỏ qua cho đến sau khi ứng dụng đã được thực hiện.

Các thay đổi có thể phân loại theo một số cách.

- Thứ nhất, chúng được phân theo kiểu như loại bỏ lỗi, cải tiến thực hiện hoặc thay đổi chức năng.

- Thứ hai, thay đổi phân loại thành yêu cầu và lựa chọn.

- Thứ ba, phân theo độ ưu tiên như khẩn cấp, lệnh với một ngày kết thúc yêu cầu, lệnh với ngày bắt đầu yêu cầu hoặc ưu tiên thấp.

- Thông thường, kiểu loại bỏ lỗi là khẩn cấp theo yêu cầu, trong khi thay đổi chức năng là bảo dưỡng lệnh theo yêu cầu, và cải tiến thực hiện là lựa chọn và có thể không có ưu tiên.

Việc biết được loại yêu cầu thay đổi quyết định xem liệu nó có cần phải chịu điều khiển thay đổi hay không. Các thay đổi khẩn cấp thường phá vỡ thủ tục điều khiển thay đổi do các công việc thực hiện tuần tự nhưng chúng lại được tài liệu hoá sau khi thay đổi đã kết thúc. Tất cả các loại thay đổi khác đều phải tuân theo các điều khiển thay đổi. Ví dụ như thay đổi về yêu cầu chức năng có thể xảy ra bất cứ lúc nào, nhưng khi quy định yêu cầu chức năng được thông qua thì nó đóng băng cho đến khi ứng dụng hoạt động. Các thay đổi phải chịu sự điều khiển thay đổi: chúng được thêm vào danh sách yêu cầu thay đổi để xem xét trong tương lai trừ khi đó là một thiết kế khẩn cấp.

Một thủ tục điều khiển thay đổi yêu cầu người sử dụng phải đệ trình một lời yêu cầu thay đổi chính thức cho người điều hành dự án:

- Người sử dụng gửi cho người điều hành dự án và người chủ một mẫu yêu cầu thay đổi.

- Người điều hành dự án và kỹ sư phần mềm triển khai một khai báo tự động. Vào lúc đó, danh sách kiểm soát của người điều hành dự án được dùng để xác định tất cả các hoạt động và thay đổi công việc có liên quan tới yêu cầu.

- Yêu cầu thay đổi được thảo luận với chủ sử dụng để vạch ra các thay đổi về ưu tiên, tiến trình và chi phí.

-
- Thoả thuận được chính thức hoá và chủ sử dụng thông qua thay đổi về tiến trình và chi phí.
 - Sử dụng khai báo tác động, ứng dụng và tất cả các tài liệu có liên quan được thay đổi.
 - Thực hiện thay đổi: khi các nhiệm vụ hoàn thành, xoá nhiệm vụ trong danh sách kiểm soát của người điều hành dự án.
 - Chủ sử dụng thông qua việc đóng yêu cầu và yêu cầu được đóng.
 - Người điều hành dự án và kỹ sư phần mềm định nghĩa các tác động tiến trình và chi phí của thay đổi. Sau đó các thay đổi được bàn bạc với người sử dụng. Dựa trên thương lượng với người sử dụng, thay đổi được gán một ưu tiên hoạt động, và chi phí và tiến trình được thay đổi.

Yêu cầu, ngày dự định hoạt động, thay đổi tiến trình và tăng chi phí được thêm vào một file quá trình dự án. Các thay đổi có thể được quản lý bởi một nhân viên điều khiển thay đổi, là một người có nhiệm vụ bảo dưỡng quá trình dự án và các bản ghi điều khiển thay đổi, và hàng tháng in ra một bản báo cáo điều khiển thay đổi. Một file điều khiển thay đổi chứa tất cả các yêu cầu, thư từ và tài liệu về các thay đổi. Một yêu cầu thay đổi mở có thể được tạo ra khi yêu cầu được đưa ra và một số lượng thay đổi được gán. Yêu cầu thay đổi mở nằm trong file cho đến khi yêu cầu được hoàn thành, đóng và được báo cáo.

Khi thay đổi được thực hiện, các mục có ảnh hưởng được cập nhật, bao gồm tư liệu tương ứng, mã, ... Một danh sách kiểm soát của người điều hành dự án được dùng để loại bỏ các hoạt động đã được yêu cầu. Tài liệu mới được nhân viên điều khiển thay đổi sắp xếp và phân phối nó cho những người có quan tâm. Ngày hoàn thành thay đổi được đưa vào file điều khiển thay đổi. Thay đổi được xác định khi được đóng trong báo cáo tình trạng tới và yêu cầu mở được chuyển từ file điều khiển thay đổi sang.

Dựa trên tổ chức này, người điều hành hệ thống có thể theo dõi các yêu cầu thay đổi của dự án để nhận biết sự thành công trong nhóm các yêu cầu. Chi phí thay đổi chung của một năm thường được sử dụng như là một chỉ tiêu để chỉ ra xem ứng dụng đang có triển vọng hay cần vứt bỏ hay cần công nghệ hoá lại. Trong những trường hợp này, cả chi phí và số lượng các yêu cầu thay đổi đều được theo dõi thông qua quá trình điều khiển thay đổi. Các báo cáo tổng kết bởi dự án thay đổi trong một thời kỳ nhất định, hoặc so sánh theo thời kỳ có thể được triển khai.

6.5.2. Ghi quyết định theo thời gian

Khi bắt đầu một dự án, người điều hành dự án và kỹ sư phần mềm quyết định sử dụng các công cụ để lưu trữ quá trình quyết định. Có nghĩa là có thể dùng công cụ điện tử hoặc một phiên bản viết tay và các quyết định được duy trì dưới dạng văn bản. Với công cụ điện tử, các bản sao điện tử được lưu trữ. Với công cụ ghi tay, phiên bản cũ được cập nhật và đổi tên khi một tài liệu thay đổi. Ví dụ như, các quy định chức năng của công ty ABC có thể được đặt tên là ABCFS-mmddyy, trong đó ABC là công ty, FS là viết tắt của

quy định chức năng (Functional Specification) và mmdyy là ngày tháng. Phần ngày tháng của tên sẽ thay đổi do bất kỳ một thay đổi quan trọng nào của tài liệu.

6.5.3. Quản lý thay đổi tài liệu

Các thay đổi tài liệu có thể được xác định bởi một bảng nội dung các thay đổi tại đầu mỗi tài liệu. Bảng nội dung các thay đổi bao gồm ngày hiệu lực, các phân bị ảnh hưởng của tài liệu và một tóm tắt về thay đổi. Mục đích của bảng nội dung các thay đổi là để tóm tắt tất cả các thay đổi cho người đọc.

Các thay đổi nên được đánh dấu đỏ trong văn bản để xác định được bộ phận thay đổi. Nếu nội dung cũ là quan trọng thì nó có thể được đưa vào trong chú ý, được ghi ngày tháng, và được dán nhãn là phiên bản trước. Cần nhớ rằng phải giữ tài liệu phiên bản cũ để dùng cho quá trình phát triển.

Câu hỏi và bài tập vận dụng

Câu hỏi thảo luận:

Câu 1: Chuyển giao phần mềm là gì? Phân tích các hoạt động chính của chuyển giao phần mềm?

Câu 2: Bảo trì phần mềm là gì? Có những loại bảo trì phần mềm nào?

Câu 3: Hãy đánh giá các hoạt động chính yếu trong bảo trì phần mềm?

Câu 4: Phân tích một số hình thức bảo trì phần mềm?

Câu 5: Quản lý thay đổi phần mềm là gì? Hãy phân tích các công việc của quá trình quản lý thay đổi phần mềm?

Câu hỏi trắc nghiệm:

Câu 1: Trong quá trình phát triển phần mềm, giai đoạn bảo trì phần mềm có nghĩa là:

- a. Thay đổi và phát triển hệ thống đã được xây dựng
- b. Xây dựng không gian giải pháp cho vấn đề
- c. Xây dựng một hệ thống thực hiện được dựa vào thiết kế
- d. Thẩm định lại tính đúng đắn của giải pháp theo yêu cầu ban đầu đã đặt ra

Câu 2: Bảo trì có cấu trúc là:

- a. Bảo trì cấu trúc dữ liệu của phần mềm
- b. Bảo trì phần mềm theo cấu trúc từ trên xuống dưới
- c. Bảo trì phần mềm theo cấu trúc từ dưới lên trên
- d. Bảo trì có hệ thống được phát triển theo đúng trình tự của kỹ nghệ phần mềm (phân tích, thiết kế, lập trình, kiểm định)

Câu 3: Một chương trình con bị xóa hay thay đổi có thể dẫn tới dạng hiệu ứng lè nào của công việc bảo trì?

- a. Hiệu ứng lè của việc thay đổi mã nguồn

-
- b. Hiệu ứng lề của việc thay đổi tài liệu
 - c. Hiệu ứng lề của việc thay đổi dữ liệu
 - d. Hiệu ứng lề của việc thay đổi chương trình con

Câu 4: Nếu thành phần có được duy nhất của một cấu hình phần mềm là mã nguồn, hoạt động bảo trì sẽ diễn ra:

- a. Khá đơn giản.
- b. Bình thường như những phần mềm có đầy đủ thành phần khác.
- c. Khá phức tạp.
- d. Không thể tiến hành hoạt động bảo trì.

Câu 5: Trong bảo trì phần mềm, người ta đã ghi nhận sự giảm sút năng suất lao động theo tỷ lệ 40:1, có nghĩa là:

- a. Chi phí cho việc phát triển trị giá \$25.00 trên một dòng lệnh sẽ có thể trị giá tới \$1000.00 cho việc bảo trì mỗi dòng lệnh.
- b. Chi phí cho việc phát triển trị giá \$1000.00 trên một dòng lệnh sẽ có thể trị giá tới \$25.00 cho việc bảo trì mỗi dòng lệnh.
- c. Chi phí cho việc phát triển trị giá \$25.00 trên một dòng lệnh sẽ có thể trị giá tới \$1000.00 cho việc bảo trì mỗi chương trình.
- d. Chi phí cho việc phát triển trị giá \$1000.00 trên một chương trình sẽ có thể trị giá tới \$25.00 cho việc bảo trì mỗi dòng lệnh.

Bài tập vận dụng:

Bài 1: Bạn được giao nhiệm vụ bảo trì phần mềm “Quản lý sinh viên của một trường Đại học”, bạn hãy phân tích định hướng và các công việc, phương pháp để thực hiện nhiệm vụ này?

Bài 2: Bạn được giao nhiệm vụ bảo trì phần mềm “Game 3D”, bạn hãy phân tích định hướng và các công việc, phương pháp để thực hiện nhiệm vụ này?

Bài 3: Bạn được giao nhiệm vụ bảo trì phần mềm “Website quản lý bán hàng online của 1 hệ thống bán quần áo”, bạn hãy phân tích định hướng và các công việc, phương pháp để thực hiện nhiệm vụ này?

Bài 4: Bạn được giao nhiệm vụ bảo trì ứng dụng dạng “Diệt virus”, bạn hãy phân tích định hướng và các công việc, phương pháp để thực hiện nhiệm vụ này?

Bài 5: Hãy chọn một phần mềm mà bạn từng tham gia phát triển hoặc bạn có định hướng tìm hiểu, hãy phân tích định hướng, các công việc và phương pháp để thực hiện nhiệm vụ bảo trì phần mềm này?

TÀI LIỆU THAM KHẢO

- [1]. Roger S. Pressman, Ph.D, “*Software Engineering*”, FIFTH EDITION, McGraw-Hill Higher Education, 2017.
- [2]. Thạc Bình Cường, “*Nhập môn Công nghệ Phần mềm*”, Nhà xuất bản Giáo dục, 2011.
- [3]. Bộ môn HTTT, Khoa CNTT, “*Giáo trình Công nghệ Phần mềm*”, Đại học Hàng Hải Việt Nam, 2021.
- [4]. QuickTes Pro, Junit, Selenium IDE, Selenium RC / Phạm Quang Huy, Phạm Quang Hiển, “*Giáo trình thực hành Kiểm thử phần mềm*”, Nhà xuất bản Thanh niên, 2020.
- [5]. Nguyễn Thị Kiêm Ái, “*Kiểm thử phần mềm từng bước trở thành Tester chuyên nghiệp*”, Nhà xuất bản Thanh niên, 2020.